

永磁同步电机 控制解析

整理编辑：梁江枫

编制日期：2020-12-30

版本：v2.0

目录

电机 FOC 控制	5
1. 角度处理模块 AngleModule	6
1.1 电角度与零位 AngleDeal	6
1.2 机械转速计算 CalMotorVelo	9
2. 电流 Id 和 Iq 反馈 CalIdqFb	11
2.1 坐标变换	11
2.2 滑动平均滤波处理	16
3. 转矩电流补偿 TorqueCurCom	17
3.1 电流矢量的分解	17
3.2 稳态下的转矩补偿	17
4. 电流 PI 调节 CurrentPIAdjust	19
4.1 位置式 PID 模拟控制的离散算法	19
4.2 改进的数字 PI 算法	20
4.3 积分量化误差的消除	21
5. 计算电机电压 CalUdUq	23
5.1 PMSM 的数学模型	23
5.2 PMSM 矢量控制系统的标幺化设计	24
6. 电压 PI 调节与弱磁 VoltagePIAdjust	27
6.1 弱磁升速策略	27
6.2 电压环输出实现自动弱磁	28
7. 死区补偿 DeadTimeCom	31
7.1 死区效应分析	31
7.2 SVPWM 逆变器时间补偿法	31
7.3 电流极性的判断	35
8. 七段 SVPWM 调制 SevenSvpwm	39
8.1 基本电压空间矢量	39
8.2 参考矢量分解及作用时间分配	44
8.3 各扇区矢量的先后作用顺序及占空比	54
8.4 电压标幺化的基值变换处理	56
8.5 矢量所在扇区判断	59
8.6 矢量切换时刻及对应占空比	63
1. TI 官方例程占空比解析	72
1.1 参考矢量分解及作用时间分配	72
1.2 矢量所在扇区判断	82
1.3 矢量切换时刻及对应占空比	84

1.4 占空比解析	89
1. 导入工程	98
2. 设定两相旋转坐标的电压量	98
3. 逆 Park 变换计算两相静止坐标的电压量	99
4. SVGENDQ_MACRO(svgen1) 解释	101
5. PWM_MACRO(1, 2, 3, pwm1) 解释	109
6. 我的疑问	110
参考资料	112
参考资料	112
芯片 TI28x 系列驱动配置	113
【1】编码器 QEP 配置	113
硬件接口及说明	113
电角度的计算	113
速度的计算	117
位置的计算	118
外设配置	119
【2】位置脉冲 QEP 配置	121
外设配置	122
【3】PWM 配置	122
ePWM1,2,3 电机控制	122
ePWM4 母线泻放	132
ePWM5 逐波限流	133
ePWM6 模拟输出	134
【4】FPGA 外部接口 XINTF 配置	134
硬件接口及说明	134
外设配置	135
【5】CAN 通讯	138
硬件接口及说明	138
外设配置	138
【6】模拟量采样 ADC	142
硬件接口及说明	142
外设配置	143
功能函数	145
【备注】	147
【1】编码器 QEP 配置	147
硬件接口及说明	147
电角度的计算	147
速度的计算	151
功能函数	152
【2】位置脉冲 QEP 配置	153
外设配置	154
【3】PWM 配置	154
ePWM1,2,3 电机控制	154

ePWM4 母线泻放	162
ePWM5 逐波限流	163
ePWM6 模拟输出	164
【4】FPGA 外部接口 XINTF 配置	164
硬件接口及说明	164
外设配置	165
【5】CAN 通讯	168
硬件接口及说明	168
功能函数	168
【备注】	169
PWM 模块	169
主控模块 EPwm1Regs 的配置	169
从动模块 EPwm2Regs 和 EPwm3Regs 的配置	171
数据类型转换	174
用户底层	179
电机弱磁标定	180
控制策略和通讯	184
无位置传感器控制	184
1. 用 PWM 模块实现数模转换	184
1.1 放大倍数	184
1.2 滤波用的电阻电容及截止频率	185
2. 差分放大电路与电流采样	185
2.1 放大倍数与偏置电压	185
2.2 电流对称采样以及标幺化	185
3. 查表法计算反正切	185
3.1 从原理及分析到实现	186
3.2 数值的精度和象限的判断	189
4. 有传感器下的电角度的获取	189
4.1 光电编码器 Z 脉冲与磁链 d 轴夹角的关系	189
4.2 用求模取余的方法对电角度进行转换	190
4.3 霍尔 UVW 信号与磁极的初始定位	190
5. 初始位置的获取原理与电控	190
6. 七段 SVPWM 占空比分析	190
6.1 不变的数值计算分析	190
6.2 正三角与倒三角载波对应的占空比	190
7. 弱磁阈值与线性计算	190
9. 外同步启动用的周期弧度增量计算	190
10. 反电动势的获取及分析	190
10.1 反电动势采样的分析	191
10.2 反电动势采样的滤波与补偿	194
10.3 逆时针（负序）相与线反电动势关系	195
11. 无传感器 SMO 调试	195
11.1 外同步启动和 Icur/frq 控制	195

11.2 角度估算误差与转速电流双闭环.....	196
11.3 脉振高频注入法.....	196
参考资料.....	196
12. 扩展卡尔曼滤波器 EKF 算法	196
1 公式推导	196
2 在永磁同步电机无位置传感器控制的应用	201
3 矩阵求逆	203
4 其他的矩阵运算	206
参考资料.....	206
书签笔记	207
1. 电流检测中 AD 采样值处理的高效方法.....	207
2. 两种结构体成员初始化方式：顺序和乱序	209

格式说明

各级标题用相应的字号并加粗加红色

正文中文小四，代码小五

版本说明：

版本	日期	修改内容
V1.2	2017-03-30	电动汽车驱动
V2.0	2020-01-12	无传感控制

电机FOC控制

电机控制底层主要函数如下：

void MotorDrive(void)

```
{
    AngleModule();
    CalIdqFb();
    TorqueCurCom();
    CurrentPIAdjust();
    CalUdUq();
    VoltagePIAdjust();
    DeadTimeCom();
    SevenSvpwm();
}
```

1. 获得对 π 标么化，范围为-1到+1的Q15格式角度值，计算与角度值相关的量，如机械速度，电频率。

2. 把对CURRENT_SENSOR进行标幺化的Q15格式采样电流值（两相或三相）和旋变解码芯片（AD2S1205）得到的Q15格式的电角度值，经过克拉克变换和帕克变换后得到旋转两相坐标系的 I_d ， I_q 当前采样反馈值。

3. 通过前面的查表（有电机标定数据）或是人为设定（进行电机标定）得出相应转矩下对应的Q15格式电流矢量参考值和与d轴的夹角（Q15格式），再根据静止坐标系下的电压方程以及电压环的输出做调整，计算当前执行的 I_d ， I_q 给定值（Q15格式）。

4. 对电流进行PI调节，使之满足与之对应的转矩大小。

5. 计算电机的静止两相坐标系下的电压，输入量来自电流PI控制器的输出即 I_{dOut} 和 I_{qOut} ，通过电压方程与标幺化的变量计算给定的电压值 U_{dGd} 和 U_{qGd} （Q15格式）用于后续的电压环PI调节和七段SVPWM调制。

6. 电压环的计算与调整。若参考电压矢量大于最大限制值，则把电压环的输出用于调节当前执行的 I_d ， I_q 给定值。

7. 按照用当前静止坐标系下的电流大小计算出的电角度，判断三相电流极性后进行死区补偿。

8. 根据 U_d ， U_q 实现七段SVPWM调制。

1. 角度处理模块AngleModule

该函数获得对 π 标幺化，范围为-1到+1的Q15格式角度值，计算与角度值相关的量，如机械速度，电频率。

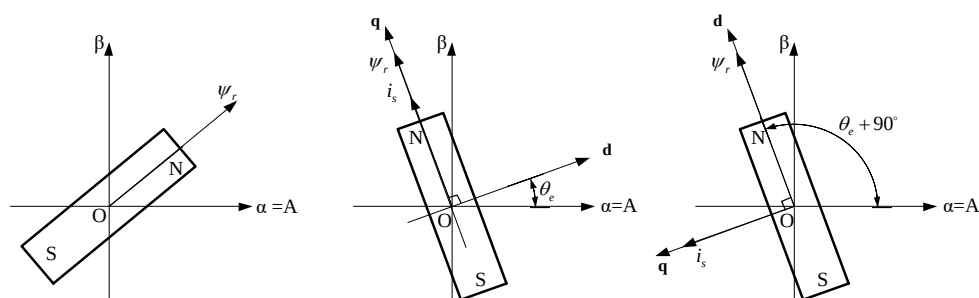
1.1 电角度与零位AngleDeal

1.1.1 转子相位初始化

因为在电动机转动之前。转子的位置是未知的。而转子磁场定向控制要求转子的位置必须是已知的，所以在电动机转动之前必须要对转子的相位进行初始化。

转子相位初始化采用磁定位的方法，它是通过给定子通以一个已知大小和方向的直流电，这样使定子产生一个恒定的磁场，这个磁场与转子的恒定磁场相互作用，迫使转子转到两个磁链成一线的位置而停正，从而得到转子的相位。

转子相位初始化还可以通过图1-1-1来进行详细说明。



(a) 转子在任意位置 (b) 通电后转子的位置 (c) dq 坐标系旋转 90°

图1-1-1 转子相位初始化

图1-1-1(a)表示转子处于一个未知位置。这时给定子通一个直流电流 i_s ， i_s

的d轴分量 $i_d = 0$ ，q轴分量 $i_q = i_n$ （ i_n 为额定电流），并且dq坐标系的相位为 θ_e 。

i_s 所产生的磁场与转子磁场作用，使转子转到图1-1-1(b)的位置。但是这个位置并不是我们希望的转子磁场定向位置，因为转子的磁场方向没有与d轴重合，所以需要将dq坐标系在 θ_e 的基础上再转 90° ，如图1-1-1(c)，实现转子磁场定向。

根据上面的道理，给定子通一个 $i_d = 0$ ， $i_q = i_n$ ， $\theta_e = -90^\circ$ 的直流电，就可以使转子转到d轴、A轴、 α 轴三轴都重合的位置，实现了转子相位的初始化。

如果转子转动，只要当前位置角度数值减去零位角度数值就可以得到转子磁极磁链的位置，再根据电机的极对数和旋转变压器的极对数计算出当前电机的电角度。

程序如下：

```
void AngleDeal(void)
{
    int16 AngleCom = 0;
    AngleCom = (MotorDriveInStr.AngleRdc - MotorDriveParaStr.MotorAngle0) << 4;
    MotorDriveVarStr.EleAngle = (int32)AngleCom * MotorDriveParaStr.MotorPole /
MotorDriveParaStr.RdcPole;
//电机零位测试, -90du, ud=0, uq=小值-----
    if (MotorDriveInStr.Debug == 1)
    {
        MotorDriveVarStr.EleAngle = -16384;
    }
}
```

1.1.2 高效获得Q15格式的角度值

以下代码高效处理使之变为Q15格式角度数值，参考图1-1-2。

```
AngleCom = (MotorDriveInStr.AngleRdc - MotorDriveParaStr.MotorAngle0) << 4;
```

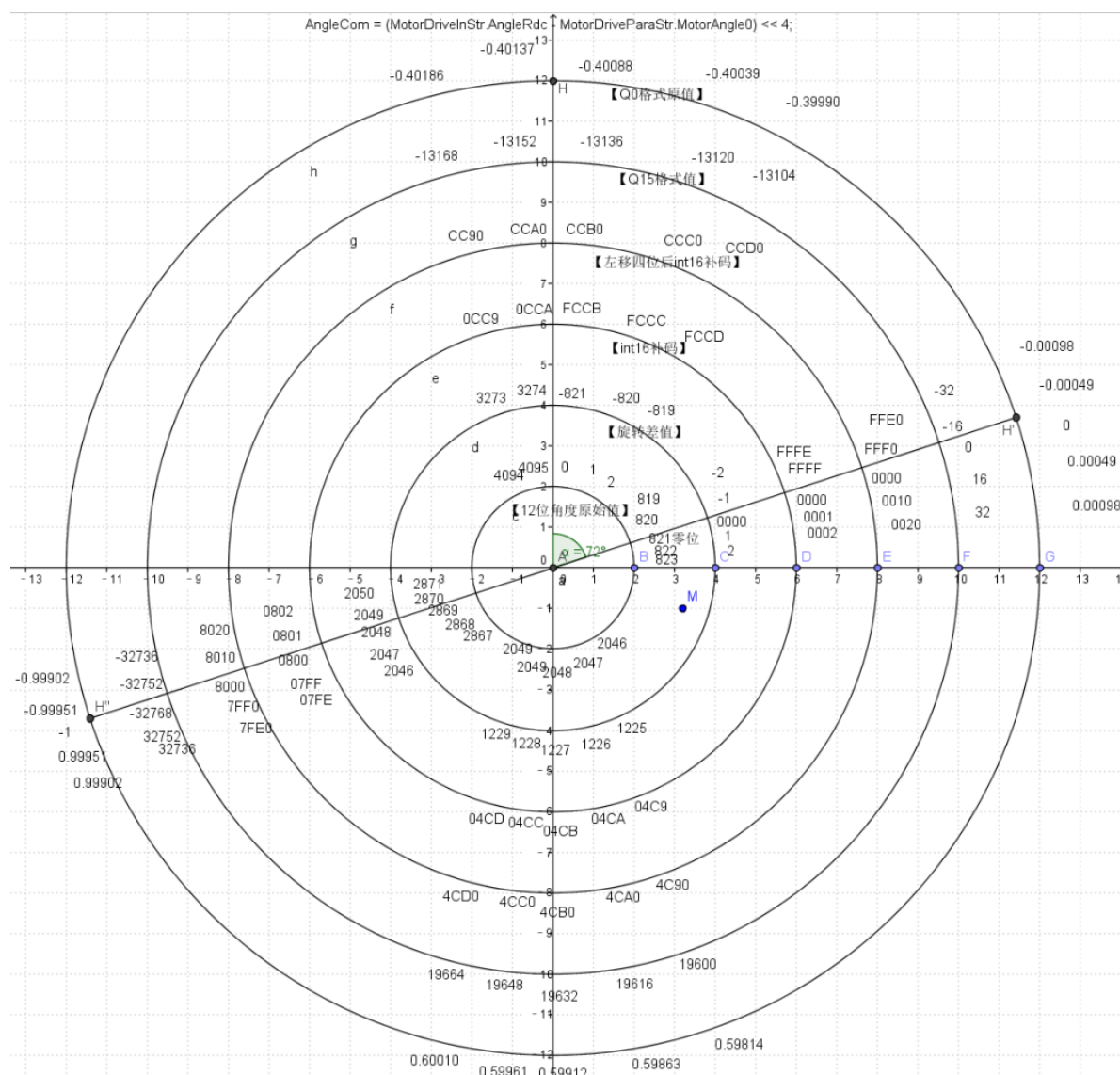
1，采集当前位置角度数值，AD2S1205芯片解码出来的12位值角位置数据格式为无符号二进制，全0表示0度，全1（4095）表示360度- 1 LSB。减去零位后以二进制补码的方式存储。

2，存储的值用16位有符号二进制补码形式存放。

3，左移4位变成对 180° 标幺化后的Q15格式，范围为-32768~32767。

4，方便观察图像，已经计算出标幺化后的Q0格式，范围为-1~1。

5，这里假设零位为821，旋变极对数为1，电机极对数为1。（电机极对数是旋变极对数的倍数关系，若两者极对数相同，获得的机械角度即为电角度）



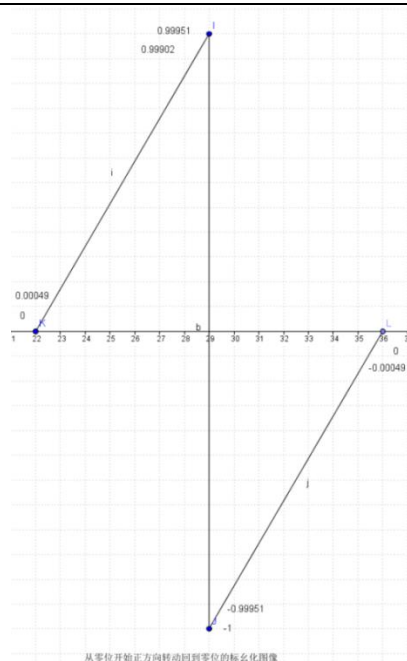


图1-1-3从零位开始按正转方向匀速转一圈回到零位的图像

然而实际中电机转子极对数是旋变极对数的整数倍，电角度的处理如下：

```
MotorDriveVarStr.EleAngle = (int32)AngleCom * MotorDriveParaStr.MotorPole /
MotorDriveParaStr.RdcPole;
```

这里将int32类型的变量赋值给int16类型的变量，数据截断即只取int32类型变量的低16位。假设电机6极对，旋变3极对，电机转子相对于旋变从零位开始按正转方向转1/3圈，分析如下：

RDC读取值	左移四位后int16补码	原Q15	原Q0	乘以2(int32)	截断为int16	Q15	Q0
0x000	0x0000	0	0.0000	0x0000 0000	0x0000	0	0.0000
0x001	0x0010	16	0.0005	0x0000 0020	0x0020	32	0.0010
0x002	0x0020	32	0.0010	0x0000 0040	0x0040	64	0.0020
.....							
0x3FF	0x3FF0	16368	0.4995	0x0000 7FE0	0x7FE0	32736	0.9990
0x400	0x4000	16384	0.5000	0x0000 8000	0x8000	-32768	-1.0000
0x401	0x4010	16400	0.5005	0x0000 8020	0x8020	-32736	-0.9990
.....							
0x7FE	0x7FE0	32736	0.9990	0x0000 FFC0	0xFFC0	-64	-0.0020
0x7FF	0x7FF0	32752	0.9995	0x0000 FFE0	0xFFE0	-32	-0.0010
0x800	0x8000	-32768	-1.0000	0x0001 0000	0x0000	0	0.0000
0x801	0x8010	-32752	-0.9995	0x0001 0020	0x0020	32	0.0010
0x802	0x8020	-32736	-0.9990	0x0001 0040	0x0040	64	0.0020
.....							
0xBFF	0xBFF0	-16400	-0.5005	0x0001 7FE0	0x7FE0	32736	0.9990
0xC00	0xC000	-16384	-0.5000	0x0001 8000	0x8000	-32768	-1.0000
0xC01	0xC010	-16368	-0.4995	0x0001 8020	0x8020	-32736	-0.9990
.....							
0xFFE	0xFFE0	-32	-0.0010	0x0001 FFC0	0xFFC0	-64	-0.0020
0xFFF	0xFFFF	-16	-0.0005	0x0001 FFE0	0xFFE0	-32	-0.0010
0x000	0x0000	0	0.0000	0x0000 0000	0x0000	0	0.0000

如果试转之后标定几次电机零位，则出现两个RDC原始读取的值，都可以正确表示电机的零位，它们相差2048。它们经过补码左移和数据截断后各自的Q0格式数值是一样的。如零位在-1(Q0)时可以是0x400也可以是0xC00，相差2048。

1.2 机械转速计算CalMotorVelo

1.2.1 计算机械转速

单位为rpm。需要的变量为转(单位为r)和时间(单位为min)。

先计算对180° 标幺化的Q15格式电角度变化差值再转为Q0格式(单位为 π)，按照比例可以换算成转(单位为r)

$$\frac{x(r)}{1(r)} = \frac{\Delta_{Q0}\pi}{2\pi} \Rightarrow x(r) = \frac{\Delta_{Q15} / 2^{15}}{2} \Rightarrow x(r) = \frac{\Delta_{Q15}}{2^{16}}$$

Δ_{Q15} 的计算代码为: AngleDiff = MotorDriveVarStr.EleAngle - AngleEleLast;

$x(r)$ 的计算代码为: (Temp32 * AngleDiff >> 16)

计算时间: 中断周期为MotorDriveInStr.IsrPer(单位为us), 前面有延时4个中断周期, 按照比例可以转为时间(单位为min)

$$\frac{x(\min)}{1(\min)} = \frac{4 * T_{isr} * 10^{-6}(s)}{60(s)} \Rightarrow x(\min) = \frac{T_{isr}}{15 * 10^6}$$

作为分母变换为因子为: $\frac{1}{x(\min)} = \frac{15 * 10^6}{T_{isr}}$

$\frac{1}{x(\min)}$ 的计算代码为: Temp32 = 15000000 / MotorDriveInStr.IsrPer;

前面计算的是电角度变化差值, 需要除以电机极对数才能得到机械转速, 综上所述计算的代码(单位为rpm)如下:

AngleArr[j] = (Temp32 * AngleDiff >> 16) / MotorDriveParaStr.MotorPole;

1.2.2 滑动平均滤波处理

然而这里的速度计算是一种对时间的微分, 难免存在有干扰, 需要对其进行滤波处理, 这里采用的是滑动平均滤波。

滑动平均滤波算法

只采样一次, 将这一次采样值和过去的若干次采样值一起求平均, 得到的有效采样值即可投入使用。如果取若干个采样值求平均, RAM中必须开辟若干个数据的暂存区。每新采集一个数据便存入暂存区, 同时去掉一个最老最旧的数据, 保持这个数据始终是最近的数据。这种数据存放方式可以用环形队列结构方便地实现, 每存入一个新数据便自动冲去一个最老的数据。

说明:

1. 采样值: (Temp32 * AngleDiff >> 16) / MotorDriveParaStr.MotorPole为机械转速(单位为rpm)并将其放入AngleArr[j]内存单元

代码如下: AngleArr[j] = (Temp32 * AngleDiff >> 16) / MotorDriveParaStr.MotorPole;

2. 数据的暂存区: AngleArr[64]数组, 控制循环的变量j

3. 求64个机械转速之和, 每次中断逐渐累加完。

RpmSum = RpmSum + AngleArr[j];

4. 对64个数值之和RpmSum取平均: $RPM2 = RpmSum \gg 6$;这里方便微机快速运算, 等价于除以 $2^6=64$,
5. 自动冲去一个最老的数据: 在此之前需要把j在0~63范围内循环自增以实现“自动”, 这里j已经自增。代码如下 $RpmSum = RpmSum - AngleArr[j]$;

1.2.3 类滑动平均滤波

类滑动平均滤波算法, 这里存储区只是一个变量RpmSum2。

```
RPM2 = RpmSum >> 6; //LJF 【第一次处理】: 转速滑动平均滤波完毕
//LJF 【第二次处理】: 再次求和平均, 这个是类滑动平均滤波
RpmSum2 -= RpmSum2 >> 6; //先右移再做减法最后赋值, 看作为去掉一个旧值
RpmSum2 += RPM2; //LJF 求和
MotorDriveOutStr.MotorRpm = RpmSum2 >> 6; //LJF 64个数值取平均
```

1.2.4 计算电流频率

不管正反转, 取其绝对值。

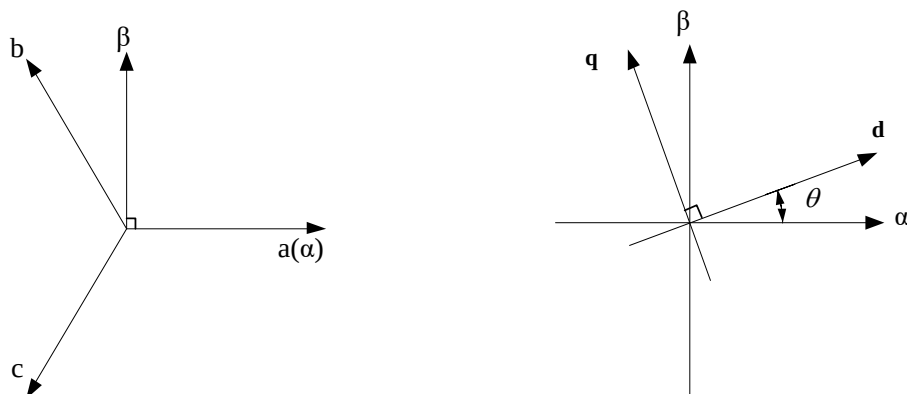
```
//LJF 电流频率we计算: 有机械角速度we *电机极对数np /60: we(rpm->rps)=np*wm/60
CurFreq = (int32)MotorDriveOutStr.MotorRpm * MotorDriveParaStr.MotorPole / 60;
MotorDriveOutStr.CurFreq = _IQabs(CurFreq);
```

2. 电流Id和Iq反馈CalIdqFb

该函数把对CURRENT_SENSOR进行标么化的Q15格式采样电流值(两相或三相)和旋变解码芯片(AD2S1205)得到的Q15格式的电角度值, 经过克拉克变换和帕克变换后得到旋转两相坐标系的Id, Iq当前采样反馈值。

2.1 坐标变换

以下都为等幅值变换, 即各坐标系下合成的矢量(大小和方向)相等。
三相静止abc坐标系(3S)-两相静止αβ坐标系(2S)-两相同步旋转坐标系(2R)



(a) 三相静止坐标系与两相静止坐标系 (b) 两相静止坐标系与两相旋转坐标系

图2-1-1 坐标系的定义

(1) 三相静止坐标系(3S)——两相静止坐标系(2S)

三相合成的矢量大小

定子相电流 I_a 、 I_b 、 I_c 为三相平衡正弦电流时，三相合成矢量 $\vec{I}_s = \vec{I}_a + \vec{I}_b + \vec{I}_c$

I_a 、 I_b 、 I_c 分别为相电流的大小。考虑矢量的大小和方向，三相各自的电流大小为幅值是 I ，互差120度电角度的余弦电流，方向在空间上（三相静止坐标）也互差120度电角度，用欧拉公式进行矢量的分解与合成分析，那么有

$$\begin{aligned}
 I_s &= I \cos \xi + I \cos(\xi - \frac{2\pi}{3}) e^{j\frac{2\pi}{3}} + I \cos(\xi + \frac{2\pi}{3}) e^{-j\frac{2\pi}{3}} \\
 &= I \cos \xi + I \cos(\xi - \frac{2\pi}{3}) (\cos \frac{2\pi}{3} + j \sin \frac{2\pi}{3}) + I \cos(\xi + \frac{2\pi}{3}) (\cos \frac{2\pi}{3} - j \sin \frac{2\pi}{3}) \\
 &= I \cos \xi + I (\cos \xi \cos \frac{2\pi}{3} + \sin \xi \sin \frac{2\pi}{3}) (-\frac{1}{2} + j \frac{\sqrt{3}}{2}) \\
 &\quad + I (\cos \xi \cos \frac{2\pi}{3} - \sin \xi \sin \frac{2\pi}{3}) (-\frac{1}{2} - j \frac{\sqrt{3}}{2}) \\
 &= I \cos \xi + I (-\frac{1}{2} \cos \xi + \frac{\sqrt{3}}{2} \sin \xi) (-\frac{1}{2} + j \frac{\sqrt{3}}{2}) + I (-\frac{1}{2} \cos \xi - \frac{\sqrt{3}}{2} \sin \xi) (-\frac{1}{2} - j \frac{\sqrt{3}}{2}) \\
 &= I (\cos \xi + \frac{1}{4} \cos \xi - j \frac{\sqrt{3}}{4} \cos \xi - \frac{\sqrt{3}}{4} \sin \xi + j \frac{3}{4} \sin \xi \\
 &\quad + \frac{1}{4} \cos \xi + j \frac{\sqrt{3}}{4} \cos \xi + \frac{\sqrt{3}}{4} \sin \xi + j \frac{3}{4} \sin \xi) \\
 &= I (\frac{3}{2} \cos \xi + j \frac{3}{2} \sin \xi) \\
 &= \frac{3}{2} I e^{j\xi}
 \end{aligned}$$

这里三相合成的矢量大小是两相静止坐标系合成的矢量大小的3/2倍。若为等幅值变换，即各坐标系下合成的矢量（大小和方向）相等，则需要**在变换矩阵中再乘以2/3这个系数**。

$$\begin{aligned}
 C_{3S/2S} &= \frac{2}{3} \begin{bmatrix} \cos \lambda & \cos(\lambda - 2\pi/3) & \cos(\lambda + 2\pi/3) \\ \sin \lambda & \sin(\lambda - 2\pi/3) & \sin(\lambda + 2\pi/3) \end{bmatrix} \\
 C_{2S/3S} &= C_{3S/2S}^T
 \end{aligned}$$

式中： λ 表示坐标轴 α 与坐标轴 a 的夹角；

当坐标轴 α 与坐标轴 a 重合时， $\lambda = 0$ ，则有：

$$C_{3S/2S} = \frac{2}{3} \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix}$$

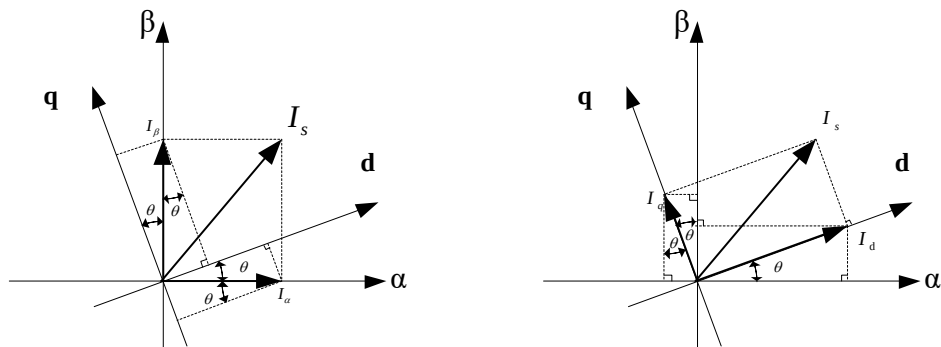
$$C_{2S/3S} = C_{3S/2S}^T = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \quad (\text{Clarke正向变换有} \frac{2}{3} \text{这个系数, 逆向变换没有!})$$

三相平衡的时候可以对Clarke变换作变形, 将 $i_c = -i_a - i_b$ 代入即可消去 i_c :

$$\begin{cases} i_\alpha = \frac{2}{3}(i_a - \frac{1}{2}i_b - \frac{1}{2}i_c) \\ i_\beta = \frac{2}{3}(\frac{\sqrt{3}}{2}i_b - \frac{\sqrt{3}}{2}i_c) \end{cases} \Rightarrow \begin{cases} i_\alpha = \frac{2}{3}[i_a - \frac{1}{2}i_b - \frac{1}{2}(-i_a - i_b)] \\ i_\beta = \frac{2}{3}[\frac{\sqrt{3}}{2}i_b - \frac{\sqrt{3}}{2}(-i_a - i_b)] \end{cases}$$

$$\Rightarrow \begin{cases} i_\alpha = \frac{2}{3}(i_a + \frac{1}{2}i_a) = i_a \\ i_\beta = \frac{2}{3}(\sqrt{3}i_b + \frac{\sqrt{3}}{2}i_a) = \frac{(2i_b + i_a)}{\sqrt{3}} \end{cases}$$

(2) 两相静止坐标系 (2S) —— 两相同步旋转坐标系 (2R)



(a) 正向Park变换的矢量分解

(b) 逆向Park变换的矢量分解

图2-1-2 不同坐标系的分解

由图2-1-2可以分析出以下关系:

$$\begin{aligned} i_d &= i_\alpha \cos \theta + i_\beta \sin \theta \\ i_q &= -i_\alpha \sin \theta + i_\beta \cos \theta \end{aligned}$$

$$\begin{aligned} i_\alpha &= i_d \cos \theta - i_q \sin \theta \\ i_\beta &= i_d \sin \theta + i_q \cos \theta \end{aligned}$$

转换为矩阵的表达方式:

$$C_{2S/2R} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$$

$$C_{2R/2S} = C_{2S/2R}^T = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

(3) 三相静止坐标系 (3S) —— 两相同步旋转坐标系 (2R)

$$C_{3S/2R} = C_{3S/2S} * C_{2S/2R} = \frac{2}{3} \begin{bmatrix} \cos \theta & \cos(\theta - 2\pi/3) & \cos(\theta + 2\pi/3) \\ -\sin \theta & -\sin(\theta - 2\pi/3) & -\sin(\theta + 2\pi/3) \end{bmatrix}$$

$$C_{2R/3S} = C_{3S/2R}^T = \begin{bmatrix} \cos \theta & -\sin \theta & 1/2 \\ \cos(\theta - 2\pi/3) & -\sin(\theta - 2\pi/3) & 1/2 \\ \cos(\theta + 2\pi/3) & -\sin(\theta + 2\pi/3) & 1/2 \end{bmatrix}$$

电流AD采样得到的数值适当处理后进行以下的坐标变换:

$$\begin{cases} i_a = I_s \cos \xi \\ i_b = I_s \cos(\xi - \frac{2\pi}{3}) \\ i_c = I_s \cos(\xi + \frac{2\pi}{3}) \end{cases} \xrightarrow{\text{等幅值Clarke变换}} \begin{cases} i_\alpha = I_s \cos \xi \\ i_\beta = \frac{2I_s \cos(\xi - \frac{2\pi}{3}) + I_s \cos \xi}{\sqrt{3}} \end{cases}$$

$$\begin{cases} i_\alpha = I_s \cos \xi \\ i_\beta = \frac{2I_s \cos(\xi - \frac{2\pi}{3}) + I_s \cos \xi}{\sqrt{3}} \\ \xi = \varphi + \theta \end{cases} = I_s \sin \xi \xrightarrow[\text{电角度}\theta]{\text{Park变换}} \begin{cases} i_d = I_s \cos(\xi - \theta) = I_s \cos \varphi \\ i_q = I_s \sin(\xi - \theta) = I_s \sin \varphi \end{cases}$$

若标定电机或判断电流极性则进行逆变换:

$$\begin{cases} I_s \\ \varphi \end{cases} \longrightarrow \begin{cases} i_d = I_s \cos \varphi \\ i_q = I_s \sin \varphi \end{cases} \xrightarrow[\text{电角度}\theta]{\text{逆Park变换}} \begin{cases} i_\alpha = I_s \cos(\varphi + \theta) \\ i_\beta = I_s \sin(\varphi + \theta) \\ \xi = \varphi + \theta \end{cases}$$

$$\xrightarrow{\text{逆Clarke变换}} \begin{cases} i_a = I_s \cos \xi \\ i_b = -\frac{1}{2} I_s \cos \xi + \frac{\sqrt{3}}{2} I_s \sin \xi = I_s \cos(\xi - \frac{2\pi}{3}) \\ i_c = -\frac{1}{2} I_s \cos \xi - \frac{\sqrt{3}}{2} I_s \sin \xi = I_s \cos(\xi + \frac{2\pi}{3}) \end{cases}$$

若A相为 $I_s \cos \xi$ ，B相为 $I_s \cos(\xi - 2\pi/3)$ ，C相为 $I_s \cos(\xi + 2\pi/3)$ ，那么 α 轴

为 $I_s \cos \xi$, β 轴为 $I_s \sin \xi$; d 轴为 $I_s \cos(\xi - \theta)$, q 轴为 $I_s \sin(\xi - \theta)$, 其中 $\varphi = \xi - \theta$,

符合常规的 dq 轴坐标定义, 能用电角度 θ 确定 d 轴分量的位置。

综上所述, 电流为余弦的空间关系如图2-1-3所示:

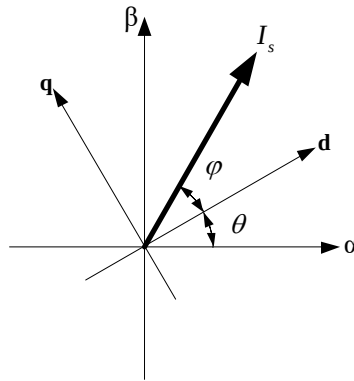


图2-1-3 电流为余弦的空间关系

程序中选择了Q15格式, 对应的电流均对电流基值标么化, 再加入运算。

//坐标变换角度=电角度-电流采样延迟角度(Q15格式)

AngElecTemp = MotorDriveVarStr.EleAngle - CURRENT_DELAY_ANGLE;

//LJF*****三相静止坐标系(3S)——>两相同步旋转坐标系(2R)

//LJF 计算坐标变换所需的三个余弦函数和三个正弦函数值

//LJF 现在的电角度

Cosx = CosUser(AngElecTemp);

//LJF 减120度: 5555(HEX)==>21845(DEC)/32768

Cosx_Sub120 = CosUser(AngElecTemp - 0x5555);

//LJF 加120度, 21845/32768

Cosx_Add120 = CosUser(AngElecTemp + 0x5555);

Sinx = SinUser(AngElecTemp);

Sinx_Sub120 = SinUser(AngElecTemp - 0x5555);

Sinx_Add120 = SinUser(AngElecTemp + 0x5555);

//LJF*****

//LJF Iu为Q15格式的值, 乘以三角函数再右移仍为Q15格式

TempA = (int32)MotorDriveInStr.Iu * Cosx >> 15;

TempB = (int32)MotorDriveInStr.Iv * Cosx_Sub120 >> 15;

TempC = (int32)MotorDriveInStr.Iw * Cosx_Add120 >> 15;

//LJF 三相电流转换为【d轴】

Temp32 = (int32)TempA + TempB + TempC;

//乘以幅值守恒变换系数 2/3(Q15格式)

Temp32 = Temp32 * 21845 >> 15;

IdFbNew = _IQsat(Temp32, 32767, -32768);

//LJF*****

```
//LJF Iu寄存器偏置后的有符号值
TempA = (int32)MotorDriveInStr.Iu * Sinx >> 15;
TempB = (int32)MotorDriveInStr.Iv * Sinx_Sub120 >> 15;
TempC = (int32)MotorDriveInStr.Iw * Sinx_Add120 >> 15;
//LJF 三相电流转换为【q轴】
Temp32 = -((int32)TempA + TempB + TempC);
//乘以幅值守恒变换系数 2/3(Q15格式)
Temp32 = Temp32 * 21845 >> 15;
IqFbNew = _IQsat(Temp32, 32767, -32768);
//LJF*****
```

2.2 滑动平均滤波处理

具体算法参见1.2.2节，分别对IdFbNew和IqFbNew进行滤波处理

这里只取IdFbNew进行说明：

1. 采样值：IdFbNew为d轴电流的Q15格式采样反馈值(单位为pu)并将其放入IdFbArr[i]内存单元。

代码如下：IdFbArr[i] = IdFbNew;

2. 数据的暂存区：IdFbArr[64]数组，控制循环的变量i

3. 求64个d轴电流的Q15格式采样反馈值之和，每次中断逐渐累加完。

代码如下：IdFbSum = IdFbSum + IdFbArr[i];

4. 对64个数值之和IdFbSum取平均：MotorDriveOutStr.IdFb = IdFbSum >> 6;这里方便微机快速运算，等价于除以 $2^6=64$,

5. 自动冲去一个最老的数据：在此之前需要把j在0~63范围内循环自增以实现“自动”，这里j已经自增。

代码如下：IdFbSum = IdFbSum - IdFbArr[i];

整体的功能代码如下：这里每次中断都进行以下处理，从而实现滑动

```
IdFbArr[i] = IdFbNew;
IqFbArr[i] = IqFbNew;
IdFbSum = IdFbSum + IdFbArr[i];
IqFbSum = IqFbSum + IqFbArr[i];
MotorDriveOutStr.IdFb = IdFbSum >> 6;
MotorDriveOutStr.IqFb = IqFbSum >> 6;
i++;
if (i >= 64)
{
    i = 0;
}
IdFbSum = IdFbSum - IdFbArr[i];
IqFbSum = IqFbSum - IqFbArr[i];
```

在第7节中，死区补偿需要取当前 i_d 和 i_q 的夹角，寻求一种简便、适合死区时间补偿的方案至关重要。这里采用了一种新方案:通过在旋转坐标系下进行变换，三相定子电流的基波分量在同步旋转坐标系中则表现为直流分量，对该直

流分量进行滤波处理，对幅值，相位均不会产生影响。从而可得到电流矢量的空间角，用它来判断三相定子电流的极性。该方法避免了上述直接检测定子电流并进行滤波处理所带来的问题。

3. 转矩电流补偿TorqueCurCom

该函数通过前面的查表（有电机标定数据）或是人为设定（进行电机标定）得出相应转矩下对应的Q15格式电流矢量参考值和与d轴的夹角（Q15格式），再根据静止坐标系下的电压方程以及电压环的输出做调整，计算当前执行的Id，Iq给定值（Q15格式）。

3.1 电流矢量的分解

如图3-1-1所示，当给定了电流矢量 I_s ，以及与直轴d轴的夹角 φ 时，便可以计算出 i_d 和 i_q 。实现的代码（选取Q15格式）如下：

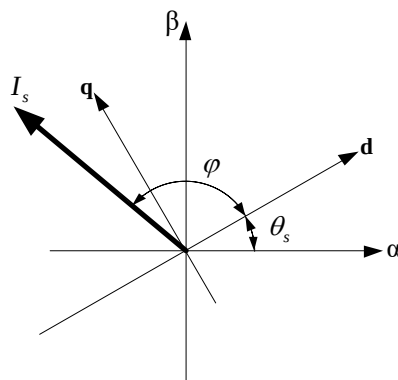


图3-1-1 定子电流矢量位置

```
IsGdTemp = (int32)MotorDriveInStr.IsGd * 32767 / MotorDriveParaStr.CurSensor;
ThetaTemp = (int32)MotorDriveInStr.Theta * 32767 / 1800;
IdNew = (int32)IsGdTemp * CosUser(ThetaTemp) >> 15;
IqNew = (int32)IsGdTemp * SinUser(ThetaTemp) >> 15;
```

3.2 稳态下的转矩补偿

为避免 i_d 的持续阶跃变化对电机电磁转矩的影响进而造成电机速度波动，本策略在调节 i_d 的同时进行交轴电流 i_q 的同步调节来维持电磁转矩的恒定，这样可以保持PMSM系统的稳态，提高效率优化速度。下面具体叙述调整过程。

直轴电流调整前，PMSM的电磁转矩 T_{em} 为

$$T_{em} = 1.5p[\psi_f i_q + (L_d - L_q)i_d i_q]$$

式中 L_d, L_q ——电机的直、交轴电感

ψ_f ——空载永磁磁链

p ——极对数

直轴电流变化 Δi_d 后，交轴电流值相应调整，保证电流调整前后PMSM电磁转矩不变，则调整后的电磁转矩可以表示为

$$T_{em} = 1.5p[\psi_f i_q' + (L_d - L_q)(i_d + \Delta i_d)i_q']$$

因此，交轴电流实际给定值为

$$T_{em} = 1.5p[\psi_f i_q + (L_d - L_q)i_d i_q] = 1.5p[\psi_f i_q' + (L_d - L_q)(i_d + \Delta i_d)i_q']$$

$$i_q' = \frac{\psi_f i_q + (L_d - L_q)i_d i_q}{\psi_f + (L_d - L_q)(i_d + \Delta i_d)} = \frac{\psi_f + (L_d - L_q)i_d}{\psi_f + (L_d - L_q)(i_d + \Delta i_d)} i_q$$

调整后的交、直轴电流分量即为系统实际给定控制量。

程序中把电压环的输出作为直轴电流变化量 Δi_d ，意义在于当期望电压矢量大于直流电压的输出最大值 $U_{dc}/\sqrt{3}$ 时减小 i_d （负增大），调节 i_q ，即进行了弱磁，使得 i_d 一直为负值。 Δi_d 调整到一定程度就把 i_q 置零。详细的控制原理与方法见6.2节。

假设 I_s 矢量在 dq 坐标轴的第三象限，若出现负转矩（制动），则查表得出电流给定值后再取负值，此时 I_s 矢量在 dq 坐标轴的第四象限，但仍需弱磁，只能手动强制 i_d 电流大小为负。而 i_q 大小不变， I_s 矢量最后在 dq 坐标轴的第三象限。

```
if (IdNew > 0)
{
    IdNew = -IdNew;
}
IdExe = IdNew - MotorDriveOutStr.VolPiOut;
TCLsub = MotorDriveVarStr.MotorLd - MotorDriveVarStr.MotorLq;
TorqueCom = -(((int32)TCLsub * IqNew >> 15) * MotorDriveOutStr.VolPiOut /
(((int32)TCLsub * IdExe >> 15) + MotorDriveVarStr.MotorFlux));
// TorqueCom = (int32)TorqueCom * 2;
if (IqNew >= 0)
{
    if (TorqueCom > IqNew)
    {
```

```

        TorqueCom = IqNew;
    }
}
else
{
    if (TorqueCom < IqNew)
    {
        TorqueCom = IqNew;
    }
}
IqExe = IqNew - TorqueCom;
MotorDriveOutStr.IdGd = IdExe;
MotorDriveOutStr.IqGd = IqExe;

```

4. 电流PI调节CurrentPIAdjust

该函数对电流进行PI调节，使之满足与之对应的转矩大小。

4.1 位置式PID模拟控制的离散算法

一般的位置式PID模拟控制器的基本算式为

$$u(k) = K_p(e(t) + \frac{1}{T_I} \int e(t)dt + T_D \frac{de(t)}{dt})$$

将上式各项近似离散为：

$$t \approx kT$$

$$k = 0, 1, 2, \dots$$

$$e(t) \approx e(kT)$$

$$\int e(t)dt \approx \sum_{j=0}^k e(jT)T = T \sum_{j=0}^k e(jT)$$

$$\frac{de(t)}{dt} \approx \frac{e(kT) - e[(k-1)T]}{T}$$

kT均用k简化表示，那么离散化的PID位置式算法表示为：

$$\begin{aligned}
 u(k) &= K_p \left\{ e(k) + \frac{T}{T_I} \sum_{j=0}^k e(j) + \frac{T_D}{T} [e(k) - e(k-1)] \right\} \\
 &= K_p e(k) + K_I \sum_{j=0}^k e(j) + K_D [e(k) - e(k-1)]
 \end{aligned}$$

其中：

$$K_I = K_p \frac{T}{T_I}$$

$$K_D = K_p \frac{T_D}{T}$$

这里参数的定义如下：

T_D :微分常数； T_I :积分常数； K_P :比例常数；

T:采样周期； $e(k)$:k时刻的偏差

4.2 改进的数字PI算法

程序中只用到了PI控制器，那么对应的离散算法为：

$$u(k) = K_P e(k) + K_I \sum_{i=0}^k e(i)$$

在微机数字控制系统中，很容易把P和I分开。这里使用了改进的数字PI算法，当输出限幅时，不对当前误差进行积分，而是使用上一次的积分值，这样既可最终消除稳态偏差，又能避免较大的退饱和超调，这就是积分分离算法的基本思想。积分分离法能有效抑制振荡或减小超调，常用于转速调节器。

综上所述， I_q 的PI控制输入为由查表或电流指令得出的 I_q 给定值与采样变换后的 I_q 反馈值之差。控制输出到电压方程中进行 U_d ， U_q 的计算，最后经过SVPWM算法调节相应的占空比。流程图如图4-2-1所示：

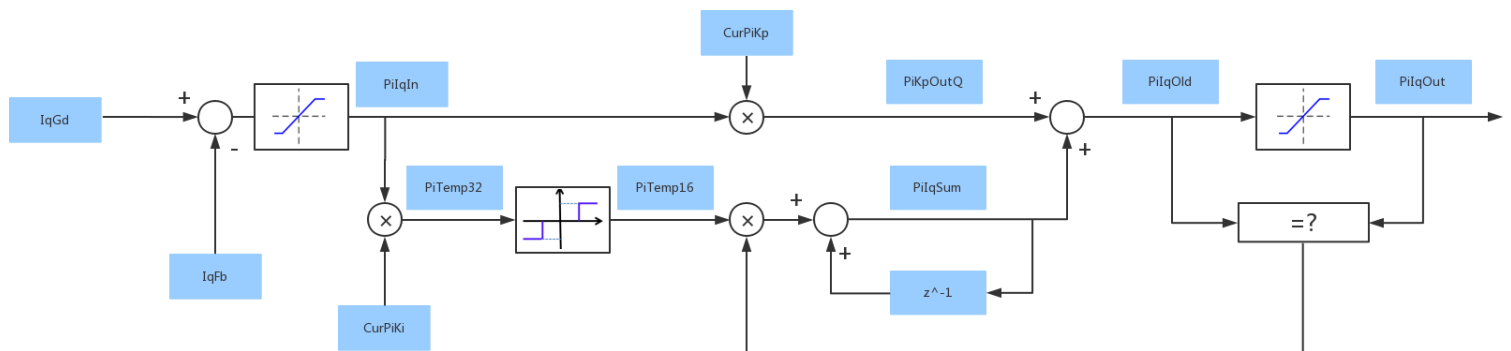


图4-2-1 电流 I_q 的PI控制流程图

实现以上基本功能的代码如下：

```
PiIqIn = (int32)MotorDriveOutStr.IqGd - MotorDriveOutStr.IqFb;
```

```
PiIqIn = _IQsat(PiIqIn, 10000, -10000);
```

```
PiKpOutQ = (int32)PiIqIn * MotorDriveParaStr.CurPiKp >> 15;
```

```
if (PiIqPer > 0)
```

```
{
```

```
    PiIqPer--;
```

```
}
```

```
else
```

```
{
```

```
    PiIqPer = MotorDriveParaStr.CurPiPer;
```

```

PiTemp32 = (int32)PiIqIn * MotorDriveParaStr.CurPiKi;
PiTemp16 = PiTemp32 >> 15;

if (PiTemp16 == 0)
{
    if (PiTemp32 > MotorDriveParaStr.CurPiPrecision)
    {
        PiTemp16 = 1;
    }
}
else if (PiTemp16 == -1)
{
    if (PiTemp32 > -MotorDriveParaStr.CurPiPrecision)
    {
        PiTemp16 = 0;
    }
}

if (PiIqOld == PiIqOut)
{
    PiIqSum += PiTemp16;
}

}

PiIqOld = PiKpOutQ + PiIqSum;
PiIqOut = _IQsat(PiIqOld, PiIqOutLim, -PiIqOutLim);

```

4.3 积分量化误差的消除

积分部分是偏差的积累，当采样周期较小，且偏差 $e(k)$ 也较小时，当前的积分项可能很小，在运算时被计算机整量化而舍掉，从而产生积分量化误差。扩大运算变量的字长，提高计算精度和分辨率，都能有效地减小积分量化误差，但这会使存储空间和运算复杂程度成倍地增加。为了解决这个矛盾，可以只增加积分项的有效字长，并将它分为整数与尾数两部分，整数与比例部分构成调节器输出，尾数保留下来作为下一拍累加的基数值。这样做的好处是，可以减小积分量化误差，而存储空间和运算复杂程度增加得并不多。

程序中只增加积分项的有效字长，存储的变量为PiTemp32，并将它分为精度分辨率以上和精度分辨率以下两部分，精度分辨率以上部分赋值为1与比例部分构成调节器输出，精度以下部分赋值为零即误差为零。

Q15	Q0	PiTemp16 补码	左移前 PiTemp32 最小	左移前 PiTemp32 最大
-1	-0.00049	0xFFFF	0xFFFF 8000	0xFFFF FFFF
0	0	0x0000	0x0000 0000	0x0000 7FFF

```
test = shift>>15;
```

根据上述的移位和溢出分析，PiTemp32只能是用**30位**表示数值大小，**2位**表示符号的**Q30**格式二进制补码，这样才能保证不会溢出，所以对PiIdIn进行限幅处理。CurPiKi也不宜太大。

对精度的相关计算推导如下:

PiTemp16	int16补码形式	对应PiTemp32最小值	CurPiPrecision	CurPiPrecision	对应PiTemp32最大值	有作用的PiIdIn最小值	有作用的PiIdIn最大值
0	0x0000	0x0000 0000	0x0000 2710(+10000)	0x0000 2710(+10000)	0x0000 7FFF	CurPiPrecision/CurPiKi	0x0000 7FFF/CurPiKi
		PiTemp16=0		PiTemp16=1		±0.003(Q0格式)	±0.01(Q0格式)
-1	0xFFFF	0xFFFF 8000	0xFFFF D8F0(-10000)	0xFFFF D8F0(-10000)	0xFFFF FFFF	基值=595(A)	
		PiTemp16=0		PiTemp16=-1		+1.785A	+5.95A

用图4-3-1直观表示:

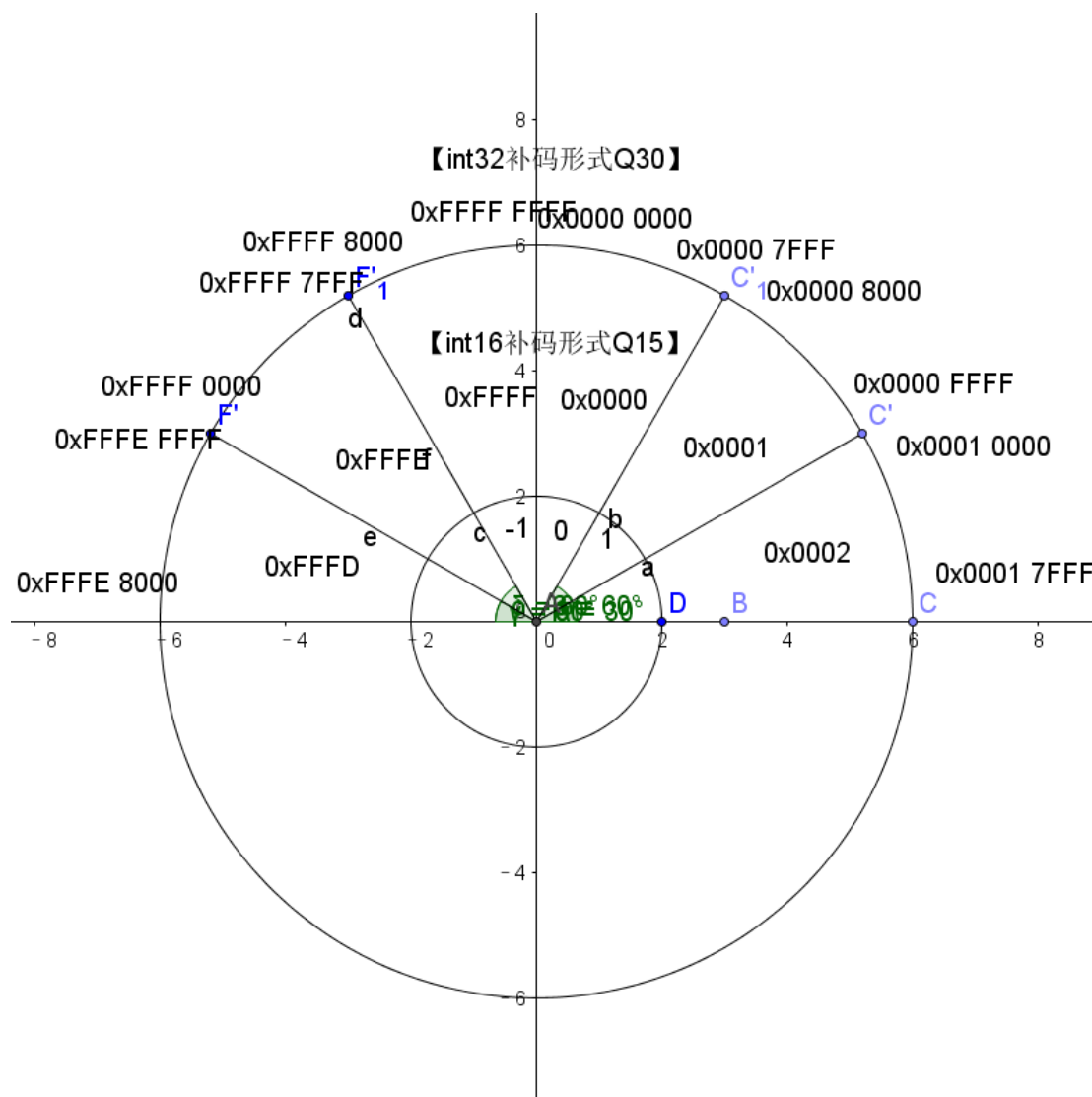


图4-3-1 量化精度的数值分析

实现以上功能的代码如下：

```
PiTemp32 = (int32)PiIdIn * MotorDriveParaStr.CurPiKi;
PiTemp16 = PiTemp32 >> 15;
//LJF 精度的不同T16(Q15)为0但T32(Q30)不为零右移之后变为0!
if (PiTemp16 == 0)
{
    //LJF 精度的设置的范围为0~32767
    //LJF 正精度之外为1，这里需要对PiTemp16赋值为1.
    if (PiTemp32 > MotorDriveParaStr.CurPiPrecision)
    {
        PiTemp16 = 1;
    }
    //LJF 正精度之内为0，这里不需要对PiTemp16赋值为0.
    //LJF 这里无else语句
}
else if (PiTemp16 == -1)
{
    //LJF 负精度之内为0
    if (PiTemp32 > -MotorDriveParaStr.CurPiPrecision)
    {
        PiTemp16 = 0;
    }
    //LJF 负精度之外为-1，这里不需要对PiTemp16赋值为-1.
    //LJF 这里无else语句
}
```

5. 计算电机电压Ca1UdUq

该函数计算电机的静止两相坐标系下的电压，输入量来自电流PI控制器的输出即IdOut和IqOut，通过电压方程与标幺化的变量计算给定的电压值UdGd和UqGd（Q15格式）用于后续的电压环PI调节和七段SVPWM调制。

在永磁同步电机的高性能控制中，无论是矢量控制还是直接转矩控制，控制算法多由定点DSP来实现。众所周知，定点DSP并不擅长浮点数运算，在运算之前需要对浮点操作数进行定标处理。在永磁同步电机的矢量控制中，由于参与计算的物理量数值相差较大，采用同一Q格式(操作数的一种定标方式，如Q15格式)定标出来的结果也相差较大，因此在DSP处理过程中会出现运算精度低、数据溢出等问题。

文中研究了一种PMSM矢量控制系统的标么化设计方法，给出了PMSM的数学模型，介绍了模型中各物理量基准值的选取方法，并详细推导了PMSM矢量控制系统的标么化设计过程。

5.1 PMSM的数学模型

永磁同步电机在dq同步旋转坐标系下的电压方程和电磁转矩方程分别为：

$$\begin{cases} u_d = R_d i_d + L_d p i_d - \omega_r L_q i_q \\ u_q = R_q i_q + L_q p i_q + \omega_r L_d i_d + \omega_r \psi_f \end{cases} \quad (1)$$

$$T_e = \frac{3}{2} P_n [\psi_f i_q - (L_q - L_d) i_d i_q] \quad (2)$$

式中， i_d 、 i_q 和 u_d 、 u_q 分别为永磁同步电机的定子电流和定子电压在d、q轴上的分量； L_d 、 L_q 分别为电机的直轴、交轴同步电感； ω_r 为电机电角速度； p 为微分算子； R_d 、 R_q 分别为电机定子q、d轴电阻； ψ_f 为电机转子磁通。

其运动方程为

$$J \frac{d\Omega}{dt} + B\Omega + T_L = T_e \quad (3)$$

式中， J 为电机转动惯量， B 为电机摩擦系数， T_L 为负载转矩。

5.2 PMSM矢量控制系统的标么化设计

其基本思想是先选定一些基本物理量，并确定相应的基准值，再通过一定的约束条件(如电路理论、电机理论)推导出控制系统中其他物理量的基准值，进而对PMSM矢量控制系统进行标么化处理，使标么化后的大部分参数在1附近，同时保证采用标么化表示的数学模型与实际值表示形式一致，也使得DSP中的运算公式与理论推导公式一致。

5.2.1 基值的选取

文中选用母线电压采样计算量程和电流传感器计算量程作为PMSM矢量控制系统的基本物理量，其他物理量均可由这两个物理量推算出来。需要说明的是，为了较直观地观测各个物理量随时间的变化情况，时间值不采用标么值，计量单位为秒。各变量的标么值采用变量加上标“*”表示。下表给出了永磁同步电机各变量的基准值选取情况。

变量	基准值公式	原单位
功率	系统功率	W
电压	MotorDriveParaStr.VolSensor	V
电流	MotorDriveParaStr.CurSensor	A
阻抗	VolSensor/ CurSensor	Ω
机械角速度	MotorRpmPeak	r/min
电角频率	$\text{MotorRpmPeak} * 2\pi / 60 * N_p$ (或 $2\pi f_b$)	rad/s (或 Hz * 2π)
电感(由 $X_L = \omega L$ 推)	VolSensor / CurSensor / BaseValueW	H

磁链(见楷体字说明)	VolSensor / BaseValueW	V/(Hz*2 π)
频率	未使用	Hz
转矩	未使用	Nm
转动惯量	未使用	kg·m ²
角度	未使用	rad

电阻基值(Ω): VolSensor / CurSensor;

电阻标么值: MotorRes / (VolSensor / CurSensor)

机械转速基值(r/min): MotorRpmPeak

机械转速标么值: MotorRpm / MotorRpmPeak

电角频率基值(rad/s): MotorRpmPeak * 2 * 3.14159 / 60 * MotorPole;

但这里直接用机械转速的标么值, 以减少运算量, 原因如下:

$$\text{电角频率标么值} = \text{机械转速标么值} = \frac{\text{当前转速} \frac{\text{r}}{\text{min}} * \frac{2\pi \frac{\text{rad}}{\text{s}}}{60 \frac{\text{s}}{\text{min}}} * N_p}{\text{峰值转速} \frac{\text{r}}{\text{min}} * \frac{2\pi \frac{\text{rad}}{\text{s}}}{60 \frac{\text{s}}{\text{min}}} * N_p} = \frac{\text{当前转速} \frac{\text{rad}}{\text{s}}}{\text{峰值转速} \frac{\text{rad}}{\text{s}}}$$

电感基值: VolSensor / CurSensor / BaseValueW;

电感标么值: MotorLq / BaseValueL

磁链基值: VolSensor / BaseValueW

磁链标么值: MotorFlux / BaseValueP

磁链单位V/Hz的物理关系推导

电压、反电势相当于磁链的变化率, 也就是磁链的变化量除以时间变化量, 磁链变化量与磁链是同一量纲的量, 它们的基值相同。所以磁链的基值除以时间的基值等于电压的基值。

还可以从另一个角度理解, 电压基值等于阻抗基值乘以电流基值, 阻抗基值等于电感基值乘以角频率基值, 这样, 电压基值等于电感基值乘以角频率基值再乘以电流基值。而电感基值乘以电流基值又等于磁链基值, 所以电压基值等于磁链基值乘以角频率基值, 又因角频率基值与时间基值互为倒数, 因此, 电压基值等于磁链基值除以时间基值。

由反电动势 (V/krpm) 也可以求出磁链大小, 转换如下:

$$\frac{V \frac{1}{\sqrt{3}}}{\frac{k * r}{\text{min}} \frac{1 \text{ min}}{60 \text{ s}} \frac{1000}{1 \text{ k}} \frac{2\pi}{1 \text{ r}} N_p} = V / \text{Hz} = \text{Wb}$$

上式中的转换关系如下:

1, V为线电压 换为 相电压 (线电压=相电压*1.732)

2, 转每分 换为 弧度每秒 (1转转过2π弧度)

3, 机械量 换为 电控量 (电机极对数)

由于精度的原因, 部分变量标幺化后定标为Q12格式。

综上所述, 选取表格所述的基值进行标幺化的代码如下:

```
void MotorParameter(void)
{
    float32 BaseValueW = 0.0;
    float32 BaseValueR = 0.0;
    float32 BaseValueL = 0.0;
    float32 BaseValueP = 0.0;

    BaseValueW = (float32)MotorDriveParaStr.MotorRpmPeak * 2 * 3.14159 / 60 *
    MotorDriveParaStr.MotorPole;
    BaseValueR = (float32)MotorDriveParaStr.VolSensor / MotorDriveParaStr.CurSensor;
    BaseValueL = (float32)MotorDriveParaStr.VolSensor / MotorDriveParaStr.CurSensor /
    BaseValueW;
    BaseValueP = (float32)MotorDriveParaStr.VolSensor / BaseValueW;

    MotorDriveVarStr.MotorRes = MotorDriveParaStr.MotorRes * 32768 / BaseValueR;
    MotorDriveVarStr.MotorLd = MotorDriveParaStr.MotorLd * 4096 / BaseValueL;
    MotorDriveVarStr.MotorLq = MotorDriveParaStr.MotorLq * 4096 / BaseValueL;
    MotorDriveVarStr.MotorFlux = MotorDriveParaStr.MotorFlux * 4096 / BaseValueP;
}
```

5.2.2 电压方程的标幺化

对式(1)进行标幺化, 方程两边同时除以 U_b 可得:

$$\begin{cases} \frac{u_d}{U_b} = \frac{R_d}{Z_b} \frac{i_d}{I_b} + \frac{1}{\omega_b} \frac{L_d}{L_b} \frac{di_d}{dt} - \frac{\omega_r}{\omega_b} \frac{L_q}{L_b} \frac{i_q}{I_b} \\ \frac{u_q}{U_b} = \frac{R_q}{Z_b} \frac{i_q}{I_b} + \frac{1}{\omega_b} \frac{L_q}{L_b} \frac{di_q}{dt} + \frac{\omega_r}{\omega_b} \frac{L_d}{L_b} \frac{i_d}{I_b} + \frac{\omega_r}{\omega_b} \frac{\psi_f}{\psi_b} \end{cases} \quad (4)$$

将式(4)以标幺值表示为:

$$\begin{cases} u_d^* = R_d^* i_d^* + \frac{1}{\omega_b} L_d^* \frac{di_d^*}{dt} - \omega_r^* L_q^* i_q^* \\ u_q^* = R_q^* i_q^* + \frac{1}{\omega_b} L_q^* \frac{di_q^*}{dt} + \omega_r^* L_d^* i_d^* + \omega_r^* \psi_f^* \end{cases} \quad (5)$$

程序中认为电机达到稳态, 亦即 i_d 和 i_q 对时间的变化量为零, 微分项为零。

忽略处理之后，表示为：

$$\begin{cases} u_d^* = R_d^* i_d^* - \omega_r^* L_q^* i_q^* \\ u_q^* = R_q^* i_q^* + \omega_r^* L_d^* i_d^* + \omega_r^* \psi_f^* \end{cases} \quad (6)$$

实现以上标么化计算的代码如下：

```
void UdUqEqu(void)
{
    int16 MotorVelo = 0;
    int16 Temp1 = 0;
    int16 Temp2 = 0;
    MotorVelo = (int32)MotorDriveOutStr.MotorRpm * 32768 /
MotorDriveParaStr.MotorRpmPeak;
    Temp1 = (int32)MotorDriveOutStr.IdOut * MotorDriveVarStr.MotorRes >> 15;
    Temp2 = ((int32)MotorVelo * MotorDriveOutStr.IqOut >> 15) *
MotorDriveVarStr.MotorLq >> 12;
    MotorDriveOutStr.UdGd = Temp1 - Temp2;
    Temp1 = (int32)MotorDriveOutStr.IqOut * MotorDriveVarStr.MotorRes >> 15;
    Temp2 = ((int32)MotorDriveOutStr.IdOut * MotorDriveVarStr.MotorLd >> 15) +
MotorDriveVarStr.MotorFlux;
    Temp2 = (int32)Temp2 * MotorVelo >> 12;
    MotorDriveOutStr.UqGd = Temp1 + Temp2;
    if (MotorDriveInStr.Debug != 0)
    {
        MotorDriveOutStr.UdGd = MotorDriveInStr.UdGdDbg;
        MotorDriveOutStr.UqGd = MotorDriveInStr.UqGdDbg;
    }
}
```

6. 电压PI调节与弱磁VoltagePIAdjust

（由于程序执行的先后顺序影响，这里应以第八节的内容为背景。）

该函数功能是电压环的计算与调整。若参考电压矢量大于最大限制值，则把电压环的输出用于调节当前执行的Id，Iq给定值。也是弱磁控制的体现。

6.1 弱磁升速策略

由于永磁同步电动机的反电势与转速成正比，因此当电动机端电压随转速升高到逆变器能够输出的最大电压之后，电动机电枢绕组电流不能再增大，如果不采用特殊控制，电动机不能产生满足车辆机械特性驱动要求的转矩、转速，无法在高速下维持恒功率调速。为了使电机转速能宽速运行以满足电动汽车的复杂工况，许多弱磁扩速方法已被提出。

永磁同步电机的控制可分为额定转速以下的恒转矩控制和额定转速以上的弱磁控制。在基速以下，对其采取的控制方案为最大转矩/电流（MTPA）控制，使其工作在恒转矩起动区域。随着转速的提升，定子反电势增加，当反电势高于逆变器端部所能提供的最大电压时，逆变器无法将能量馈入电机，此时电机无

法继续往高速运行。为使电机工作在较宽的可调高速恒功率区域，在变流器容量不变的前提下，只有通过调节定子电流，即增加定子反向直轴去磁电流分量来维持高速运行时电压的平衡。

6.2 电压环输出实现自动弱磁

自动弱磁原理框图中，PartI用来实现最大转矩电流比控制，基速以下运行时主要采取最大转矩电流比运行控制策略。基速以上时，要采取弱磁控制，也就是原理图中的PartII和PartIII。弱磁的开通时刻主要由控制系统的同步PI电流调节器的输出参考电压来决定。当永磁同步电机的转速不断升高时，控制系统电流调节器输出的电压接近电压PWM调制的边界，这时候如果继续升速，电流调节器输出的电压就会很大，会导致电流调节器控制性能变差，从而失去了对系统的控制能力。为了解决这个问题，就需要避免电流调节器的饱和，可以通过改变直轴电流的大小来解决此问题，也就是说要减小直轴的电流。

永磁同步电机控制系统中引入电压环调节，归根到底是对电机磁链的调节，通过调节判断电流调节器输出电压的大小，使弱磁控制的开通可以自动完成。流程见图6-2-1。在基速以下，电机的电流调节器输出的电压幅值 U_d ， U_q 一般情况下不会高于 U_{smax} ，也就是说电流调节器输出电压的幅值小于给定的参考值，这时候PartII不被激活， $\Delta i_d = 0$ 。当电机继续升速，转速达到基速时，电流调节器输出的交、直轴的电压幅值 U_d ， U_q 会大于 U_{smax} ，此时PartII自动激活，电压外环PI输出的 Δi_d 作为d轴的去磁电流分量，这个去磁电流分量和PartI输出 i_d^* 叠加起来，作为d轴电流指令。PartIII用来调节永磁同步电机的交轴电流的大小。

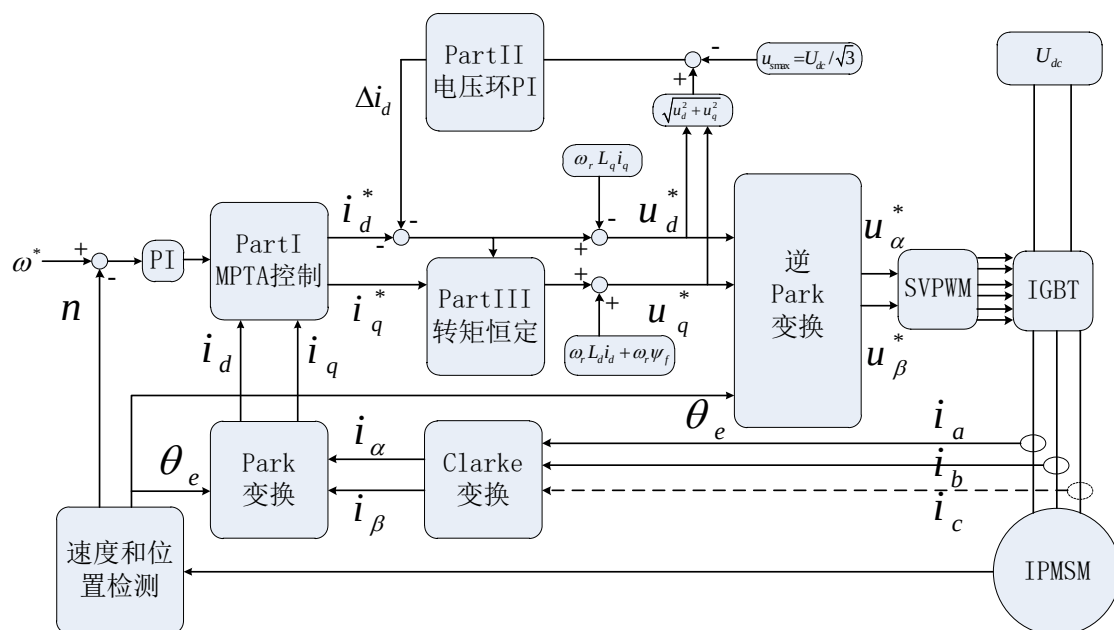


图6-2-1 自动弱磁原理框图

因为程序中不对电压进行过调制，所以最大相电压矢量形成的轨迹即为矢量围成的正六边形内切圆，又因六个基本电压矢量的幅值为 $2U_{dc}/3$ ，那么根据几何关系可以知道其半径为 $U_{dc}/\sqrt{3}$ 。如图6-2-2所示：

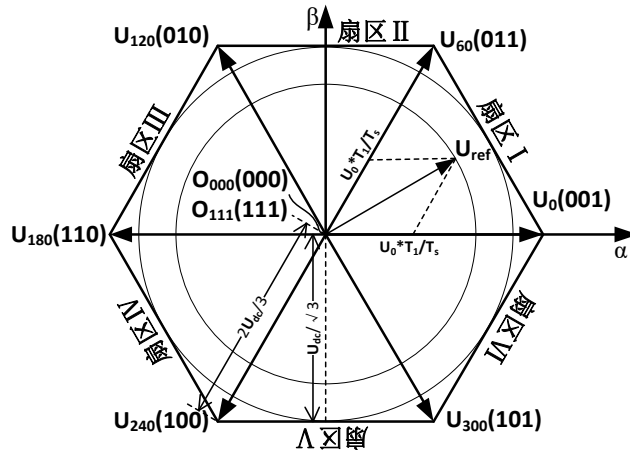


图6-2-2 扇区的矢量大小关系

用另外一个的方法也可以求出线电压的幅值：相电压幅值为 $2U_{dc}/3$ ，那么线电压幅值可以根据向量的加减求出为 $U_{dc}/\sqrt{3}$ 。如图6-2-3所示：

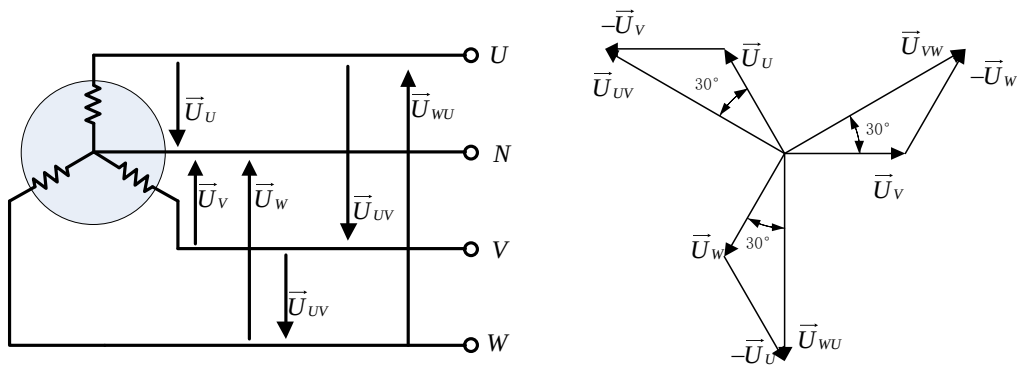


图6-2-3 线电压与相电压的关系

不对电压进行过调制，母线的电压为线电压 U_{dc} ，那么相电压为 $\frac{U_{dc}}{\sqrt{3}}$ 。由电压方程计算得到的 U_α 和 U_β 计算当前的 U_{ref} （为相电压），当期望电压参考矢量 U_{ref} 大于由直流电压输出限制的最大值 $U_{dc}/\sqrt{3}$ 时，把电压环的PI输出作用到上 i_d ，而 i_d 本身为负值，再减小 i_d （即增大负电流，加大弱磁效果），由于要控制

转矩不变，则按照第3.2节的公式调节 i_q 的大小。

注意到PI的输入为反馈值减去给定值，而不是给定值减去反馈值，且当PI调节运算后，若期望电压参考矢量 U_{ref} 小于输出限制的最大值 $U_{dc}/\sqrt{3}$ 时，PI输出为负数，这里把它限幅到0，意义为不超出电压限制最大值，则不进行电流大小的调整。可以与第3.2节的转矩电流补偿对应，这里PI输出为正，则减小 i_d 。

实现上述功能的代码如下：

```
UdcTemp = MotorDriveInStr.Udc >> 1;
UdGdTemp = MotorDriveOutStr.UdGd >> 1;
UqGdTemp = MotorDriveOutStr.UqGd >> 1;
UsMax = (int32)UdcTemp * MotorDriveParaStr.ModulationRatio >> 15;
Us2 = (int32)UdGdTemp * UdGdTemp + (int32)UqGdTemp * UqGdTemp;
Us = SqrtUser(Us2);
PiVolIn = (int32)Us - UsMax;
PiVolIn = _IQsat(PiVolIn, 10000, -10000);
PiVolKpOut = (int32)PiVolIn * MotorDriveParaStr.VolPiKp >> 15;
if (PiVolPer > 0)
{
    PiVolPer--;
}
else
{
    PiVolPer = MotorDriveParaStr.VolPiPer;
    PiTemp32 = (int32)PiVolIn * MotorDriveParaStr.VolPiKi;
    PiTemp16 = PiTemp32 >> 15;
    if (PiTemp16 == 0)
    {
        if (PiTemp32 > MotorDriveParaStr.VolPiPrecision)
        {
            PiTemp16 = 1;
        }
    }
    else if (PiTemp16 == -1)
    {
        if (PiTemp32 > -MotorDriveParaStr.VolPiPrecision)
        {
            PiTemp16 = 0;
        }
    }
    if (PiVolOld == PiVolOut)
    {
```

```

        PiVolSum += PiTemp16;
    }
}
PiVolOld = PiVolKpOut + PiVolSum;
PiVolOut = _IQsat(PiVolOld, MotorDriveParaStr.VolPiOutLimit, 0);
if (MotorDriveInStr.MotorStatus == 0)
{
    PiVolSum = 0;
    PiVolOld = 0;
    PiVolOut = 0;
}
MotorDriveOutStr.VolPiOut = PiVolOut;

```

7. 死区补偿DeadTimeCom

（由于程序执行的先后顺序影响，这里应以第八节的内容为背景。）

该程序按照用当前静止坐标系下的电流大小计算出的电角度，判断三相电流极性后进行死区补偿。

7.1 死区效应分析

以逆变器中一桥臂的A相为例进行死区效应分析（图7-1-1和图7-1-2），其负载可为同步电机或异步电机。受死区效应的影响，元件导通或关断时，逆变器的输出电压影响电流的极性。在理想状态下，功率管VT1和VT2的开关状态是互补的。但是置入死区时间 T_d 后，功率管要延迟 T_d 才能导通（这里忽略了开关管的导通和关断时间）。因此，在死区时间内，VT1和VT2都处于关断状态。由于同步电机或异步电机是感性负载，输出电流经过二极管续流。假设电流流进电机的方向为正，流出方向为负。在死区时间内， $i > 0$ 时，VD2导通，相当于下桥臂开关管导通，a点连接到中间电压的负端（图7-1-1）。同理， $i < 0$ 时，VD1导通，a点连接到中间电压的正端（图7-1-2）。

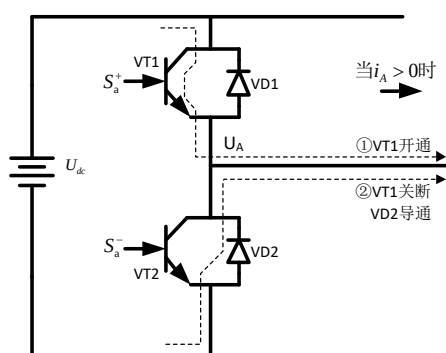


图7-1-1 VT1开通和关断时的电流流向

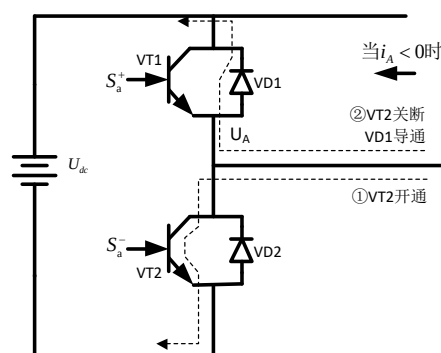


图7-1-2 VT2开通和关断时的电流流向

7.2 SVPWM逆变器时间补偿法

7.2.1 电机单相的时间补偿

一个载波周期内，开关管的理想和实际开关状态如图7-2-1所示。图中， S_a^+ 和 S_a^- 分别为上下管的触发导通信号，（a）和（b）为上、下桥臂的理想触发信号；（c）和（d）为实际开关信号，阴影部分时，上、下管均处于关断状态。 $i > 0$ 时，下桥臂二极管导通，就上管而言，输出正电压的脉冲宽度减少了 T_d ，即实际导通时间比理想导通时间缩短了 T_d ，而下管的实际导通时间则比理想导通时间延长了 T_d 。（前后两段死区时间相当于下桥导通，下管延长了两倍的 T_d ，与理想信号比较下管只延长了 T_d 。）需要做的补偿是将上管的理想导通时间延长 T_d ，由于信号的互补性，下管的理想导通时间相应缩短了 T_d ，达到了实际导通时间与理想的相一致的目的，从而保证了实际脉冲信号与给定脉冲信号的一样性。同理可知， $i < 0$ 时，补偿方法是将上管的理想导通时间缩短 T_d 。

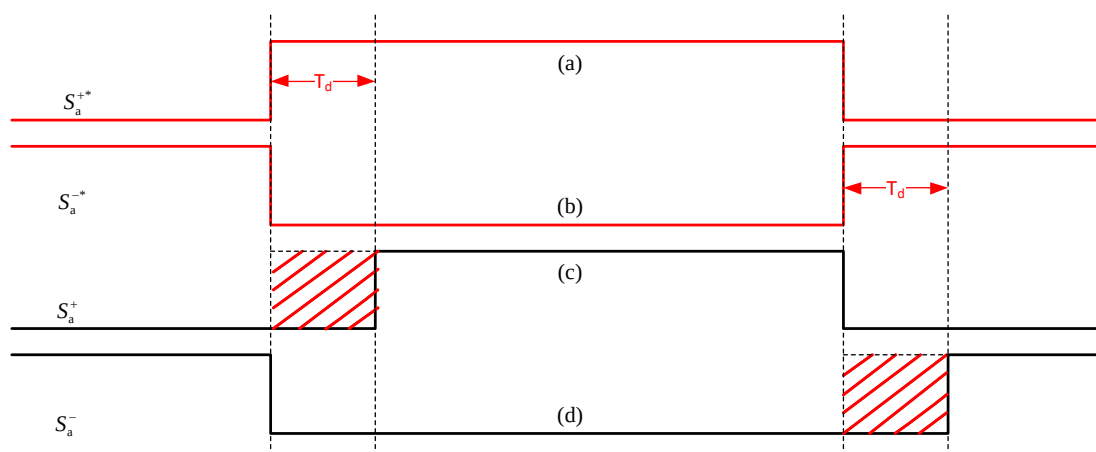


图7-2-1 理想触发信号和实际开关信号

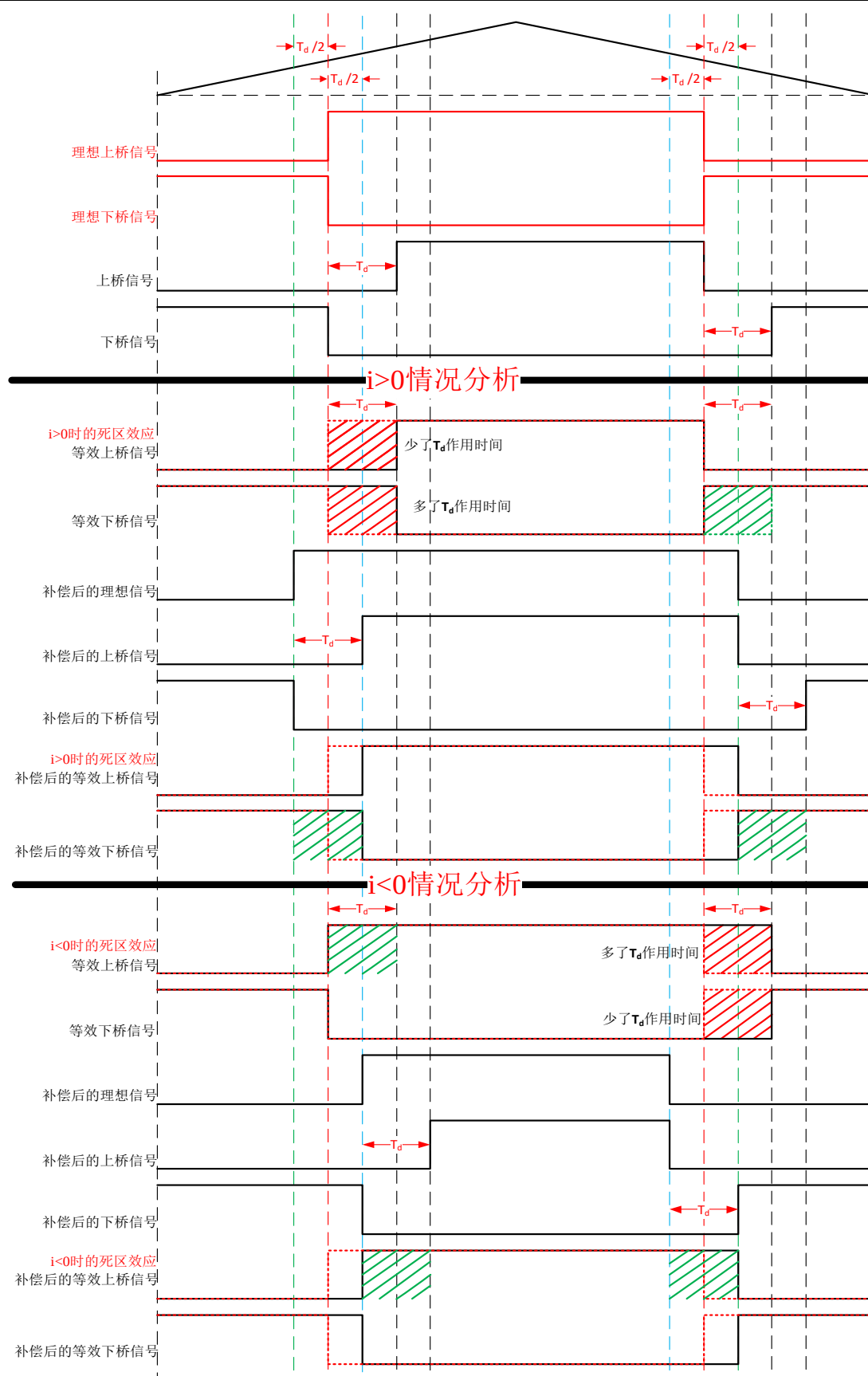


图7-2-2 单相电流不同极性的死区补偿

在一个调制周期内，以 t_{aon} 表示a相上桥臂开关管信号触发导通时刻，补偿后的触发导通时间表示为 t'_{aon} ，结合图7-2-2，那么综上所述有：

若 $i > 0$ 时，补偿方法是将上管的理想导通时间延长 T_d 。即半周期时刻 $-T_d/2$ ：

$$t'_{aon} = t_{aon} - T_d/2$$

若 $i < 0$ 时，补偿方法是将上管的理想导通时间缩短 T_d 。即半周期时刻 $+T_d/2$ ：

$$t'_{aon} = t_{aon} + T_d/2$$

其中， t_{aon} 为原来理想的触通时刻；b、c相的补偿方法与a相相同。该方法从触发信号着手，通过时间补偿使开关管实际开通时间等于理想给定时间，可用于 SPWM 和 SVPWM 逆变器的死区补偿。

7.2.2 三相综合的时间补偿

SVPWM方法综合三相而采用矢量合成的方法。补偿中，通过零矢量 O_{000} 和 O_{111} 的分配，可得到更为简化的算法。以abc电流极性正负负、电压矢量合成的第1扇区为例分析，如图7-2-3所示。

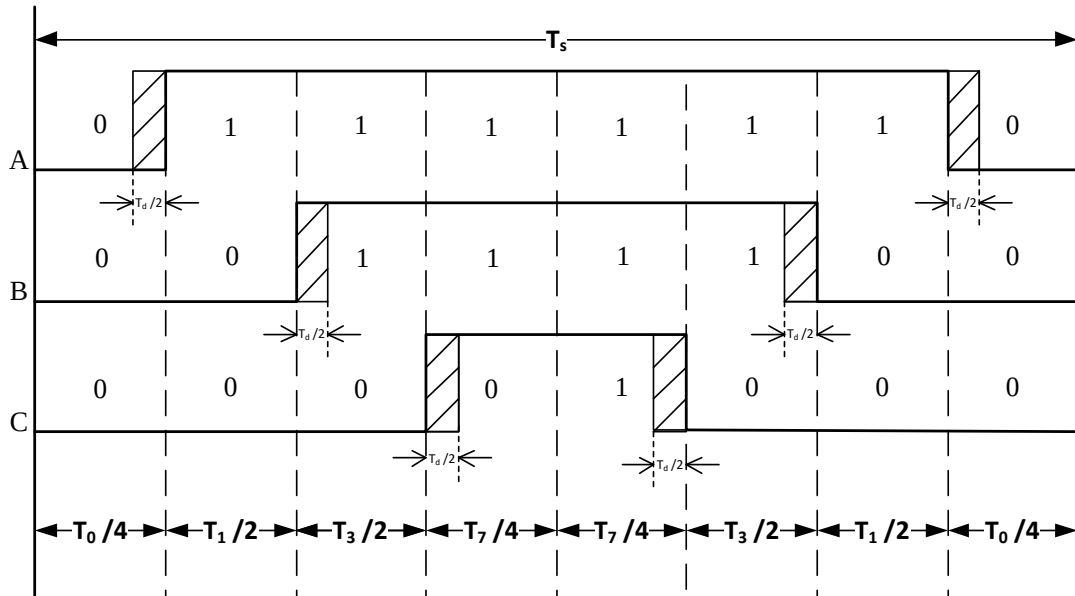


图7-2-3 扇区1下ABC相电流正负负时间补偿

虚线划分出原来各理想电压矢量的作用区间，按照上述补偿方法，是将 S_a^+ 高电平延长 T_d 时间，将 S_b^+ 、 S_c^+ 缩短 T_d 时间，阴影部分为补偿后的效果。综合三相得：矢量 U_0 作用时间比理想给定时间增加了 $2T_d$ 、矢量 U_{60} 作用时间没有变化，零矢量作用时间减少了 $2T_d$ 。若保持bc相理想触发信号不变，而将 S_a^+ 高电平延长 $2T_d$ 时间，则具有相同的矢量分配效果，且简化了算法，此时A相半周期的导通时刻为：

$$t'_{aon} = t_{aon} - T_d$$

在 abc 电流极性正负负下，分析其它电压矢量合成扇区的时间补偿，有相

同的结果，可以同理分析其它电流极性扇区。

补偿时刻对应的占空比 T_d^* 计算如下：

$$T_d = \frac{T_{PWM}}{2} * T_d^* \Rightarrow T_d^* = \frac{2T_d}{T_{PWM}}$$

该占空比 T_d^* 与七段SVPWM计算出的占空比一样为Q15格式。理论计算出的矢量占空比之后，经过上述的时间补偿法一起得出最终的占空比，再乘以比较寄存器最大值得到去标么化后的Q0格式值，最后将其赋值给比较寄存器。

实现上述功能代码如下：

```
#define DEAD_TIME    14745        //Q12,    3.6(us)
//LJF 半周期调整Td，全周期占空比为2Td/T_isr并转为Q15格式
DeadTemp16 = ((int32)DEAD_TIME << 4) / MotorDriveInStr.IsrPer;
DeadTime = DeadTemp16;
```

7.3 电流极性的判断

在旋转两相坐标系下进行电流矢量 I_s 所在“扇区”（角度）的判断较为简单，而不是像第8.5节电压矢量所在扇区判断（在静止两相坐标系下）那样那么复杂。在旋转两相坐标系下，用当前执行的 I_d 和 I_q 的反正切函数计算出电流矢量 I_s 与直轴d轴的夹角 φ 时，再加上当前由旋变得到的电角度 θ ，便可以计算出电流矢量 I_s 与 α 轴（或A轴）的夹角 $\xi = \varphi + \theta$ 。那么就可以得出静止三相坐标系下的三相电流。

这样的做法只需求夹角 $\xi = \varphi + \theta$ ，便可以判断电流的极性，且提高了程序执行的速度，减少DSP微机的运算。避免了用 I_d 和 I_q 的值进行坐标变换，省掉了与三角函数相关的多次运算。而两种方法计算出来的判断结果是相同的。

$$\begin{cases} I_s \\ \xi \\ \theta \end{cases} \xrightarrow[\text{电角度}\theta]{\text{逆Park变换}} \begin{cases} i_\alpha = I_s \cos \xi \\ i_\beta = I_s \sin \xi \end{cases} \xrightarrow{\text{逆Clarke变换}} \begin{cases} i_a = I_s \cos \xi \\ i_b = -\frac{1}{2} I_s \cos \xi + \frac{\sqrt{3}}{2} I_s \sin \xi = I_s \cos(\xi - \frac{2\pi}{3}) \\ i_c = -\frac{1}{2} I_s \cos \xi - \frac{\sqrt{3}}{2} I_s \sin \xi = I_s \cos(\xi + \frac{2\pi}{3}) \end{cases}$$

借助绘图工具，画出图7-3-1，找出一个周期内三相各自的过零点就可以判断三相电流的极性了。由于DSP微机芯片的IQ数学运算库中_IQatan2函数的计算结果范围为 $-\pi$ 到 $+\pi$ ，我们就选这个特定的周期进行分析：

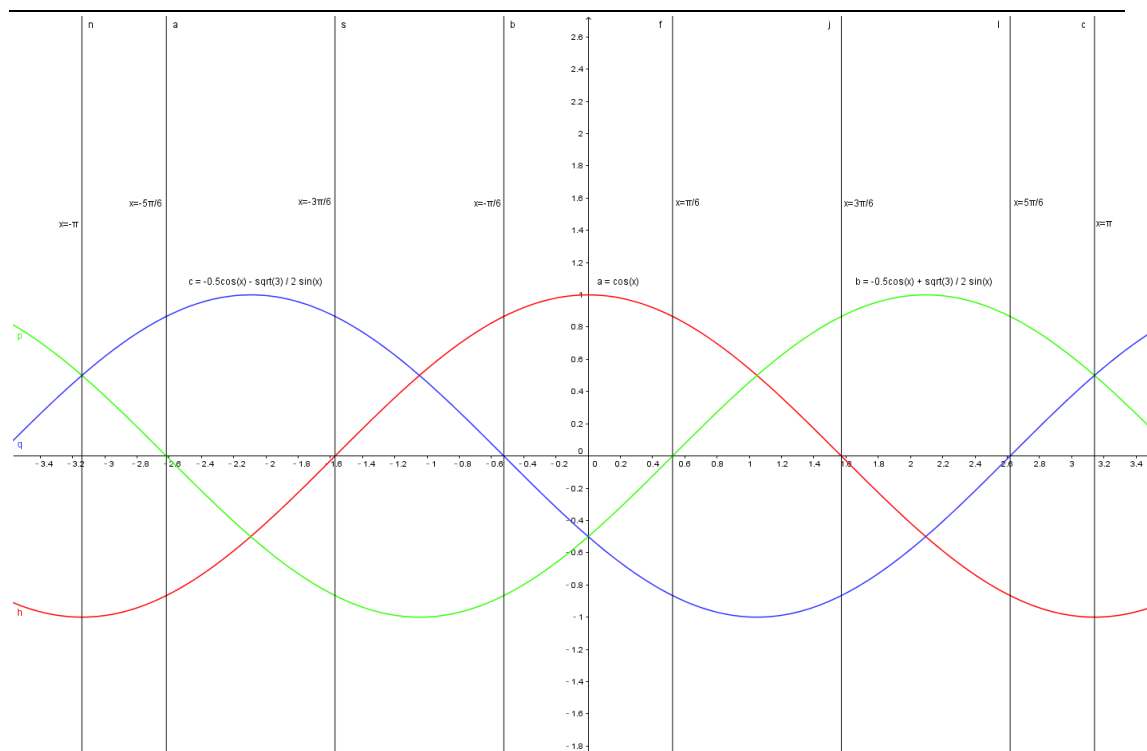


图7-3-1 按照电角度判断电流极性

判断出电流极性和对应的角度区间,并映射到空间矢量六边形上,如图7-3-2所示

整理归类如表7-3-1所示:

表7-3-1 SVPWM 时间补偿法

电夹角 ξ	i_a	i_b	i_c	半周期的导通时刻补偿方法
$[-5\pi/6, -\pi/2)$	-	-	+	$t'_{con} = t_{con} - T_d$
$[-\pi/2, -\pi/6)$	+	-	+	$t'_{bon} = t_{bon} + T_d$
$[-\pi/6, \pi/6)$	+	-	-	$t'_{aon} = t_{aon} - T_d$
$[\pi/6, \pi/2)$	+	+	-	$t'_{con} = t_{con} + T_d$
$[\pi/2, 5\pi/6)$	-	+	-	$t'_{bon} = t_{bon} - T_d$
$[5\pi/6, \pi)$ 或 $[-\pi, -5\pi/6)$	-	+	+	$t'_{aon} = t_{aon} + T_d$

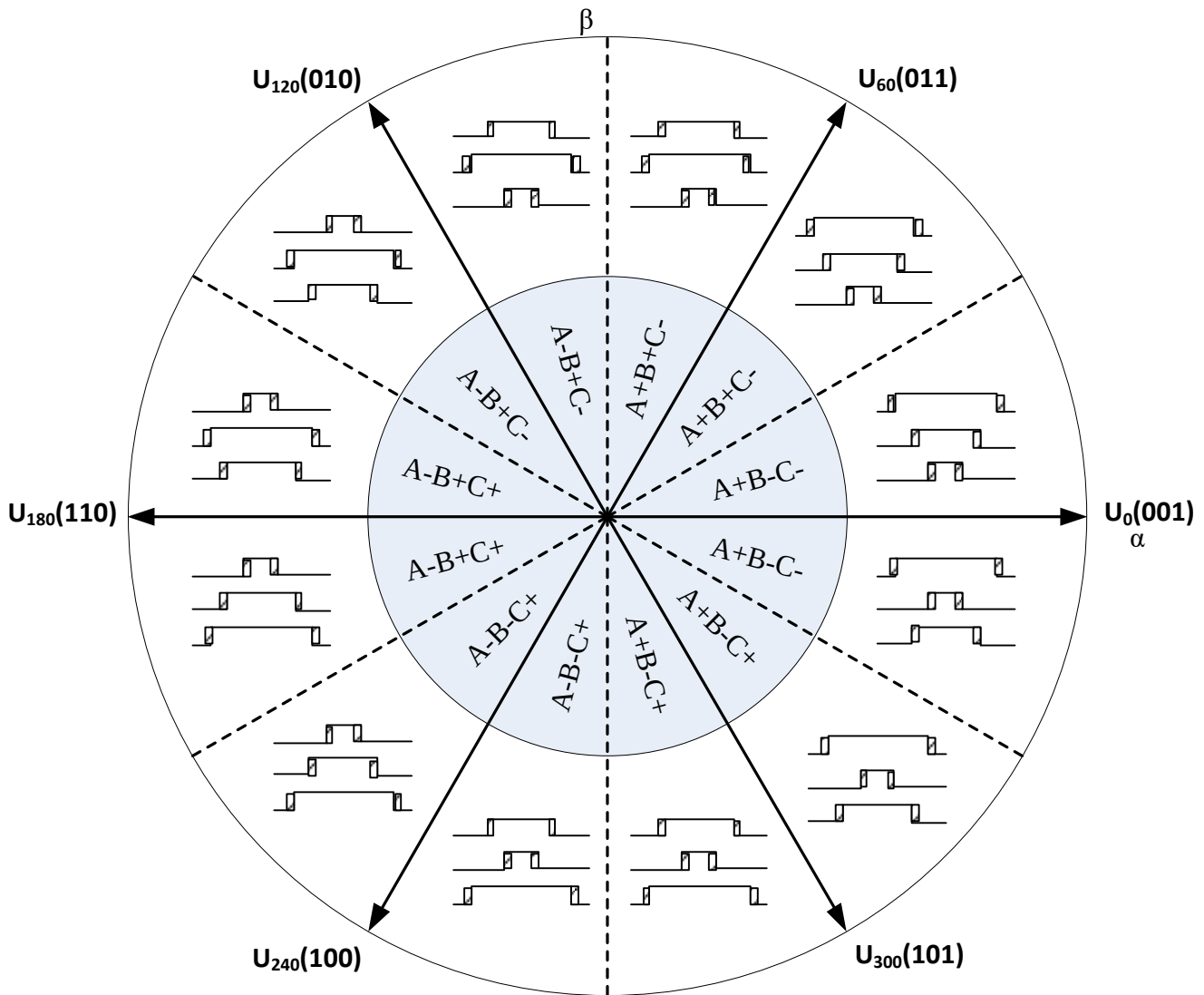


图7-3-2 按照电角度和电流极性的死区补偿

实现上述功能代码如下：

//IQNatan2输出范围为 $-\pi$ 到 $+\pi$ 。角度比较值需要调整

#define CUR_VEC_ANG1 5460//LJF 30

#define CUR_VEC_ANG2 16383//LJF 90

#define CUR_VEC_ANG3 27306//LJF 150

#define CUR_VEC_ANG4 -27307//LJF -150

#define CUR_VEC_ANG5 -16384//LJF -90

#define CUR_VEC_ANG6 -5462//LJF -30

//LJF 求取Is与d轴的夹角，需要以下两个参数

Id = MotorDriveOutStr.IdGd;

Iq = MotorDriveOutStr.IqGd;

//LJF 输入参数为全局Q15格式，第一个参数为sin值。求得的输出参数为全局Q15格式且范围为 $-\pi$ 到 $+\pi$ 。

```
AtanValue = _IQatan2(Iq,Id);
```

//LJF 转换为对 π 的比值仍为Q15格式， $\pi=3.14159$ 的Q13格式为25735

```
CurAngle = (AtanValue << 13) / 25735;
```

//LJF 将Is与d轴的夹角加上当前的电角度 即为电流矢量与A轴（零位）的夹角，对 π 的比值Q15格式

```
CurVecAng = MotorDriveVarStr.EleAngle + CurAngle;
```

//LJF 以下的死区补偿对各相进行调整，不影响带死区时间的SVPWM计算及实现

```
if (CurVecAng <= CUR_VEC_ANG4 || CurVecAng > CUR_VEC_ANG3)
```

```
    //-++
```

```
{
```

```
    ComA = DeadTime;
```

```
    ComB = 0;
```

```
    ComC = 0;
```

```
}
```

```
else if (CurVecAng > CUR_VEC_ANG4 && CurVecAng <= CUR_VEC_ANG5)
```

```
    //--+
```

```
{
```

```
    ComA = 0;
```

```
    ComB = 0;
```

```
    ComC = -DeadTime;
```

```
}
```

```
else if (CurVecAng > CUR_VEC_ANG5 && CurVecAng <= CUR_VEC_ANG6)
```

```
    //++-
```

```
{
```

```
    ComA = 0;
```

```
    ComB = DeadTime;
```

```
    ComC = 0;
```

```
}
```

```
else if (CurVecAng > CUR_VEC_ANG6 && CurVecAng <= CUR_VEC_ANG1)
```

```
    //+-
```

```
{
```

```
    ComA = -DeadTime;
```

```
    ComB = 0;
```

```
    ComC = 0;
```

```
}
```

```
else if (CurVecAng > CUR_VEC_ANG1 && CurVecAng <= CUR_VEC_ANG2)
```

```
    //++-
```

```
{
```

```
    ComA = 0;
```

```
    ComB = 0;
```

```
    ComC = DeadTime;
```

```
}
```

```

else if (CurVecAng > CUR_VEC_ANG2 && CurVecAng <= CUR_VEC_ANG3)
    //+-
{
    ComA = 0;
    ComB = -DeadTime;
    ComC = 0;
}
MotorDriveVarStr.DeadComU = ComA;
MotorDriveVarStr.DeadComV = ComB;
MotorDriveVarStr.DeadComW = ComC;

```

8. 七段SVPWM调制SevenSvpwm

该函数根据 U_d ， U_q 实现七段SVPWM调制。电压方程算出的 U_d 和 U_q 经过克拉克变换得到的 U_α 和 U_β 就是该函数的输入，而输出则为电机三相绕组接入电压母线的顺序和相应的导通时间。

8.1 基本电压空间矢量

8.1.1 逆变器与开关状态

逆变器实现了直流到交流的转换。如图8-8-1所示为三相功率逆变器，其六个功率开关元件组成的特定的开关模式从而产生的脉宽调制波，能够使输出电流波形尽可能接近于理想的正弦波形。

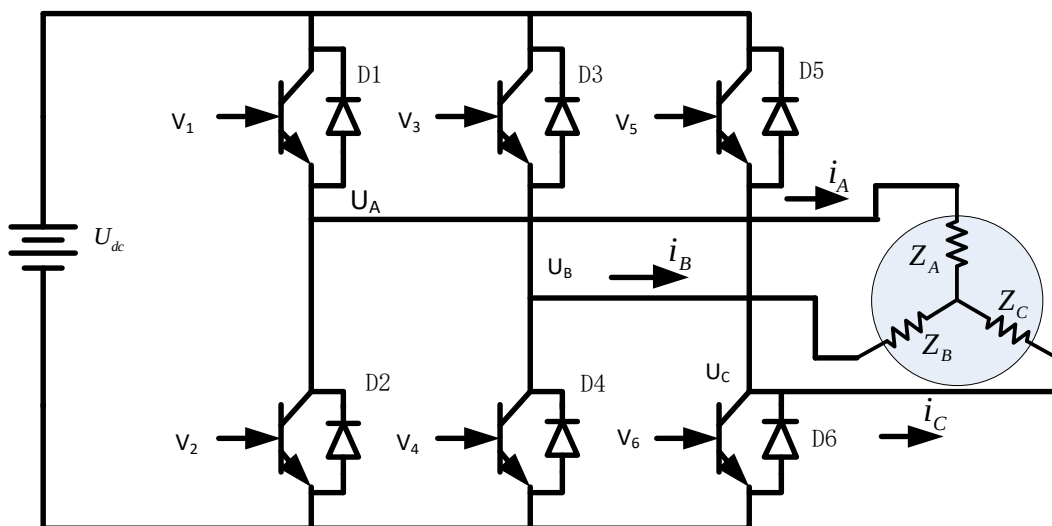


图8-1-1三相逆变器与异步电机主电路示意图

SVPWM指的是一种应用于像感应电机和永磁同步电机的代表三相电压型逆变器的三个电力器件开关的特别开关切换顺序。这种用于电力器件的特别开关方案

可以在定子上生成类似正弦的电流。

假定电机三相阻抗相等为 Z ，对于电机接入直流母线电压为一相进两相出的情况，如图8-1-2所示。例如开关管a-V1,b'-V4,c'-V6导通，那么各相的相电压计算如下：

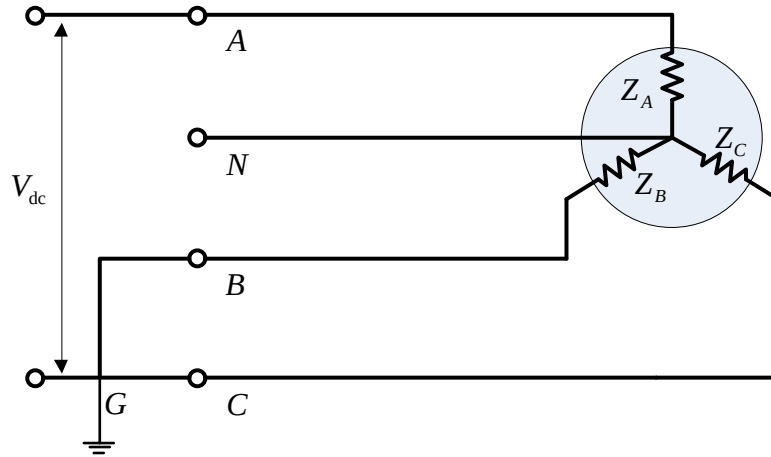


图8-1-2 “一相进两相出”的情况

对于并联的B和C相阻抗： $Z \square Z = \frac{Z^2}{Z+Z} = \frac{1}{2}Z$

N点到地的电压： $V_{NG} = \frac{\frac{1}{2}Z}{\frac{1}{2}Z + Z} V_{dc} = \frac{1}{3}V_{dc}$

那么由图可得：

$$V_{AN} = V_{AG} - V_{NG} = V_{dc} - \frac{1}{3}V_{dc} = \frac{2}{3}V_{dc}$$

$$V_{BN} = -V_{NG} = -\frac{1}{3}V_{dc}$$

$$V_{CN} = -V_{NG} = -\frac{1}{3}V_{dc}$$

$$V_{AB} = V_{AN} - V_{BN} = \frac{2}{3}V_{dc} - (-\frac{1}{3}V_{dc}) = V_{dc}$$

$$V_{BC} = V_{BN} - V_{CN} = -\frac{1}{3}V_{dc} - (-\frac{1}{3}V_{dc}) = 0$$

$$V_{CA} = V_{CN} - V_{AN} = -\frac{1}{3}V_{dc} - \frac{2}{3}V_{dc} = -V_{dc}$$

假定电机三相阻抗相等为 Z ，对于电机接入直流母线电压为两相进一相出的情况，如图8-1-3所示。例如开关管a-V1, b-V3, c'-V6导通，那么各相的相电压计算如下：

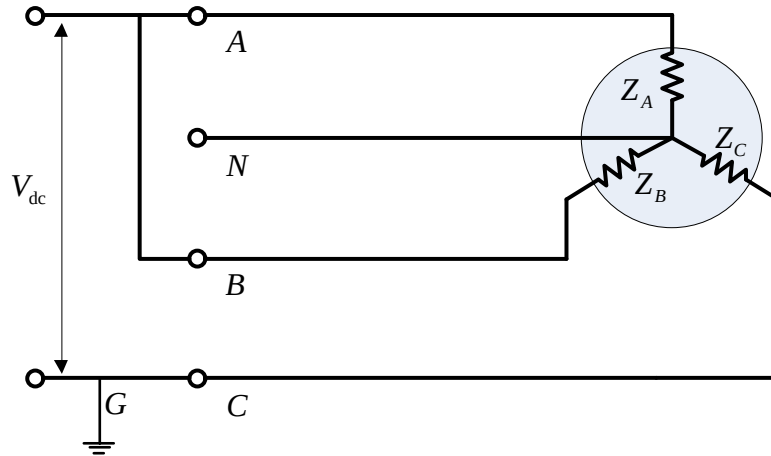


图8-1-3 “两相进一相出”的情况

对于并联的A和B相阻抗： $Z \parallel Z = \frac{1}{2}Z$

$$N\text{点到地的电压：} V_{NG} = \frac{Z}{\frac{1}{2}Z + Z} V_{dc} = \frac{2}{3}V_{dc}$$

那么由图可得

$$V_{AN} = V_{AG} - V_{NG} = V_{dc} - \frac{2}{3}V_{dc} = \frac{1}{3}V_{dc}$$

$$V_{BN} = V_{BG} - V_{NG} = V_{dc} - \frac{2}{3}V_{dc} = \frac{1}{3}V_{dc}$$

$$V_{CN} = -V_{NG} = -\frac{2}{3}V_{dc}$$

$$V_{AB} = V_{AN} - V_{BN} = \frac{1}{3}V_{dc} - \frac{1}{3}V_{dc} = 0$$

$$V_{BC} = V_{BN} - V_{CN} = \frac{1}{3}V_{dc} - (-\frac{2}{3}V_{dc}) = V_{dc}$$

$$V_{CA} = V_{CN} - V_{AN} = -\frac{2}{3}V_{dc} - \frac{1}{3}V_{dc} = -V_{dc}$$

若上桥臂开关管导通记为“1”，关断记为“0”，并且每相上桥臂开关管开关状态以c、b、a排列那么综上所述，对于电机接入直流母线电压为“一相进两相出”的情况有（0,0,1），（0,1,0），（1,0,0）。对于异步电机接入直流母线电压为“两相进一相出”的情况有（0,1,1），（1,1,0），（1,0,1）。

对于异步电机没有接入直流母线电压的情况有（0,0,0），（1,1,1）。以上已经列出3个开关管8种不同的开关组合状态，对于不同的开关组合状态，同理可用上面的分析方法，我们列出了相电压与线电压如表8-1-1所示：

表8-1-1 不同开关状态组合对应三相逆变器的相电压与线电压

c	b	a	V_{AN}	V_{BN}	V_{CN}	V_{AB}	V_{BC}	V_{CA}
0	0	0	0	0	0	0	0	0
0	0	1	$2V_{DC}/3$	$-V_{DC}/3$	$-V_{DC}/3$	V_{DC}	0	$-V_{DC}$
0	1	0	$-V_{DC}/3$	$2V_{DC}/3$	$-V_{DC}/3$	$-V_{DC}$	V_{DC}	0
0	1	1	$V_{DC}/3$	$V_{DC}/3$	$-2V_{DC}/3$	0	V_{DC}	$-V_{DC}$
1	0	0	$-V_{DC}/3$	$-V_{DC}/3$	$2V_{DC}/3$	0	$-V_{DC}$	V_{DC}
1	0	1	$V_{DC}/3$	$-2V_{DC}/3$	$V_{DC}/3$	V_{DC}	$-V_{DC}$	0
1	1	0	$-2V_{DC}/3$	$V_{DC}/3$	$V_{DC}/3$	$-V_{DC}$	0	V_{DC}
1	1	1	0	0	0	0	0	0

8.1.2 Clarke变换与空间矢量

Clarke变换的数学意义是经过变换后波形的幅值、频率和原来三相波形一致，合成矢量的大小不变，只是相位由原来的三相互差120度变成了两个互差90度的波形。

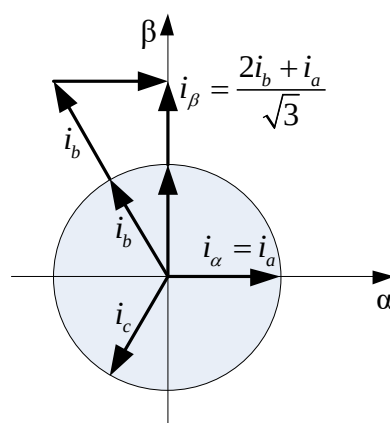


图8-1-4 等效向量的转换

如图8-1-4所示，分别把 i_a 、 i_b 、 i_c 视为单位向量。 α 轴和 β 轴方向上的单位向量分别可以由三个基础向量相加得到，根据向量的运算可得：

$$\begin{aligned}\vec{i}_\alpha &= k_1 * \vec{i}_a \\ \vec{i}_\beta &= k_2 * (2\vec{i}_b + \vec{i}_a)\end{aligned}$$

因为最后得到的 α 轴和 β 轴方向上两个向量相等都为单位向量。由此可得：

$$\begin{aligned}k_1 &= 1 \\ 1 &= k_2 * \sqrt{3} \Rightarrow k_2 = \frac{1}{\sqrt{3}}\end{aligned}$$

代入上面式子可以得到大小关系如下：

$$\begin{aligned}i_\alpha &= i_a \\ i_\beta &= \frac{2i_b + i_a}{\sqrt{3}}\end{aligned}$$

其实也是第2.1节的Clarke变换的变形，三相平衡的时候可以对Clarke变换作变形，将 $i_c = -i_a - i_b$ 代入即可消去 i_c ：

$$\begin{aligned}\begin{cases} i_\alpha = \frac{2}{3}(i_a - \frac{1}{2}i_b - \frac{1}{2}i_c) \\ i_\beta = \frac{2}{3}(\frac{\sqrt{3}}{2}i_b - \frac{\sqrt{3}}{2}i_c) \end{cases} &\Rightarrow \begin{cases} i_\alpha = \frac{2}{3}[i_a - \frac{1}{2}i_b - \frac{1}{2}(-i_a - i_b)] \\ i_\beta = \frac{2}{3}[\frac{\sqrt{3}}{2}i_b - \frac{\sqrt{3}}{2}(-i_a - i_b)] \end{cases} \\ \Rightarrow \begin{cases} i_\alpha = \frac{2}{3}i_a + \frac{1}{2}i_a = i_a \\ i_\beta = \frac{2}{3}(\sqrt{3}i_b + \frac{\sqrt{3}}{2}i_a) = \frac{(2i_b + i_a)}{\sqrt{3}} \end{cases} \end{aligned}$$

把它运用到定子电压计算上，计算的等式如下：

$$\begin{aligned}V_\alpha &= V_{AN} \\ V_\beta &= (2V_{BN} + V_{AN}) / \sqrt{3}\end{aligned}$$

上小节已经列出3个开关管8种不同的开关组合状态，对于不同的开关组合状态，列出了相电压与线电压的大小。这里为了方便DSP程序计算，改用Clark变换之后的两相静止正交坐标系($\alpha-\beta$)中，其相应的大小如表8-1-2所示。

表8-1-2 不同开关状态下对应($\alpha-\beta$)轴的电压空间矢量

c	b	a	V_{AN}	V_{BN}	V_{CN}	V_α	V_β	矢量
0	0	0	0	0	0	0	0	O_0
0	0	1	$2V_{DC}/3$	$-V_{DC}/3$	$-V_{DC}/3$	$2V_{DC}/3$	0	U_0
0	1	1	$V_{DC}/3$	$V_{DC}/3$	$-2V_{DC}/3$	$V_{DC}/3$	$V_{DC}/\sqrt{3}$	U_{60}

0	1	0	$-V_{DC}/3$	$2V_{DC}/3$	$-V_{DC}/3$	$-V_{DC}/3$	$V_{DC}/\sqrt{3}$	U_{120}
1	1	0	$-2V_{DC}/3$	$V_{DC}/3$	$V_{DC}/3$	$-2V_{DC}/3$	0	U_{180}
1	0	0	$-V_{DC}/3$	$-V_{DC}/3$	$2V_{DC}/3$	$-V_{DC}/3$	$-V_{DC}/\sqrt{3}$	U_{240}
1	0	1	$V_{DC}/3$	$-2V_{DC}/3$	$V_{DC}/3$	$V_{DC}/3$	$-V_{DC}/\sqrt{3}$	U_{300}
1	1	1	0	0	0	0	0	O_{111}

通过Clark坐标变换后，我们更容易求任意非零电压矢量的幅值 V ，只需要用勾股定理便可求得：

$$V = \sqrt{V_{\alpha}^2 + V_{\beta}^2} = \frac{2}{3} V_{dc} \quad (\text{式3-5-1})$$

由此可以定义六个不同的非零电压空间矢量，下标可以用相对于 α 轴的夹角命名，（角度计算为 $\arctan(V_{\beta}/V_{\alpha})$ ）分别为 U_0 ， U_{60} ， U_{120} ， U_{180} ， U_{240} ， U_{300} 。其

中两个零矢量命名为 O_{000} ， O_{111} 。将8种开关状态组合对应相应的相电压映射到二维坐标平面，得到六个非零矢量和两个零矢量。六个非零矢量按照开关状态不同分布，相邻之间夹角为60度，其中两个零矢量位于原点，由此构成了一个正六边形。如图8-1-5所示：

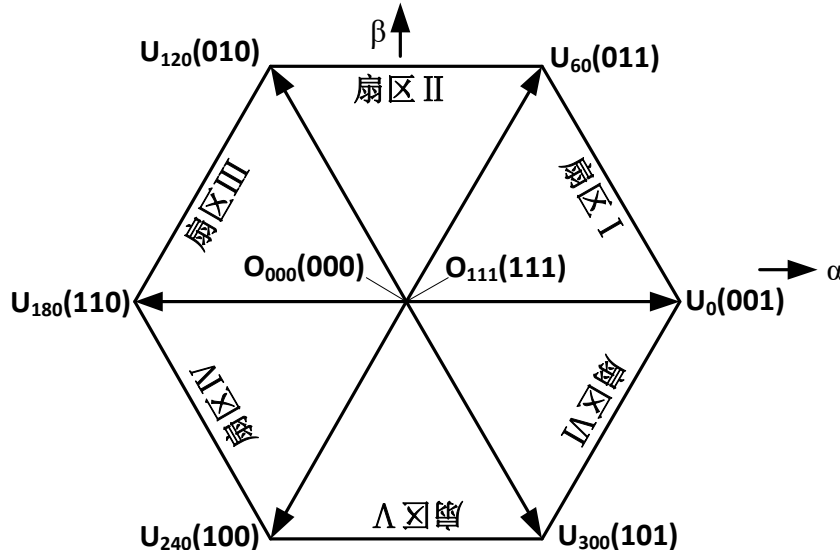


图8-1-5 不同开关状态构成的基本空间矢量

8.2 参考矢量分解及作用时间分配

为了方便统一符号和分析，现有定义：

1. 开关顺序为C，B，A。状态开为1，关为0。对应的作用时间标号为其构成的二进制转为十进制的数值。如 T_3 的开关状态为C-0，B-1，A-1，可以推出是

电压空间矢量 U_{60} 的作用时间。

2. 各个电压空间矢量大小对相电压 $\frac{U_{dc}}{\sqrt{3}}$ 进行标幺化为 $\frac{\frac{2}{3}U_{dc}}{\frac{U_{dc}}{\sqrt{3}}} = \frac{2}{\sqrt{3}}$ 。电压矢量

大小对相电压 $\frac{U_{DC}}{\sqrt{3}}$ 标幺化的目的是为了化简矢量作用时间的计算，即与60度sin

和cos三角函数值对消约去。这里的计算技巧应该要体会得到。

3. 静止两相坐标系下的两个电压大小对相电压 $\frac{U_{dc}}{\sqrt{3}}$ 进行标幺化为 U_{α}^* 和 U_{β}^*

4. 矢量方向：规定静止两相坐标系下的代数值符号：沿各坐标轴的正值增加为正，负值减小的方向为负。

5. 标幺化后的物理量均附带上标“*”。如 U_{α}^* 为 α 轴电压对相电压 $\frac{U_{dc}}{\sqrt{3}}$ 的标

幺值。

由于非零矢量的基本电压矢量只有6个，因此只能形成一个正六边形的磁链轨迹。为获得近似圆形的磁链轨迹，一种方法是利用这几个非零的基本电压空间矢量的线性时间组合来得到更多的开关状态。如扇区1图所示，任意时刻输出参考相电压矢量 U_{ref} 可由相邻的两个基本电压空间矢量 U_0 和 U_{60} 的线性时间组合来合成，它等于 T_1/T 倍的 U_0 与 T_3/T 倍的 U_{60} 的矢量和。其中 T_1 和 T_3 分别是 U_0 和 U_{60} 作用时间， T 是 U_{ref} 作用的时间。另外为了减少开关切换次数，插入零矢量进行调整。

$$\begin{cases} O_0 t_0 + U_1 t_1 + U_2 t_2 = U_{ref} T \\ t_0 + t_1 + t_2 = T \end{cases}$$

上式中 U_1, t_1 是先作用的电压矢量和时间， U_2, t_2 是后作用的电压矢量和时间，

O_0, t_0 是指两个不同的零矢量和作用时间。后续会继续分析。

扇区1分析

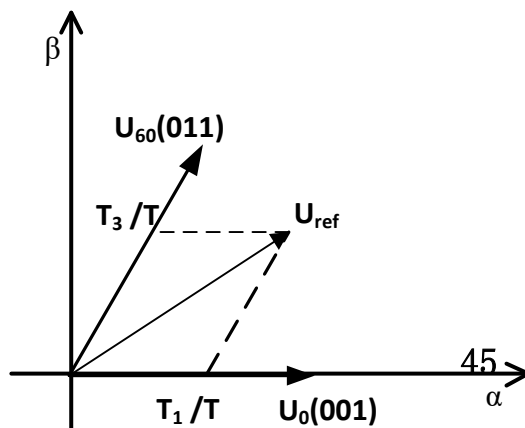


图8-2-1 参考矢量在扇区1的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 1} = \frac{T_1}{T} U_0 + \frac{T_3}{T} U_{60} \cos 60^\circ$$

$$U_{\beta 1} = \frac{T_3}{T} U_{60} \sin 60^\circ$$

计算出时间如下

$$T_3 = \frac{T U_{\beta 1}}{U_{60} \sin 60^\circ} = \frac{T \frac{U_{\beta 1}}{U_{DC} / \sqrt{3}}}{\frac{U_{60}}{U_{DC} / \sqrt{3}} \frac{\sqrt{3}}{2}} = \frac{T \frac{U_{\beta 1}}{U_{DC} / \sqrt{3}}}{\frac{2 U_{DC} / 3 \sqrt{3}}{U_{DC} / \sqrt{3}} \frac{\sqrt{3}}{2}} = \frac{T U_{\beta 1}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = T U_{\beta 1}^*$$

$$\begin{aligned} T_1 &= \frac{T(U_{\alpha 1} - \frac{T_3}{T} U_{60} \cos 60^\circ)}{U_0} \\ &= \frac{T(U_{\alpha 1} \frac{\sqrt{3}}{U_{DC}} - \frac{T_3}{T} \frac{\sqrt{3}}{U_{DC}} U_{60} \cos 60^\circ)}{U_0 \frac{\sqrt{3}}{U_{DC}}} \\ &= \frac{T(U_{\alpha 1}^* - U_{\beta 1}^* \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\ &= \frac{T \sqrt{3} U_{\alpha 1}^* - T U_{\beta 1}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\ &= \frac{T(\sqrt{3} U_{\alpha 1}^* - U_{\beta 1}^*)}{2} \end{aligned}$$

对应的占空比为

$$T_3^* = \frac{T_3}{T} = U_{\beta 1}^*$$

$$T_1^* = \frac{T_1}{T} = \frac{-(-\sqrt{3} U_{\alpha 1}^* + U_{\beta 1}^*)}{2}$$

扇区2分析

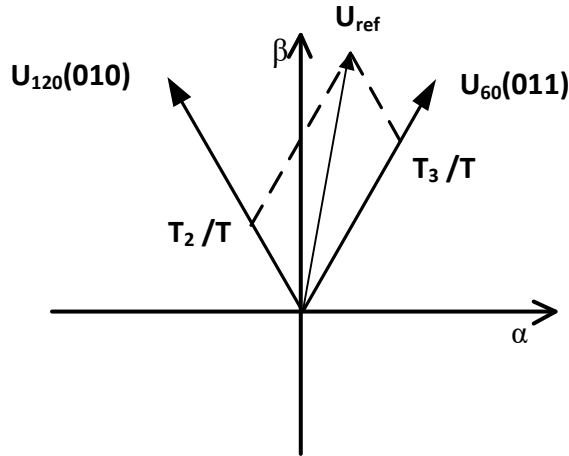


图8-2-2 参考矢量在扇区2的分解

根据矢量作用时间和几何关系可得

$$\text{式1: } U_{\alpha 2} = \frac{T_3}{T} U_{60} \cos 60^\circ - \frac{T_2}{T} U_{120} \cos 60^\circ$$

$$\text{式2: } U_{\beta 2} = \frac{T_3}{T} U_{60} \sin 60^\circ + \frac{T_2}{T} U_{120} \sin 60^\circ$$

计算出时间如下

由式1得:

$$\begin{aligned} T_3 &= \frac{T(U_{\alpha 2} + \frac{T_2}{T} U_{120} \cos 60^\circ)}{U_{60} \cos 60^\circ} \\ T_3 &= \frac{T(U_{\alpha 2} \frac{\sqrt{3}}{U_{DC}} + \frac{T_2}{T} \frac{\sqrt{3}}{U_{DC}} U_{120} \cos 60^\circ)}{\frac{2}{\sqrt{3}} \frac{1}{2}} \\ &= \frac{T(U_{\alpha 2}^* + \frac{T_2}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}} \frac{1}{2}} \\ &= T\sqrt{3}U_{\alpha 2}^* + T_2\sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2} \\ &= T\sqrt{3}U_{\alpha 2}^* + T_2 \end{aligned}$$

由式2得：

$$T_2 = \frac{T(U_{\beta 2} - \frac{T_3}{T} U_{60} \sin 60^\circ)}{U_{120} \sin 60^\circ}$$

$$= \frac{T(U_{\beta 2} \frac{\sqrt{3}}{U_{DC}} - \frac{T_3}{T} \frac{\sqrt{3}}{U_{DC}} U_{60} \sin 60^\circ)}{U_{120} \frac{\sqrt{3}}{U_{DC}} \sin 60^\circ}$$

$$= \frac{T(U_{\beta 2}^* - \frac{T_3}{T} \frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2})}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}}$$

$$= TU_{\beta 2}^* - T_3$$

再由上式解得：

$$T_3 = T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^* - T_3$$

$$= \frac{T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^*}{2}$$

$$T_2 = TU_{\beta 2}^* - \frac{T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^*}{2}$$

$$= \frac{-T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^*}{2}$$

对应的占空比为

$$T_3^* = \frac{T_3}{T} = \frac{\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$$

$$T_2^* = \frac{T_2}{T} = \frac{-\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$$

扇区3分析

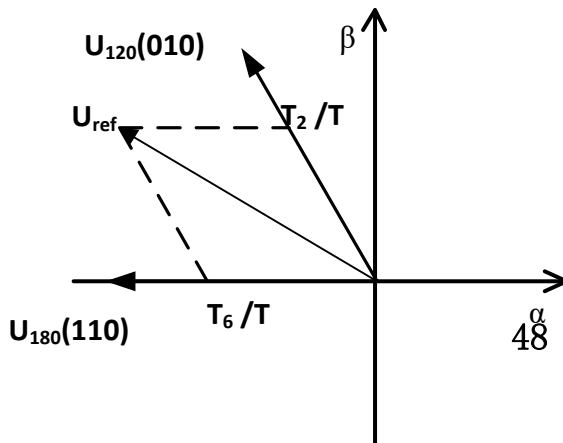


图8-2-3 参考矢量在扇区3的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 3} = -\frac{T_6}{T} U_{180} - \frac{T_2}{T} U_{120} \cos 60^\circ$$

$$U_{\beta 3} = \frac{T_2}{T} U_{120} \sin 60^\circ$$

计算出时间如下

$$T_2 = \frac{T U_{\beta 3}}{U_{120} \sin 60^\circ} = \frac{T U_{\beta 1}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = T U_{\beta 1}^*$$

$$\begin{aligned} T_6 &= \frac{T(-U_{\alpha 3} - \frac{T_2}{T} U_{120} \cos 60^\circ)}{U_{180}} \\ &= \frac{T(-U_{\alpha 3} \frac{\sqrt{3}}{U_{DC}} - \frac{T_2}{T} U_{120} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_{180} \frac{\sqrt{3}}{U_{DC}}} \\ &= \frac{T(-U_{\alpha 3}^* - \frac{T_2}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\ &= \frac{-T \sqrt{3} U_{\alpha 3}^* - T U_{\beta 3}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\ &= \frac{T(-\sqrt{3} U_{\alpha 3}^* - U_{\beta 3}^*)}{2} \end{aligned}$$

对应的占空比为

$$T_2^* = \frac{T_2}{T} = U_{\beta 3}^*$$

$$T_6^* = \frac{T_6}{T} = \frac{-(\sqrt{3} U_{\alpha 3}^* + U_{\beta 3}^*)}{2}$$

扇区4分析

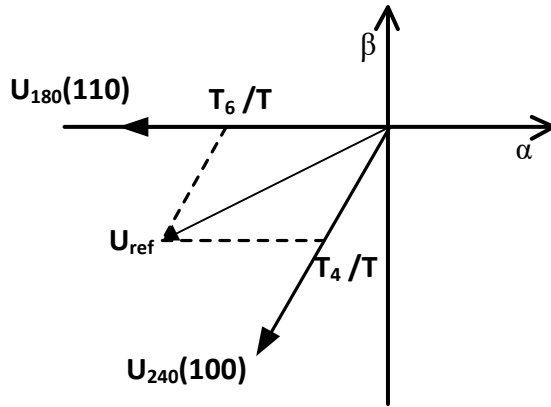


图8-2-4 参考矢量在扇区4的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 4} = -\frac{T_6}{T} U_{180} - \frac{T_4}{T} U_{240} \cos 60^\circ$$

$$U_{\beta 4} = -\frac{T_4}{T} U_{240} \sin 60^\circ$$

计算出时间如下

$$T_4 = -\frac{T U_{\beta 4}}{U_{240} \sin 60^\circ} = -\frac{T U_{\beta 4}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = -T U_{\beta 4}^*$$

$$\begin{aligned} T_6 &= \frac{T(-U_{\alpha 4} - \frac{T_4}{T} U_{240} \cos 60^\circ)}{U_{180}} \\ &= \frac{T(-U_{\alpha 4} \frac{\sqrt{3}}{U_{DC}} - \frac{T_4}{T} U_{240} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_{180} \frac{\sqrt{3}}{U_{DC}}} \\ &= \frac{T(-U_{\alpha 4}^* - \frac{T_4}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\ &= \frac{-T \sqrt{3} U_{\alpha 4}^* + T U_{\beta 4}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\ &= \frac{T(-\sqrt{3} U_{\alpha 4}^* + U_{\beta 4}^*)}{2} \end{aligned}$$

对应的占空比为

$$T_4^* = \frac{T_4}{T} = -U_{\beta 4}^*$$

$$T_6^* = \frac{T_6}{T} = \frac{-\sqrt{3}U_{\alpha 4}^* + U_{\beta 4}^*}{2}$$

扇区5分析

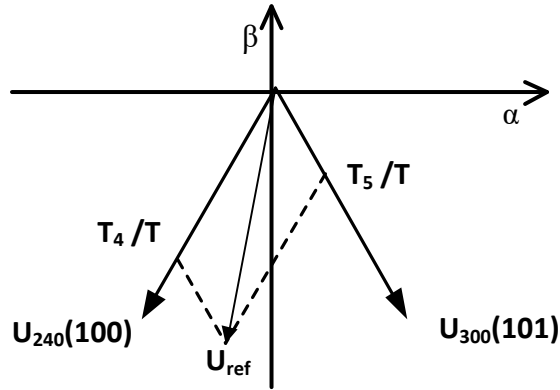


图8-2-5 参考矢量在扇区5的分解

根据矢量作用时间和几何关系可得

$$\text{式1: } U_{\alpha 5} = \frac{T_5}{T} U_{300} \cos 60^\circ - \frac{T_4}{T} U_{240} \cos 60^\circ$$

$$\text{式2: } U_{\beta 5} = -\frac{T_5}{T} U_{300} \sin 60^\circ - \frac{T_4}{T} U_{240} \sin 60^\circ$$

计算出时间如下

由式1得:

$$\begin{aligned}
 T_5 &= \frac{T(U_{\alpha 5} + \frac{T_4}{T} U_{240} \cos 60^\circ)}{U_{300} \cos 60^\circ} \\
 &= \frac{T(U_{\alpha 5} \frac{\sqrt{3}}{U_{DC}} + \frac{T_4}{T} U_{240} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_{300} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ} \\
 &= \frac{T(U_{\alpha 5}^* + \frac{T_4}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}} \frac{1}{2}} \\
 &= T\sqrt{3}U_{\alpha 5}^* + T_4\sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2} \\
 &= T\sqrt{3}U_{\alpha 5}^* + T_4
 \end{aligned}$$

由式2得:

$$\begin{aligned}
 T_4 &= \frac{T(-U_{\beta 5} - \frac{T_5}{T} U_{300} \sin 60^\circ)}{U_{240} \sin 60^\circ} \\
 &= \frac{T(-U_{\beta 5} \frac{\sqrt{3}}{U_{DC}} - \frac{T_5}{T} U_{300} \frac{\sqrt{3}}{U_{DC}} \sin 60^\circ)}{U_{240} \frac{\sqrt{3}}{U_{DC}} \sin 60^\circ} \\
 &= \frac{T(-U_{\beta 5}^* - \frac{T_5}{T} \frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2})}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} \\
 &= -TU_{\beta 5}^* - T_5
 \end{aligned}$$

再由上式解得:

$$\begin{aligned}
 T_5 &= T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^* - T_5 \\
 &= \frac{T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^*}{2} \\
 T_4 &= -TU_{\beta 5}^* - \frac{T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^*}{2} \\
 &= \frac{-T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^*}{2}
 \end{aligned}$$

对应的占空比为

$$T_5^* = \frac{T_5}{T} = \frac{-(-\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$$

$$T_4^* = \frac{T_4}{T} = \frac{-(\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$$

扇区6分析

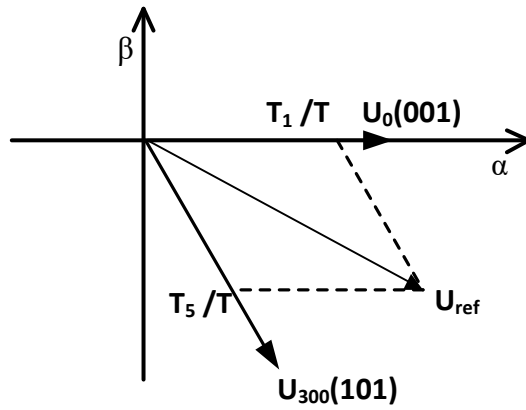


图8-2-6 参考矢量在扇区6的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 6} = \frac{T_1}{T}U_0 + \frac{T_5}{T}U_{300} \cos 60^\circ$$

$$U_{\beta 6} = -\frac{T_5}{T}U_{300} \sin 60^\circ$$

计算出时间如下

$$T_5 = -\frac{TU_{\beta 6}}{U_{300} \sin 60^\circ} = \frac{-TU_{\beta 6}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = -TU_{\beta 6}^*$$

$$\begin{aligned}
T_1 &= \frac{T(U_{\alpha 6} - \frac{T_5}{T} U_{300} \cos 60^\circ)}{U_0} \\
&= \frac{T(U_{\alpha 6} \frac{\sqrt{3}}{U_{DC}} - \frac{T_5}{T} U_{300} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_0 \frac{\sqrt{3}}{U_{DC}}} \\
&= \frac{T(U_{\alpha 6}^* + U_{\beta 6}^* \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\
&= \frac{T\sqrt{3}U_{\alpha 6}^* + TU_{\beta 6}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\
&= \frac{T(\sqrt{3}U_{\alpha 6}^* + U_{\beta 6}^*)}{2}
\end{aligned}$$

对应的占空比为

$$\begin{aligned}
T_5^* &= \frac{T_5}{T} = -U_{\beta 6}^* \\
T_1^* &= \frac{T_1}{T} = \frac{\sqrt{3}U_{\alpha 6}^* + U_{\beta 6}^*}{2}
\end{aligned}$$

8.3 各扇区矢量的先后作用顺序及占空比

8.3.1 作用顺序的确定

先确定对应基本矢量的先后作用。因为要按照开关切换次数最少，利用率最高的原则，在非零电压空间矢量作用的前后插入起调整作用的零矢量。

排列的具体顺序是：每个参考矢量分解后以零矢量 O_{000} (000) 开始作用，接着是开关状态只含有一个“1”的矢量作用，紧跟着作用的矢量其开关状态含有两个“1”，最后作用的为另外一个零矢量 O_{111} (111)，这里是上半个PWM周期的基本矢量作用顺序。

因为采用对称的七段调制，那么下半个PWM周期则以零矢量 O_{111} (111) 开始作用，接着是开关状态含有两个“1”的矢量作用，紧跟着作用的矢量其开关状态只含有一个“1”，最后作用的为零矢量 O_0 (000)。

显而易见，开关状态只含有一个“1”的矢量有 U_0 、 U_{120} 、 U_{240} 。开关状态

含有两个“1”的矢量有 U_{60} 、 U_{180} 、 U_{300} 。

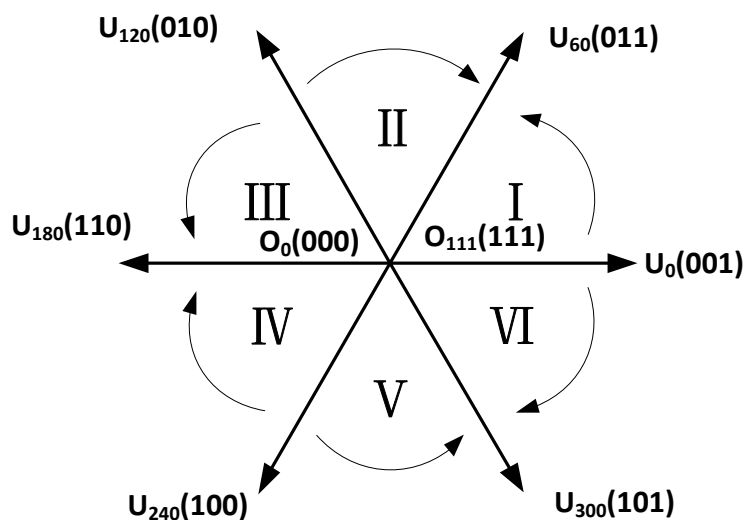


图8-3-1 各扇区相邻矢量的先后作用

如图8-3-1所示，圆弧箭头从先作用矢量指向后作用矢量；按照上述方法可以大致排列好基本矢量的作用顺序，整理可以得出表8-3-1如下：

表8-3-1 扇区与电压空间矢量作用顺序的关系

扇区编号	先作用矢量	后作用矢量	采用对称七段调制的矢量作用顺序
S1	U_0	U_{60}	$O_0 \rightarrow U_0 \rightarrow U_{60} \rightarrow O_{111} \rightarrow U_{60} \rightarrow U_0 \rightarrow O_0$
S2	U_{120}	U_{60}	$O_0 \rightarrow U_{120} \rightarrow U_{60} \rightarrow O_{111} \rightarrow U_{60} \rightarrow U_{120} \rightarrow O_0$
S3	U_{120}	U_{180}	$O_0 \rightarrow U_{120} \rightarrow U_{180} \rightarrow O_{111} \rightarrow U_{180} \rightarrow U_{120} \rightarrow O_0$
S4	U_{240}	U_{180}	$O_0 \rightarrow U_{240} \rightarrow U_{180} \rightarrow O_{111} \rightarrow U_{180} \rightarrow U_{240} \rightarrow O_0$
S5	U_{240}	U_{300}	$O_0 \rightarrow U_{240} \rightarrow U_{300} \rightarrow O_{111} \rightarrow U_{300} \rightarrow U_{240} \rightarrow O_0$
S6	U_0	U_{300}	$O_0 \rightarrow U_0 \rightarrow U_{300} \rightarrow O_{111} \rightarrow U_{300} \rightarrow U_0 \rightarrow O_0$

8.3.2 占空比通用参考量

现定义先作用矢量的占空比为 t_1 ，后作用矢量的占空比为 t_2 。根据上一节的

推导分析，定义几个作用时间的标么值亦即占空比：

$$X = U_{\beta}^* \quad Y = \frac{\sqrt{3}U_{\alpha}^* + U_{\beta}^*}{2} \quad Z = \frac{-\sqrt{3}U_{\alpha}^* + U_{\beta}^*}{2}$$

那么可以得出表8-3-2（与TI官方文档对应）：

表8-3-2 先后顺序及占空比（TI官方）

Sector	U_0, U_{60}	U_{120}, U_{60}	U_{120}, U_{180}	U_{180}, U_{240}	U_{240}, U_{300}	U_{300}, U_0
先作用矢量 量占空 t_1	$(U_0 : T_1^*)$ -Z	$(U_{120} : T_2^*)$ Z	$(U_{120} : T_2^*)$ X	$(U_{240} : T_4^*)$ -X	$(U_{240} : T_4^*)$ -Y	$(U_0 : T_1^*)$ Y
后作用矢量 量占空 t_2	$(U_{60} : T_3^*)$ X	$(U_{60} : T_3^*)$ Y	$(U_{180} : T_6^*)$ -Y	$(U_{180} : T_6^*)$ Z	$(U_{300} : T_5^*)$ -Z	$(U_{300} : T_5^*)$ -X

程序中定义的跟上面的定义不同，程序相应的参考值为：

$$X = U_{\beta}^* \quad Y = \frac{\sqrt{3}U_{\alpha}^* - U_{\beta}^*}{2} \quad Z = \frac{-\sqrt{3}U_{\alpha}^* - U_{\beta}^*}{2}$$

那么可以得出表8-3-3（与项目程序对应）：

表8-3-3 先后顺序及占空比（项目程序）

SectorTemp	3	1	5	4	6	2
先作用矢量的占空 比 t_1	$(U_0 : T_1^*)$ Y	$(U_{120} : T_2^*)$ -Y	$(U_{120} : T_2^*)$ X	$(U_{240} : T_4^*)$ -X	$(U_{240} : T_4^*)$ Z	$(U_0 : T_1^*)$ -Z
后作用矢量的占空 比 t_2	$(U_{60} : T_3^*)$ X	$(U_{60} : T_3^*)$ -Z	$(U_{180} : T_6^*)$ Z	$(U_{180} : T_6^*)$ -Y	$(U_{300} : T_5^*)$ Y	$(U_{300} : T_5^*)$ -X

8.4 电压标么化的基值变换处理

到了这一步应该注意到这里提及的UaOut, UbOut, Udc都是对母线电压量程倍数VOLTAGE_SENSOR标么化的Q15格式数值。规定 U_{α}^{*b} 代表对 U_{base} 标么的值，

U_{α}^* 代表对 $\frac{U_{DC}}{\sqrt{3}}$ 标么的值，其他的量也用同样的表示方法。

在8.3节中分析各扇区后定义的值：

$$X = U_{\beta}^* \quad Y = \frac{\sqrt{3}U_{\alpha}^* - U_{\beta}^*}{2} \quad Z = \frac{-\sqrt{3}U_{\alpha}^* - U_{\beta}^*}{2}$$

这些都是要求静止两坐标的电压量 U_α 和 U_β 要对相电压 $\frac{U_{DC}}{\sqrt{3}}$ 标么化，而程序中静止两坐标的电压量 U_α 和 U_β 是对 U_{base} （程序中的VOLTAGE_SENSOR）标么化的，这里需要改为对相电压 $\frac{U_{DC}}{\sqrt{3}}$ 标么化且为Q15格式！这样才可以得出相应的占空比。

转换前的代码如下：

//LJF 扇区判断的3个参考电压或是作用时间【相关量】！不是占空比！！

```
Temp1 = (int32)MotorDriveOutStr.UaOut * 28378 >> 15; //28378:2分之根号3
```

```
Temp2 = MotorDriveOutStr.UbOut >> 1;
```

```
VolTemp1 = MotorDriveOutStr.UbOut;
```

```
VolTemp2 = Temp1 - Temp2;
```

```
VolTemp3 = -Temp1 - Temp2;
```

用规定的符号表示为

$$U_\beta^{*b} = \text{VolTemp1};$$

$$\frac{\sqrt{3}U_\alpha^{*b} - U_\beta^{*b}}{2} = \text{VolTemp2};$$

$$\frac{-\sqrt{3}U_\alpha^{*b} - U_\beta^{*b}}{2} = \text{VolTemp3};$$

【1】先明确这里的Q15格式标么值是对 U_{base} （程序中的VOLTAGE_SENSOR）标么的。 U_α^{*b} 代表对 U_{base} 标么的值， U_α^* 代表对 $\frac{U_{DC}}{\sqrt{3}}$ 标么的值，其他的量也用同样的表示方法。与程序对应关系为：

$$U_\alpha^{*b} = \text{MotorDriveOutStr.UaOut}$$

$$U_\beta^{*b} = \text{MotorDriveOutStr.UbOut}$$

$$U_{dc}^{*b} = \text{MotorDriveInStr.Udc}$$

$$U_{base} = \text{VOLTAGE_SENSOR}$$

【2】由标么值转为实际值。

$$U_{\alpha}^{*b} = \frac{U_{\alpha}}{U_{base}} * 2^{15} \rightarrow U_{\alpha} = \frac{U_{\beta}^{*b} U_{base}}{2^{15}}$$

$$U_{\beta}^{*b} = \frac{U_{\beta}}{U_{base}} * 2^{15} \rightarrow U_{\beta} = \frac{U_{\beta}^{*b} U_{base}}{2^{15}}$$

$$U_{dc}^{*b} = \frac{U_{dc}}{U_{base}} * 2^{15} \rightarrow U_{dc} = \frac{U_{dc}^{*b} U_{base}}{2^{15}}$$

【3】把 U_{α} ， U_{β} ， U_{DC} 分别对 $\frac{U_{DC}}{\sqrt{3}}$ 标么化且转为Q15格式整理好关系就可以求出

X,Y,Z三个Q15格式的“占空比”。

求X的过程：

$$\begin{aligned} X = U_{\beta}^{*} &= \frac{U_{\beta}}{\frac{U_{DC}}{\sqrt{3}}} * 2^{15} = \frac{\frac{U_{\beta}^{*b} U_{base}}{2^{15}}}{(\frac{U_{dc}^{*b} U_{base}}{2^{15}}) / 56756 * 2^{-15}} * 2^{15} \\ &= \frac{U_{\beta}^{*b} * 56756 * 2^{-15}}{U_{dc}^{*b}} * 2^{15} = \frac{U_{\beta}^{*b} * 56756}{U_{dc}^{*b}} \end{aligned}$$

实现上述功能的代码如下：

```
Time1 = VolTemp1 * 56756 / MotorDriveInStr.Udc;
```

求Y的过程：

$$\begin{aligned} Y &= \frac{\sqrt{3}U_{\alpha}^{*} - U_{\beta}^{*}}{2} = \frac{\sqrt{3}U_{\alpha} / \frac{U_{dc}}{\sqrt{3}} - U_{\beta} / \frac{U_{dc}}{\sqrt{3}}}{2} * 2^{15} \\ &= \frac{U_{\alpha} * \sqrt{3} / 2 - U_{\beta} / 2}{\frac{U_{dc}}{\sqrt{3}}} * 2^{15} = \frac{\frac{U_{\alpha}^{*b} * U_{base}}{2^{15}} * 28378 * 2^{-15} - \frac{U_{\beta}^{*b} U_{base}}{2^{15}} * \frac{1}{2}}{(\frac{U_{dc}^{*b} U_{base}}{2^{15}}) / 56756 * 2^{-15}} * 2^{15} \\ &= \frac{(28378 * 2^{-15} * U_{\alpha}^{*b} - \frac{1}{2} * U_{\beta}^{*b}) * 56756 * 2^{-15}}{U_{dc}^{*b}} * 2^{15} \\ &= \frac{(28378 * 2^{-15} * U_{\alpha}^{*b} - \frac{1}{2} * U_{\beta}^{*b}) * 56756}{U_{dc}^{*b}} \end{aligned}$$

实现上述功能的代码如下：

```
Time2 = VolTemp2 * 56756 / MotorDriveInStr.Udc;
```

求Z的过程：

$$\begin{aligned}
Z &= \frac{-\sqrt{3}U_{\alpha}^* - U_{\beta}^*}{2} = \frac{-\sqrt{3}U_{\alpha} / \frac{U_{dc}}{\sqrt{3}} - U_{\beta} / \frac{U_{dc}}{\sqrt{3}}}{2} * 2^{15} \\
&= \frac{-U_{\alpha} * \sqrt{3} / 2 - U_{\beta} / 2}{\frac{U_{dc}}{\sqrt{3}}} * 2^{15} = \frac{-\frac{U_{\alpha}^* U_{base}}{2^{15}} * 28378 * 2^{-15} - \frac{U_{\beta}^* U_{base}}{2^{15}} * \frac{1}{2}}{\left(\frac{U_{dc}^* U_{base}}{2^{15}}\right) / 56756 * 2^{-15}} * 2^{15} \\
&= \frac{(-28378 * 2^{-15} * U_{\alpha}^* - \frac{1}{2} * U_{\beta}^*) * 56756 * 2^{-15}}{U_{dc}^*} * 2^{15} \\
&= \frac{(-28378 * 2^{-15} * U_{\alpha}^* - \frac{1}{2} * U_{\beta}^*) * 56756}{U_{dc}^*}
\end{aligned}$$

实现上述功能的代码如下：

```
Time3 = VolTemp3 * 56756 / MotorDriveInStr.Udc;
```

8.5 矢量所在扇区判断

经过前几节的说明，已经介绍了基本电压空间矢量，参考电压矢量分解成基本电压空间矢量，以及推导出对称七段调制的矢量作用顺序和非零电压矢量的Q15格式占空比。以上所述并没有判断参考电压矢量所在的扇区，所以也不知道实际的作用顺序。

不管 U_{ref} 大小、是否过调制都满足 $U_{ref} = \sqrt{U_{\alpha}^2 + U_{\beta}^2}$ ，那么在静止两相坐标系

下则有 $U_{\alpha} = U_{ref} \cos \alpha$ 和 $U_{\beta} = U_{ref} \sin \alpha$ 这样的关系，参考图8-5-1，按照矢量间60度递增和正余弦关系分析扇区如下：

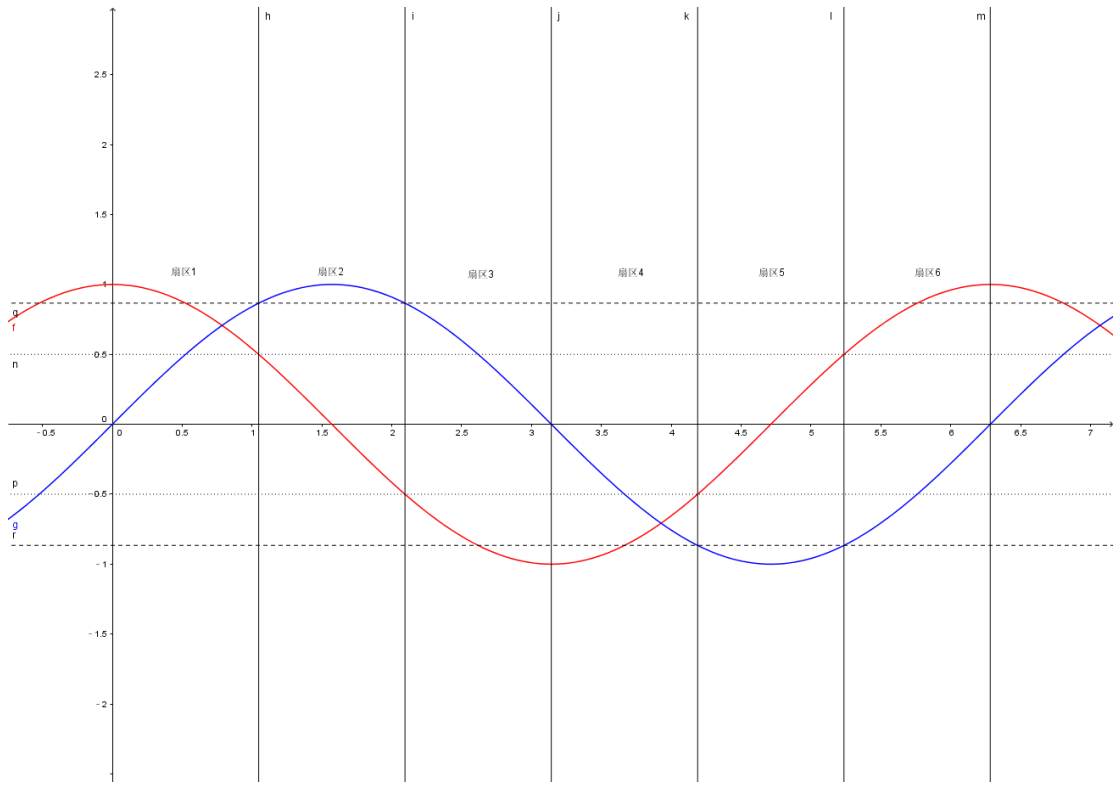


图8-5-1 各扇区正交量的大小

当 U_{ref} 处于扇区1中： $U_{\alpha} > 0$ 且 $|U_{\alpha}| \geq |U_{\text{ref}}| \cos 60^{\circ}$ 即 $|U_{\alpha}| \geq \frac{1}{2}|U_{\text{ref}}|$ 。 $U_{\beta} > 0$ 且

$|U_{\beta}| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}|$ 。实际上，若进一步结合矢量几何关系分析，条件可表述为：

$$|U_{\alpha}| \geq \frac{1}{2}|U_{\text{ref}}| \Rightarrow \sqrt{3}|U_{\alpha}| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 且 } |U_{\beta}| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 那么 } \sqrt{3}|U_{\alpha}| - |U_{\beta}| \geq 0$$

当 $U_{\beta} > 0$ ，且 $\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta} > 0$ 时，则 U_{ref} 处于扇区1

当 U_{ref} 处于扇区2中： $U_{\alpha} > 0$ 或 $U_{\alpha} < 0$ 且 $|U_{\alpha}| \leq \frac{1}{2}|U_{\text{ref}}|$ ， $U_{\beta} > 0$ 且

$|U_{\beta}| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}|$ 。实际上，若进一步结合矢量几何关系分析，条件可表述为：

$$|U_{\alpha}| \leq \frac{1}{2}|U_{\text{ref}}| \Rightarrow \sqrt{3}|U_{\alpha}| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 且 } |U_{\beta}| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 那么 } \sqrt{3}|U_{\alpha}| - |U_{\beta}| \leq 0$$

当 $U_{\beta} > 0$ ，且 $\frac{\sqrt{3}}{2}|U_{\alpha}| - \frac{1}{2}U_{\beta} < 0$ 时，则 U_{ref} 处于扇区2

当 U_{ref} 处于扇区3中： $U_\alpha < 0$ 且 $|U_\alpha| \geq \frac{1}{2}|U_{\text{ref}}|$ 。 $U_\beta > 0$ 且 $|U_\beta| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}|$ 。实际上，若进一步结合矢量几何关系分析，条件可表述为：

$$|U_\alpha| \geq \frac{1}{2}|U_{\text{ref}}| \Rightarrow \sqrt{3}|U_\alpha| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 且 } |U_\beta| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 那么 } \sqrt{3}|U_\alpha| - |U_\beta| \geq 0$$

当 $U_\beta > 0$ ，且 $-\frac{\sqrt{3}}{2}U_\alpha - \frac{1}{2}U_\beta > 0$ 时，则 U_{ref} 处于扇区3

当 U_{ref} 处于扇区4中： $U_\alpha < 0$ 且 $|U_\alpha| \geq \frac{1}{2}|U_{\text{ref}}|$ 。 $U_\beta < 0$ 且 $|U_\beta| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}|$ 。实际上，若进一步结合矢量几何关系分析，条件可表述为：

$$|U_\alpha| \geq \frac{1}{2}|U_{\text{ref}}| \Rightarrow \sqrt{3}|U_\alpha| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 且 } |U_\beta| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 那么 } \sqrt{3}|U_\alpha| - |U_\beta| \geq 0$$

当 $U_\beta < 0$ ，且 $\frac{\sqrt{3}}{2}U_\alpha - \frac{1}{2}U_\beta < 0$ 时，则 U_{ref} 处于扇区4

当 U_{ref} 处于扇区5中： $U_\alpha > 0$ 或 $U_\alpha < 0$ 且 $|U_\alpha| \leq \frac{1}{2}|U_{\text{ref}}|$ ， $U_\beta < 0$ 且 $|U_\beta| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}|$ 。实际上，若进一步结合矢量几何关系分析，条件可表述为：

$$|U_\alpha| \leq \frac{1}{2}|U_{\text{ref}}| \Rightarrow \sqrt{3}|U_\alpha| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 且 } |U_\beta| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 那么 } \sqrt{3}|U_\alpha| - |U_\beta| \leq 0$$

当 $U_\beta < 0$ ，且 $-\frac{\sqrt{3}}{2}|U_\alpha| - \frac{1}{2}U_\beta > 0$ 时，则 U_{ref} 处于扇区5

当 U_{ref} 处于扇区6中： $U_\alpha > 0$ 且 $|U_\alpha| \geq \frac{1}{2}|U_{\text{ref}}|$ ， $U_\beta < 0$ 且 $|U_\beta| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}|$ 。实际上，若进一步结合矢量几何关系分析，条件可表述为：

$$|U_\alpha| \geq \frac{1}{2}|U_{\text{ref}}| \Rightarrow \sqrt{3}|U_\alpha| \geq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 且 } |U_\beta| \leq \frac{\sqrt{3}}{2}|U_{\text{ref}}| \text{ 那么 } \sqrt{3}|U_\alpha| - |U_\beta| \geq 0$$

当 $U_\beta < 0$ ，且 $-\frac{\sqrt{3}}{2}U_\alpha - \frac{1}{2}U_\beta < 0$ 时，则 U_{ref} 处于扇区6

不等式两边同时除以2的原因是为了跟第8.3.2节占空比计算统一形式。

运用条件等效，这样扇区的判断与 U_{ref} 无关，完全避免了计算非线性函数，实现起来就容易得多了。

整理得出：

当 $U_{\beta} > 0$ ，且 $\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta} > 0$ 时，则 U_{ref} 处于扇区1

当 $U_{\beta} > 0$ ，且 $\frac{\sqrt{3}}{2}|U_{\alpha}| - \frac{1}{2}U_{\beta} < 0$ 时，则 U_{ref} 处于扇区2

当 $U_{\beta} > 0$ ，且 $-\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta} > 0$ 时，则 U_{ref} 处于扇区3

当 $U_{\beta} < 0$ ，且 $\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta} < 0$ 时，则 U_{ref} 处于扇区4

当 $U_{\beta} < 0$ ，且 $-\frac{\sqrt{3}}{2}|U_{\alpha}| - \frac{1}{2}U_{\beta} > 0$ 时，则 U_{ref} 处于扇区5

当 $U_{\beta} < 0$ ，且 $-\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta} < 0$ 时，则 U_{ref} 处于扇区6

采用上述条件，只需经过简单的加减及逻辑运算即可确定所在的扇区，避免了计算复杂的非线性函数，对于简化运算和提高系统的响应速度很有实际的意义，但这还不是最简练的表述，若对上述条件作进一步的分析，判断方法可进一步简化。

由所推导出的条件可以看出， U_{ref} 所处所在的扇区完全可由 U_{β} ， $\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta}$ ，

$-\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta}$ 三式与0的关系决定，由此，可定义以下变量：

$$U_{\text{ref}1} = U_{\beta}$$

$$U_{\text{ref}2} = \frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta}$$

$$U_{\text{ref}3} = -\frac{\sqrt{3}}{2}U_{\alpha} - \frac{1}{2}U_{\beta}$$

再定义：

若 $U_{\text{ref}1} > 0$ ，则 $A=1$ ，否则 $A=0$ ；

若 $U_{ref2} > 0$ ，则 $B=1$ ，否则 $B=0$ ；

若 $U_{ref3} > 0$ ，则 $C=1$ ，否则 $C=0$ ；

A, B, C 之间共有八种组合，但由判断扇区的公式可知 A, B, C 不会同时为1或同时为0，所以实际的组合是六种， A, B, C 组合取不同的值对应着不同的扇区，并且是一一对应的，因此完全可以由 A, B, C 的组合判断所在的扇区。为区别六种状态，令：

$$S = A + 2B + 4C$$

则 S 可为1至6六个整数值，正好与六个扇区一一对应，只是在具体数值顺序上与扇区实际顺序有所差别，用上式判断出的数值与实际扇区 N 的对应关系如下表8-5-1所示：

表8-5-1 各扇区的 S 值

实际扇区	1	2	3	4	5	6
对应的 S	3	1	5	4	6	2

用上述方法判断参考电压矢量 U_{ref} 所处的扇区非常简单，只要在具体分配作用矢量时注意将计算出的 S 值与实际扇区号 N 对应即可。

实现以上功能的代码如下：

//LJF 扇区判断的3个参考电压或是作用时间【相关量】！不是占空比！！

Temp1 = (int32)MotorDriveOutStr.UaOut * 28378 >> 15; //28378:2分之根号3

Temp2 = MotorDriveOutStr.UbOut >> 1;

VolTemp1 = MotorDriveOutStr.UbOut;

VolTemp2 = Temp1 - Temp2;

VolTemp3 = -Temp1 - Temp2;

```

if (VolTemp1 > 0)
{
    SectorTemp = 1;
}
if (VolTemp2 > 0)
{
    SectorTemp = SectorTemp + 2;
}
if (VolTemp3 > 0)
{
    SectorTemp = SectorTemp + 4;
}

```

8.6 矢量切换时刻及对应占空比

上一节求出了参考电压矢量所在的扇区，那么可以知道先后作用的矢量。假定在DSP芯片上配置好的PWM模块为增减计数。在增计数时计数值等于比较值时

候，对应控制上桥信号置高，在减计数时计数值等于比较值时候，对应控制上桥信号置低，用七段波形调制。由于波形的对称性，各个矢量作用时间分成2份，分开后用于增计数作用和减计数作用。为了满足开关切换最少，以及方便后续的死区补偿（见第7节）添加了两个零矢量，又因为作用时间不变，所以该模式下只能把零矢量作用时间再分成2份。方便区别和分析，这里规定小写 t 为占空比，大写的 T 为实际作用时间。

按照上述的逻辑，可以得出下表8-6-1：

表8-6-1 矢量作用时刻表

增减计数状态	作用的电压空间矢量	持续时间	开始时刻	结束时刻
增计数	零矢量 O_{000} (000)	$\frac{T_0}{4}$	0	$\frac{T_0}{4}$
	开关状态只含有一个“1”的矢量	$\frac{T_1}{2}$	$\frac{T_0}{4}$	$\frac{T_0}{4} + \frac{T_1}{2}$
	开关状态含有两个“1”的矢量	$\frac{T_2}{2}$	$\frac{T_0}{4} + \frac{T_1}{2}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2}$
	零矢量 O_{111} (111)	$\frac{T_0}{4}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} + \frac{T_0}{4} = \frac{T_{PWM}}{2}$
减计数	零矢量 O_{111} (111)	$\frac{T_0}{4}$	$\frac{T_{PWM}}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4}$
	开关状态含有两个“1”的矢量	$\frac{T_2}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2}$
	开关状态只含有一个“1”的矢量	$\frac{T_1}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2} + \frac{T_1}{2}$
	零矢量 O_{000} (000)	$\frac{T_0}{4}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2} + \frac{T_1}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} + \frac{T_0}{4} = T_{PWM}$

按照上表，三个切换时刻的分析计算如下：

【1】先作用的零矢量作用完毕，先作用的非零矢量作用开始的时刻及占空比：

$$\begin{aligned}
 \frac{T_0}{4} &= \frac{T_{PWM} - T_1 - T_2}{4} \\
 &= \frac{T_{PWM}}{2} * \frac{1 - T_1^* - T_2^*}{2} \\
 &= \frac{T_{PWM}}{2} * \frac{1 - t_1 - t_2}{2} = \frac{T_{PWM}}{2} * t_{aon}
 \end{aligned}$$

根据上述定义 $t_{aon} = \frac{1 - t_1 - t_2}{2}$

【2】先作用的非零矢量作用完毕，后作用的非零矢量作用开始的时刻及占空比：

$$\begin{aligned}
\frac{T_0}{4} + \frac{T_1}{2} &= \frac{T_{PWM} - T_1 - T_2}{4} + \frac{T_1}{2} \\
&= \frac{T_{PWM}}{2} * \left(\frac{1 - T_1^* - T_2^*}{2} + T_1^* \right) \\
&= \frac{T_{PWM}}{2} * \left(\frac{1 - t_1 - t_2}{2} + t_1 \right) = \frac{T_{PWM}}{2} * t_{bon}
\end{aligned}$$

根据上述定义 $t_{bon} = t_{aon} + t_1$

【3】 后作用的非零矢量作用完毕，后作用的零矢量作用开始的时刻及占空比：

$$\begin{aligned}
\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} &= \frac{T_{PWM} - T_1 - T_2}{4} + \frac{T_1}{2} + \frac{T_2}{2} \\
&= \frac{T_{PWM}}{2} * \left(\frac{1 - T_1^* - T_2^*}{2} + T_1^* + T_2^* \right) \\
&= \frac{T_{PWM}}{2} * \left(\frac{1 - t_1 - t_2}{2} + t_1 + t_2 \right) = \frac{T_{PWM}}{2} * t_{con}
\end{aligned}$$

根据上述定义 $t_{con} = t_{bon} + t_2$

明显可以得到 $t_{aon} < t_{bon} < t_{con}$ 这样的关系。这里的abc字母不代表电机三相序号，而是代表导通的优先顺序，a代表最先导通，b代表接着导通，c代表最后导通。以上求出了切换时刻对应的占空比，再次整理如下：

$$\begin{aligned}
t_{aon} &= \frac{1 - t_1 - t_2}{2} \\
t_{bon} &= t_{aon} + t_1 \\
t_{con} &= t_{bon} + t_2
\end{aligned}$$

以第一扇区为例，按照以上逻辑排列可以得出大致波形如图8-6-1所示：

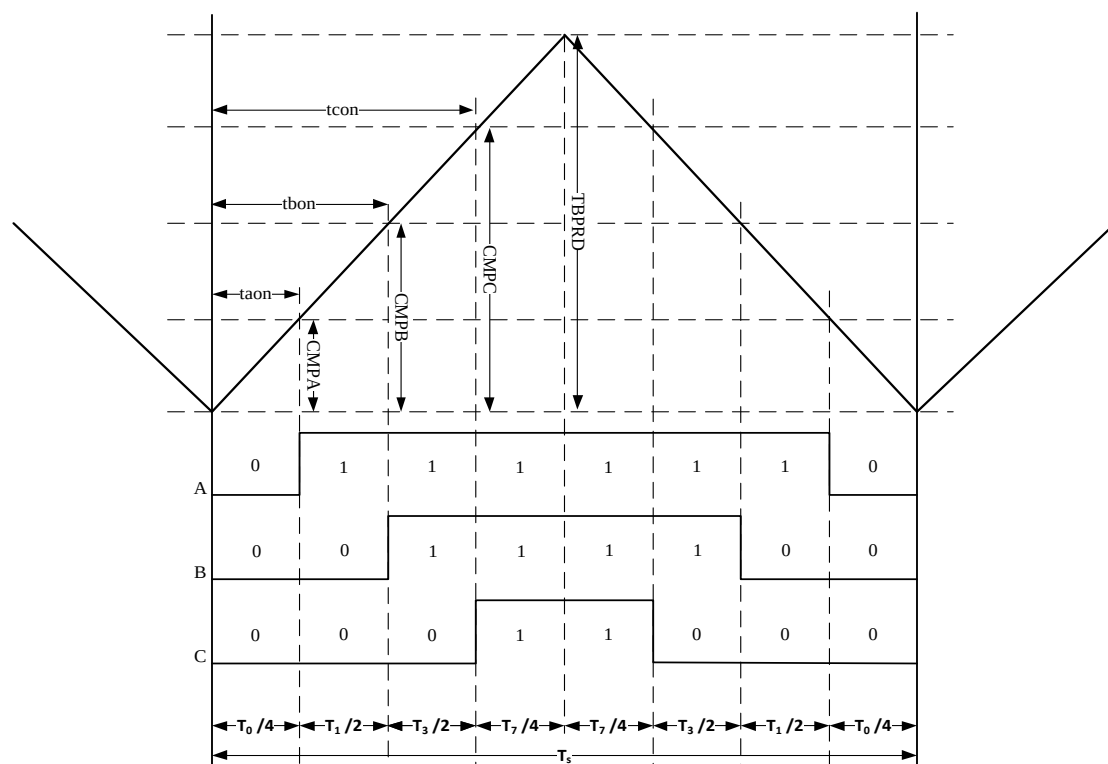


图8-6-1 第一扇区大致波形

以第三扇区为例，按照以上逻辑排列可以得出大致波形如下图8-6-2所示：

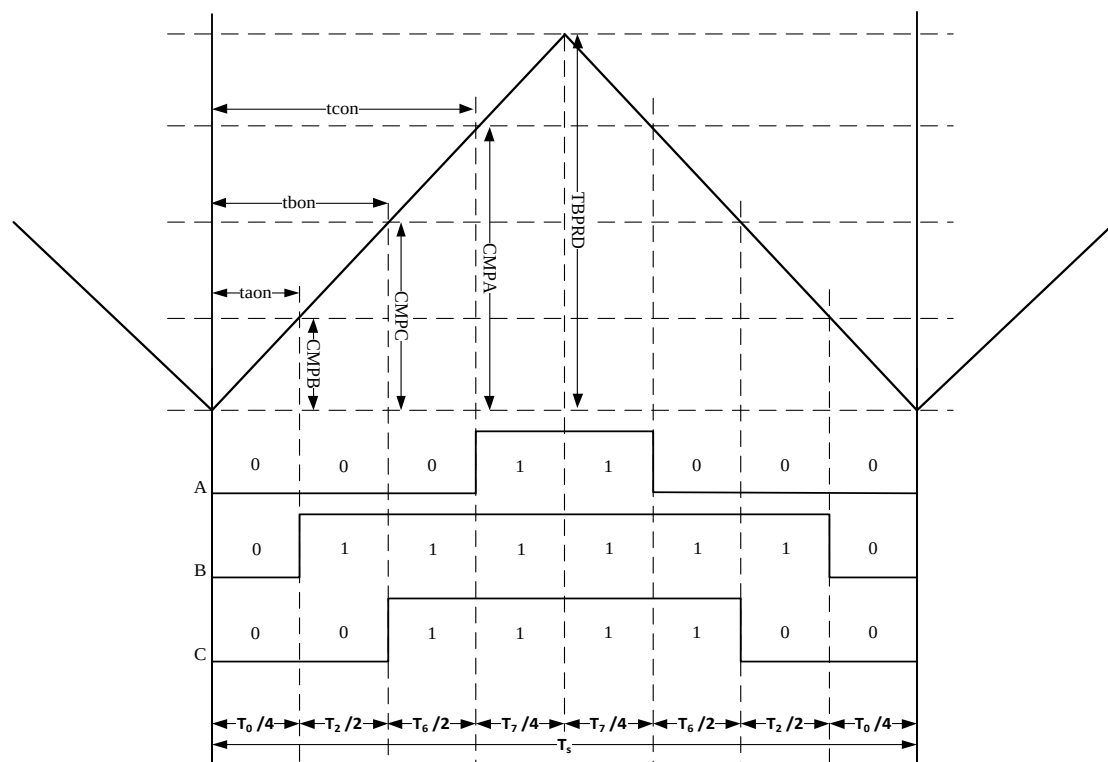


图8-6-2 第三扇区大致波形

分配基本矢量的作用顺序合成对应的对称PWM波形。根据上述规律可以得出扇区与电压空间矢量作用顺序的关系，那么不同扇区的PWM大致波形如图8-6-3所示。因为要加入作用时间，波形的占空比会有所变化，下面会继续分配和计算作用时间。

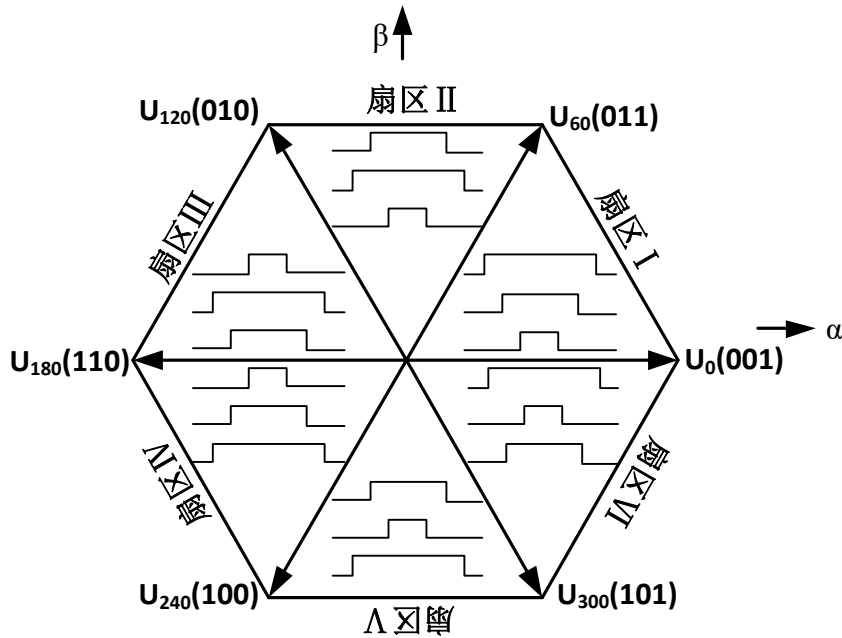


图8-6-3 各扇区大致波形

我们先定义：在前半个增计数的PWM周期内，控制电机A,B,C三相上桥臂导通的时刻对应的占空比分别为Ta, Tb, Tc。对比图3-8和图3-9的差别可以得出一定的规律，电机三相导通的每一路波形从左往右看并互相对比，导通最快的那一相的导通时刻即为 t_{aon} ，接着导通的那一相的导通时刻为 t_{bon} ，最后导通的那一相的导通时刻为 t_{con} 。再乘以最大计数值TBPRD就可以得到各相的比较值。按照这样的逻辑，可以得到每个扇区对应的三相导通时刻如下表8-6-1所示：

表8-6-1 每个扇区对应的三相导通时刻

扇区编号	大致波形	导通最快	接着导通	最后导通
S1		$Ta = t_{aon}$	$Tb = t_{bon}$	$Tc = t_{con}$

S2		$T_b = t_{aon}$	$T_a = t_{bon}$	$T_c = t_{con}$
S3		$T_b = t_{aon}$	$T_c = t_{bon}$	$T_a = t_{con}$
S4		$T_c = t_{aon}$	$T_b = t_{bon}$	$T_a = t_{con}$
S5		$T_c = t_{aon}$	$T_a = t_{bon}$	$T_b = t_{con}$
S6		$T_a = t_{aon}$	$T_c = t_{bon}$	$T_b = t_{con}$

到了这里， T_a 、 T_b 、 T_c 已经对PWM周期 T 归一化处理，那么这样的 T_a 、 T_b 、 T_c 就是占空比了，这里的 T_a 、 T_b 、 T_c 是一个小于 $_{IQ}(1)$ 的比例数值，这个比例是相对于 T 的。 T 是定时器的PWM主中断周期。若乘以PWM半周期就可以得出赋值于比较寄存器的实际值CMP1，CMP2和CMP3，亦即三相导通的时刻，整理可得下表8-6-2：

表8-6-2 各扇区三相导通时刻

Sector	U0,U60	U60,U120	U120,U180	U180,U240	U240,U300	U300,U0
Ta(CmrA)	t_{aon}	t_{bon}	t_{con}	t_{con}	t_{bon}	t_{aon}
Tb(CmrB)	t_{bon}	t_{aon}	t_{aon}	t_{bon}	t_{con}	t_{con}
Tc(CmrC)	t_{con}	t_{con}	t_{bon}	t_{aon}	t_{aon}	t_{bon}

这里理清各种表示的标号：

T_1^* （公式）= t_1 （官方文档）= T_x （程序）→先作用矢量时间占空比

T_2^* （公式）= t_2 （官方文档）= T_y （程序）→后作用矢量时间占空比

T_{PWM} 为主中断周期

t_{aon} 为最先导通时间占空比

t_{bon} 为中间导通时间占空比

t_{con} 为最后导通时间占空比

t_a 为A相导通时刻= CmrA（程序）= T_a （本文）

t_b 为B相导通时刻= CmrB（程序）= T_b （本文）

t_c 为C相导通时刻= CmrC（程序）= T_c （本文）

综上所述，实现以上功能的代码如下：

//LJF 根据矢量判断先后作用时间。 T_x 为先作用矢量的作用时间， T_y 为先作用矢量的作用时间，两者为对T标幺化的Q15格式！

//LJF 亦即占空比的Q15格式！

```
switch(SectorTemp)
{
    case 1:
    {
        Tx = -Time2;
```

```
        Ty = -Time3;
        break;
    }
    case 2:
    {
        Tx = -Time3;
        Ty = -Time1;
        break;
    }
    case 3:
    {
        Tx = Time2;
        Ty = Time1;
        break;
    }
    case 4:
    {
        Tx = -Time1;
        Ty = -Time2;
        break;
    }
    case 5:
    {
        Tx = Time1;
        Ty = Time3;
        break;
    }
    case 6:
    {
        Tx = Time3;
        Ty = Time2;
        break;
    }
    default:
        break;
}
Temp32 = Tx + Ty;
if (Temp32 > 32767)
{
    Tx = (Tx * 32767) / Temp32;
    Ty = (Ty * 32767) / Temp32;
}
OverModulate(Temp32);
switch(SectorTemp)
```

```
{
    case 1:
    {
        CmrB = (32767 - Tx - Ty) >> 1;
        CmrA = CmrB + Tx;
        CmrC = CmrA + Ty;
        break;
    }
    case 2:
    {
        CmrA = (32767 - Tx - Ty) >> 1;
        CmrC = CmrA + Tx;
        CmrB = CmrC + Ty;
        break;
    }
    case 3:
    {
        CmrA = (32767 - Tx - Ty) >> 1;
        CmrB = CmrA + Tx;
        CmrC = CmrB + Ty;
        break;
    }
    case 4:
    {
        CmrC = (32767 - Tx - Ty) >> 1;
        CmrB = CmrC + Tx;
        CmrA = CmrB + Ty;
        break;
    }
    case 5:
    {
        CmrB = (32767 - Tx - Ty) >> 1;
        CmrC = CmrB + Tx;
        CmrA = CmrC + Ty;
        break;
    }
    case 6:
    {
        CmrC = (32767 - Tx - Ty) >> 1;
        CmrA = CmrC + Tx;
        CmrB = CmrA + Ty;
        break;
    }
    default:
```

```

    {
        CmrA = 16384;
        CmrB = 16384;
        CmrC = 16384;
        break;
    }
}

//比较值Cmr越小，占空比Duty越大。
Temp32 = (int32)CmrA + MotorDriveVarStr.DeadComU;
CmrOutA = _IQsat(Temp32, 32767, 0);
Temp32 = (int32)CmrB + MotorDriveVarStr.DeadComV;
CmrOutB = _IQsat(Temp32, 32767, 0);
Temp32 = (int32)CmrC + MotorDriveVarStr.DeadComW;
CmrOutC = _IQsat(Temp32, 32767, 0);
MotorDriveOutStr.SvpwmCmrU = CmrOutA;
MotorDriveOutStr.SvpwmCmrV = CmrOutB;
MotorDriveOutStr.SvpwmCmrW = CmrOutC;

理论计算出的矢量占空比之后，经过上述的时间补偿法一起得出最终的占空比，再乘以比较寄存器最大值得到去标么化后的Q0格式值，最后将其赋值给比较寄存器。即计算出的占空比需要去标么化。函数如下：
void PwmDutySet(int16 Cmp1, int16 Cmp2, int16 Cmp3)
{
    EPwm1Regs.CMPA.half.CMPA = (int32)Cmp1 * EPwm1Regs.TBPRD / Q15_MAX;
    EPwm2Regs.CMPA.half.CMPA = (int32)Cmp2 * EPwm2Regs.TBPRD / Q15_MAX;
    EPwm3Regs.CMPA.half.CMPA = (int32)Cmp3 * EPwm3Regs.TBPRD / Q15_MAX;
}

```

1. TI官方例程占空比解析

鉴于目前见到的七段 SVPWM 程序都使用倒三角形的比较模式，具体分析芯片 PWM 模块的配置，特此查找一段代码用于解析和说明。

为了方便对照，以及代码的解读，请先从 CCS 里导入官方套件 controlSUITE(v3.4.9-2018.03.28)的工程，然后编译一下用于 C 程序和头文件的关联。工程路径如下：

C:\ti\controlSUITE\development_kits\HVMotorCtrl+PfcKit_v2.1\HVPM_Sensored_Servo

请根据需要选择章节阅读，比如1.4和1.6节

该函数根据 U_d ， U_q 实现七段 SVPWM 调制。电压方程算出或者电流 PI 调节输

出的 U_d 和 U_q 经过克拉克变换得到的 U_α 和 U_β 就是该函数的输入，而输出则为电机三相绕组相应的导通时刻计数值。

1.1 参考矢量分解及作用时间分配

为了方便统一符号和分析，现有定义：

1. 开关顺序为C, B, A。状态开为1，关为0。对应的作用时间标号为其构成的二进制转为十进制的数值。如 T_3 的开关状态为C-0, B-1, A-1，可以推出是电压空间矢量 U_{60} 的作用时间。

2. 各个电压空间矢量大小对相电压 $\frac{U_{dc}}{\sqrt{3}}$ 进行标么化为 $\frac{\frac{2}{3}U_{dc}}{\frac{U_{dc}}{\sqrt{3}}} = \frac{2}{\sqrt{3}}$ 。电压矢量

大小对相电压 $\frac{U_{DC}}{\sqrt{3}}$ 标么化的目的是为了化简矢量作用时间的计算，即与60度sin和cos三角函数值对消约去。这里的计算技巧应该要体会得到。

3. 静止两相坐标系下的两个电压大小对相电压 $\frac{U_{dc}}{\sqrt{3}}$ 进行标么化为 U_α^* 和 U_β^*

4. 矢量方向：规定静止两相坐标系下的代数值符号：沿各坐标轴的正值增加为正，负值减小的方向为负。

5. 标么化后的物理量均附带上标“*”。如 U_α^* 为 α 轴电压对相电压 $\frac{U_{dc}}{\sqrt{3}}$ 的标么值。

由于非零矢量的基本电压矢量只有6个，因此只能形成一个正六边形的磁链轨迹。为获得近似圆形的磁链轨迹，一种方法是利用这几个非零的基本电压空间矢量的线性时间组合来得到更多的开关状态。如扇区1图所示，任意时刻输出参考相电压矢量 U_{ref} 可由相邻的两个基本电压空间矢量 U_0 和 U_{60} 的线性时间组合来合成，它等于 T_1/T 倍的 U_0 与 T_3/T 倍的 U_{60} 的矢量和。其中 T_1 和 T_3 分别是 U_0 和 U_{60} 作用时间， T 是 U_{ref} 作用的时间。另外为了减少开关切换次数，插入零矢量进行调整。

$$\begin{cases} O_0 t_0 + U_1 t_1 + U_2 t_2 = U_{ref} T \\ t_0 + t_1 + t_2 = T \end{cases}$$

上式中 U_1, t_1 是先作用的电压矢量和时间， U_2, t_2 是后作用的电压矢量和时间，

O_0, t_0 是指两个不同的零矢量和作用时间。后续会继续分析。

扇区1分析

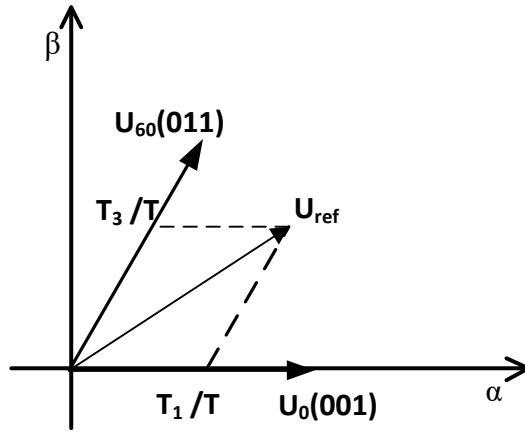


图8-2-1 参考矢量在扇区1的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 1} = \frac{T_1}{T} U_0 + \frac{T_3}{T} U_{60} \cos 60^\circ$$

$$U_{\beta 1} = \frac{T_3}{T} U_{60} \sin 60^\circ$$

计算出时间如下

$$T_3 = \frac{T U_{\beta 1}}{U_{60} \sin 60^\circ} = \frac{T \frac{U_{\beta 1}}{U_{DC} / \sqrt{3}}}{\frac{U_{60}}{U_{DC} / \sqrt{3}} \frac{\sqrt{3}}{2}} = \frac{T \frac{U_{\beta 1}}{U_{DC} / \sqrt{3}}}{\frac{2 U_{DC} / 3 \sqrt{3}}{U_{DC} / \sqrt{3}} \frac{\sqrt{3}}{2}} = \frac{T U_{\beta 1}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = T U_{\beta 1}^*$$

$$\begin{aligned}
T_1 &= \frac{T(U_{\alpha 1} - \frac{T_3}{T} U_{60} \cos 60^\circ)}{U_0} \\
&= \frac{T(U_{\alpha 1} \frac{\sqrt{3}}{U_{DC}} - \frac{T_3}{T} \frac{\sqrt{3}}{U_{DC}} U_{60} \cos 60^\circ)}{U_0 \frac{\sqrt{3}}{U_{DC}}} \\
&= \frac{T(U_{\alpha 1}^* - U_{\beta 1}^* \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\
&= \frac{T\sqrt{3}U_{\alpha 1}^* - TU_{\beta 1}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\
&= \frac{T(\sqrt{3}U_{\alpha 1}^* - U_{\beta 1}^*)}{2}
\end{aligned}$$

对应的占空比为

$$T_3^* = \frac{T_3}{T} = U_{\beta 1}^*$$

$$T_1^* = \frac{T_1}{T} = \frac{-(-\sqrt{3}U_{\alpha 1}^* + U_{\beta 1}^*)}{2}$$

扇区2分析

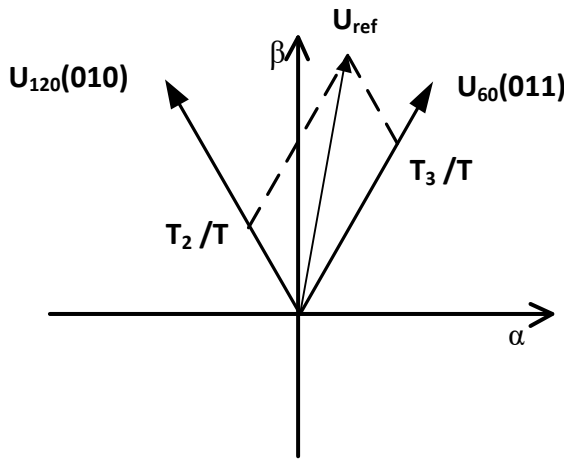


图8-2-2 参考矢量在扇区2的分解

根据矢量作用时间和几何关系可得

$$\text{式1: } U_{\alpha 2} = \frac{T_3}{T} U_{60} \cos 60^\circ - \frac{T_2}{T} U_{120} \cos 60^\circ$$

$$\text{式2: } U_{\beta 2} = \frac{T_3}{T} U_{60} \sin 60^\circ + \frac{T_2}{T} U_{120} \sin 60^\circ$$

计算出时间如下

由式1得：

$$\begin{aligned}
 T_3 &= \frac{T(U_{\alpha 2} + \frac{T_2}{T} U_{120} \cos 60^\circ)}{U_{60} \cos 60^\circ} \\
 T_3 &= \frac{T(U_{\alpha 2} \frac{\sqrt{3}}{U_{DC}} + \frac{T_2}{T} \frac{\sqrt{3}}{U_{DC}} U_{120} \cos 60^\circ)}{\frac{2}{\sqrt{3}} \frac{1}{2}} \\
 &= \frac{T(U_{\alpha 2}^* + \frac{T_2}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}} \frac{1}{2}} \\
 &= T\sqrt{3}U_{\alpha 2}^* + T_2\sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2} \\
 &= T\sqrt{3}U_{\alpha 2}^* + T_2
 \end{aligned}$$

由式2得：

$$\begin{aligned}
 T_2 &= \frac{T(U_{\beta 2} - \frac{T_3}{T} U_{60} \sin 60^\circ)}{U_{120} \sin 60^\circ} \\
 &= \frac{T(U_{\beta 2} \frac{\sqrt{3}}{U_{DC}} - \frac{T_3}{T} \frac{\sqrt{3}}{U_{DC}} U_{60} \sin 60^\circ)}{U_{120} \frac{\sqrt{3}}{U_{DC}} \sin 60^\circ} \\
 &= \frac{T(U_{\beta 2}^* - \frac{T_3}{T} \frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2})}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} \\
 &= TU_{\beta 2}^* - T_3
 \end{aligned}$$

再由上式解得：

$$\begin{aligned}
 T_3 &= T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^* - T_3 \\
 &= \frac{T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^*}{2} \\
 T_2 &= TU_{\beta 2}^* - \frac{T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^*}{2} \\
 &= \frac{-T\sqrt{3}U_{\alpha 2}^* + TU_{\beta 2}^*}{2}
 \end{aligned}$$

对应的占空比为

$$T_3^* = \frac{T_3}{T} = \frac{\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$$

$$T_2^* = \frac{T_2}{T} = \frac{-\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$$

扇区3分析

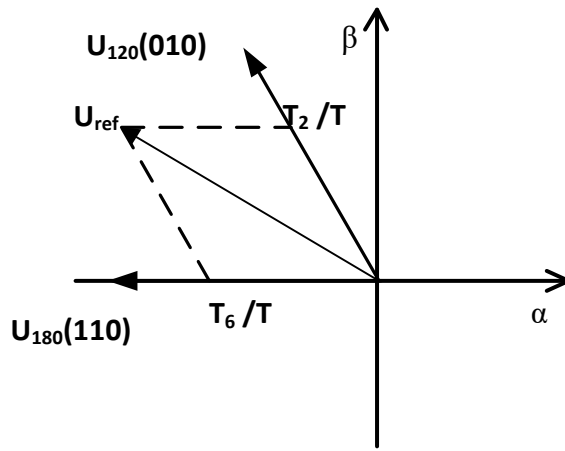


图8-2-3 参考矢量在扇区3的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 3} = -\frac{T_6}{T}U_{180} - \frac{T_2}{T}U_{120} \cos 60^\circ$$

$$U_{\beta 3} = \frac{T_2}{T}U_{120} \sin 60^\circ$$

计算出时间如下

$$T_2 = \frac{TU_{\beta 3}}{U_{120} \sin 60^\circ} = \frac{TU_{\beta 1}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = TU_{\beta 1}^*$$

$$\begin{aligned}
T_6 &= \frac{T(-U_{\alpha 3} - \frac{T_2}{T} U_{120} \cos 60^\circ)}{U_{180}} \\
&= \frac{T(-U_{\alpha 3} \frac{\sqrt{3}}{U_{DC}} - \frac{T_2}{T} U_{120} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_{180} \frac{\sqrt{3}}{U_{DC}}} \\
&= \frac{T(-U_{\alpha 3}^* - \frac{T_2}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\
&= \frac{-T\sqrt{3}U_{\alpha 3}^* - TU_{\beta 3}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\
&= \frac{T(-\sqrt{3}U_{\alpha 3}^* - U_{\beta 3}^*)}{2}
\end{aligned}$$

对应的占空比为

$$T_2^* = \frac{T_3}{T} = U_{\beta 3}^*$$

$$T_6^* = \frac{T_6}{T} = \frac{-(\sqrt{3}U_{\alpha 3}^* + U_{\beta 3}^*)}{2}$$

扇区4分析

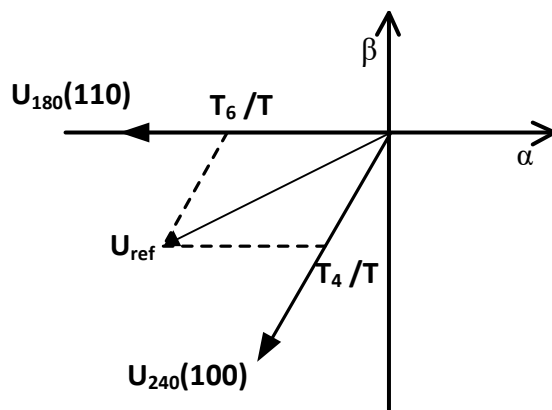


图8-2-4 参考矢量在扇区4的分解
根据矢量作用时间和几何关系可得

$$U_{\alpha 4} = -\frac{T_6}{T}U_{180} - \frac{T_4}{T}U_{240} \cos 60^\circ$$

$$U_{\beta 4} = -\frac{T_4}{T}U_{240} \sin 60^\circ$$

计算出时间如下

$$T_4 = -\frac{TU_{\beta 4}}{U_{240} \sin 60^\circ} = -\frac{TU_{\beta 4}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = -TU_{\beta 4}^*$$

$$\begin{aligned} T_6 &= \frac{T(-U_{\alpha 4} - \frac{T_4}{T}U_{240} \cos 60^\circ)}{U_{180}} \\ &= \frac{T(-U_{\alpha 4} \frac{\sqrt{3}}{U_{DC}} - \frac{T_4}{T}U_{240} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_{180} \frac{\sqrt{3}}{U_{DC}}} \\ &= \frac{T(-U_{\alpha 4}^* - \frac{T_4}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\ &= \frac{-T\sqrt{3}U_{\alpha 4}^* + TU_{\beta 4}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\ &= \frac{T(-\sqrt{3}U_{\alpha 4}^* + U_{\beta 4}^*)}{2} \end{aligned}$$

对应的占空比为

$$T_4^* = \frac{T_4}{T} = -U_{\beta 4}^*$$

$$T_6^* = \frac{T_6}{T} = \frac{-\sqrt{3}U_{\alpha 4}^* + U_{\beta 4}^*}{2}$$

扇区5分析

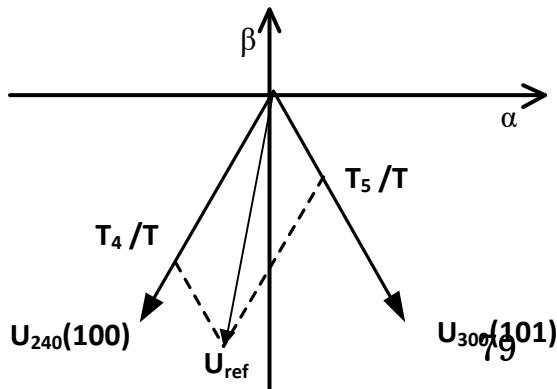


图8-2-5 参考矢量在扇区5的分解

根据矢量作用时间和几何关系可得

$$\text{式1: } U_{\alpha 5} = \frac{T_5}{T} U_{300} \cos 60^\circ - \frac{T_4}{T} U_{240} \cos 60^\circ$$

$$\text{式2: } U_{\beta 5} = -\frac{T_5}{T} U_{300} \sin 60^\circ - \frac{T_4}{T} U_{240} \sin 60^\circ$$

计算出时间如下

由式1得:

$$\begin{aligned} T_5 &= \frac{T(U_{\alpha 5} + \frac{T_4}{T} U_{240} \cos 60^\circ)}{U_{300} \cos 60^\circ} \\ &= \frac{T(U_{\alpha 5} \frac{\sqrt{3}}{U_{DC}} + \frac{T_4}{T} U_{240} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_{300} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ} \\ &= \frac{T(U_{\alpha 5}^* + \frac{T_4}{T} \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}} \frac{1}{2}} \\ &= T\sqrt{3}U_{\alpha 5}^* + T_4\sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2} \\ &= T\sqrt{3}U_{\alpha 5}^* + T_4 \end{aligned}$$

由式2得:

$$\begin{aligned} T_4 &= \frac{T(-U_{\beta 5} - \frac{T_5}{T} U_{300} \sin 60^\circ)}{U_{240} \sin 60^\circ} \\ &= \frac{T(-U_{\beta 5} \frac{\sqrt{3}}{U_{DC}} - \frac{T_5}{T} U_{300} \frac{\sqrt{3}}{U_{DC}} \sin 60^\circ)}{U_{240} \frac{\sqrt{3}}{U_{DC}} \sin 60^\circ} \\ &= \frac{T(-U_{\beta 5}^* - \frac{T_5}{T} \frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2})}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} \\ &= -TU_{\beta 5}^* - T_5 \end{aligned}$$

再由上式解得:

$$\begin{aligned} T_5 &= T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^* - T_5 \\ &= \frac{T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^*}{2} \end{aligned}$$

$$T_4 = -TU_{\beta 5}^* - \frac{T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^*}{2}$$

$$= \frac{-T\sqrt{3}U_{\alpha 5}^* - TU_{\beta 5}^*}{2}$$

对应的占空比为

$$T_5^* = \frac{T_5}{T} = \frac{-(-\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$$

$$T_4^* = \frac{T_4}{T} = \frac{-(\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$$

扇区6分析

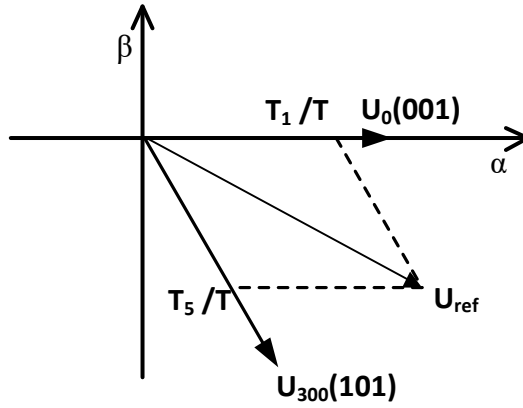


图8-2-6 参考矢量在扇区6的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 6} = \frac{T_1}{T}U_0 + \frac{T_5}{T}U_{300} \cos 60^\circ$$

$$U_{\beta 6} = -\frac{T_5}{T}U_{300} \sin 60^\circ$$

计算出时间如下

$$T_5 = -\frac{TU_{\beta 6}}{U_{300} \sin 60^\circ} = \frac{TU_{\beta 6}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = -TU_{\beta 6}^*$$

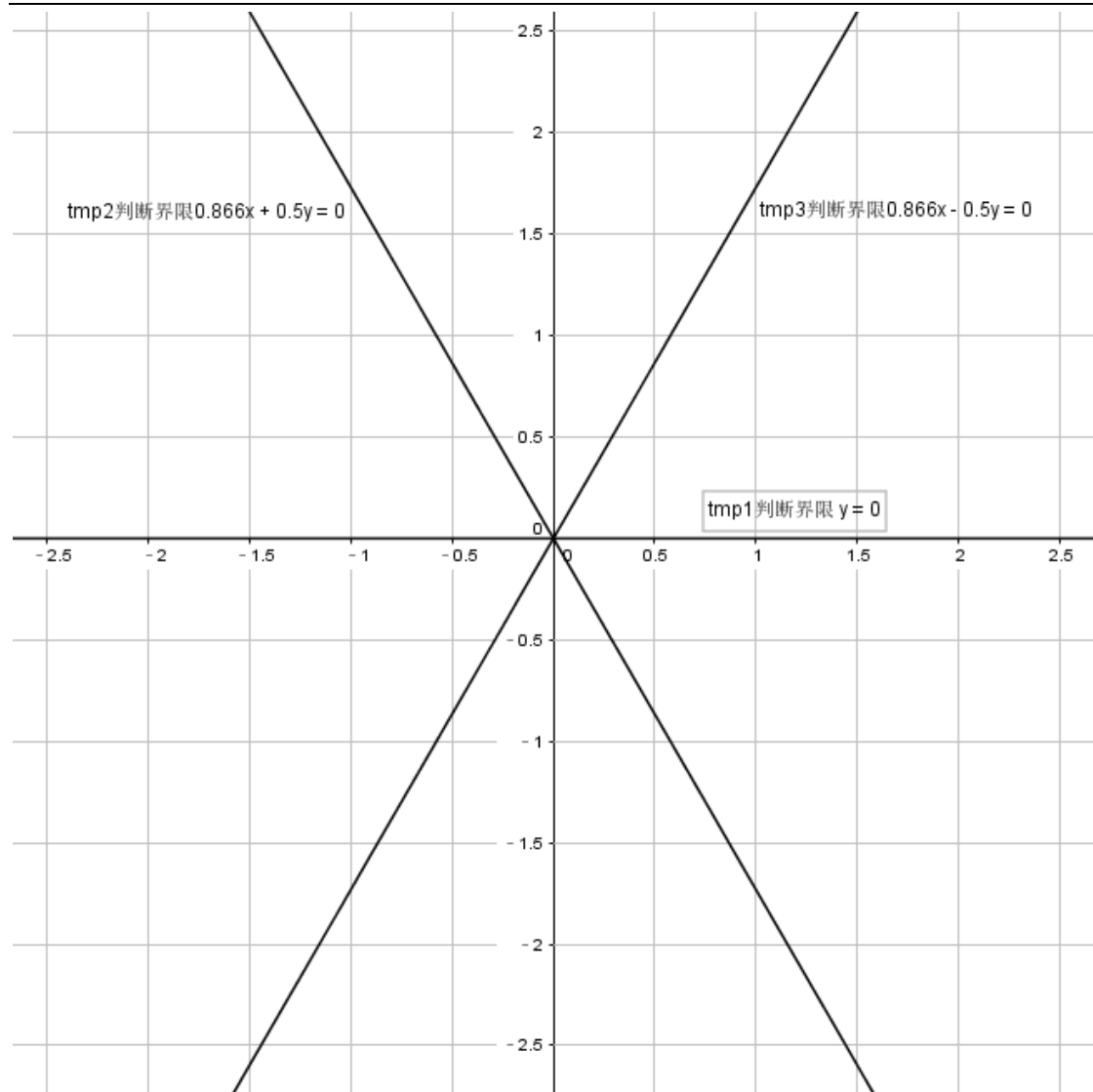
$$\begin{aligned}
T_1 &= \frac{T(U_{\alpha 6} - \frac{T_5}{T} U_{300} \cos 60^\circ)}{U_0} \\
&= \frac{T(U_{\alpha 6} \frac{\sqrt{3}}{U_{DC}} - \frac{T_5}{T} U_{300} \frac{\sqrt{3}}{U_{DC}} \cos 60^\circ)}{U_0 \frac{\sqrt{3}}{U_{DC}}} \\
&= \frac{T(U_{\alpha 6}^* + U_{\beta 6}^* \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\
&= \frac{T\sqrt{3}U_{\alpha 6}^* + TU_{\beta 6}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\
&= \frac{T(\sqrt{3}U_{\alpha 6}^* + U_{\beta 6}^*)}{2}
\end{aligned}$$

对应的占空比为

$$\begin{aligned}
T_5^* &= \frac{T_5}{T} = -U_{\beta 6}^* \\
T_1^* &= \frac{T_1}{T} = \frac{\sqrt{3}U_{\alpha 6}^* + U_{\beta 6}^*}{2}
\end{aligned}$$

1.2 矢量所在扇区判断

其实就是线性规划的运用，根据三条直线大于零和小于零的组合确定扇区，



程序判断的代码如下：

```

v.tmp1= v.Ubeta;
v.tmp2= _IQdiv2(v.Ubeta) + (_IQmpy(_IQ(0.866),v.Ualpha));
v.tmp3= v.tmp2 - v.tmp1;

```

等式表示为：

$$tmp1=U_{\beta}^*$$

$$tmp2=\frac{U_{\beta}^* + \sqrt{3}U_{\alpha}^*}{2}$$

$$tmp3=\frac{-U_{\beta}^* + \sqrt{3}U_{\alpha}^*}{2}$$

如果单纯判断大于零小于零的时候，其实是以下三条直线：

$$y=0$$

$$y + \sqrt{3}x = 0$$

$$y - \sqrt{3}x = 0$$

这里坐标轴的纵坐标 y 为 U_{β} ，横坐标 x 为 U_{α}

进行不等式区域判断可以辨别电压落在哪个扇区。横纵坐标对应静止两相正交坐标。

整理如下表

扇区\判断条件	tmp2	Tmp3	Tmp1
1	>0	>0	>0
2	>0	<0	>0
3	<0	<0	>0
4	<0	<0	<0
5	<0	>0	<0
6	>0	>0	<0
三个条件无交集	>0	<0	<0
三个条件无交集	<0	>0	>0

遍历所有情况：

第 I 步 v.VecSector=(v.tmp2>0)?(v.VecSector-1):v.VecSector;							
v.tmp2 > 0 v.VecSector = 3 - 1 = 2				v.tmp2 < 0 v.VecSector = 3			
第 II 步 v.VecSector=(v.tmp3>0)?(v.VecSector-1):v.VecSector;							
v.tmp3 > 0 v.VecSector = 2 - 1 = 1		v.tmp3 < 0 v.VecSector = 2		v.tmp3 > 0 v.VecSector = 3 - 1 = 2		v.tmp3 < 0 v.VecSector = 3	
第 III 步 v.VecSector=(v.tmp1<0)?(7-v.VecSector) :v.VecSector;							
v.tmp1 < 0 v.VecSector = 7 - 1 = 6	v.tmp1 > 0 v.VecSector r = 1	v.tmp1 < 0 v.VecSector r = 7 - 2 = 5 三个条件无交集， 不属于任何扇区	v.tmp1 > 0 0 v.VecSector or = 2	v.tmp1 < 0 v.VecSector r = 7 - 2 = 5	v.tmp1 > 0 v.VecSector r = 2 三个条件无交集， 不属于任何扇区	v.tmp1 < 0 0 v.VecSector or = 7 - 3 = 4	v.tmp1 > 0 0 v.VecSector or = 3

三个条件大于和小于分成了八种情况，其中两种没有交集，不属于任何扇区。这里的VecSector数值就是扇区编号，并非3,1,5,4,6,2对应扇区1,2,3,4,5,6。

1.3 矢量切换时刻及对应占空比

假定在DSP芯片上配置好的PWM模块为增减计数。在增计数时计数值等于比较值时候，对应控制上桥信号置高，在减计数时计数值等于比较值时候，对应控制上桥信号置低，用七段波形调制。由于波形的对称性，各个矢量作用时间分成2份，分开后用于增计数作用和减计数作用。为了满足开关切换最少，以及方便后续的死区补偿添加了两个零矢量，又因为作用时间不变，所以该模式下只能把零矢量作用时间再分成2份。方便区别和分析，这里规定小写 t 为占空比，大写的 T 为实际作用时间。

按照上述的逻辑，可以得出下表8-6-1：

表8-6-1 矢量作用时刻表

增减计数状态	作用的电压空间矢量	持续时间	开始时刻	结束时刻
增计数	零矢量 $O_{000} (000)$	$\frac{T_0}{4}$	0	$\frac{T_0}{4}$
	开关状态只含有一个“1”的矢量	$\frac{T_1}{2}$	$\frac{T_0}{4}$	$\frac{T_0}{4} + \frac{T_1}{2}$
	开关状态含有两个“1”的矢量	$\frac{T_2}{2}$	$\frac{T_0}{4} + \frac{T_1}{2}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2}$
	零矢量 $O_{111} (111)$	$\frac{T_0}{4}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} + \frac{T_0}{4} = \frac{T_{PWM}}{2}$
减计数	零矢量 $O_{111} (111)$	$\frac{T_0}{4}$	$\frac{T_{PWM}}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4}$
	开关状态含有两个“1”的矢量	$\frac{T_2}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2}$
	开关状态只含有一个“1”的矢量	$\frac{T_1}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2} + \frac{T_1}{2}$
	零矢量 $O_{000} (000)$	$\frac{T_0}{4}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2} + \frac{T_1}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} + \frac{T_0}{4} = T_{PWM}$

按照上述方法可以大致排列好基本矢量的作用顺序，整理可以得出表8-3-1如下：

表8-3-1 扇区与电压空间矢量作用顺序的关系

扇区编号	先作用矢量	后作用矢量	采用对称七段调制的矢量作用顺序
S1	U_0	U_{60}	$O_0 \rightarrow U_0 \rightarrow U_{60} \rightarrow O_{111} \rightarrow U_{60} \rightarrow U_0 \rightarrow O_0$
S2	U_{120}	U_{60}	$O_0 \rightarrow U_{120} \rightarrow U_{60} \rightarrow O_{111} \rightarrow U_{60} \rightarrow U_{120} \rightarrow O_0$
S3	U_{120}	U_{180}	$O_0 \rightarrow U_{120} \rightarrow U_{180} \rightarrow O_{111} \rightarrow U_{180} \rightarrow U_{120} \rightarrow O_0$
S4	U_{240}	U_{180}	$O_0 \rightarrow U_{240} \rightarrow U_{180} \rightarrow O_{111} \rightarrow U_{180} \rightarrow U_{240} \rightarrow O_0$
S5	U_{240}	U_{300}	$O_0 \rightarrow U_{240} \rightarrow U_{300} \rightarrow O_{111} \rightarrow U_{300} \rightarrow U_{240} \rightarrow O_0$
S6	U_0	U_{300}	$O_0 \rightarrow U_0 \rightarrow U_{300} \rightarrow O_{111} \rightarrow U_{300} \rightarrow U_0 \rightarrow O_0$

按照上表8-6-1，三个切换时刻及对应的占空比的分析计算如下：

【2】先作用的零矢量作用完毕，先作用的非零矢量作用开始的时刻及占空比：

$$\begin{aligned}\frac{T_0}{4} &= \frac{T_{PWM} - T_1 - T_2}{4} \\ &= \frac{T_{PWM}}{2} * \frac{1 - T_1^* - T_2^*}{2} \\ &= TBPRD * \frac{1 - t_1 - t_2}{2} = TBPRD * t_{aon}\end{aligned}$$

根据上述定义 $t_{aon} = \frac{1 - t_1 - t_2}{2}$

【2】先作用的非零矢量作用完毕，后作用的非零矢量作用开始的时刻及占空比：

$$\begin{aligned}\frac{T_0}{4} + \frac{T_1}{2} &= \frac{T_{PWM} - T_1 - T_2}{4} + \frac{T_1}{2} \\ &= \frac{T_{PWM}}{2} * \left(\frac{1 - T_1^* - T_2^*}{2} + T_1^* \right) \\ &= TBPRD * \left(\frac{1 + t_1 - t_2}{2} \right) = TBPRD * t_{bon}\end{aligned}$$

根据上述定义 $t_{bon} = t_{aon} + t_1 = \frac{1 + t_1 - t_2}{2}$

【3】后作用的非零矢量作用完毕，后作用的零矢量作用开始的时刻及占空比：

$$\begin{aligned}\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} &= \frac{T_{PWM} - T_1 - T_2}{4} + \frac{T_1}{2} + \frac{T_2}{2} \\ &= \frac{T_{PWM}}{2} * \left(\frac{1 - T_1^* - T_2^*}{2} + T_1^* + T_2^* \right) \\ &= TBPRD * \left(\frac{1 - t_1 - t_2}{2} + t_1 + t_2 \right) = TBPRD * t_{con}\end{aligned}$$

根据上述定义 $t_{con} = t_{bon} + t_2 = \frac{1 + t_1 + t_2}{2}$

明显可以得到 $t_{aon} < t_{bon} < t_{con}$ 这样的关系。这里的abc字母不代表电机三相序号，而是代表导通的优先顺序，**a**代表最先导通，**b**代表接着导通，**c**代表最后导通。以上求出了切换时刻对应的占空比，再次整理如下：

$$t_{aon} = \frac{1-t_1-t_2}{2}$$

$$t_{bon} = t_{aon} + t_1 = \frac{1+t_1-t_2}{2}$$

$$t_{con} = t_{Bon} + t_2 = \frac{1+t_1+t_2}{2}$$

以第一扇区为例，按照以上逻辑排列可以得出大致波形如图8-6-1所示：

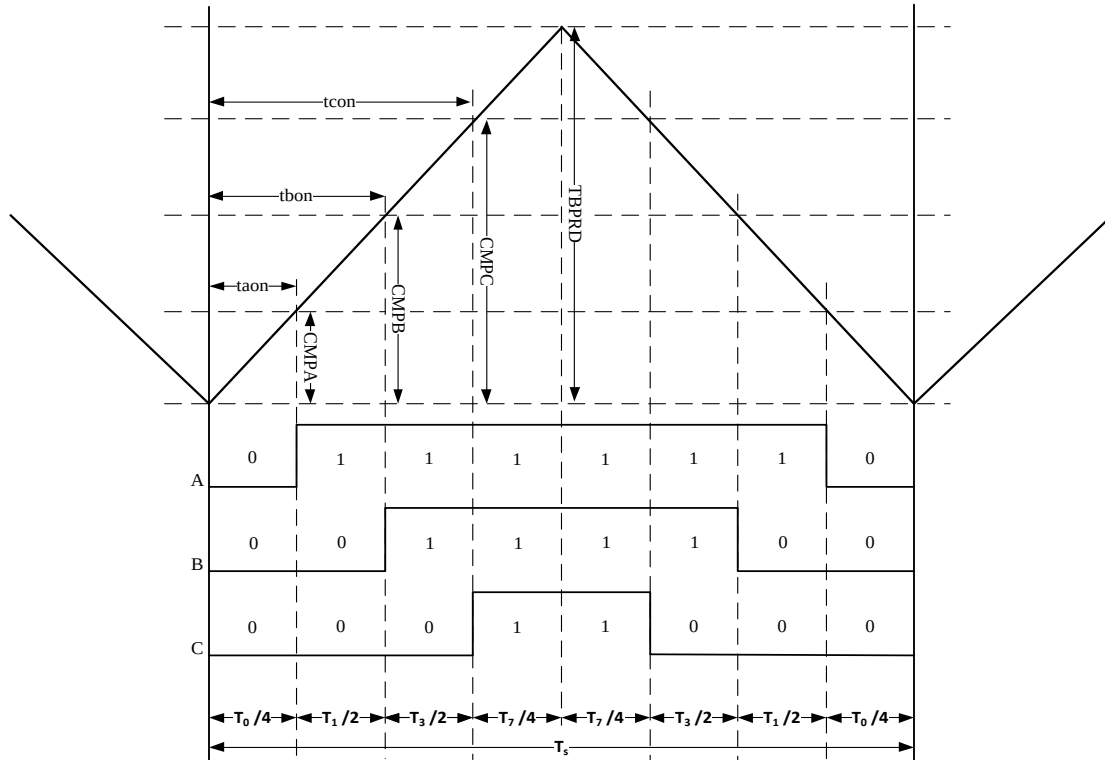


图8-6-1 第一扇区大致波形

以第三扇区为例，按照以上逻辑排列可以得出大致波形如下图8-6-2所示：

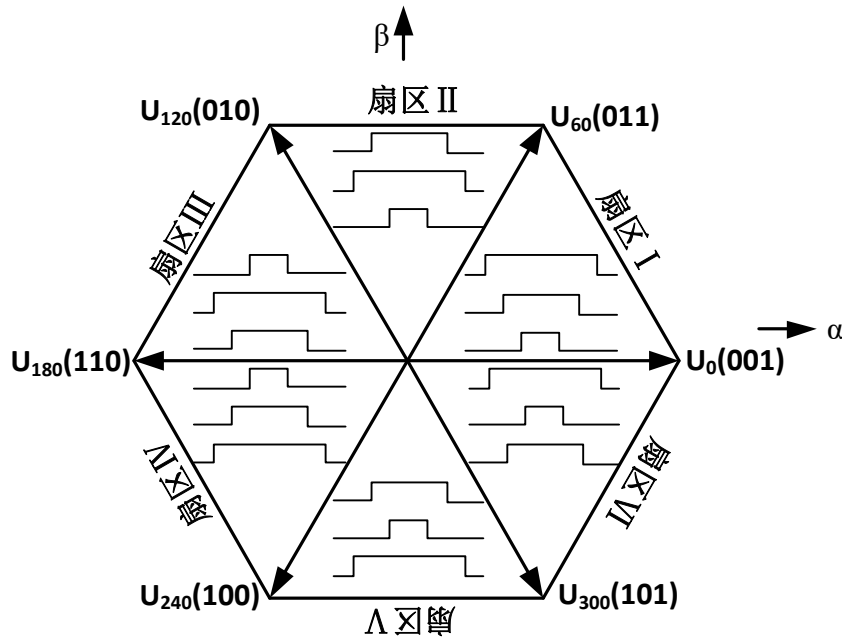


图8-6-3 各扇区大致波形

1.4 占空比解析

根据以上章节可以整理如下

根据1.1节，整理出各个扇区两个非零矢量作用时间的比例，如下：

扇区1占空比

$$T_3^* = \frac{T_3}{T} = U_{\beta 1}^*$$

$$T_1^* = \frac{T_1}{T} = \frac{-(-\sqrt{3}U_{\alpha 1}^* + U_{\beta 1}^*)}{2}$$

扇区2占空比

$$T_3^* = \frac{T_3}{T} = \frac{\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$$

$$T_2^* = \frac{T_2}{T} = \frac{-\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$$

扇区3占空比

$$T_2^* = \frac{T_2}{T} = U_{\beta 3}^*$$

$$T_6^* = \frac{T_6}{T} = \frac{-(\sqrt{3}U_{\alpha 3}^* + U_{\beta 3}^*)}{2}$$

扇区4占空比

$$T_4^* = \frac{T_4}{T} = -U_{\beta 4}^*$$

$$T_6^* = \frac{T_6}{T} = \frac{-\sqrt{3}U_{\alpha 4}^* + U_{\beta 4}^*}{2}$$

扇区5占空比

$$T_5^* = \frac{T_5}{T} = \frac{-(-\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$$

$$T_4^* = \frac{T_4}{T} = \frac{-(\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$$

扇区6占空比

$$T_5^* = \frac{T_5}{T} = -U_{\beta 6}^*$$

$$T_1^* = \frac{T_1}{T} = \frac{\sqrt{3}U_{\alpha 6}^* + U_{\beta 6}^*}{2}$$

按照七段矢量调制开关切换最少原则，矢量先后作用决定了电机三相的导通时刻排列顺序如下：

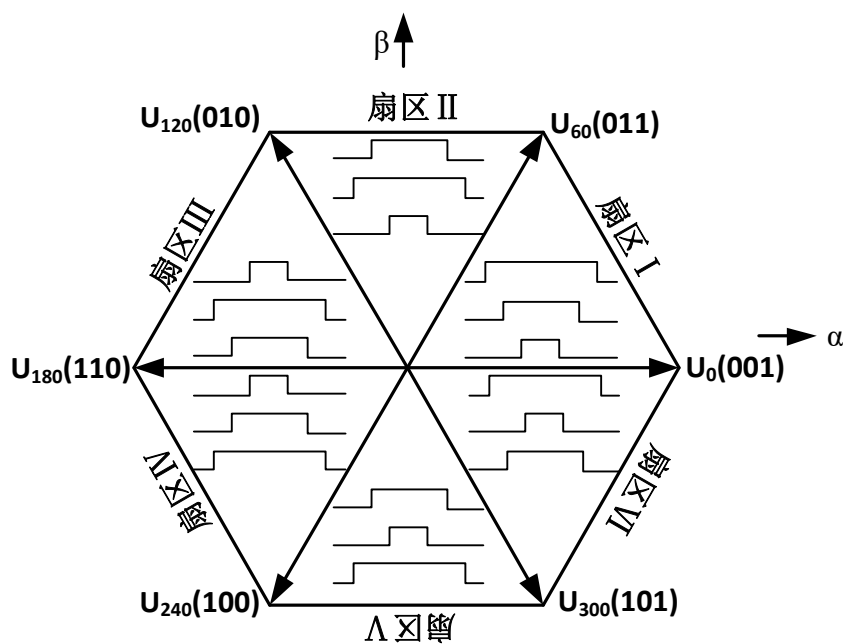


图8-6-3 各扇区大致波形

有如下表格

表X-X-1 先后顺序及占空比

扇区	1=U ₀ ,U ₆₀	2=U ₆₀ ,U ₁₂₀	3=U ₁₂₀ ,U ₁₈₀	4=U ₁₈₀ ,U ₂₄₀	5=U ₂₄₀ ,U ₃₀₀	6=U ₃₀₀ ,U ₀
先作用矢量 占空 t_1	$(U_0 : T_1^*)$ $\frac{-(-\sqrt{3}U_{\alpha 1}^* + U_{\beta 1}^*)}{2}$	$(U_{120} : T_2^*)$ $\frac{-\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$	$(U_{120} : T_2^*)$ $U_{\beta 3}^*$	$(U_{240} : T_4^*)$ $-U_{\beta 4}^*$	$(U_{240} : T_4^*)$ $\frac{-(\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$	$(U_0 : T_1^*)$ $\frac{\sqrt{3}U_{\alpha 6}^* + U_{\beta 6}^*}{2}$
后作用矢量 占空 t_2	$(U_{60} : T_3^*)$ $U_{\beta 1}^*$	$(U_{60} : T_3^*)$ $\frac{\sqrt{3}U_{\alpha 2}^* + U_{\beta 2}^*}{2}$	$(U_{180} : T_6^*)$ $\frac{-(\sqrt{3}U_{\alpha 3}^* + U_{\beta 3}^*)}{2}$	$(U_{180} : T_6^*)$ $\frac{-\sqrt{3}U_{\alpha 4}^* + U_{\beta 4}^*}{2}$	$(U_{300} : T_5^*)$ $\frac{-(-\sqrt{3}U_{\alpha 5}^* + U_{\beta 5}^*)}{2}$	$(U_{300} : T_5^*)$ $-U_{\beta 6}^*$

正三角增减计数模式，向上计数相等时置位，向下计数相等时置零。

表X-X-2 切换时刻的占空比

扇区	最先导通占空比 $t_{aon} = \frac{1-t_1-t_2}{2}$	其次导通占空比 $t_{bon} = \frac{1+t_1-t_2}{2}$	最后导通占空比 $t_{con} = \frac{1+t_1+t_2}{2}$
1=U0, U60	$1 - \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}$	$1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2} - U_\beta^*$	$1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}$
2=U60, U120	$\frac{1-U_\beta^*}{2}$	$\frac{1-\sqrt{3}U_\alpha^*}{2}$	$\frac{1+U_\beta^*}{2}$
3=U120, U180	$1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2}$	$1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2} + U_\beta^*$	$1 + \frac{U_\beta^* - \sqrt{3}U_\alpha^*}{2}$
4=U180, U240	$1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}$	$1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2} - U_\beta^*$	$1 - \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}$
5=U240, U300	$\frac{1+U_\beta^*}{2}$	$\frac{1-\sqrt{3}U_\alpha^*}{2}$	$\frac{1-U_\beta^*}{2}$
6=U300, U0	$1 + \frac{U_\beta^* - \sqrt{3}U_\alpha^*}{2}$	$1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2} + U_\beta^*$	$1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2}$

按照七段矢量调制开关切换最少原则，矢量先后作用决定了电机三相的导通时刻排列顺序如下：

表8-6-2 各扇区三相导通时刻

Sector	U0, U60	U60, U120	U120, U180	U180, U240	U240, U300	U300, U0
Ta(CmrA)	t_{aon}	t_{bon}	t_{con}	t_{con}	t_{bon}	t_{aon}
Tb(CmrB)	t_{bon}	t_{aon}	t_{aon}	t_{bon}	t_{con}	t_{con}
Tc(CmrC)	t_{con}	t_{con}	t_{bon}	t_{aon}	t_{aon}	t_{bon}

那么整理得出下表

表X-X-3 各扇区电机三相上桥导通时刻对应的占空比

扇区	电机 A 相占空比 $T_a(CmrA)$	电机 B 相占空比 $T_b(CmrB)$	电机 C 相占空比 $T_c(CmrC)$
1=U0, U60	$\frac{t_{aon}}{1 - \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{bon}}{1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2} - U_\beta^*}$	$\frac{t_{con}}{1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}}$
2=U60, U120	$\frac{t_{bon}}{1 - \frac{\sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{aon}}{1 - U_\beta^*}$	$\frac{t_{con}}{1 + U_\beta^*}$
3=U120, U180	$\frac{t_{con}}{1 + \frac{U_\beta^* - \sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{aon}}{1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{bon}}{1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2} + U_\beta^*}$
4=U180, U240	$\frac{t_{con}}{1 - \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{bon}}{1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2} - U_\beta^*}$	$\frac{t_{aon}}{1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2}}$
5=U240, U300	$\frac{t_{bon}}{1 - \frac{\sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{con}}{1 - U_\beta^*}$	$\frac{t_{aon}}{1 + U_\beta^*}$
6=U300, U0	$\frac{t_{aon}}{1 + \frac{U_\beta^* - \sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{con}}{1 + \frac{-U_\beta^* + \sqrt{3}U_\alpha^*}{2}}$	$\frac{t_{bon}}{1 + \frac{U_\beta^* + \sqrt{3}U_\alpha^*}{2} + U_\beta^*}$

若程序定义的参考量如下：

```

v.tmp1= v.Ubeta;
v.tmp2= _IQdiv2(v.Ubeta) + (_IQmpy(_IQ(0.866),v.Ualpha));
v.tmp3= v.tmp2 - v.tmp1;

```

即为

$$tmp1=U_{\beta}^*$$

$$tmp2=\frac{U_{\beta}^*+\sqrt{3}U_{\alpha}^*}{2}$$

$$tmp3=\frac{-U_{\beta}^*+\sqrt{3}U_{\alpha}^*}{2}$$

HalfPerMax 为全周期计数值的四分之一即 $TBPRD/2$

PeriodMax 为全周期计数值的二分之一即赋值到 $TBPRD$ 的值

表X-X-4 正三角增减计数模式下用程序定义的参考量表示的各扇区电机三相上桥导通时刻对应的占空比

扇区	电机 A 相占空比 $Ta(CmrA)$	电机 B 相占空比 $Tb(CmrB)$	电机 C 相占空比 $Tc(CmrC)$
1=U0,U60	$\frac{t_{aon}}{2}$ $\frac{1-tmp2}{2}$	$\frac{t_{bon}}{2}$ $\frac{1+tmp3-tmp1}{2}$	$\frac{t_{con}}{2}$ $\frac{1+tmp2}{2}$
2=U60,U120	$\frac{t_{bon}}{2}$ $\frac{1-tmp2-tmp3}{2}$	$\frac{t_{aon}}{2}$ $\frac{1-tmp1}{2}$	$\frac{t_{con}}{2}$ $\frac{1+tmp1}{2}$
3=U120,U180	$\frac{t_{con}}{2}$ $\frac{1-tmp3}{2}$	$\frac{t_{aon}}{2}$ $\frac{1+tmp3}{2}$	$\frac{t_{bon}}{2}$ $\frac{1+tmp2+tmp1}{2}$
4=U180,U240	$\frac{t_{con}}{2}$ $\frac{1-tmp2}{2}$	$\frac{t_{bon}}{2}$ $\frac{1+tmp3-tmp1}{2}$	$\frac{t_{aon}}{2}$ $\frac{1+tmp2}{2}$
5=U240,U300	$\frac{t_{bon}}{2}$ $\frac{1-tmp2-tmp3}{2}$	$\frac{t_{con}}{2}$ $\frac{1-tmp1}{2}$	$\frac{t_{aon}}{2}$ $\frac{1+tmp1}{2}$
6=U300,U0	$\frac{t_{aon}}{2}$ $\frac{1-tmp3}{2}$	$\frac{t_{con}}{2}$ $\frac{1+tmp3}{2}$	$\frac{t_{bon}}{2}$ $\frac{1+tmp2+tmp1}{2}$

计算出的矢量占空比之后，再乘以比较寄存器最大值 $TBPRD$ （切换时刻是

以其作为标准的)得到去标么化后的Q0格式值计数值,最后将其赋值给控制电机对应相的比较寄存器。

需要注意的是 T_a , T_b , T_c 为占空比,是对半个载波周期时间标么化的Q格式小数,需要程序中用__IQmpy(A,B,GLOBAL_Q)这个函数将以下的算式去标么化。

根据上述得出

(*ePWM[1]).CMPA.half.CMPA 的赋值为 $TBPRD * T_a$

(*ePWM[2]).CMPA.half.CMPA 的赋值为 $TBPRD * T_b$

(*ePWM[3]).CMPA.half.CMPA 的赋值为 $TBPRD * T_c$

程序中对PWM模块的配置如下:

```
#define AQCTLA_INIT_STATE ( CAD_SET + CAU_CLEAR )
```

仔细查看TI 的代码,发现在PWM 模块初始化被设置为增减计数模式,且“PWM 输出设置为在减计数时当计数值等于比较器值时输出高电平,在增计数时当计数值等于比较器值时输出低电平”,前面的比较器值计算的设定刚好相反。

这时候需要将上面的表格,对TBPRD进行取补,就相当于倒计时。对括号里面的占空比进行取补计算,用1减去括号内各自的占空比:

(*ePWM[1]).CMPA.half.CMPA 的赋值为

$TBPRD * (1 - T_a)$

(*ePWM[2]).CMPA.half.CMPA 的赋值为

$TBPRD * (1 - T_b)$

(*ePWM[3]).CMPA.half.CMPA 的赋值为

$TBPRD * (1 - T_c)$

以第三扇区为例,按照该模式可以得出大致波形如下图8-6-2所示:

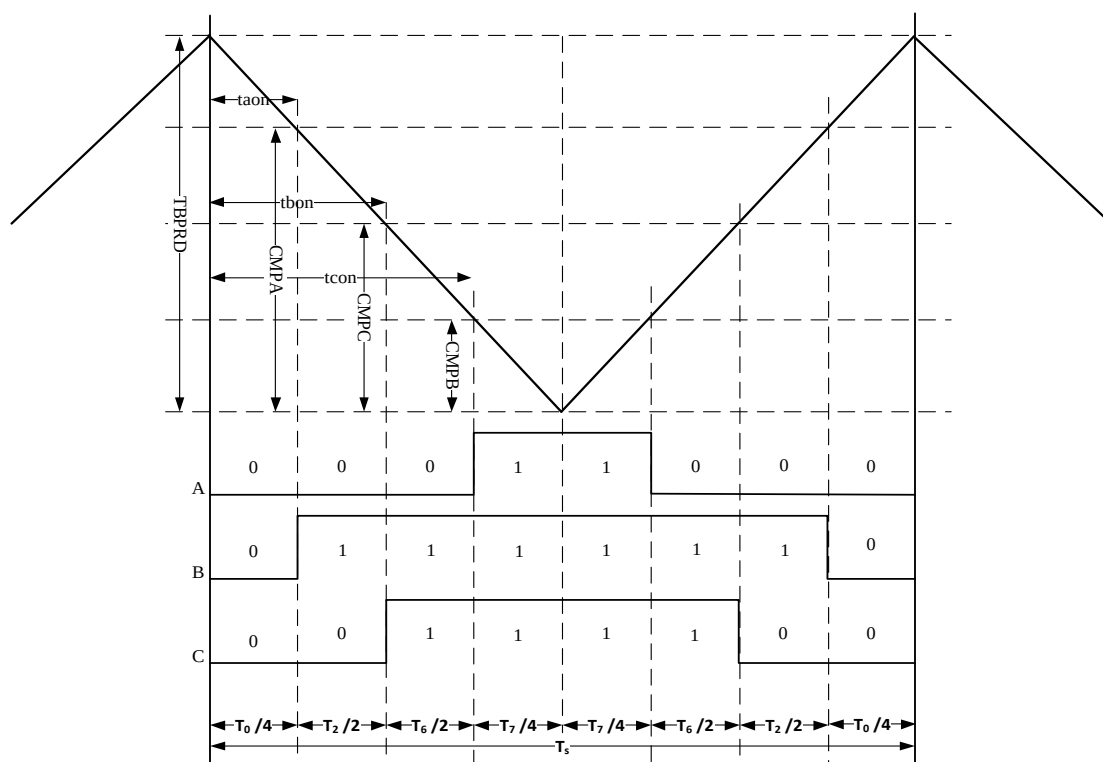


图8-6-2 第三扇区大致波形

表X-X-5 倒三角增减计数模式下用程序定义的参考量表示的各扇区电机三相上桥导通时刻对应的占空比

扇区	电机 A 相占空比 T_a	电机 B 相占空比 T_b	电机 C 相占空比 T_c
1=U0, U60	t_{aon} $\frac{1+tmp2}{2}$	t_{bon} $\frac{1+tmp1-tmp3}{2}$	t_{con} $\frac{1-tmp2}{2}$
2=U60, U120	t_{bon} $\frac{1+tmp3+tmp2}{2}$	t_{aon} $\frac{1+tmp1}{2}$	t_{con} $\frac{1-tmp1}{2}$
3=U120, U180	t_{con} $\frac{1+tmp3}{2}$	t_{aon} $\frac{1-tmp3}{2}$	t_{bon} $\frac{1-(tmp1+tmp2)}{2}$
4=U180, U240	t_{con} $\frac{1+tmp2}{2}$	t_{bon} $\frac{1+tmp1-tmp3}{2}$	t_{aon} $\frac{1-tmp2}{2}$

5=U240,U300	t_{bon} $\frac{1+tmp3+tmp2}{2}$	t_{con} $\frac{1+tmp1}{2}$	t_{aon} $\frac{1-tmp1}{2}$
6=U300,U0	t_{aon} $\frac{1+tmp3}{2}$	t_{con} $\frac{1-tmp3}{2}$	t_{bon} $\frac{1-(tmp1+tmp2)}{2}$

```

if (v.VecSector==1 || v.VecSector==4) \
{
    v.Ta= v.tmp2; \
    v.Tb= v.tmp1-v.tmp3; \
    v.Tc=-v.tmp2; \
} \
\
else if(v.VecSector==2 || v.VecSector==5) \
{
    v.Ta= v.tmp3+v.tmp2; \
    v.Tb= v.tmp1; \
    v.Tc=-v.tmp1; \
} \
\
else \
{
    v.Ta= v.tmp3; \
    v.Tb=-v.tmp3; \
    v.Tc=-(v.tmp1+v.tmp2); \
} \

```

最后看到

```

#define PWM_MACRO(ch1,ch2,ch3,m) \
\
/* Mfuncx range is (-1,1) */ \
/* The code below changes PeriodMax*Mfuncx range .... */ \
/* from (-PeriodMax,PeriodMax) to (0,PeriodMax) where HalfPerMax=PeriodMax/2*/ \
\
(*ePWM[ch1]).CMPA.half.CMPA = _IQmpy(m.HalfPerMax,m.MfuncC1)+ m.HalfPerMax; \
(*ePWM[ch2]).CMPA.half.CMPA = _IQmpy(m.HalfPerMax,m.MfuncC2)+ m.HalfPerMax; \
(*ePWM[ch3]).CMPA.half.CMPA = _IQmpy(m.HalfPerMax,m.MfuncC3)+ m.HalfPerMax; \

```

根据上面的代码把电机 ABC 三相上桥的计数值用等式表示为：

$$CMPA_{PhaseA} = \frac{TBPRD}{2} * T_a + \frac{TBPRD}{2} = \frac{TBPRD * (1 + T_a)}{2}$$

$$CMPA_{PhaseB} = \frac{TBPRD}{2} * T_b + \frac{TBPRD}{2} = \frac{TBPRD * (1 + T_b)}{2}$$

$$CMPA_{PhaseC} = \frac{TBPRD}{2} * T_c + \frac{TBPRD}{2} = \frac{TBPRD * (1 + T_c)}{2}$$

对应地就实现了上述的 **CMPA** 赋值公式，这里需要说明一下

HalfPerMax 为全周期计数值的四分之一即 $TBPRD / 2$

PeriodMax 为全周期计数值的二分之一即赋值到 $TBPRD$ 的值

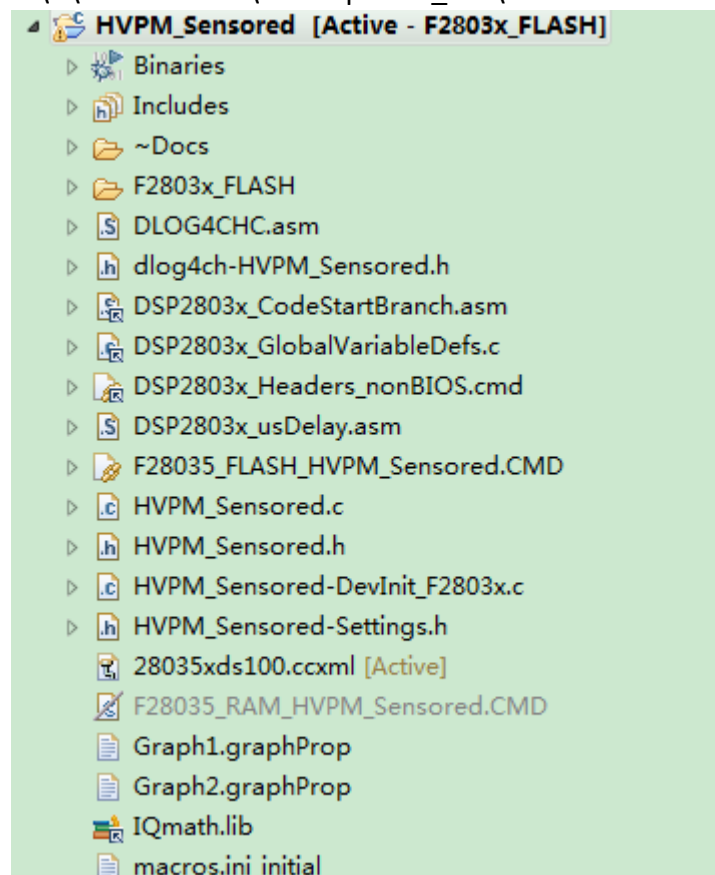
从以下代码可以看出：

```
pwm1.PeriodMax = SYSTEM_FREQUENCY*1000000*T/2; // Prescaler X1 (T1),ISR period = T x 1
pwm1.HalfPerMax=pwm1.PeriodMax/2;
```

1. 导入工程

先从 CCS 里导入官方套件 controlSUITE 的工程：

C:\ti\controlSUITE\development_kits\HVMotorCtrl+PfcKit_v2.1\HVPM_Sensored



2. 设定两相旋转坐标的电压量

看到HVPM_Sensored.c这个文件的82行代码如下

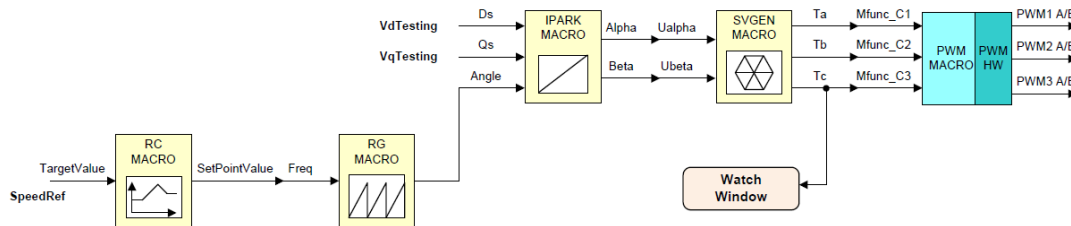
```
_iq VdTesting = _IQ(0.0); // Vd reference (pu)
```

```
_iq VqTesting = _IQ(0.15); // Vq reference (pu)
```

这里先设定 U_d 和 U_q 电压（单位为 V）对电压基值 $U_{dc}/\sqrt{3}$ （单位为 V）进行标么化的无量纲的小数数值，再转化为全局 Q 格式的数值亦即标么化后的 U_d^* 和 U_q^* 。

3. 逆 Park 变换计算两相静止坐标的电压量

再看到主中断函数里的 BUILDLEVEL 1，经过以下框图



经过逆 park 变换运算后（IPARK_MACRO）仍不改变电压基值，svgen1.Ualphah 和 svgen1.Ubeta 均为静止两相坐标系下的 U_α 和 U_β 电压（单位为 V）对电压基值 $U_{dc}/\sqrt{3}$ （单位为 V）进行标么化的无量纲的小数数值，再转化为全局 Q 格式的数值亦即标么化后的 U_α^* 和 U_β^* 。

```
svgen1.Ualpha = ipark1.Alpha;
```

```
svgen1.Ubeta = ipark1.Beta;
```

代码见红色加粗的部分和中文注释：

```
#if (BUILDLEVEL==LEVEL1)
```

```
//
```

```
-----
```

```
-----
```

```
// Connect inputs of the RMP module and call the ramp control macro
```

```
//
```

```
-----
```

```
-----
```

```
rc1.TargetValue = SpeedRef;
```

```
RC_MACRO(rc1)
```

```
//
```

```
-----
```

```
-----
```

```
// Connect inputs of the RAMP_GEN module and call the ramp generator
macro
//
-----
-----
    rg1.Freq = rc1.SetpointValue;
    RG_MACRO(rg1)

//
-----
-----
// Connect inputs of the INV_PARK module and call the inverse park
trans. macro
// There are two option for trigonometric functions:
// IQ sin/cos look-up table provides 512 discrete sin and cos
points in Q30 format
// IQsin/cos PU functions interpolate the data in the lookup table
yielding higher resolution.
//
-----
-----
    ipark1.Ds = VdTesting;//设定标么化的Ud和Uq
    ipark1.Qs = VqTesting;

    //ipark1.Sine  =_IQ30toIQ(IQsinTable[_IQtoIQ9(rg1.Out)]);
    //ipark1.Cosine=_IQ30toIQ(IQcosTable[_IQtoIQ9(rg1.Out)]);

    ipark1.Sine=_IQsinPU(rg1.Out);
    ipark1.Cosine=_IQcosPU(rg1.Out);
    IPARK_MACRO(ipark1)//电压的逆帕克变换,两相旋转 化为 两相静止

//
-----
-----
// Connect inputs of the SVGEN_DQ module and call the space-vector
gen. macro
//
-----
-----
    svgen1.Ualpha = ipark1.Alpha; //计算出标么化的Ualpha和Ubeta
    svgen1.Ubeta = ipark1.Beta;
    SVGENDQ_MACRO(svgen1)
```

```
//
-----
// Connect inputs of the PWM_DRV module and call the PWM signal
generation macro
//
-----
pwm1.MfuncC1 = svgen1.Ta;
pwm1.MfuncC2 = svgen1.Tb;
pwm1.MfuncC3 = svgen1.Tc;
PWM_MACRO(1,2,3,pwm1) // Calculate the new
PWM compare values
```

4.SVGENDQ_MACRO(svgen1)解释

函数定义代码如下:

```
#define SVGENDQ_MACRO(v) \
    v.tmp1= v.Ubeta; \
    v.tmp2= _IQdiv2(v.Ubeta) + (_IQmpy(_IQ(0.866),v.Ualpha)); \
    v.tmp3= v.tmp2 - v.tmp1; \
\
    v.VecSector=3; \
\
    v.VecSector=(v.tmp2> 0)?( v.VecSector-1):v.VecSector; \
    v.VecSector=(v.tmp3> 0)?( v.VecSector-1):v.VecSector; \
    v.VecSector=(v.tmp1< 0)?(7-v.VecSector) :v.VecSector; \
\
    if (v.VecSector==1 || v.VecSector==4) \
    { \
        v.Ta= v.tmp2; \
        v.Tb= v.tmp1-v.tmp3; \
        v.Tc=-v.tmp2; \
    } \
\
    else if(v.VecSector==2 || v.VecSector==5) \
    { \
        v.Ta= v.tmp3+v.tmp2; \
        v.Tb= v.tmp1; \
        v.Tc=-v.tmp1; \
    } \
\
    else \
    { \
        v.Ta= v.tmp3; \
        v.Tb=-v.tmp3; \
    }
```

```
v.Tc=-(v.tmp1+v.tmp2);  
}  
}
```

4.1判断扇区。

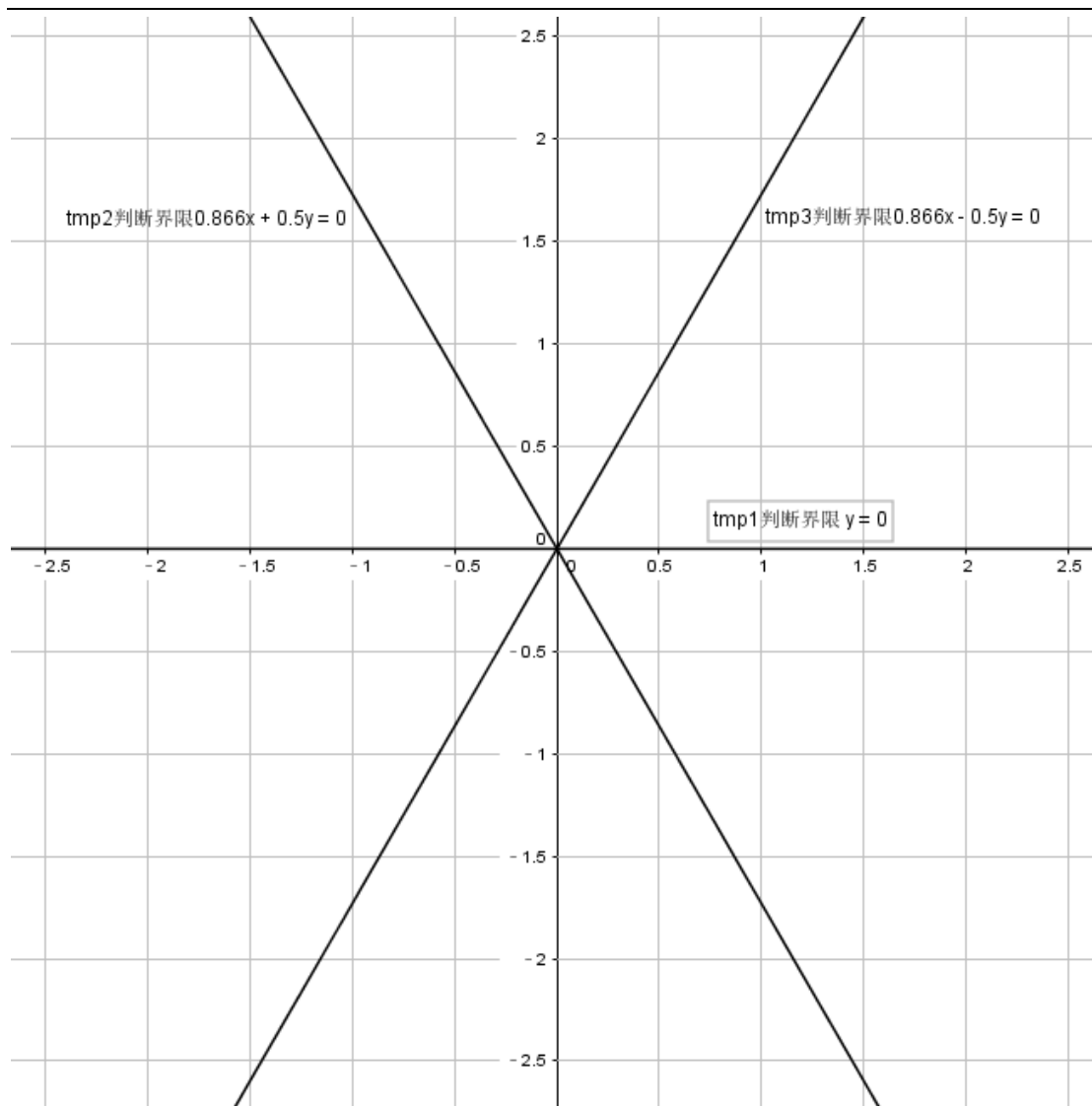
利用以下等式

```
v.tmp1= v.Ubeta;  
v.tmp2= _IQdiv2(v.Ubeta) + (_IQmpy(_IQ(0.866),v.Ualpha));  
v.tmp3= v.tmp2 - v.tmp1;
```

进行不等式区域判断可以辨别电压落在哪个扇区。横纵坐标对应静止两相正交坐标。

整理如下表

扇区\判断条件	tmp2	Tmp3	Tmp1
1	>0	>0	>0
2	>0	<0	>0
3	<0	<0	>0
4	<0	<0	<0
5	<0	>0	<0
6	>0	>0	<0
三个条件无交集	>0	<0	<0
三个条件无交集	<0	>0	>0



遍历所有情况：

第 I 步 $v.\text{VecSector}=(v.\text{tmp2}>0)?(v.\text{VecSector}-1):v.\text{VecSector};$							
$v.\text{tmp2}>0$ $v.\text{VecSector}=3-1=2$				$v.\text{tmp2}<0$ $v.\text{VecSector}=3$			
第 II 步 $v.\text{VecSector}=(v.\text{tmp3}>0)?(v.\text{VecSector}-1):v.\text{VecSector};$							
$v.\text{tmp3}>0$ $v.\text{VecSector}=2-1=1$		$v.\text{tmp3}<0$ $v.\text{VecSector}=2$		$v.\text{tmp3}>0$ $v.\text{VecSector}=3-1=2$		$v.\text{tmp3}<0$ $v.\text{VecSector}=3$	
第 III 步 $v.\text{VecSector}=(v.\text{tmp1}<0)?(7-v.\text{VecSector}):v.\text{VecSector};$							
$v.\text{tmp1}<0$ $v.\text{VecSector}$ $r=7-1=$ 6	$v.\text{tmp1}>0$ $v.\text{VecSector}$ $r=$ 1	$v.\text{tmp1}<0$ $v.\text{VecSector}$ $r=7-2=$ 5 三个条件无交集， 不属于任何扇区	$v.\text{tmp1}>0$ $v.\text{VecSector}$ $r=$ 2	$v.\text{tmp1}<0$ $v.\text{VecSector}$ $r=7-2=$ 5	$v.\text{tmp1}>0$ $v.\text{VecSector}$ $r=$ 2 三个条件无交集， 不属于任何扇区	$v.\text{tmp1}<0$ $v.\text{VecSector}$ $r=7-3=$ 4	$v.\text{tmp1}>0$ $v.\text{VecSector}$ $r=$ 3

三个条件大于和小于分成了八种情况，其中两种没有交集，不属于任何扇区。这里的 VecSector 数值就是扇区编号，并非 3,1,5,4,6,2 对应扇区 1,2,3,4,5,6。

4.2 计算占空比

以扇区 1 为例如图 8-2-1 所示，任意时刻输出参考相电压矢量 $\mathbf{U}_{\text{ref}}$ 可由相邻的两个基本电压空间矢量 $\mathbf{U}_0$ 和 $\mathbf{U}_{60}$ 的线性时间组合来合成，它等于 T_1/T 倍的 $\mathbf{U}_0$ 与 T_3/T 倍的 $\mathbf{U}_{60}$ 的矢量和。其中 T_1 和 T_3 分别是 $\mathbf{U}_0$ 和 $\mathbf{U}_{60}$ 作用时间， T 是 $\mathbf{U}_{\text{ref}}$ 作用的时间。另外为了减少开关切换次数，插入零矢量进行调整。

$$\begin{cases} O_0 t_0 + U_1 t_1 + U_2 t_2 = U_{\text{ref}} T \\ t_0 + t_1 + t_2 = T \end{cases}$$

上式中 U_1, t_1 是先作用的电压矢量和时间， U_2, t_2 是后作用的电压矢量和时间，

O_0, t_0 是指两个不同的零矢量和作用时间。后续会继续分析。

扇区1分析

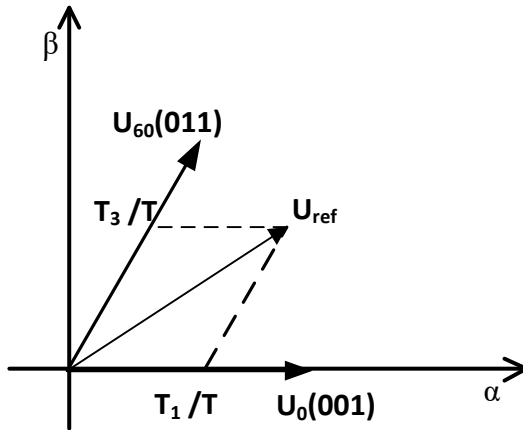


图8-2-1 参考矢量在扇区1的分解

根据矢量作用时间和几何关系可得

$$U_{\alpha 1} = \frac{T_1}{T} U_0 + \frac{T_3}{T} U_{60} \cos 60^\circ$$

$$U_{\beta 1} = \frac{T_3}{T} U_{60} \sin 60^\circ$$

(注意这里 $U_{\alpha 1}$ 和 $U_{\beta 1}$ 单位为V)

计算出时间如下：

对式子 $T_3 = \frac{T U_{\beta 1}}{U_{60} \sin 60^\circ}$ 中分子分母同时除以 $\frac{U_{dc}}{\sqrt{3}}$ 实现了两个量的标么化：电压空

间矢量 U_{60} 大小 对 相电压 $\frac{U_{dc}}{\sqrt{3}}$ 进行标么化为 $\frac{\frac{2}{3} U_{dc}}{\frac{U_{dc}}{\sqrt{3}}} = \frac{2}{\sqrt{3}}$ 。静止两相坐标系下的

电压矢量 $U_{\beta 1}$ 大小 对 相电压 $\frac{U_{DC}}{\sqrt{3}}$ 标么化为 $U_{\beta 1}^*$ 。这样选取 $\frac{U_{DC}}{\sqrt{3}}$ 为电压基值的目的是为了化简矢量作用时间的计算，即与60度sin和cos三角函数值对消约去。计算技巧见下式：

$$T_3 = \frac{T U_{\beta 1}}{U_{60} \sin 60^\circ} = \frac{T \frac{U_{\beta 1}}{U_{DC} / \sqrt{3}}}{\frac{U_{60}}{U_{DC} / \sqrt{3}} \frac{\sqrt{3}}{2}} = \frac{T \frac{U_{\beta 1}}{U_{DC} / \sqrt{3}}}{\frac{2 U_{DC} / 3 \sqrt{3}}{U_{DC} / \sqrt{3}} \frac{\sqrt{3}}{2}} = \frac{T U_{\beta 1}^*}{\frac{2}{\sqrt{3}} \frac{\sqrt{3}}{2}} = T U_{\beta 1}^*$$

$$\begin{aligned} T_1 &= \frac{T(U_{\alpha 1} - \frac{T_3}{T} U_{60} \cos 60^\circ)}{U_0} \\ &= \frac{T(U_{\alpha 1} \frac{\sqrt{3}}{U_{DC}} - \frac{T_3}{T} \frac{\sqrt{3}}{U_{DC}} U_{60} \cos 60^\circ)}{U_0 \frac{\sqrt{3}}{U_{DC}}} \\ &= \frac{T(U_{\alpha 1}^* - U_{\beta 1}^* \frac{2}{\sqrt{3}} \frac{1}{2})}{\frac{2}{\sqrt{3}}} \\ &= \frac{T \sqrt{3} U_{\alpha 1}^* - T U_{\beta 1}^* \sqrt{3} \frac{2}{\sqrt{3}} \frac{1}{2}}{2} \\ &= \frac{T(\sqrt{3} U_{\alpha 1}^* - U_{\beta 1}^*)}{2} \end{aligned}$$

对应的占空比为

$$T_3^* = \frac{T_3}{T} = U_{\beta 1}^* \quad \text{等于判断条件tmp1, 开关状态含有两个“1”的矢量作用时间占空比}$$

$$T_1^* = \frac{T_1}{T} = \frac{-(-\sqrt{3} U_{\alpha 1}^* + U_{\beta 1}^*)}{2} \quad \text{等于判断条件 tmp3, 开关状态只有一个“1”的矢量作用时间占空比}$$

4.3 矢量切换时刻

上一节求出了参考电压矢量所在的扇区，那么可以知道先后作用的矢量。假定在DSP芯片上配置好的PWM模块为增减计数。在增计数时计数值等于比较值时候，对应控制上桥信号置高，在减计数时计数值等于比较值时候，对应控制上桥

信号置低。死区模块设置为互补波形，且根据实际电路设置电平高或低有效，用七段波形调制。由于波形的对称性，各个矢量作用时间分成2份，分开后用于增计数作用和减计数作用。为了满足开关切换最少，以及方便后续的死区补偿添加了两个零矢量，又因为作用时间不变，所以该模式下只能把零矢量作用时间再分成2份。方便区别和分析，这里规定小写 t 和 T^* 为占空比，大写的 T 为实际作用时间。

按照上述的逻辑，可以得出下表8-6-1：

表8-6-1 矢量作用时刻表

增减计数状态	作用的电压空间矢量	持续时间	开始时刻	结束时刻
增计数	零矢量 $O_{000} (000)$	$\frac{T_0}{4}$	0	$\frac{T_0}{4}$
	开关状态只含有一个“1”的矢量	$\frac{T_1}{2}$	$\frac{T_0}{4}$	$\frac{T_0}{4} + \frac{T_1}{2}$
	开关状态含有两个“1”的矢量	$\frac{T_2}{2}$	$\frac{T_0}{4} + \frac{T_1}{2}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2}$
	零矢量 $O_{111} (111)$	$\frac{T_0}{4}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2}$	$\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} + \frac{T_0}{4} = \frac{T_{PWM}}{2}$
减计数	零矢量 $O_{111} (111)$	$\frac{T_0}{4}$	$\frac{T_{PWM}}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4}$
	开关状态含有两个“1”的矢量	$\frac{T_2}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2}$
	开关状态只含有一个“1”的矢量	$\frac{T_1}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2} + \frac{T_1}{2}$
	零矢量 $O_{000} (000)$	$\frac{T_0}{4}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_2}{2} + \frac{T_1}{2}$	$\frac{T_{PWM}}{2} + \frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} + \frac{T_0}{4} = T_{PWM}$

按照上表，三个切换时刻的分析计算如下：

【3】先作用的零矢量作用完毕，先作用的非零矢量作用开始的时刻及占空比：

$$\begin{aligned}
 \frac{T_0}{4} &= \frac{T_{PWM} - T_1 - T_2}{4} \\
 &= \frac{T_{PWM}}{2} * \frac{1 - T_1^* - T_2^*}{2} \\
 &= \frac{T_{PWM}}{2} * \frac{1 - t_1 - t_2}{2} = \frac{T_{PWM}}{2} * t_{aon}
 \end{aligned}$$

根据上述定义 $t_{aon} = \frac{1 - t_1 - t_2}{2}$

扇区1则为 $t_{aon} = \frac{1 - T_1^* - T_3^*}{2}$

【2】先作用的非零矢量作用完毕，后作用的非零矢量作用开始的时刻及占空比：

$$\begin{aligned}
\frac{T_0}{4} + \frac{T_1}{2} &= \frac{T_{PWM} - T_1 - T_2}{4} + \frac{T_1}{2} \\
&= \frac{T_{PWM}}{2} * (\frac{1 - T_1^* - T_2^*}{2} + T_1^*) \\
&= \frac{T_{PWM}}{2} * (\frac{1 - t_1 - t_2}{2} + t_1) = \frac{T_{PWM}}{2} * t_{bon}
\end{aligned}$$

根据上述定义 $t_{bon} = t_{aon} + t_1$

扇区1则为 $t_{bon} = t_{aon} + T_1^*$

【3】后作用的非零矢量作用完毕，后作用的零矢量作用开始的时刻及占空比：

$$\begin{aligned}
\frac{T_0}{4} + \frac{T_1}{2} + \frac{T_2}{2} &= \frac{T_{PWM} - T_1 - T_2}{4} + \frac{T_1}{2} + \frac{T_2}{2} \\
&= \frac{T_{PWM}}{2} * (\frac{1 - T_1^* - T_2^*}{2} + T_1^* + T_2^*) \\
&= \frac{T_{PWM}}{2} * (\frac{1 - t_1 - t_2}{2} + t_1 + t_2) = \frac{T_{PWM}}{2} * t_{con}
\end{aligned}$$

根据上述定义 $t_{con} = t_{bon} + t_2$

扇区1则为 $t_{con} = t_{bon} + T_3^*$

明显可以得到 $t_{aon} < t_{bon} < t_{con}$ 这样的关系。这里的abc字母不代表电机三相序号，而是代表导通的优先顺序，a代表最先导通，b代表接着导通，c代表最后导通。以上求出了切换时刻对应的占空比，再次整理如下：

$$\begin{aligned}
t_{aon} &= \frac{1 - t_1 - t_2}{2} \xrightarrow{\text{扇区1}} t_{aon} = \frac{1 - T_1^* - T_3^*}{2} = \frac{1 - tmp3 - tmp1}{2} \\
t_{bon} &= t_{aon} + t_1 \xrightarrow{\text{扇区1}} t_{bon} = t_{aon} + tmp3 \\
t_{con} &= t_{bon} + t_2 \xrightarrow{\text{扇区1}} t_{con} = t_{bon} + tmp1
\end{aligned}$$

以第一扇区为例，按照以上逻辑排列可以得出大致波形如图8-6-1所示：

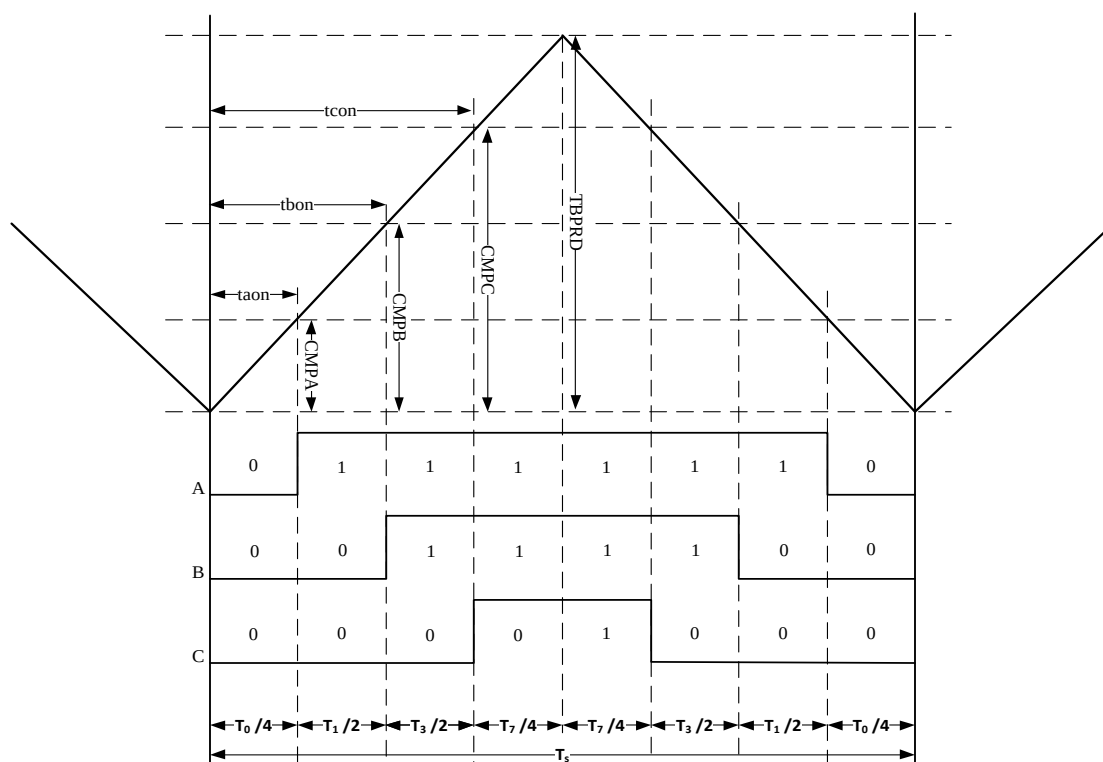


图8-6-1 第一扇区大致波形

比较寄存器值 $CMPA = (t_{aon} / IQ_MAX) * TBPRD$; t_{aon} 为占空比也是对参考矢量作用时间标么的Q格式

小数, IQ_MAX 是1在全局Q格式下的大小。括号内算出的就是小数, 整个算式为去标么化。

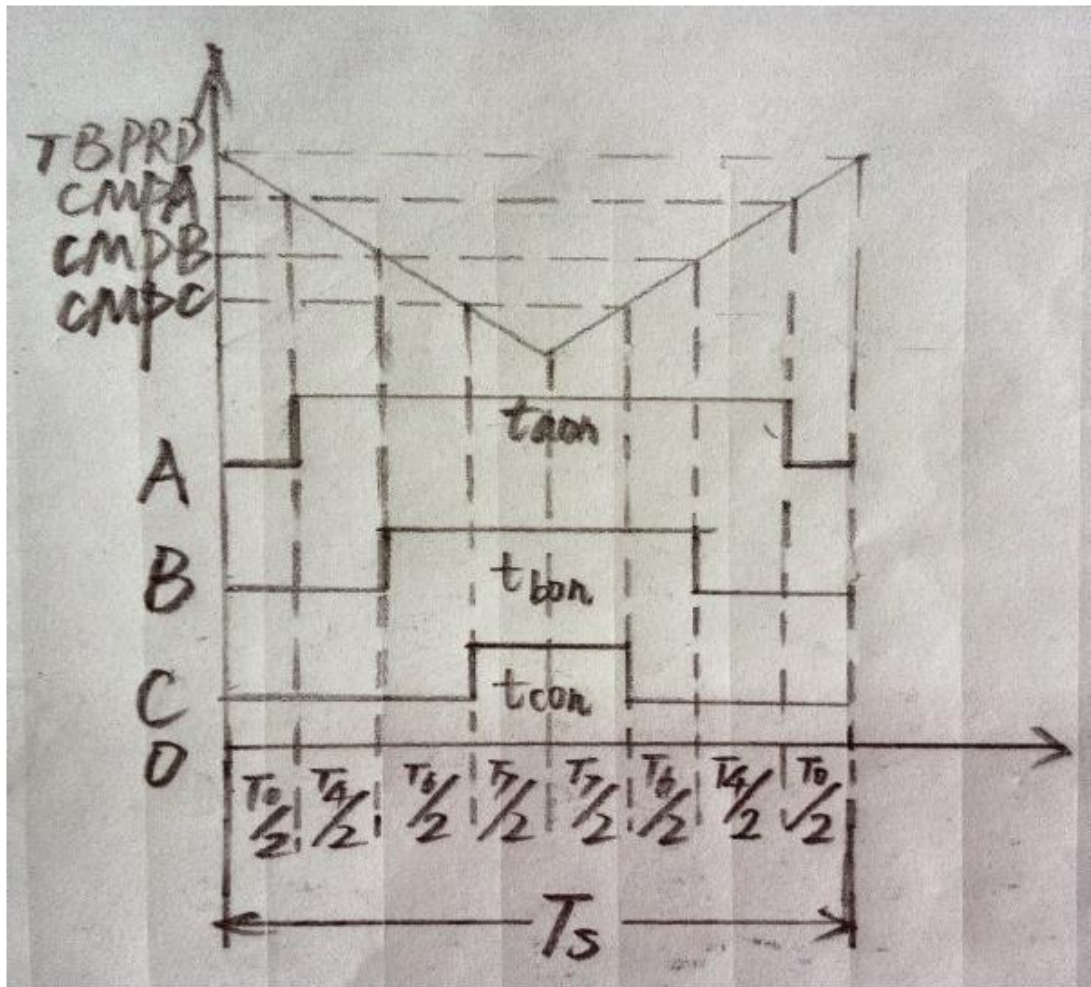
根据上述得出

$$(*ePWM[1]).CMPA.half.CMPA \text{ 的赋值为 } TBPRD * \left(\frac{1 - tmp3 - tmp1}{2} \right)$$

$$(*ePWM[2]).CMPA.half.CMPA \text{ 的赋值为 } TBPRD * \left(\frac{1 + tmp3 - tmp1}{2} \right)$$

$$(*ePWM[3]).CMPA.half.CMPA \text{ 的赋值为 } TBPRD * \left(\frac{1 + tmp3 + tmp1}{2} \right)$$

仔细查看TI 的代码, 发现在PWM 模块初始化被设置为增减计数模式, 且
“PWM 输出设置为在减计数时当计数值等于比较器值时输出高电平, 在增计数时当计数值等于比较器值时输出低电平”, 前面的比较器值计算的设定刚好相反, 如下图:



那么则对括号里面的占空比进行互补计算，用 1 减去括号内各自的占空比：

`(*ePWM[1]).CMPA.half.CMPA` 的赋值为

$$TBPRD * (1 - \frac{1 - tmp3 - tmp1}{2}) = TBPRD * (\frac{1 + tmp3 + tmp1}{2}) = \frac{TBPRD}{2} * (1 + tmp2)$$

`(*ePWM[2]).CMPA.half.CMPA` 的赋值为

$$TBPRD * (1 - \frac{1 + tmp3 - tmp1}{2}) = \frac{TBPRD}{2} * (1 + tmp1 - tmp3)$$

`(*ePWM[3]).CMPA.half.CMPA` 的赋值为

$$TBPRD * (1 - \frac{1 + tmp3 + tmp1}{2}) = \frac{TBPRD}{2} * [1 + (-tmp2)]$$

```

if (v.VecSector==1 || v.VecSector==4)
{
    v.Ta= v.tmp2;
    v.Tb= v.tmp1-v.tmp3;
    v.Tc=-v.tmp2;
}

```

5. PWM_MACRO(1,2,3,pwm1)解释

最后看到

```
#define PWM_MACRO(ch1,ch2,ch3,m)
\
\
\
/* Mfuncx range is (-1,1)
*/ \
/* The code below changes PeriodMax*Mfuncx range .... */
\
/* from (-PeriodMax,PeriodMax) to (0,PeriodMax) where HalfPerMax=PeriodMax/2 */
\
\
(*ePWM[ch1]).CMPA.half.CMPA = _IQmpy(m.HalfPerMax,m.MfuncC1)+ m.HalfPerMax; \
(*ePWM[ch2]).CMPA.half.CMPA = _IQmpy(m.HalfPerMax,m.MfuncC2)+ m.HalfPerMax; \
(*ePWM[ch3]).CMPA.half.CMPA = _IQmpy(m.HalfPerMax,m.MfuncC3)+ m.HalfPerMax; \
```

对应地就实现了上述的 **CMPA** 赋值公式，这里需要说明一下

HalfPerMax 为全周期计数值的四分之一即 $TBPRD/2$

PeriodMax 为全周期计数值的二分之一即赋值到 **TBPRD** 的值

从以下代码可以看出：

```
pwm1.PeriodMax = SYSTEM_FREQUENCY*1000000*T/2; // Prescaler X1 (T1),ISR period = T x 1
pwm1.HalfPerMax=pwm1.PeriodMax/2;
```

其他扇区判断请自行推导。

6.我的疑问

不过实际代码设置不跟以上所述。以上推导有问题！有待讨论！

设定如下：

主中断由**ADC**模块产生主中断

TBCTL

15:14 自由运行

13 同步信号到来时增计数

12:10 分频*1

9:7 分频*1

6 不触发软同步信号

5:4 同步信号到来时输出同步信号（同步顺流模式）

3 直接写入工作 **TBPRD** 寄存器，不用影子寄存器

2 不把相位 **TBPHS** 装入 **TBCTR**

1:0 增减计数

CMPCTL

15:10 保留

9 略

8 略

7 保留

6 比较值 B 使用影子寄存器(激活但不用)

5 保留

4 比较值 A 使用影子寄存器

3:2 当 TBCTR 计数到 0 从 CMPB 影子寄存器装入工作寄存器(激活但不用)

1:0 当 TBCTR 计数到 0 从 CMPA 影子寄存器装入工作寄存器

AQCTLA

15:12 保留

11:10 略，无动作

9:8 略，无动作

7:6 当 TBCTR 减计数且等于 CMPA 时，将 EPWMxA 输出置高

5:4 当 TBCTR 增计数且等于 CMPA 时，将 EPWMxA 输出置低

3:2 略，无动作

1:0 略，无动作

DBCTL

15 DB 模块时钟与 TB 模块时钟一致

14:6 保留

5:4 只用上一个模块的 EPWMxA 作为单相两桥臂的控制信号输入源

3:2 高有效，EPWMxA 输出不翻转，EPWMxB 输出翻转

1:0 上一个模块的输出接入死区模块

PC 模块关闭

TZ 模块关闭

使能EPWMxSOCA脉冲信号：

EPwm1Regs.ETSEL.bit.SOCAEN = 1;

当TBCTR = TBPRD时将会生成EPWMxSOCA脉冲信号：

EPwm1Regs.ETSEL.bit.SOCASEL = 2;

事件发生一次就生成EPWMxSOCA脉冲信号

EPwm1Regs.ETPS.bit.SOCAPRD = 1;

清除SOCA标志位

EPwm1Regs.ETCLR.bit.SOCA = 1;

参考资料

文献与书籍:

德州仪器PPT资料TI-MotorControlCompendium-2010

德州仪器controlSUITE软件库

电动机的DSP控制技术与实践_任志斌

德州仪器TMS320x2803x Piccolo ePWM Module Reference Guide_spruge9e

软件工具:

Visio_Premium_2010-32bits

MathType-6.9a

controlSUITE-v3.4.9-2018.03.28

参考资料

文献与书籍:

1.1.1-电动机的dsp控制——ti公司dsp应用(第2版)_王晓明

1.1.2-C语言程序设计(第二版)_谭浩强

2.1-现代永磁同步电机控制原理及MATLAB仿真_袁雷等

2.2-逆变器死区补偿中一种提高三相电流极性检测精度的方法_徐凯

3.2-电动汽车驱动用永磁同步电动机系统效率优化控制研究_许家群等

4-电力拖动自动控制系统(第3版)_陈伯时

5.1-基于标幺化模型的PMSM矢量控制系统设计_宋建国

5.2-永磁同步电机矢量控制系统的标幺化设计_李阳

6.1-电动汽车用永磁同步电机超前角弱磁控制_罗德荣等

7.1- SVPWM逆变器死区补偿的研究与实现_魏凯

7.2-正弦和空间矢量PWM逆变器死区效应分析与补偿_吴茂刚

7.3- SVPWM的简化算法以及死区补偿_吴爱国

8-TI-MotorControlCompendium-2010_德州仪器

8-math blocks_德州仪器controlSUITE软件库

8-电动机的DSP控制技术与实践_任志斌

8.3-永磁同步电机矢量控制系统弱磁控制策略研究_丁强

8.6-TMS320x2803x Piccolo ePWM Module Reference Guide_spruge9e

软件工具:

GeoGebra-Windows-Installer-5-0-332-0

Visio_Premium_2010-32bits

MathType-6.9a

controlSUITE-v3.3.3

芯片TI28x系列驱动配置

正文

【1】编码器 QEP 配置

硬件接口及说明

GPIO50,51,53 对应 A,B,Z 信号。GPIO52 暂时不用做 QEPS 提示信号脚,定义为 2kHz 采样,与速度环频率一致。

电角度的计算

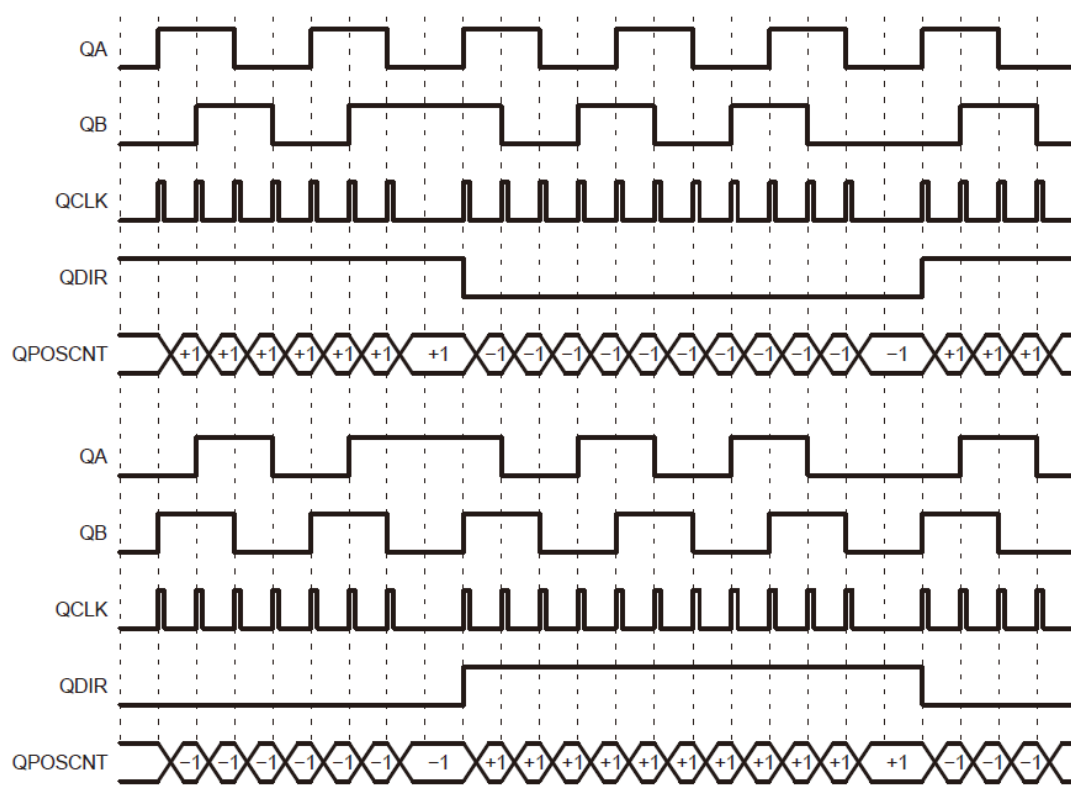
一般的旋转编码器本身只能提供相对位置信息,不过配合零点信号(即 Z 相信号)即可确定绝对位置,下面结合 TI C2000 系列单片机来讨论具体实现方法。

基本原理

在 C2000 系列单片机中,使用 EQEP 模块对旋转编码器输入信号进行处理,先根据输入的 A、B、Z 三相信号生成 QCLK 内部时钟信号、QDIR 方向信号及 QI 同步信号,这部分工作是由 QDU (Quadrature Decoder Unit) 子模块完成的。输入信号的极性,输出的来源等均可配置,具体详见数据手册。

生成的 QCLK 信号来源于 A、B 两相信号跳变沿,此信号与 QDIR 方向信号一起确定 QPOSCNT 寄存器的计数增减,具体时序图如下:

Figure 7. Quadrature-clock and Direction Decoding



从理论上说，只要确定了一点的绝对位置，就可以通过读取 QPOSCNT 寄存器获取任意时刻的绝对位置，假设 QPOSCNT 的初值是 0，电机完整的顺时针转过一圈后 QPOSCNT 中的值应该是 $4 * \text{MOTOR_LINE}$ 。在 C2000 单片机中，可以通过 QPOS MAX 寄存器来设置这个最大值，示例代码如下：

```
EQep1Regs.QEPCTL.bit.PCRM = 1; // 1: QPOSCNT reset on the maximum position
EQep1Regs.QPOS MAX = 4 * MOTOR_LINE - 1; // Set the maximum position
```

然而，这有两个问题，首先是初始时刻的绝对位置如何确定；其次是如何消除累积误差。解决方法是：初始时刻的绝对位置通过霍尔 UVW 信号进行估测，累积误差通过 Z 相同步信号来消除(暂时不使用)。下面分别来讨论这两个问题。

初始时刻定位

初始时刻的位置确定比较简单，只要预先测定出 Ha、Hb、Hc 三个霍尔信号输出与绝对角度间的关系，在启动时通过查表的方法即可大致估算出初始角度。因为这一估算过程本来就不要求也不可能精确，故也无需测定霍尔信号跳变沿的准确位置，粗略的使用示波器观察信号估计一下即可。

示例性代码如下：

```
EQep1Regs.QPOSCNT = GetHall();
```

这句代码在初始化 EQEP 模块的时候调用一次，GetHall()函数可返回根据 Hall 信号猜测出的 QPOSCNT 寄存器的初值：

```
// Hall signal delt
```

```
#define HALL_DELT (QEP_LINE / 3)
```

```
// Hall signal offset
```

```
#define HALLOFFSET (HALL_DELT / 2)
```

```
Uint32 GetHall(void)
{
    Uint16 Ha, Hb, Hc;
    Uint16 HallSignal;

    // 输入斯密特触发器处进行了取反
    // 此处再次取反获取原信号
    Ha = ~GpioDataRegs.GPADAT.bit.GPIO9 & 0x01;
    Hb = ~GpioDataRegs.GPADAT.bit.GPIO23 & 0x01;
    Hc = ~GpioDataRegs.GPADAT.bit.GPIO10 & 0x01;

    HallSignal = (Ha << 2) + (Hb << 1) + Hc;

    switch (HallSignal)
    {
        case 0x00: // 000
            return HALLOFFSET + 0;

        case 0x01: // 001
            return HALLOFFSET + HALL_DELT;

        case 0x03: // 011
            return HALLOFFSET + 2 * HALL_DELT;

        case 0x07: // 111
            return HALLOFFSET + 3 * HALL_DELT;

        case 0x06: // 110
            return HALLOFFSET + 4 * HALL_DELT;

        case 0x04: // 100
            return HALLOFFSET + 5 * HALL_DELT;

        default:
            __asm ("      ESTOP0");
    }

    return 0;
}
```

z 相同步信号

Z 相信号用于对 QPOSCNT 寄存器进行同步定位，这个信号需要仔细考虑下，否则会造成之后得到的绝对位置存在误差。首先需要考虑的是，Z 相信号本身是一个存在一定宽度的脉冲信号，从旋转编码器的原理上来看，正转和反转时 Z 相信号的上升沿和下降沿对应的绝对位置是不同的，正转时的上升沿对应反转时的下降沿，反之亦然。这就需要在正转和反转时使用不同的边沿触发同步事件，C2000 单片机中可以很方便的实现这一功能，只需要设置 IEI 寄存器即可：

```
EQep1Regs.QEPCTL.bit.IEI = 3; // 0 : Does nothing
                                // 2 : Initializes QPOSCNT on rising edge of QEPI
                                // 3 : Initializes QPOSCNT on rising or falling edge of
```

QEPI depends on direction

同步事件发生时写入 QPOSCNT 中的值由 QPOSINIT 寄存器确定：

```
EQep1Regs.QPOSINIT = INDEX_OFFSET; // eQEP Position Counter Initialization Value
```

这里就涉及到 INDEX_OFFSET 这个值怎么确定的问题，因为 Z 相脉冲并不是严格的出现于绝对零点位置处，故需要进行角度校正。

角度校正

角度校正的基本思想是，记录下 Z 相信号对应的绝对角度值，将其写入 QPOSINIT 寄存器即可，即上述代码中的 INDEX_OFFSET 宏定义的值。要确定绝对角度，必须要有一个基准，在进行角度校正时，使用预定位的方式确定绝对角度的零点。

具体做法是，使能三相逆变器，使用 SVPWM 算法生成一个 $U_{\alpha} > 0$; $U_{\beta} = 0$ 的矢量， U_{α} 不宜设置得过大(对应 A 相上桥导通占空比，BC 两相上桥不导通)，以免电流过大，此时电机旋转至绝对零点处。之后使能 EQEP 模块，将 QPOSCNT 的值设为 0，SEI 设为 0，以禁用 QI 信号信号的自动同步复位功能；与此同时，使能 IEL，以便在 QI 信号发生时自动锁存 QPOSCNT 寄存器的值。以上初始化过程用代码表示如下：

```
EQep1Regs.QPOSCNT = 0;
EQep1Regs.QEPCTL.bit.IEI = 0; // Does nothing on rising or falling edge of QEPI
EQep1Regs.QEPCTL.bit.IEL = 3; // Latch QPOSCNT on rising or falling edge of QEPI
depends on direction
```

按照以上步骤初始化 EQEP 模块后，禁用三相逆变器，此时电机可以自由旋转。用手旋转电机以产生 QI 信号，此时 QPOSCNT 寄存器中的值会被自动锁存在 QPOSILAT 寄存器中，可以正反多旋转几次电机，记录下若干次 QPOSILAT 寄存器中的值取平均值即可。读取寄存器的值可以通过使用仿真器进入调试模式实现。

计算得到的平均值即 QI 信号对应的绝对角度值，将此值定义为 INDEX_OFFSET 宏写入 QPOSINIT 寄存器中即可。

注：上述叙述假设编码器 Z 相信号连接至了 QEPI 输入端上，若连接至了 QEPS 输入端处，只需将 QI 相关部分改为 QS 即可，二者的功能基本相同。

电角度计算

正余弦用 256 个数组元素查表获得。

Z 脉冲未到来，由 QEP 计数和 UVW 霍尔粗略得到电角度

Z 脉冲触发中断，由 QEP 计数，Z 锁存值和 Z 和磁铁 N 极夹角三个参数得到电角

度，角度更为连续以及精确。

//LJF 当前d轴位置，即编码器读取位置与旋转坐标d轴的夹角

/*-----*/

**gstr_Encoder.g_QPosCnt从编码器读取位置开始转动后的角度，配置好QEP后软件自动记录在QPOSCNT

**gstr_Encoder.g_ZLatch从编码器读取位置相对于Z脉冲的角度，配置好QEP后软件自动记录在QPOSILAT

**gstr_Encoder.g_ZToBetaTheta旋转坐标d轴相对于Z脉冲的电角度脉冲，由电机厂商给出，或者定子产生磁场吸转子磁铁，关波后转动一圈读取Z锁存的POSCNT

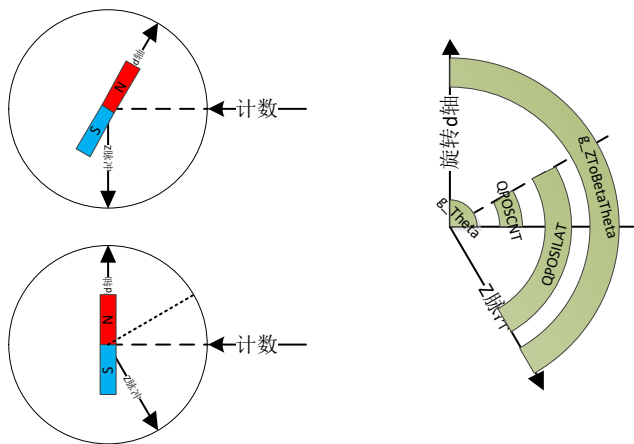
-----*/

//LJF =从编码器读取位置开始转动后的角度+ (旋转坐标d轴相对于Z脉冲的电角度脉冲-从编码器读取位置相对于Z脉冲的角度)

//LJF =从编码器读取位置开始转动后的角度+ 旋转坐标d轴相对于编码器读取位置的电角度

//LJF =编码器读取位置与旋转坐标d轴的夹角

```
gstr_Encoder.g_Theta = (int32)gstr_Encoder.g_QPosCnt
                      - (int32)gstr_Encoder.g_ZLatch
                      + gstr_Encoder.g_ZToBetaTheta;
```



速度的计算

系统时钟设定为 150MHz

高速时用 M 法测频法：由 QUPRD 控制固定时间。读取的变量为位置脉冲 QPOSLAT 前后的差值,因为编码器有一定的精度，计算时可适当放大（100 倍）后单位为 0.01rpm，公式为

$$100 * \frac{1}{\frac{1}{60} \left(\frac{\text{min}}{\text{s}} \right) * \frac{QUPRD}{150M(Hz)}} * \Delta POSLAT$$

低速时用 T 法测周法：由 UPPS 控制固定旋转角度，由 CCPS 控制计时时钟快慢精度。读取的变量为时间计数差值 QCPRDLAT。放大 100 倍后单位为 0.01rpm，公式为

$$100 * \frac{\frac{1}{2500 * 4(pluse)} * 2^{UPPS}}{\frac{1}{60}(\frac{\min}{s}) * \frac{2^{CCPS}}{150M(Hz)}} * \frac{1}{QCPRDLAT}$$

位置的计算

位置环的速度前馈加入滤波器
一阶低通滤波器如下式表示

$$G(s) = \frac{Y(s)}{R(s)} = \frac{1}{T_c s + 1}$$

$$\text{也可以写成时域的形式 } \frac{dY(t)}{dt} = \frac{1}{T_c} [R(t) - Y(t)]$$

这里 $T_c = \frac{1}{\omega_c} = \frac{1}{2\pi f_c}$ 为低通滤波器的时间常数亦即与截止角频率有关的常量

系数，单位为秒； f_c 为截止频率,单位为 Hz。

这里需要离散化，以便在 DSP 上实现，采用向后差分那么上式可以写成：

$$\frac{Y(k) - Y(k-1)}{T} = \frac{1}{T_c} R(k) - \frac{1}{T_c} Y(k)$$

$$(\frac{1}{T} + \frac{1}{T_c})Y(k) = \frac{1}{T_c} R(k) + \frac{1}{T} Y(k-1)$$

$$Y(k) = \frac{T_c}{T_c + T} Y(k-1) + \frac{T}{T_c + T} R(k)$$

这里 T 为采样时间，截止角频率系数 $T_c = \frac{1}{\omega_c} = \frac{1}{2\pi f_c}$

$$\text{令 } K_2 = \frac{T_c}{T_c + T} = \frac{1}{1 + \frac{T}{T_c}} = \frac{1}{1 + 2\pi f_c T}, \quad K_3 = \frac{T}{T_c + T} \text{ 同时有 } K_2 + K_3 = 1。$$

那么也对包含在 speed1 结构体中的速度计算参数进行设定。设定截止频率为 30Hz，则初始化的 K2，K3 编程如下：

```
speed1.K2 = _IQ(1/(1+T*2*PI*30)); // 低通滤波器，截止频率 30Hz
speed1.K3 = _IQ(1)-speed1.K2;
```

计算出的速度标么值进行滤波，程序代码段如下：
Tmp1 = _IQmpy(v->K2,_IQtoIQ21(v->Speed))+_IQmpy(v->K3,Tmp1);

外设配置

```
void InitEQep1(void)
{
    //初始化 eQEP1
    //QEP的模式配置

    //仿真环境配置
    //LJF 2:仿真挂起对其无影响
    EQep1Regs.QEPCTL.bit.FREE_SOFT = 2;    //调试中断时不复位EQEP

    //LJF 0-正交编码器四倍频计数模式
    EQep1Regs.QDECCTL.bit.QSRC = 0;        // 四倍频计数模式; (freq. measurement)

    //-----
    //【1】配置同步输出管脚，配置管脚的STROBE功能
    //LJF 不使用？选择提示信号引脚作为同步信号的输出 1-Strobe pin is used for sync output
    EQep1Regs.QDECCTL.bit.SPSEL = 1;        //1- strobe pin for sync out,0-index pin
    //LJF 该值将和QPOSCNT进行比较，以产生同步脉冲信号(或中断)
    EQep1Regs.QPOSCMP          = 0x0;        //位置比较寄存器值，同步信号输出比较点，即与Z
    信号的相位差
    //LJF 使能位置比较同步信号输出，1-Enable position-compare sync output
    EQep1Regs.QDECCTL.bit.SOEN = 1;        //使能sync out输出功能

    //比较功能控制寄存器设置
    //LJF 禁止位置比较寄存器映射功能
    EQep1Regs.QPOSCCTL.bit.PCSHDW = 0;    //禁止映射寄存器，立即加载；映射寄存器的加载
    方式先不调；
    //LJF 使能位置比较功能
    EQep1Regs.QPOSCCTL.bit.PCE      = 1;    //1-使能比较功能，输出同步信号sync out
    //LJF 同步脉冲宽度 = (PCSPW_255 + 1) * 4 * T_SYSCLKOUT_10ns = 10.24us
    EQep1Regs.QPOSCCTL.bit.PCSPW   = 0xFF;    //配置sync out脉冲宽度：= (PCSPW(0x0-0xFF)
    +1) * 4 * T_SYSCLKOUT(6.67ns)

    //-----
    //【1】通过交换A和B可以更改电机旋转方向
    //LJF 0-内部不交换正交脉冲信号
    EQep1Regs.QDECCTL.bit.SWAP = 0;        //0-默认，不更改，1-AB交换，反相
```

```
//-----  
//【1=计数和锁存配置】选通 Index pin使用方式：初始化位置计数器，锁存当前计数器值，触发  
中断  
//周期和初始寄存器初始值  
//LJF EQEP计数器的最大值，编码器线数*4倍频  
EQep1Regs.QPOSMAX = (Uint16)gstr_Encoder.g_PEncoder-1; //EQEP计数器的最大  
值，编码器线数?  
//LJF 1-最大值时复位位置计数器  
EQep1Regs.QEPCTL.bit.PCRM = 1; //位置计数器复位方式：1-最大值时复位；0-index  
事件触发QPOS CNT 复位  
  
//LJF 0-禁止索引脉冲门控功能  
EQep1Regs.QDECCTL.bit.IGATE = 0; //0-默认，禁止选通index管脚脉冲，需要与AB信  
号相与以保证正转翻转锁存脉冲相同；  
  
//LJF 在下一行指定特定事件（外部提示信号事件，索引事件，软件初始化事件）下用来初始化位置  
计数器  
EQep1Regs.QPOSINIT = gstr_Encoder.g_ZToBetaTheta; //Z信号到来时，加载QPOSINIT寄存  
器值到位置计数器，为当前电角度对应的脉冲数  
//LJF 0-无动作，索引脉冲不初始化位置计数器  
EQep1Regs.QEPCTL.bit.IEI = 0; //保留，Do nothing；先不考虑利用Z信号的上升  
沿来初始化位置计数器（QPOS CNT = QPOSINIT）  
//LJF 1-在QEPI信号上升沿来锁存位置计数器 QPOSILAT <= QPOS CNT  
EQep1Regs.QEPCTL.bit.IEL = 1; //利用Z的下降沿?来锁存位置计数器 QPOSILAT =  
QPOS CNT  
//LJF 使能索引事件锁存【中断】Isr_Zgo() //中断配置，配置Z中断，指示Z何时到来  
EQep1Regs.QEINT.bit.IEL = 1; //使能Index中断  
  
//-----  
//【2=高速M法测频法QEP配置】QEP控制寄存器和UTIME触发锁存配置  
//LJF 【M:固定的T】高速M法测速定周期应该为0.5ms(2kHz)读取位置脉冲计数  
EQep1Regs.QUPRD = gstr_Encoder.g_FIXT-1; //定义单位计时器 for 2KHz at 150?MHz  
SYSCLKOUT，速度环调节周期  
//LJF gep定时器基准单元使能  
EQep1Regs.QEPCTL.bit.UTE = 1; //单位时间计数器使能  
//LJF 在定时器基准单元UTE超时事件（QUTMR=QUPRD）时锁存  
EQep1Regs.QEPCTL.bit.QCLM = 1; //锁存模式：1-UTtimer溢出锁存，0 - CPU读取时  
锁存  
//LJF 使能QEP位置计数器  
EQep1Regs.QEPCTL.bit.QPEN = 1; //QEP 四倍频计数器使能  
  
//-----
```

```
//【3=QEP看门狗】的操作，比较适合变频器，伺服意义不大
EQep1Regs.QWDPDRD = 0xffff;           //QWDPDRD设置四倍频脉冲停止发生的故障时间：
// QWDPDRD * 64 * T_SYSCCLKOUT对应最低转速
//LJF 0-关闭QEP的WD
EQep1Regs.QEPCTL.bit.WDE = 0;          //QEP看门狗使能，防止电机不正常堵转
                                         //调试时先禁止；在正常工作时在启动看门狗使能位，
零速指令时则禁止看门狗
//-----
//【4=低速T法测周法，捕获单元】配置
//LJF【T:固定的x】单元位置事件对位置正交编码QCLK分频，取值为UPEVNT = QCLK/(2^EQEP1_UPPS)
=QCLK/2；低速下T法测速。
EQep1Regs.QCAPCTL.bit.UPPS = EQEP1_UPPS; // 位置计数器的输入脉冲不分频（1-2048）
//LJF【T:计时时钟1.28us】QEP捕获单元定时器时钟分频CAPCLK = SYSCCLKOUT/(2^EQEP1_CCPS)
= 100MHz/128=781250Hz为1.28us
EQep1Regs.QCAPCTL.bit.CCPS = EQEP1_CCPS; // CAP单元时钟分频配置（1-128）
//LJF 1-使能捕获单元
EQep1Regs.QCAPCTL.bit.CEN = 1;          // QEP捕获单元 Enable
}
```

【2】位置脉冲 QEP 配置

暂不使用

伺服电机编码器每转 2500 个脉冲,额定转速 3000r/min（50Hz）；电机与丝杠直连，丝杠螺距为 5mm，伺服驱动器内部电子齿轮比为 1。

计算伺服电机驱动工件在 5S 左右行走 1m 的距离：

首先计算所需要的脉冲数量：

(1).行走 1m 的距离，相当于丝杠转动 $1*1000(\text{mm}) / 5(\text{mm/r}) = 200$ 转；

(2).即伺服电机发出 $2500*200=500000$ （50 万）个脉冲；

(3).二进制的 16 位可以代表“0~65535”（6 万 5 千）或“-32768~+32768”的十进制,即 16 位数据不满足需要的脉冲数；

(4).二进制的 32 位可以代表“0~4294967295”（42 亿 9 千）的十进制,远远大于需要的 50 万个脉冲，即 32 位数据肯定满足需要的脉冲数综上，距离需要 50 万个脉冲即可行走需要的距离。

再确定伺服的行走速度：

(1).伺服电机额定转速 3000r/min（50Hz），即 50r/s；

(2).伺服电机额定发出的脉冲数， $50*2500=125000$ 脉冲/s；

(3).规定 3s 左右发出的脉冲数 $3*125000=375000$ 脉冲/s（37 万个脉冲），即需要超速或延时；

(4).规定 3s 左右完成，则 $500000/3=166667$ 脉冲/s， $166667*50/125000=67\text{Hz}$ ，有点快；

(5).理论最快时间， $500000/125000=4\text{s}$ ，即最快 4s，再加加减速 20%，5~6s 比较合适综上，125000 个脉冲，该距离，用时 5s 左右比较适合。

说明书 P90

信号处理函数

PosPulsCal 到 P_Source 到 PosControl

脉冲位置指令系数

系统时钟频率 100M/脉冲 QEP 模块频率 1k = 100,000, 计数值。

$$\begin{aligned} & \frac{60 * 1000 * \text{脉冲位置} \Delta POSLAT}{4 * 2500(\text{pluse})} * 1000 * 2^{-12} \\ &= \text{脉冲位置} \Delta POSLAT * \frac{10 * 1000 * 1.5 * 2^{12}}{2500(\text{pluse})} * 1000 * 2^{-12} \\ &= \text{脉冲位置} \Delta POSLAT * \frac{1k * 6144}{2500(\text{pluse})} * 10 * 2^{-12} * 100 \end{aligned}$$

AoConstInit 用于显示和 PWM 脉冲

使用（位置）速度前馈 ForebackP

外设配置

计数器使能，AB 不交换

索引事件锁存

单位时间计数器使能

EQep2Regs.QUPRD = EQEP2_QUPRD 设置为 1KHz at 150MHz SYSCLKOUT, 位置环调节周期

QEP 捕捉锁存模式: 1-UTtimer 溢出锁存

EQEP 计数器的最大值, 28bits

位置计数器复位方式为 0-index 事件触发 QPOSCNT 复位, 同时, 上溢或下溢时同样复位。

位置反馈来源于增量式编码器正交解码模块 EQEP1, 在速度环更新反馈值 gstr_Gvar.PosFdb 等于锁存值的差值增量

最后计算给定与反馈的误差

$\text{gstr_Gvar.PosAmplifErr} = \text{gstr_Gvar.PosRef} - \text{gstr_Gvar.PosFdb} + \text{gstr_Gvar.PosLsatErr};$

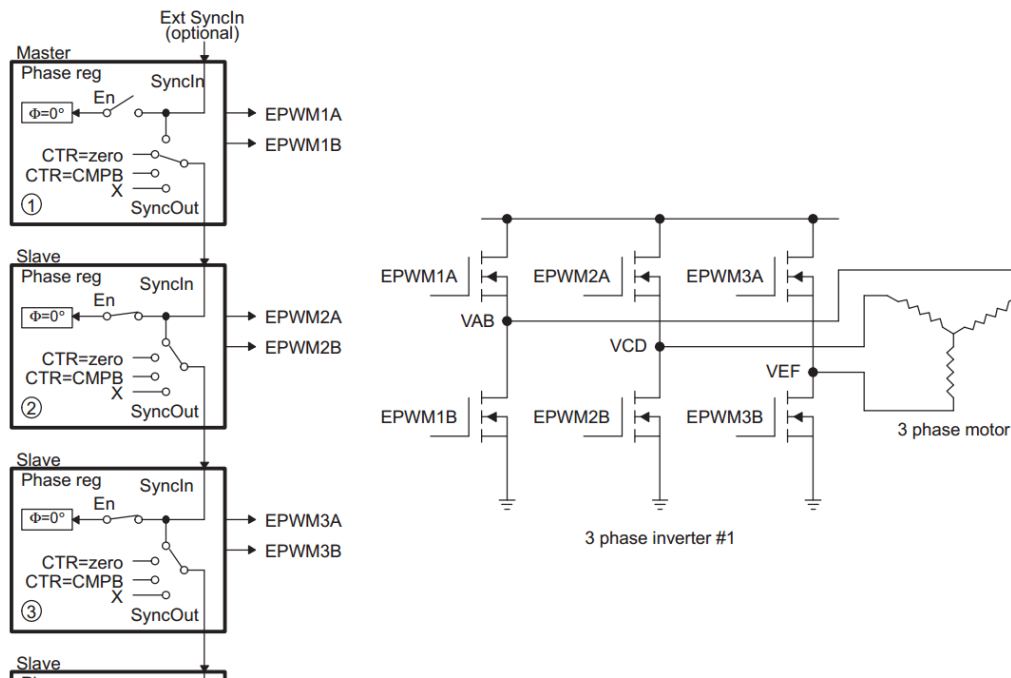
同时也保存此次误差, 最后加入累加的误差里。

【3】PWM 配置

ePWM1,2,3 电机控制

硬件接口及说明

硬件接口 GPIO0,1,2,3,4,5 分别对应 A 相上下桥, B 相上下桥和 C 相上下桥控制信号。具体实现框图如下:

Figure 55. Control of Dual 3-Phase Inverter Stages as Is Commonly Used in Motor Control

主控模块 EPwm1Regs 的配置

TB 子模块:

TBCTL

15-14 不用仿真模式

13 同步事件之后相位增计数

12-10 对来自系统的时钟分频为 1

9-7 对来自系统的时钟分频为 1

6 不触发软同步信号

5-4 当 TBCTR=0 时输出同步信号（主控模式）

3 当 TBCTR=0 时把影子寄存器写入工作 TBPRD 寄存器

2 不把相位 TBPHS 装入 TBCTR

1-0 增减计数

CMP 子模块:

CMPCTL

15:10 保留

9 略

8 略

7 保留

6 比较值 B 使用影子寄存器(激活但不用)

5 保留

4 比较值 A 使用影子寄存器

3:2 当 TBCTR 计数到 0 从 CMPB 影子寄存器装入工作寄存器(激活但不用)

1:0 当 TBCTR 计数到 0 从 CMPA 影子寄存器装入工作寄存器

AQ 模块:

AQCTLA

15:12 保留

11:10 略, 无动作

9:8 略, 无动作

7:6 当 TBCTR 减计数且等于 CMPA 时, 将 EPWMxA 输出置低

5:4 当 TBCTR 增计数且等于 CMPA 时, 将 EPWMxA 输出置高

3:2 略, 无动作

1:0 略, 无动作

DB 模块:

DBCTL

15 DB 模块时钟与 TB 模块时钟一致

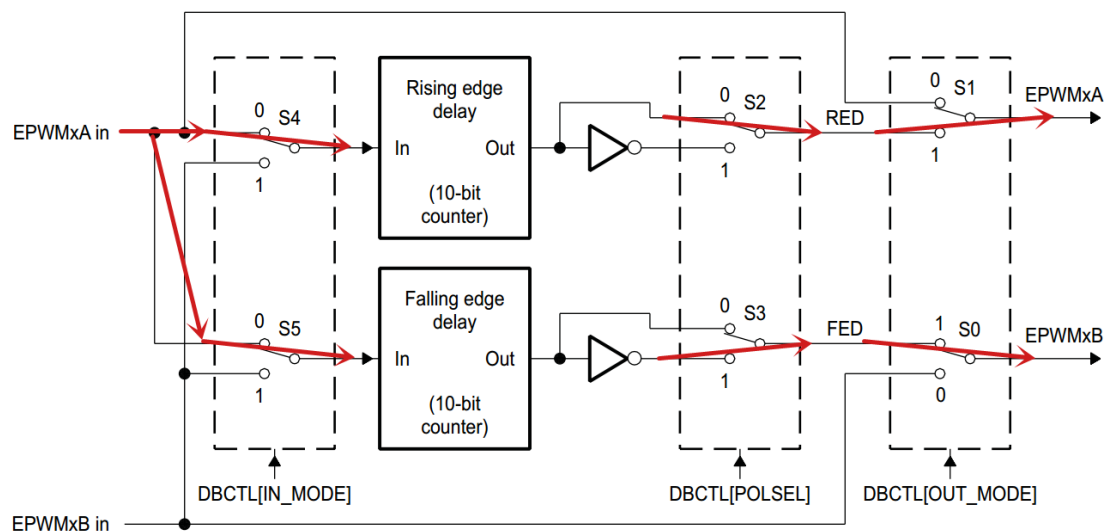
14:6 保留

5:4 只用上一个模块的 EPWMxA 输出作为单相两桥臂的两路控制信号输入源

3:2 低有效, 互补波形。EPWMxA 输出翻转, EPWMxB 输出不翻转; TBD

1:0 上一个模块的输出输入到死区模块处理后输出, 不跳过死区模块

Figure 31. Configuration Options for the Dead-Band Submodule



ET 模块:

ETSEL

15 略, 关闭

14-12 略

11 使能 EPWMxSOCA 脉冲信号

10-8 事件: 当 TBCTR = 0 时将会生成 EPWMxSOCA 脉冲信号(未分频)TBD

7-4 保留

3 生成 EPWM_INT 中断使能

2-0 事件：当 TBCTR = 0 时将会生成 EPWMxINT 中断(未分频)

ETPS

15-14 略

13-12 关闭

11-10 略

9-8 事件发生一次就生成 EPWMxSOCA 脉冲信号

7-4 保留

3-2 略

1-0 事件发生一次就生成 EPWMxINT 中断

ETCLR

15-4 保留

3-0 清除所有事件的标志位

PC 模块关闭

TZ 模块

TZSEL

默认：关闭

8 对该 PWM 模块使能 TZ1 触发 OSHT1(控制开关)

TZCTL

默认：设置为高阻态

3-2 拉高该 PWM 模块的 B 信号 TBD

1-0 拉高该 PWM 模块的 A 信号 TBD

TZEINT

默认：关闭

2 使能生成中断，OST 会触发 EPWMx_TZINT PIE 中断(控制总开关)

TZCLR

15-7 保留

6-0 清除全部触发标志位

TZFRC

默认：略

2 强制触发一次 OST 事件，OST 标志位置位，这里关波等待全局的控制命令

从动模块 EPwm2Regs 和 EPwm3Regs 的配置

TB 子模块:

TBCTL

15-14 不用仿真模式

13 同步事件之后相位增计数

12-10 对来自系统的时钟分频为 1

9-7 对来自系统的时钟分频为 1

6 不触发软同步信号

5-4 输出 EPWMxSYNC:同步信号（从动模式传递信号）

3 当 TBCTR=0 时把影子寄存器写入工作 TBPRD 寄存器

2 把相位 TBPMS 装入 TBCTR

1-0 增减计数

CMP 子模块:**CMPCTL**

15:10 保留

9 略

8 略

7 保留

6 比较值 B 使用影子寄存器(激活但不用)

5 保留

4 比较值 A 使用影子寄存器

3:2 当 TBCTR 计数到 0 从 CMPB 影子寄存器装入工作寄存器(激活但不用)

1:0 当 TBCTR 计数到 0 从 CMPA 影子寄存器装入工作寄存器

AQ 模块:**AQCTLA**

15:12 保留

11:10 略，无动作

9:8 略，无动作

7:6 当 TBCTR 减计数且等于 CMPA 时，将 EPWMxA 输出置低

5:4 当 TBCTR 增计数且等于 CMPA 时，将 EPWMxA 输出置高

3:2 略，无动作

1:0 略，无动作

DB 模块:**DBCTL**

15 DB 模块时钟与 TB 模块时钟一致

14:6 保留

5:4 只上一个模块的 EPWMxA 输出作为单相两桥臂的两路控制信号输入源

3:2 低有效，互补波形。EPWMxA 输出翻转，EPWMxB 输出不翻转

1:0 上一个模块的输出输入到死区模块处理后输出，不跳过死区模块

ET 模块:（2，3 模块不用）

PC 模块关闭

TZ 模块

TZSEL

默认：关闭

8 对该 PWM 模块使能 TZ1 触发 OSHT1(控制开关)

TZCTL

默认：设置为高阻态

3-2 拉高该 PWM 模块的 B 信号

1-0 拉高该 PWM 模块的 A 信号

TZEINT (2, 3 模块不用)

默认：关闭

2 使能生成中断，OST 会触发 EPWMx_TZINT PIE 中断(控制总开关)

TZCLR (2, 3 模块不用)

15-7 保留

6-0 清除全部触发标志位

TZFRC

默认：略

2 强制触发一次 OST 事件，OST 标志位置位。这里关波等待全局的控制命令

外设配置

```
void InitEPwmFullCmp(void)
```

```
{
```

```
//    PWM_Deadband  =2.0*SYSTEM_FREQUENCY;
```

```
    PWM_Deadband  = 2*150;//LJF 死区计数值=2(us)*150(MHz)
```

```
//-----1-----
```

```
//LJF 放到SvDataUpdate初始化
```

```
//    EPwm1Regs.TBPRD = 9375;//6000;
```

```
// 定时器周期 9375=>8k
```

```
EPwm1Regs.TBPHS.half.TBPHS = 0; // 普通和高精度公用一个联合体
```

```
EPwm1Regs.TBCTR = 0x0000; // 定时器计数清零
```

```
EPwm1Regs.CMPA.half.CMPA = EPwm1Regs.TBPRD >> 1;
```

```
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN;
```

```
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE;
```

```
EPwm1Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1;
EPwm1Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO;
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.PHSDIR = 1;

EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO;
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;

EPwm1Regs.AQCTLA.bit.CAU = AQ_SET;//AQ_CLEAR;
EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;//AQ_SET;

EPwm1Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE;
EPwm1Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC;//DB_ACTV_LOC;//LJF 修改的地方，硬件高有效，
OST也要修改为拉低的保护动作
EPwm1Regs.DBCTL.bit.IN_MODE = DBA_ALL;

//LJF 放到SvDataUpdate初始化
// EPwm1Regs.DBRED = PWM_Deadband; //2us
// EPwm1Regs.DBFED = PWM_Deadband;

EPwm1Regs.ETSEL.bit.SOCAEN = 1;/* Enable SOCA */
EPwm1Regs.ETSEL.bit.SOCASEL = ET_CTR_PRD;//ET_CTR_ZERO;//LJF修改的地方/* Enable
period event for SOCA */

EPwm1Regs.ETSEL.bit.INTSEL = ET_CTR_ZERO;
EPwm1Regs.ETSEL.bit.INTEN = 1;
EPwm1Regs.ETPS.bit.INTPRD = ET_1ST;
EPwm1Regs.ETPS.bit.SOCAPRD = ET_1ST;/* Generate SOCA on the 1st event */
EPwm1Regs.ETCLR.bit.SOCA = 1; /* Clear SOCA flag */
EPwm1Regs.ETCLR.all = 0x000F;

EALLOW;
EPwm1Regs.TZSEL.bit.OSHT1 = 1; //enable TZ1 for OSHT
EPwm1Regs.TZCTL.bit.TZA = TZ_FORCE_LO; // EPWMxA will go low硬件高有效，有故障则拉低
EPwm1Regs.TZCTL.bit.TZB = TZ_FORCE_LO; // EPWMxB will go low硬件高有效，有故障则拉低
//LJF 只在EPWM1触发中断
EPwm1Regs.TZEINT.bit.OST = 1;
EPwm1Regs.TZCLR.all = 0xFFFF;
//LJF 初始化完在这里关闭引脚，等待发波命令清除和打开逻辑控制
EPwm1Regs.TZFRC.bit.OST = 0x1; //disabPWM1
EDIS;
```

```
//-----2-----

//LJF 放到SvDataUpdate初始化
//  EPwm2Regs.TBPRD = 9375;//6000;           // 定时器周期

EPwm2Regs.TBPHS.half.TBPHS = 0;//2; // 普通和高精度公用一个联合体//LJF更改
EPwm2Regs.TBCTR = 0x0000;           // 定时器计数清零

EPwm2Regs.CMPA.half.CMPA = EPwm2Regs.TBPRD >> 1;

EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN;
EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE;
EPwm2Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1;
EPwm2Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN;
EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm2Regs.TBCTL.bit.PHSDIR = 1;

EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO;
EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;

EPwm2Regs.AQCTLA.bit.CAU = AQ_SET;//AQ_CLEAR;
EPwm2Regs.AQCTLA.bit.CAD = AQ_CLEAR;//AQ_SET;

EPwm2Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE;
EPwm2Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC;//DB_ACTV_LOC;//LJF 修改的地方
EPwm2Regs.DBCTL.bit.IN_MODE = DBA_ALL;

//LJF 放到SvDataUpdate初始化
//  EPwm2Regs.DBRED = PWM_Deadband;           //2us
//  EPwm2Regs.DBFED = PWM_Deadband;

EALLOW;
EPwm2Regs.TZSEL.bit.OSHT1 = 1; //enable TZ1 for OSHT
EPwm2Regs.TZCTL.bit.TZA = TZ_FORCE_LO; // EPWMxA will go low硬件高有效，有故障则拉低
EPwm2Regs.TZCTL.bit.TZB = TZ_FORCE_LO; // EPWMxB will go low硬件高有效，有故障则拉低
//LJF 初始化完在这里关闭引脚，等待发波命令清除和打开逻辑控制
EPwm2Regs.TZFRC.bit.OST = 0x1;           //disabPWM2
EDIS;
```

```
//-----3-----

//LJF 放到SvDataUpdate初始化
// EPwm3Regs.TBPRD = 9375;//6000; // 定时器周期

EPwm3Regs.TBPHS.half.TBPHS = 0;//2; // 普通和高精度公用一个联合体//LJF更改
EPwm3Regs.TBCTR = 0x0000; // 定时器计数清零

EPwm3Regs.CMPA.half.CMPA = EPwm3Regs.TBPRD >> 1;

EPwm3Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN;
EPwm3Regs.TBCTL.bit.PHSEN = TB_ENABLE;
EPwm3Regs.TBCTL.bit.HSPCLKDIV = TB_DIV1;
EPwm3Regs.TBCTL.bit.CLKDIV = TB_DIV1;
EPwm3Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN;
EPwm3Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm3Regs.TBCTL.bit.PHSDIR = 1;

EPwm3Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO;
EPwm3Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO;

EPwm3Regs.AQCTLA.bit.CAU = AQ_SET;//AQ_CLEAR;
EPwm3Regs.AQCTLA.bit.CAD = AQ_CLEAR;//AQ_SET;

EPwm3Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE;
EPwm3Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC;//DB_ACTV_LOC;//LJF 修改的地方
EPwm3Regs.DBCTL.bit.IN_MODE = DBA_ALL;

//LJF 放到SvDataUpdate初始化
// EPwm3Regs.DBRED = PWM_Deadband; //2us
// EPwm3Regs.DBFED = PWM_Deadband;

EALLOW;
EPwm3Regs.TZSEL.bit.OSHT1 = 1; //enable TZ1 for OSHT
EPwm3Regs.TZCTL.bit.TZA = TZ_FORCE_LO; // EPWMxA will go low硬件高有效，有故障则拉低
EPwm3Regs.TZCTL.bit.TZB = TZ_FORCE_LO; // EPWMxB will go low硬件高有效，有故障则拉低
//LJF 初始化完在这里关闭引脚，等待发波命令清除和打开逻辑控制
EPwm3Regs.TZFRC.bit.OST = 0x1; //disabPWM3
EDIS;

//-----
```

//LJF 初始化外设时钟时把它关闭了，这里配置完PWM把它打开。

EALLOW;

SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1; // Start all the timers syncd

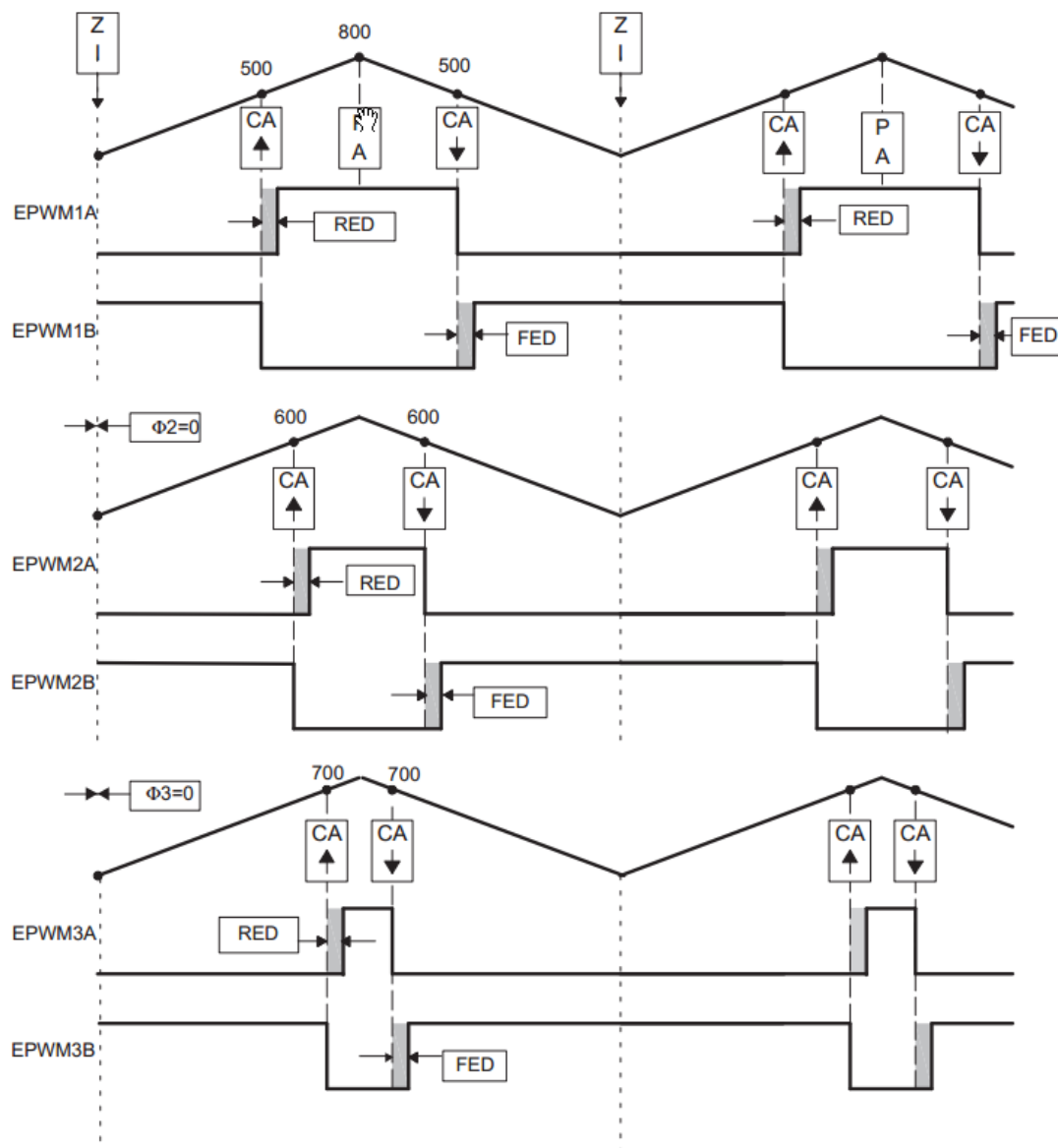
EDIS;

//LJF 初始化完在这里关闭引脚，等待发波命令清除和打开逻辑控制

ClosePwm();

}

Figure 56. 3-Phase Inverter Waveforms for Figure 55 (Only One Inverter Shown)



示例性代码作为参考如下：

Example 12. Code Snippet for Configuration in Figure 55

```

//-----
// Configuration
//-----
// Initialization Time
//-----// EPWM Module 1 config
EPwm1Regs.TBPRD = 800; // Period = 1600 TBCLK counts
EPwm1Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm1Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm1Regs.TBCTL.bit.PHSEN = TB_DISABLE; // Master module
EPwm1Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm1Regs.TBCTL.bit.SYNCSEL = TB_CTR_ZERO; // Sync down-stream module
EPwm1Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm1Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm1Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM1A
EPwm1Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm1Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
EPwm1Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
EPwm1Regs.DBFED = 50; // FED = 50 TBCLKs
EPwm1Regs.DBRED = 50; // RED = 50 TBCLKs
// EPWM Module 2 config
EPwm2Regs.TBPRD = 800; // Period = 1600 TBCLK counts
EPwm2Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm2Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm2Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Slave module
EPwm2Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm2Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
EPwm2Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm2Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm2Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM2A
EPwm2Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm2Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
EPwm2Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
EPwm2Regs.DBFED = 50; // FED = 50 TBCLKs
EPwm2Regs.DBRED = 50; // RED = 50 TBCLKs
// EPWM Module 3 config
EPwm3Regs.TBPRD = 800; // Period = 1600 TBCLK counts
EPwm3Regs.TBPHS.half.TBPHS = 0; // Set Phase register to zero
EPwm3Regs.TBCTL.bit.CTRMODE = TB_COUNT_UPDOWN; // Symmetrical mode
EPwm3Regs.TBCTL.bit.PHSEN = TB_ENABLE; // Slave module
EPwm3Regs.TBCTL.bit.PRDL = TB_SHADOW;
EPwm3Regs.TBCTL.bit.SYNCSEL = TB_SYNC_IN; // sync flow-through
EPwm3Regs.CMPCTL.bit.SHDWAMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.SHDWBMODE = CC_SHADOW;
EPwm3Regs.CMPCTL.bit.LOADAMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm3Regs.CMPCTL.bit.LOADBMODE = CC_CTR_ZERO; // load on CTR=Zero
EPwm3Regs.AQCTLA.bit.CAU = AQ_SET; // set actions for EPWM3A
EPwm3Regs.AQCTLA.bit.CAD = AQ_CLEAR;
EPwm3Regs.DBCTL.bit.OUT_MODE = DB_FULL_ENABLE; // enable Dead-band module
EPwm3Regs.DBCTL.bit.POLSEL = DB_ACTV_HIC; // Active Hi complementary
EPwm3Regs.DBFED = 50; // FED = 50 TBCLKs
EPwm3Regs.DBRED = 50; // RED = 50 TBCLKs
// Run Time (Note: Example execution of one run-time instant)
//-----
EPwm1Regs.CMPA.half.CMPA = 500; // adjust duty for output EPWM1A
EPwm2Regs.CMPA.half.CMPA = 600; // adjust duty for output EPWM2A
EPwm3Regs.CMPA.half.CMPA = 700; // adjust duty for output EPWM3A

```

ePWM4 母线泻放

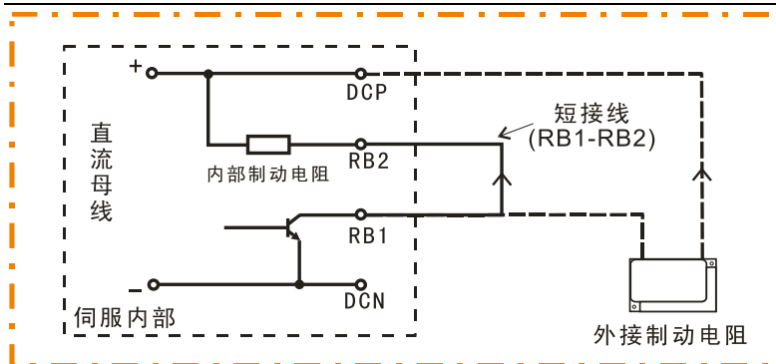
硬件接口及说明

硬件接口为 GPIO6, GPIO7 未使用; 为低电平(TBD)时制动有效。

220V 电压等级设置泻放点为 350V, 过压点为 400V

母线电压采样计算公式为 $\frac{500V * 10^{(参数H01_30)}}{4096} * ADC采样值$, 单位为

0.1V。对应函数为 MonitCal();



根据设定斜率(所有电压放大 10 倍单位为 0.1V, 最大百分比为 95%对应 400V, 最小百分比为 5%对应 350V) 超过泻放点 350V 则按该比例计算占空比:

$$\frac{\text{百分比}-5}{U_{dc}-3500} = \frac{95-5}{4000-3500} \rightarrow \text{百分比} = \frac{95-5}{4000-3500} * (U_{dc}-3500) + 5$$

最后由百分比乘以 TBPRD (赋值为 6250), 对应函数 DcVoltageBrk();

外设配置

TBCLK=150MHz, TBPRD=9375, 那么 PWM 周期为: $(2*9375)/(150*10E6)=125\mu s$, 为 8kHz,

增减计数, 无相位, 周期立即作用, CMP 为影子模式计时器到零装载, 增计数到 CMP 时拉低(制动有效), 到零时拉高。不输出同步信号。

ePWM5 逐波限流

硬件接口及说明

硬件接口为 GPIO8-9, 作用为生成一个硬件限流的模拟电平信号。

用于控制电路领域, 尤其涉及一种变频器及其逐波限流保护电路, 通过在变频器中采用包括比较模块、逻辑模块、判断模块的逐波限流保护电路, 比较模块将硬件限流信号的电平与基准电平进行比较, 根据比较结果生成第一电平信号, 并将第一电平信号输出至逻辑模块; 逻辑模块将接收到的第一电平信号的电平与直流电平信号的电平进行逻辑处理, 生成第二电平信号, 并将第二电平信号输出至判断模块; 判断模块将接收到的第二电平信号与驱动开关信号进行逻辑处理, 生成驱动使能信号, (接入 PWM1 模块的 TZ 引脚 TBD) 控制变频器输出的电流, 从而解决了现有技术中无法解决的变频器逐波限流电路结构复杂、产品可靠性低以及用户体验差的问题。

硬件满输出为 3.3V, 电压系数推导: $\text{CMP 值} / \text{SIGNTBPRD}_{1500} = \text{输出电压} / \text{满输出电压 } 3.3V=3300\text{mV}$

$$\text{CMP 值} = \text{输出电压 mV} * (\text{SIGNTBPRD}_{1500} / \text{满输出电压 } 3300\text{mV})$$

$$\text{那么电压系数 VoltCoe_Q15} = (\text{SIGNTBPRD}_{1500} / \text{满输出电压 } 3300\text{mV}) * 2^{15}$$

最后需要 $\text{Comp} = \text{SIGNTBPRD}_{1500} - (\text{VoltCoe} * \text{Volt} \gg 15)$; 原因是 PWM 模式设置为比较值越大占空比越小, 需要转换。

外设配置

TBCLK=150MHz, TBPRD=1500, 那么 PWM 周期为: $(1*1500)/(150*10E6)=10\mu s$, 为 100kHz, 增计数, 无相位, 周期立即作用, CMP 为影子模式计时器到零装载, 增计数到 CMP 时拉高(输出有效), 到零时拉低。不输出同步信号。

ePWM6 模拟输出

硬件接口及说明

硬件接口 GPIO8 和 9

硬件满输出为 10VTBD, 电压系数推导: $CMP \text{ 值} / SIGNTBPRD_1000 = \text{输出电压} / \text{满输出电压 } 10000mV$

$CMP \text{ 值} = \text{输出电压 } mV * (SIGNTBPRD_1000 / \text{满输出电压 } 10000mV)$

那么电压系数 $VoltCoe_Q15 = (SIGNTBPRD_1000 / \text{满输出电压 } 10000mV) * 2^{15}$

最后需要 $Comp = SIGNTBPRD - (VoltCoe * Volt \gg 15)$; 原因是 PWM 模式设置为比较值越大占空比越小, 需要转换。

外设配置

TBCLK=150MHz, TBPRD=1500, 那么 PWM 周期为: $(1*1500)/(150*10E6)=10\mu s$, 为 100kHz, 增计数, 无相位, 周期立即作用, CMP 为影子模式计时器到零装载, 增计数到 CMP 时拉高(输出有效), 到零时拉低。不输出同步信号。

【4】FPGA 外部接口 XINTF 配置

硬件接口及说明

配置映射空间, 在相应的 CMD 文件添加一些定义的寄存器, 这里为 Zone7。初始地址为 0x200000, 其长度大小为 1M*16。

示例如下:

MEMORY

{

PAGE 0: /* Program Memory */

PAGE 1: /* Data Memory */

.....

/* external interface registers */

FPGACODE : origin = 0x200000, length = 0x00000C /* FPGA registers */

FPGAADCMD : origin = 0x20000C, length = 0x000002 /* FPGA registers */

FPGAADFDB : origin = 0x20000E, length = 0x000004 /* FPGA registers */

```

    FPGAPOSC    : origin = 0x200012, length = 0x000007    /* FPGA registers */
.....
}

```

SECTIONS

```

{
.....
/** FPGA Data Register Structures */
    FPGACodeRegsFile : > FPGACODE,    PAGE = 1
    FPGAPosCRegsFile : > FPGAPOSC,    PAGE = 1
    FPGAAdCmdRegFile : > FPGAADCMD,    PAGE = 1
    FPGAAdFdbRegFile : > FPGAADFDB,    PAGE = 1
.....
}

```

不采用 XREADY 信号的同步采样，模块时钟 XTIMCLK = SYSCLKOUT/2。

外设配置

```
void InitXintf(void)
```

```

{
    // This shows how to write to the XINTF registers. The
    // values used here are the default state after reset.
    // Different hardware will require a different configuration.

    // For an example of an XINTF configuration used with the
    // F28335 eZdsp, refer to the examples/run_from_xintf project.

    // Any changes to XINTF timing should only be made by code
    // running outside of the XINTF.

    // All Zones-----
    // Timing for all zones based on XTIMCLK = 1/2 SYSCLKOUT
    // Make sure the XINTF clock is enabled
    SysCtrlRegs.PCLKCR3.bit.XINTFENCLK = 1;

    // Configure the GPIO for XINTF with a 16-bit data bus
    // This function is in DSP2833x_Xintf.c
    EALLOW;
    GpioCtrlRegs.GPCMUX1.bit.GPIO64 = 3; // XD15
    GpioCtrlRegs.GPCMUX1.bit.GPIO65 = 3; // XD14
    GpioCtrlRegs.GPCMUX1.bit.GPIO66 = 3; // XD13
    GpioCtrlRegs.GPCMUX1.bit.GPIO67 = 3; // XD12

```

```
GpioCtrlRegs.GPCMUX1.bit.GPIO68 = 3; // XD11
GpioCtrlRegs.GPCMUX1.bit.GPIO69 = 3; // XD10
GpioCtrlRegs.GPCMUX1.bit.GPIO70 = 3; // XD19
GpioCtrlRegs.GPCMUX1.bit.GPIO71 = 3; // XD8
GpioCtrlRegs.GPCMUX1.bit.GPIO72 = 3; // XD7
GpioCtrlRegs.GPCMUX1.bit.GPIO73 = 3; // XD6
GpioCtrlRegs.GPCMUX1.bit.GPIO74 = 3; // XD5
GpioCtrlRegs.GPCMUX1.bit.GPIO75 = 3; // XD4
GpioCtrlRegs.GPCMUX1.bit.GPIO76 = 3; // XD3
GpioCtrlRegs.GPCMUX1.bit.GPIO77 = 3; // XD2
GpioCtrlRegs.GPCMUX1.bit.GPIO78 = 3; // XD1
GpioCtrlRegs.GPCMUX1.bit.GPIO79 = 3; // XD0

GpioCtrlRegs.GPBMUX1.bit.GPIO40 = 3; // XA0/XWE1n
GpioCtrlRegs.GPBMUX1.bit.GPIO41 = 3; // XA1
GpioCtrlRegs.GPBMUX1.bit.GPIO42 = 3; // XA2
GpioCtrlRegs.GPBMUX1.bit.GPIO43 = 3; // XA3
GpioCtrlRegs.GPBMUX1.bit.GPIO44 = 3; // XA4
GpioCtrlRegs.GPBMUX1.bit.GPIO45 = 3; // XA5
GpioCtrlRegs.GPBMUX1.bit.GPIO46 = 3; // XA6
GpioCtrlRegs.GPBMUX1.bit.GPIO47 = 3; // XA7

GpioCtrlRegs.GPCMUX2.bit.GPIO80 = 3; // XA8
GpioCtrlRegs.GPCMUX2.bit.GPIO81 = 3; // XA9
GpioCtrlRegs.GPCMUX2.bit.GPIO82 = 3; // XA10
GpioCtrlRegs.GPCMUX2.bit.GPIO83 = 3; // XA11
GpioCtrlRegs.GPCMUX2.bit.GPIO84 = 3; // XA12
GpioCtrlRegs.GPCMUX2.bit.GPIO85 = 3; // XA13
GpioCtrlRegs.GPCMUX2.bit.GPIO86 = 3; // XA14
GpioCtrlRegs.GPCMUX2.bit.GPIO87 = 3; // XA15
//GpioCtrlRegs.GPBMUX1.bit.GPIO39 = 3; // XA16
//GpioCtrlRegs.GPAMUX2.bit.GPIO31 = 3; // XA17
//GpioCtrlRegs.GPAMUX2.bit.GPIO30 = 3; // XA18
//GpioCtrlRegs.GPAMUX2.bit.GPIO29 = 3; // XA19

GpioCtrlRegs.GPBMUX1.bit.GPIO34 = 3; // XREADY
GpioCtrlRegs.GPBMUX1.bit.GPIO35 = 3; // XRNW
GpioCtrlRegs.GPBMUX1.bit.GPIO38 = 3; // XWE0

//GpioCtrlRegs.GPBMUX1.bit.GPIO36 = 3; // XZCS0
GpioCtrlRegs.GPBMUX1.bit.GPIO37 = 3; // XZCS7
GpioCtrlRegs.GPAMUX2.bit.GPIO28 = 3; // XZCS6
EDIS;
```

```
EALLOW;
// All Zones-----
// Timing for all zones based on XTIMCLK = SYSCLKOUT
XintfRegs.XINTCNF2.bit.XTIMCLK = XTimClk;
// Buffer up to 3 writes
XintfRegs.XINTCNF2.bit.WRBUFF = 0; //3;
// XCLKOUT is enabled
XintfRegs.XINTCNF2.bit.CLKOFF = 0;
// XCLKOUT = XTIMCLK
XintfRegs.XINTCNF2.bit.CLKMODE = ClkMode;

// Zone 7-----
// When using ready, ACTIVE must be 1 or greater
// Lead must always be 1 or greater
// Zone write timing
XintfRegs.XTIMING7.bit.XWRLEAD = 3;
XintfRegs.XTIMING7.bit.XWRACTIVE = 7;
XintfRegs.XTIMING7.bit.XWRTRAIL = 3;
// Zone read timing
XintfRegs.XTIMING7.bit.XRDLEAD = 3;
XintfRegs.XTIMING7.bit.XRDACTIVE = 7;
XintfRegs.XTIMING7.bit.XRDTRAIL = 3;

// don't double all Zone read/write lead/active/trail timing
XintfRegs.XTIMING7.bit.X2TIMING = 1;

// Zone will not sample XREADY signal
XintfRegs.XTIMING7.bit.USEREADY = 0;
XintfRegs.XTIMING7.bit.READYMODE = 0;

// 1,1 = x16 data bus
// 0,1 = x32 data bus
// other values are reserved
XintfRegs.XTIMING7.bit.XSIZE = 3;
EDIS;

//Force a pipeline flush to ensure that the write to
//the last register configured occurs before returning.
asm(" RPT #7 || NOP");
}
```

【5】CAN 通讯

硬件接口及说明

硬件接口为 GPIO12,GPIO13，分别对应 CANTX 和 CANRX。使用的是 CAN-B 模块

系统时钟为 SYSCLK/2(参考 SPRUFB0D 文献 P33)

若无特殊要求，则为了适应上位机配置频率为 250k。

需要修改时置位 CCR 等待 CCE 允许后方可修改，最后波特率的配置如下：

表 35. 比特率 (BT = 15, TSEG1_{reg}=10, TSEG2_{reg}=2, 采样点 = 80%) 的 BRP 字段

CAN 总线速度	BRP	CAN 模块时钟
1Mbps	BRP _{reg} +1=5	15MHz
500kbps	BRP _{reg} +1=10	7.5MHz
250kbps	BRP _{reg} +1=20	3.75MHz
125kbps	BRP _{reg} +1=40	1.875MHz
100kbps	BRP _{reg} +1=50	1.5MHz
50kbps	BRP _{reg} +1=100	0.75MHz

表 36显示了如何实现与一个 15 BT 的不同采样点。

表 36. 实现与一个 15 BT 的不同采样点

TSEG1 _{reg}	TSEG2 _{reg}	SP
10	2	80%
9	3	73%
8	4	66%
7	5	60%

另外，控制和状态寄存器只允许 32 位访问。到这些寄存器的 16 位访问可能会破坏该寄存器的内容或返回错误数据 TI 发布的 DSP 头文件采用了一个有助于 32 位访问的影子寄存器结构。采用影子寄存器结构来执行 32 位读取和写入。

外设配置

```
void InitECanB(void)           // Initialize eCAN-B module
{

/* Create a shadow register structure for the CAN control registers. This is
needed, since only 32-bit access is allowed to these registers. 16-bit access
to these registers could potentially corrupt the register contents or return
false data. */

struct ECAN_REGS ECanbShadow;

EALLOW;           // EALLOW enables access to protected bits

//运行过程中初始化时先中断发送命令-----
ECanbRegs.CANTRR.all = 0xFFFFFFFF;
```

```
/* Configure eCAN RX and TX pins for CAN operation using eCAN regs*/

ECanbShadow.CANTIOC.all = ECanbRegs.CANTIOC.all;
ECanbShadow.CANTIOC.bit.TXFUNC = 1;
ECanbRegs.CANTIOC.all = ECanbShadow.CANTIOC.all;

ECanbShadow.CANRIOC.all = ECanbRegs.CANRIOC.all;
ECanbShadow.CANRIOC.bit.RXFUNC = 1;
ECanbRegs.CANRIOC.all = ECanbShadow.CANRIOC.all;

/* Configure eCAN for HECC mode - (reqd to access mailboxes 16 thru 31) */
// HECC mode also enables time-stamping feature
ECanbShadow.CANMC.all = ECanbRegs.CANMC.all;
//LJF 新加 1 After the bus-off state, the module goes back automatically into bus-on state
after 128 * 11
ECanbShadow.CANMC.bit.ABO = 1; //自动恢复
ECanbShadow.CANMC.bit.SCB = 1;
ECanbShadow.CANMC.bit.DBO = 1;
ECanbRegs.CANMC.all = ECanbShadow.CANMC.all;

/* Initialize all bits of 'Message Control Register' to zero */
// Some bits of MSGCTRL register come up in an unknown state. For proper operation,
// all bits (including reserved bits) of MSGCTRL must be initialized to zero

ECanbMboxes.MBOX0.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX1.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX2.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX3.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX4.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX5.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX6.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX7.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX8.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX9.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX10.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX11.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX12.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX13.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX14.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX15.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX16.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX17.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX18.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX19.MSGCTRL.all = 0x00000000;
```

```
ECanbMboxes.MBOX20.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX21.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX22.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX23.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX24.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX25.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX26.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX27.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX28.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX29.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX30.MSGCTRL.all = 0x00000000;
ECanbMboxes.MBOX31.MSGCTRL.all = 0x00000000;

// TAn, RMPn, GIFn bits are all zero upon reset and are cleared again
// as a matter of precaution.

ECanbRegs.CANTA.all = 0xFFFFFFFF; /* Clear all TAn bits */

ECanbRegs.CANRMP.all = 0xFFFFFFFF; /* Clear all RMPn bits */

ECanbRegs.CANGIF0.all = 0xFFFFFFFF; /* Clear all interrupt flag bits */
ECanbRegs.CANGIF1.all = 0xFFFFFFFF;

/* Configure bit timing parameters for eCANa*/

ECanbShadow.CANMC.all = ECanbRegs.CANMC.all;
ECanbShadow.CANMC.bit.CCR = 1 ;          // Set CCR = 1
ECanbRegs.CANMC.all = ECanbShadow.CANMC.all;

// Wait until the CPU has been granted permission to change the configuration registers
do
{
    ECanbShadow.CANES.all = ECanbRegs.CANES.all;
} while( ECanbShadow.CANES.bit.CCE != 1 );    // Wait for CCE bit to be set..

ECanbShadow.CANBTC.all = 0;

//LJF 配置频率配合上位机为250k = 75M/[( 1+19 )*( 2+1 + 10+1 +1 )]
ECanbShadow.CANBTC.bit.BRPREG = 19;    // (BRPREG + 1) = 10 feeds a 15 MHz CAN clock
ECanbShadow.CANBTC.bit.TSEG2REG = 2 ; // to the CAN module. (150 / 10 = 15)
ECanbShadow.CANBTC.bit.TSEG1REG = 10;  // Bit time = 15=5+1+7+1+1
```

```
ECanbShadow.CANBTC.bit.SAM = 1;

ECanbRegs.CANBTC.all = ECanbShadow.CANBTC.all;

//=====

ECanbRegs.CANME.all = 0;          // Required before writing the MSGIDs

ECanbRegs.CANGAM.all = 0x80000000;

//0x40000000: 标准帧; 0xC0000000: 扩展帧
ECanbMboxes.MBOX0.MSGID.all = 0xC0000000; //0x40000000;
ECanbMboxes.MBOX0.MSGCTRL.all = 0x00000008;
ECanbLAMRegs.LAM0.all = 0x9FFFFFFF;

ECanbMboxes.MBOX1.MSGID.all = 0x00040000;
ECanbMboxes.MBOX1.MSGCTRL.all = 0x00000008;

ECanbShadow.CANMD.all = 0x00000001;
ECanbRegs.CANMD.all = ECanbShadow.CANMD.all;

ECanbShadow.CANME.all = 0x00000003;
ECanbRegs.CANME.all = ECanbShadow.CANME.all;

//=====

ECanbShadow.CANMC.all = ECanbRegs.CANMC.all;
ECanbShadow.CANMC.bit.CCR = 0 ;          // Set CCR = 0
ECanbRegs.CANMC.all = ECanbShadow.CANMC.all;

// Wait until the CPU no longer has permission to change the configuration registers
do
{
    ECanbShadow.CANES.all = ECanbRegs.CANES.all;
} while(ECanbShadow.CANES.bit.CCE != 0 );          // Wait for CCE bit to be cleared..

/* Disable all Mailboxes */
//    ECanbRegs.CANME.all = 0;          // Required before writing the MSGIDs
```

EDIS;

```
CanMboxUserRegsR1.MboxRxFlg = 0;
CanMboxUserRegsR1.MboxNum = 0;
CanMboxUserRegsR1.MboxDLC = 0;
CanMboxUserRegsR1.MboxID = 0;
CanMboxUserRegsR1.MboxWord1 = 0;
CanMboxUserRegsR1.MboxWord2 = 0;
CanMboxUserRegsR1.MboxWord3 = 0;
CanMboxUserRegsR1.MboxWord4 = 0;
CanMboxUserRegsR2.MboxRxFlg = 0;
CanMboxUserRegsR2.MboxNum = 0;
CanMboxUserRegsR2.MboxDLC = 0;
CanMboxUserRegsR2.MboxID = 0;
CanMboxUserRegsR2.MboxWord1 = 0;
CanMboxUserRegsR2.MboxWord2 = 0;
CanMboxUserRegsR2.MboxWord3 = 0;
CanMboxUserRegsR2.MboxWord4 = 0;
```

```
CanMboxUserRegsT.MboxRxFlg = 0;
CanMboxUserRegsT.MboxNum = 0;
CanMboxUserRegsT.MboxDLC = 0;
CanMboxUserRegsT.MboxID = 0;
CanMboxUserRegsT.MboxWord1 = 0;
CanMboxUserRegsT.MboxWord2 = 0;
CanMboxUserRegsT.MboxWord3 = 0;
CanMboxUserRegsT.MboxWord4 = 0;
```

```
}
```

【6】模拟量采样 ADC

硬件接口及说明

ADCINA0/B1	输出电流 A 相
ADCINA1/B0	输出电流 B 相
ADCINA2	母线电压
ADCINA3	散热器温度
ADCINA4	模拟控制输入端口
ADCINA5	电机温度检测端口
ADCINA6	IPM 模块温度信号

ADCINA7	环境温度检测
---------	--------

在对应的端口，软件的接口改为如下：

ADCINA0=0	ADCINB0=8
ADCINA1=1	ADCINB1=9
ADCINA2=2	ADCINB2=10
ADCINA3=3	ADCINB3=11
ADCINA4=4	ADCINB4=12
ADCINA5=5	ADCINB5=13
ADCINA6=6	ADCINB6=14
ADCINA7=7	ADCINB7=15

待定：

```
ChSel[0]=1;    // Dummy meas. avoid 1st sample issue Rev0 Picollo*/
ChSel[1]=1;    // ChSelect: ADC A1-> DC Bus Current
ChSel[2]=9;    // ChSelect: ADC B1-> Phase A Current
ChSel[3]=8;    // ChSelect: ADC B0-> Phase B Current
ChSel[4]=0;    // ChSelect: ADC A0-> DC Bus Voltage
ChSel[5]=2;    // ChSelect: ADC A2-> RP3
ChSel[6]=3;    // ChSelect: ADC A3-> RP2
ChSel[7]=10;   // ChSelect: ADC B2-> Phase A Voltage
ChSel[8]=11;   // ChSelect: ADC B3-> Phase B Voltage
ChSel[9]=12;   // ChSelect: ADC B4-> Phase C Voltage
```

外设配置

初始化代码如下，已经配置成 EPWM1 模块生成的 SOCA 信号触发采样

void InitAdcSmp(void)

```
{
    //LJF 在这里配置ADC采样通道
    ChSel[0]=1;    // Dummy meas. avoid 1st sample issue Rev0 Picollo*/
    ChSel[1]=1;    // ChSelect: ADC A1-> DC Bus Current
    ChSel[2]=9;    // ChSelect: ADC B1-> Phase A Current
    ChSel[3]=8;    // ChSelect: ADC B0-> Phase B Current
    ChSel[4]=0;    // ChSelect: ADC A0-> DC Bus Voltage
    ChSel[5]=2;    // ChSelect: ADC A2-> RP3
    ChSel[6]=3;    // ChSelect: ADC A3-> RP2
    ChSel[7]=10;   // ChSelect: ADC B2-> Phase A Voltage
    ChSel[8]=11;   // ChSelect: ADC B3-> Phase B Voltage
    ChSel[9]=12;   // ChSelect: ADC B4-> Phase C Voltage
    //LJF 新添加待测试
    ChSel[10]=9;   // ChSelect: ADC B1-> Phase A Current
    ChSel[11]=8;   // ChSelect: ADC B0-> Phase B Current
```

```
ALLOW;
SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 1;
ADC_cal();
EDIS;

AdcRegs.ADCTRL3.all = 0x00E0; /* Power up bandgap/reference/ADC circuits*/
DELAY_US(ADC_usDELAY); /* Delay before converting ADC channels*/

//LJF 相对于ADCCLK采样保持长度，为8+1个时钟
AdcRegs.ADCTRL1.bit.ACQ_PS = ACQPS[0];
//LJF 对高速时钟分频Fclk=HSPCLK=75MHz;HSPCLK is directly passed on in the case when
ADCCLKPS[3-0] is 0000,
AdcRegs.ADCTRL3.bit.ADCCLKPS = 0;
//LJF 再次分频作为最后的ADC时钟ADCCLK = Fclk/2=37.5MHz
AdcRegs.ADCTRL1.bit.CPS = 1;
//LJF 双序列
AdcRegs.ADCTRL1.bit.SEQ_CASC = 0; /* 0x0 Dual Sequencer Mode, 0x1 Cascaded
Mode*/
//LJF 0=Interrupt request by INT_SEQ1 is disabled
AdcRegs.ADCTRL2.bit.INT_ENA_SEQ1 = 0x0;
//LJF Immediately reset sequencer to state CONV00
AdcRegs.ADCTRL2.bit.RST_SEQ1 = 0x1;
//LJF Immediately resets SEQ2 to an initial "pretriggered" state
AdcRegs.ADCTRL2.bit.RST_SEQ2 = 0x1;
/* enable SOC from EPWMA trigger*/
AdcRegs.ADCTRL2.bit.EPWM_SOCA_SEQ1=0x1;

AdcRegs.ADCCHSELSEQ1.bit.CONV00 = ChSel[0];
AdcRegs.ADCCHSELSEQ1.bit.CONV01 = ChSel[1];
AdcRegs.ADCCHSELSEQ1.bit.CONV02 = ChSel[2];
AdcRegs.ADCCHSELSEQ1.bit.CONV03 = ChSel[3];
AdcRegs.ADCCHSELSEQ2.bit.CONV04 = ChSel[4];
AdcRegs.ADCCHSELSEQ2.bit.CONV05 = ChSel[5];
AdcRegs.ADCCHSELSEQ2.bit.CONV06 = ChSel[6];
AdcRegs.ADCCHSELSEQ2.bit.CONV07 = ChSel[7];
AdcRegs.ADCCHSELSEQ3.bit.CONV08 = ChSel[8];
AdcRegs.ADCCHSELSEQ3.bit.CONV09 = ChSel[9];
//LJF 新添加待测试
AdcRegs.ADCCHSELSEQ3.bit.CONV10 = ChSel[10];
AdcRegs.ADCCHSELSEQ3.bit.CONV11 = ChSel[11];

//LJF MAX_CONV1 Value >7 for Dual-Sequencer Mode
```

```
    AdcRegs.ADCMAXCONV.bit.MAX_CONV1 = 9;
    EDIS;
//
//    //LJF 已经在PWM模块中设置
//    /* Set up Event Trigger with CNT_zero enable for Time-base of EPWM1 */
//    EPwm1Regs.ETSEL.bit.SOCAEN = 1;    /* Enable SOCA */
//    EPwm1Regs.ETSEL.bit.SOCASEL = 2;    /* Enable period event for SOCA */
//    EPwm1Regs.ETPS.bit.SOCAPRD = 1;    /* Generate SOCA on the 1st event */
//    EPwm1Regs.ETCLR.bit.SOCA = 1;    /* Clear SOCA flag */

}
```

功能函数

函数: void InitAdcSmp(void)

描述: 初始化 ADC 的配置, 在主函数中调用。设置成 EPWM1 模块生成的 SOCA 信号触发采样。配置相应的通道口。

函数: void AdcSample(void)

描述: 在主中断中调用。从 ADC 的镜像结果寄存器中读取右对齐的 12 位转换结果, 赋值到相应的 AdcResultStr 结构体上。可加一定的延时或校正, 保证一定的准确性。

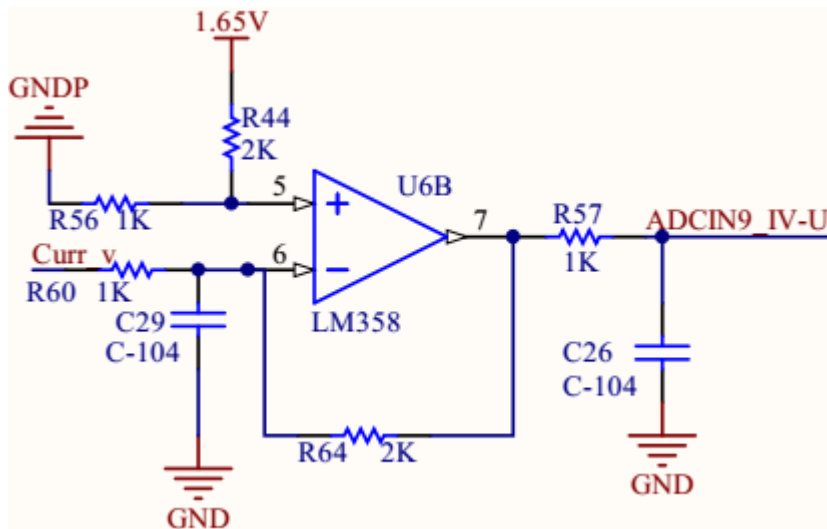
函数: void CurrentDrift(void)

描述: 在主中断发波前调用。因为三相交流永磁同步电机的定子电流分正负极性, 在发波前需要校准零电流是对应的端口电压和 ADC 数值, 作为电流采样的零点。

函数: void AdcDataInpput(void)

描述: 转化为 Q15 格式标么值, 基值需要标定。

==>电流: 原来 ADC 为 12 位, 单极性就为 Q12 格式。然而电流是双极性, 减去电流采样零点之后认为是 Q11 格式, 需要左移 4 位转为 Q15 格式。



#define CURRENT_SENSOR 8 //LJF 开发板双电阻采样，等式为 $U_{pin}=0.2I+1.65$ ，输入范围 0~3V，使用范围 0.3~3V 对称电流为 $\pm 6.75A$ ；倘若为 0~3.3V 则为 $\pm 8.25A$ 相电流 2018.11
//32767//这里作为标么化用

采样电阻为 0.1 欧姆，Curr 电压为 $0.1 \cdot i_{phase}$ ，根据运放的虚短，虚断可得：

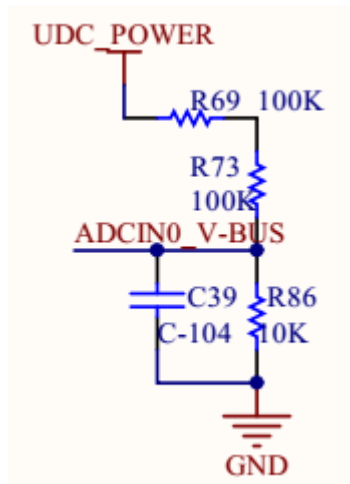
$$(U_{ADC} - 1.65 \cdot \frac{1K}{2K + 1K}) \cdot \frac{1}{2K} = (1.65 \cdot \frac{1K}{2K + 1K} - Curr) \cdot \frac{1}{1K}$$

$$U_{ADC} = -Curr \cdot 2 + 1.65 \cdot (\frac{1K}{2K + 1K} + \frac{1K \cdot 2}{2K + 1K})$$

$$U_{ADC} = -0.2i_{phase} + 1.65$$

硬件 U_{ADC} 范围 0.3~3V 对应电流 -8.25A~6.75A, 由于交流对称性则为 $\pm 6.75A$ ；软件取 6A 电流，再做校准。

=>电压：单极性。原来 ADC 为 12 位，为 Q12 格式。需要左移 3 位转为 Q15 格式。



ADC 端口电压满量程 3V

$$U_{ADC} = U_{DC} \frac{10K}{100K + 100K + 10K} = U_{DC} \frac{1}{21}$$

$$U_{DCMax} = U_{ADCMax} * 21 = 3 * 21 = 63(V)$$

在全局宏定义的时候，稍微放宽一点到 70V

```
#define VOLTAGE_SENSOR 70
```

最后可以细微的调整实际采样值

//LJF 母线电压校准补偿 UdcComArr 的 x 值为 ADC 裸值<<3, y 值为 $2^{15} * (y2 \text{ 值设为 } 0 \text{ 时显示的电压-实际电压}) / \text{电压量程}$

```
int16 UdcComArr[][2] = {{0,0},{12480,936}};
```

例如先设置 $y2=0$ ，取消补偿。外部输入 24V 时，ADC 裸值<<3 为 12480，此时软件显示的电压为 26V，那么 $y2$ 值设置为 $2^{15} * (26 - 24 / 70) = 936$

【备注】

配合 FPGA 完成测速度，测位置的功能。

通讯接口 SPI，SCI，I2C 暂不说明

【1】编码器 QEP 配置

硬件接口及说明

GPIO50,51,53 对应 A,B,Z 信号。GPIO52 暂时不用做 QEPS 提示信号脚，定义为 2kHz 采样，与速度环频率一致。

电角度的计算

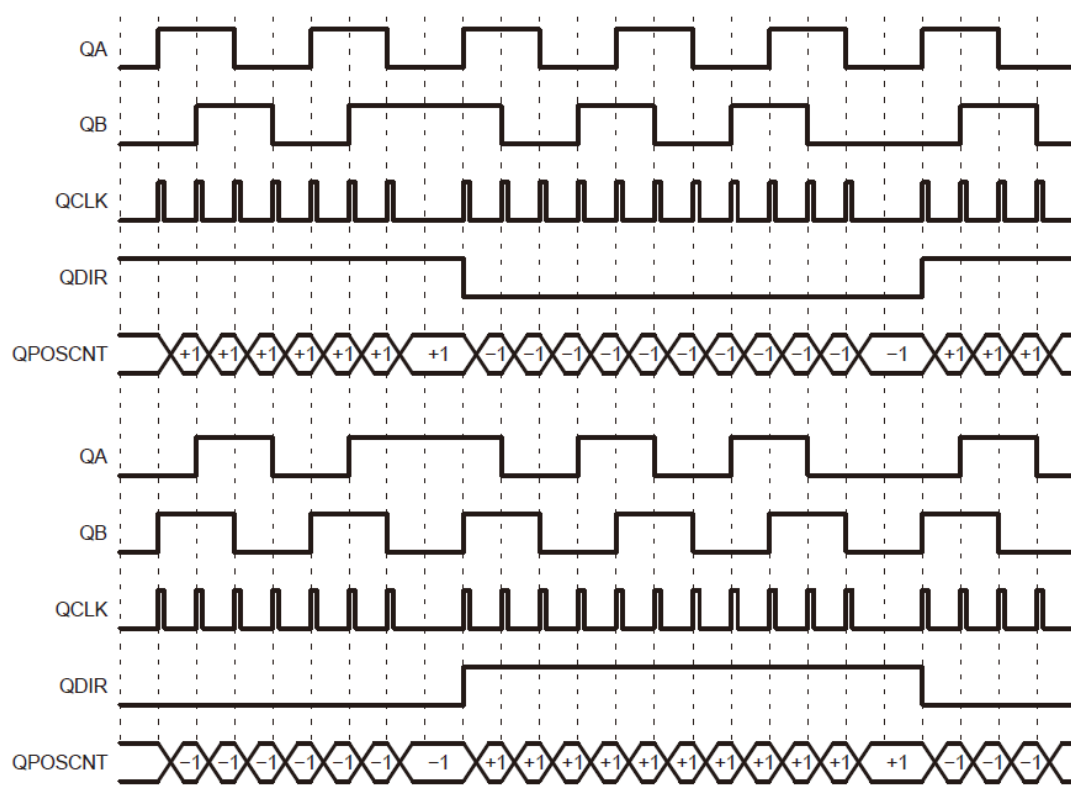
一般的旋转编码器本身只能提供相对位置信息，不过配合零点信号（即 Z 相信号）即可确定绝对位置，下面结合 TI C2000 系列单片机来讨论具体实现方法。

基本原理

在 C2000 系列单片机中，使用 EQEP 模块对旋转编码器输入信号进行处理，先根据输入的 A、B、Z 三相信号生成 QCLK 内部时钟信号、QDIR 方向信号及 QI 同步信号，这部分工作是由 QDU（Quadrature Decoder Unit）子模块完成的。输入信号的极性，输出的来源等均可配置，具体详见数据手册。

生成的 QCLK 信号来源于 A、B 两相信号跳变沿，此信号与 QDIR 方向信号一起确定 QPOSCNT 寄存器的计数增减，具体时序图如下：

Figure 7. Quadrature-clock and Direction Decoding



从理论上说，只要确定了一点的绝对位置，就可以通过读取 QPOSCNT 寄存器获取任意时刻的绝对位置，假设 QPOSCNT 的初值是 0，电机完整的顺时针转过一圈后 QPOSCNT 中的值应该是 $4 * \text{MOTOR_LINE}$ 。在 C2000 单片机中，可以通过 QPOSMAX 寄存器来设置这个最大值，示例代码如下：

```
EQep1Regs.QEPCTL.bit.PCRM = 1; // 1: QPOSCNT reset on the maximum position
EQep1Regs.QPOSMAX = 4 * MOTOR_LINE - 1; // Set the maximum position
```

然而，这有两个问题，首先是初始时刻的绝对位置如何确定；其次是如何消除累积误差。解决方法是：初始时刻的绝对位置通过霍尔 UVW 信号进行估测，累积误差通过 Z 相同步信号来消除(暂时不使用)。下面分别来讨论这两个问题。

初始时刻定位

初始时刻的位置确定比较简单，只要预先测定出 Ha、Hb、Hc 三个霍尔信号输出与绝对角度间的关系，在启动时通过查表的方法即可大致估算出初始角度。因为这一估算过程本来就不要求也不可能精确，故也无需测定霍尔信号跳变沿的准确位置，粗略的使用示波器观察信号估计一下即可。

示例性代码如下：

```
EQep1Regs.QPOSCNT = GetHall();
```

这句代码在初始化 EQEP 模块的时候调用一次，GetHall()函数可返回根据 Hall 信号猜测出的 QPOSCNT 寄存器的初值：

```
// Hall signal delt
```

```
#define HALL_DELT (QEP_LINE / 3)
```

```
// Hall signal offset
```

```
#define HALLOFFSET (HALL_DELT / 2)
```

```
Uint32 GetHall(void)
{
    Uint16 Ha, Hb, Hc;
    Uint16 HallSignal;

    // 输入斯密特触发器处进行了取反
    // 此处再次取反获取原信号
    Ha = ~GpioDataRegs.GPADAT.bit.GPIO9 & 0x01;
    Hb = ~GpioDataRegs.GPADAT.bit.GPIO23 & 0x01;
    Hc = ~GpioDataRegs.GPADAT.bit.GPIO10 & 0x01;

    HallSignal = (Ha << 2) + (Hb << 1) + Hc;

    switch (HallSignal)
    {
        case 0x00: // 000
            return HALLOFFSET + 0;

        case 0x01: // 001
            return HALLOFFSET + HALL_DELT;

        case 0x03: // 011
            return HALLOFFSET + 2 * HALL_DELT;

        case 0x07: // 111
            return HALLOFFSET + 3 * HALL_DELT;

        case 0x06: // 110
            return HALLOFFSET + 4 * HALL_DELT;

        case 0x04: // 100
            return HALLOFFSET + 5 * HALL_DELT;

        default:
            __asm ("      ESTOP0");
    }

    return 0;
}
```

z 相同步信号

Z 相信号用于对 QPOSCNT 寄存器进行同步定位，这个信号需要仔细考虑下，否则会造成之后得到的绝对位置存在误差。首先需要考虑的是，Z 相信号本身是一个存在一定宽度的脉冲信号，从旋转编码器的原理上来看，正转和反转时 Z 相信号的上升沿和下降沿对应的绝对位置是不同的，正转时的上升沿对应反转时的下降沿，反之亦然。这就需要在正转和反转时使用不同的边沿触发同步事件，C2000 单片机中可以很方便的实现这一功能，只需要设置 IEI 寄存器即可：

```
EQep1Regs.QEPCTL.bit.IEI = 3; // 0 : Does nothing
                                // 2 : Initializes QPOSCNT on rising edge of QEPI
                                // 3 : Initializes QPOSCNT on rising or falling edge of
```

QEPI depends on direction

同步事件发生时写入 QPOSCNT 中的值由 QPOSINIT 寄存器确定：

```
EQep1Regs.QPOSINIT = INDEX_OFFSET; // eQEP Position Counter Initialization Value
```

这里就涉及到 INDEX_OFFSET 这个值怎么确定的问题，因为 Z 相脉冲并不是严格的出现于绝对零点位置处，故需要进行角度校正。

角度校正

角度校正的基本思想是，记录下 Z 相信号对应的绝对角度值，将其写入 QPOSINIT 寄存器即可，即上述代码中的 INDEX_OFFSET 宏定义的值。要确定绝对角度，必须要有一个基准，在进行角度校正时，使用预定位的方式确定绝对角度的零点。

具体做法是，使能三相逆变器，使用 SVPWM 算法生成一个 $U_{\alpha} > 0$; $U_{\beta} = 0$ 的矢量， U_{α} 不宜设置得过大(对应 A 相上桥导通占空比，BC 两相上桥不导通)，以免电流过大，此时电机会旋转至绝对零点处。之后使能 EQEP 模块，将 QPOSCNT 的值设为 0，SEI 设为 0，以禁用 QI 信号信号的自动同步复位功能；与此同时，使能 IEL，以便在 QI 信号发生时自动锁存 QPOSCNT 寄存器的值。以上初始化过程用代码表示如下：

```
EQep1Regs.QPOSCNT = 0;
EQep1Regs.QEPCTL.bit.IEI = 0; // Does nothing on rising or falling edge of QEPI
EQep1Regs.QEPCTL.bit.IEL = 3; // Latch QPOSCNT on rising or falling edge of QEPI
depends on direction
```

按照以上步骤初始化 EQEP 模块后，禁用三相逆变器，此时电机可以自由旋转。用手旋转电机以产生 QI 信号，此时 QPOSCNT 寄存器中的值会被自动锁存在 QPOSILAT 寄存器中，可以正反多旋转几次电机，记录下若干次 QPOSILAT 寄存器中的值取平均值即可。读取寄存器的值可以通过使用仿真器进入调试模式实现。

计算得到的平均值即 QI 信号对应的绝对角度值，将此值定义为 INDEX_OFFSET 宏写入 QPOSINIT 寄存器中即可。

注：上述叙述假设编码器 Z 相信号连接至了 QEPI 输入端上，若连接至了 QEPS 输入端处，只需将 QI 相关部分改为 QS 即可，二者的功能基本相同。

电角度计算

正余弦用 256 个数组元素查表获得。

Z 脉冲未到来，由 QEP 计数和 UVW 霍尔粗略得到电角度

Z 脉冲触发中断，由 QEP 计数，Z 锁存值和 Z 和磁铁 N 极夹角三个参数得到电角

度，角度更为连续以及精确。

//LJF 当前d轴位置，即编码器读取位置与旋转坐标d轴的夹角

```
/*-----*/
```

****gstr_Encoder.g_QPosCnt**从编码器读取位置开始转动后的角度，配置好QEP后软件自动记录在QPOSCNT

****gstr_Encoder.g_ZLatch**从编码器读取位置相对于Z脉冲的角度，配置好QEP后软件自动记录在QPOSILAT

****gstr_Encoder.g_ZToBetaTheta**旋转坐标d轴相对于Z脉冲的电角度脉冲，由电机厂商给出，或者定子产生磁场吸转子磁铁，关波后转动一圈读取Z锁存的POSCNT

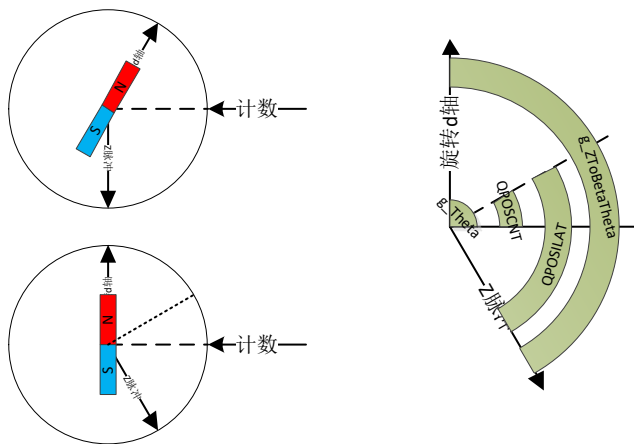
```
-----*/
```

//LJF =从编码器读取位置开始转动后的角度+ (旋转坐标d轴相对于Z脉冲的电角度脉冲-从编码器读取位置相对于Z脉冲的角度)

//LJF =从编码器读取位置开始转动后的角度+ 旋转坐标d轴相对于编码器读取位置的电角度

//LJF =编码器读取位置与旋转坐标d轴的夹角

```
gstr_Encoder.g_Theta = (int32)gstr_Encoder.g_QPosCnt
                      - (int32)gstr_Encoder.g_ZLatch
                      + gstr_Encoder.g_ZToBetaTheta;
```



速度的计算

系统时钟设定为 150MHz

高速时用 M 法测频法：由 QUPRD 控制固定时间。读取的变量为位置脉冲 QPOSLAT 前后的差值,因为编码器有一定的精度，计算时可适当放大（100 倍）后单位为 0.01rpm，公式为

$$100 * \frac{1}{\frac{1}{60} \left(\frac{\text{min}}{\text{s}} \right) * \frac{QUPRD}{150M(Hz)}} * \Delta POSLAT$$

低速时用 T 法测周法：由 UPPS 控制固定旋转角度，由 CCPs 控制计时时钟快慢精度。读取的变量为时间计数差值 QCPRDLAT。放大 100 倍后单位为 0.01rpm，公式为

$$100 * \frac{\frac{1}{2500 * 4(pluse)} * 2^{UPPS}}{\frac{1}{60} \left(\frac{\min}{s} \right) * \frac{2^{CCPS}}{150M(Hz)}} * \frac{1}{QCPRDLAT}$$

功能函数

函数：void InitEncodDat(str_ENCODER *p)

描述：初始化 EQEP 增量式编码器的相关常数用于计算，如 M 法和 T 法测速，角度转换，电周期对应的脉冲计数和 Z 脉冲对应的 D 轴夹角。

函数：void InitEnCode(void)

描述：初始化编码器的输入，选择编码器的类型：绝对值式，带霍尔 UVW 信号的增量式，省线式（不带霍尔 UVW 信号）的增量式，旋转变压器等。

函数：void InitEQep1(void)

描述：初始化 EQEP 的配置，在主函数中调用。配置最大值时复位计数，暂未设置成 Z 脉冲初始化计数。M 法固定的时间为 0.5ms（2kHz），T 法固定的角度为 QCLK/2；计数时钟为 1.28us。

函数：interrupt void Isr_Zgo()

描述：编码器 Z 脉冲中断函数，用于锁存从任意时刻任意位置旋转一圈 Z 脉冲到来时的 QPOSCNT 的计数值。从 QPOSILAT 寄存器读取。同时 Z 脉冲到来标志位置位，提示系统在转矩环时切换电角度的获取方式为光电增量式编码器，把霍尔 UVW 信号旁路。

函数：void InitPhase(str_ENCODER *p)

描述：读取霍尔 UVW 的输入引脚电平确定大致的电角度（转子磁铁 N 极）取检测扇区 60 度的一半 30 度，并转化为增量式编码器的脉冲数，以便上电时启动用。如果不在霍尔 UVW 状态内，则报错。

函数：void SpdFbCal()

描述：根据 QE 配置的 2k 中断读取 POSLAT 数值计算速度，高速时用 M 法，低速时（小于 100rpm）用 T 法，计算出的速度经过滤波后输出

函数：void SpdControl()

描述：速度控制函数，包含速度监测，根据位置模式或是速度模式来指定速度指令来源，

【2】位置脉冲 QEP 配置

暂不使用

伺服电机编码器每转 2500 个脉冲,额定转速 3000r/min (50Hz); 电机与丝杠直连, 丝杠螺距为 5mm, 伺服驱动器内部电子齿轮比为 1。

计算伺服电机驱动工件在 5S 左右行走 1m 的距离:

首先计算所需要的脉冲数量:

(1).行走 1m 的距离, 相当于丝杠转动 $1*1000(\text{mm}) / 5(\text{mm/r}) = 200$ 转;

(2).即伺服电机发出 $2500*200=500000$ (50 万) 个脉冲;

(3).二进制的 16 位可以代表 “0~65535” (6 万 5 千) 或 “-32768~+32768” 的十进制,即 16 位数据不满足需要的脉冲数;

(4).二进制的 32 位可以代表 “0~4294967295” (42 亿 9 千) 的十进制,远远大于需要的 50 万个脉冲, 即 32 位数据肯定满足需要的脉冲数
综上, 距离需要 50 万个脉冲即可行走需要的距离。

再确定伺服的行走速度:

(1).伺服电机额定转速 3000r/min (50Hz), 即 50r/s;

(2).伺服电机额定发出的脉冲数, $50*2500=125000$ 脉冲/s;

(3).规定 3s 左右发出的脉冲数 $3*125000=375000$ 脉冲/s (37 万个脉冲), 即需要超速或延时;

(4).规定 3s 左右完成, 则 $500000/3=166667$ 脉冲/s, $166667*50/125000=67\text{Hz}$, 有点快;

(5).理论最快时间, $500000/125000=4\text{s}$, 即最快 4s, 再加加减速 20%, 5~6s 比较合适
综上, 125000 个脉冲, 该距离, 用时 5s 左右比较适合。

说明书 P90

信号处理函数

PosPulsCal 到 P_Source 到 PosControl

脉冲位置指令系数

系统时钟频率 100M/脉冲 QEP 模块频率 1k = 100,000, 计数值。

$$\begin{aligned} & \frac{60*1000*脉冲位置\Delta POSLAT}{4*2500(pluse)} * 1000 * 2^{-12} \\ & = 脉冲位置\Delta POSLAT * \frac{10*1000*1.5*2^{12}}{2500(pluse)} * 1000 * 2^{-12} \\ & = 脉冲位置\Delta POSLAT * \frac{1k*6144}{2500(pluse)} * 10 * 2^{-12} * 100 \end{aligned}$$

AoConstInit 用于显示和 PWM 脉冲

使用 (位置) 速度前馈 ForebackP

外设配置

计数器使能，AB 不交换

索引事件锁存

单位时间计数器使能

EQep2Regs.QUPRD = EQEP2_QUPRD 设置为 1KHz at 150MHz SYSCLKOUT，位置环调节周期

QEP 捕捉锁存模式：1-UTtimer 溢出锁存

EQEP 计数器的最大值，28bits

位置计数器复位方式为 0-index 事件触发 QPOSCNT 复位，同时，上溢或下溢时同样复位。

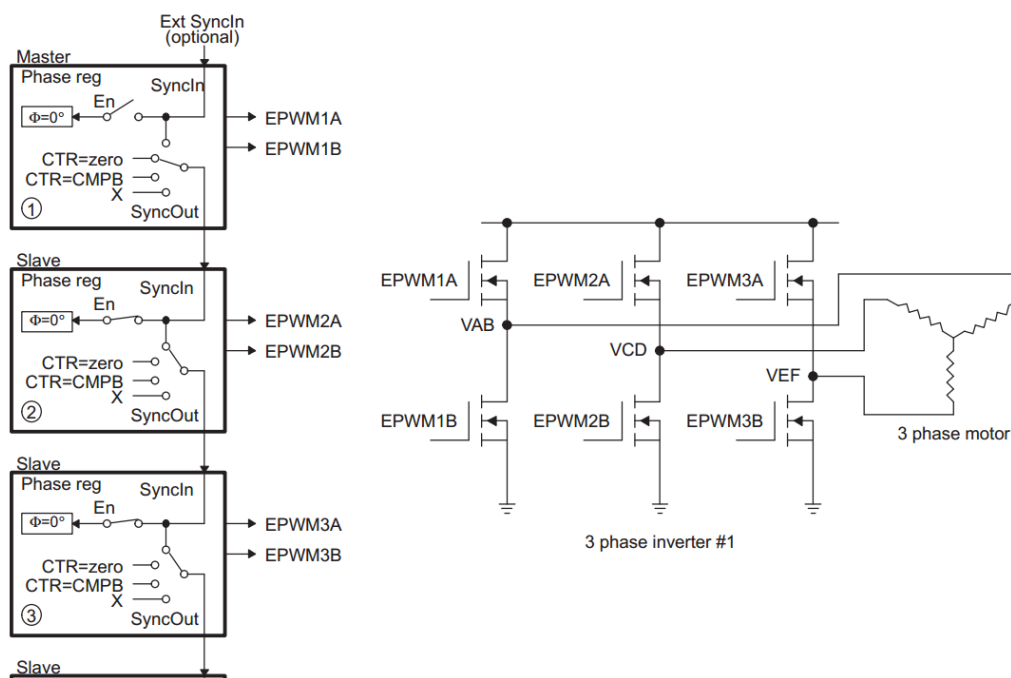
【3】PWM 配置

ePWM1,2,3 电机控制

硬件接口及说明

硬件接口 GPIO0,1,2,3,4,5 分别对应 A 相上下桥，B 相上下桥和 C 相上下桥控制信号。具体实现框图如下：

Figure 55. Control of Dual 3-Phase Inverter Stages as Is Commonly Used in Motor Control



主控模块 EPwm1Regs 的配置

TB 子模块:**TBCTL**

15-14 不用仿真模式

13 同步事件之后相位增计数

12-10 对来自系统的时钟分频为 1

9-7 对来自系统的时钟分频为 1

6 不触发软同步信号

5-4 当 TBCTR=0 时输出同步信号（主控模式）

3 当 TBCTR=0 时把影子寄存器写入工作 TBPRD 寄存器

2 不把相位 TBPHS 装入 TBCTR

1-0 增减计数

CMP 子模块:**CMPCTL**

15:10 保留

9 略

8 略

7 保留

6 比较值 B 使用影子寄存器(激活但不用)

5 保留

4 比较值 A 使用影子寄存器

3:2 当 TBCTR 计数到 0 从 CMPB 影子寄存器装入工作寄存器(激活但不用)

1:0 当 TBCTR 计数到 0 从 CMPA 影子寄存器装入工作寄存器

AQ 模块:**AQCTLA**

15:12 保留

11:10 略，无动作

9:8 略，无动作

7:6 当 TBCTR 减计数且等于 CMPA 时，将 EPWMxA 输出置低

5:4 当 TBCTR 增计数且等于 CMPA 时，将 EPWMxA 输出置高

3:2 略，无动作

1:0 略，无动作

DB 模块:**DBCTL**

15 DB 模块时钟与 TB 模块时钟一致

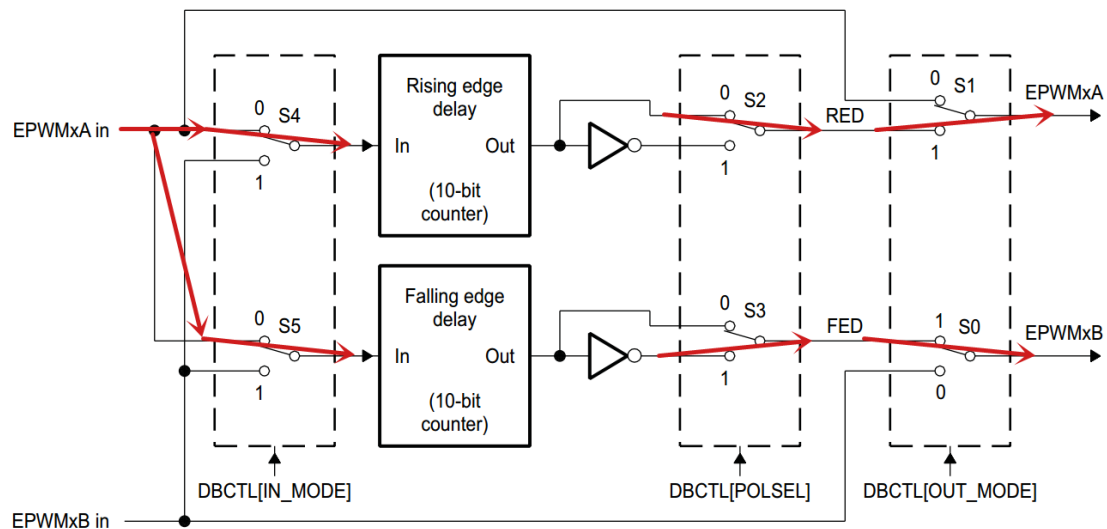
14:6 保留

5:4 只上一个模块的 EPWMxA 输出作为单相两桥臂的两路控制信号输入源

3:2 低有效，互补波形。EPWMxA 输出翻转，EPWMxB 输出不翻转；TBD

1:0 上一个模块的输出输入到死区模块处理后输出，不跳过死区模块

Figure 31. Configuration Options for the Dead-Band Submodule



ET 模块:

ETSEL

15 略，关闭

14-12 略

11 使能 EPWMxSOCA 脉冲信号

10-8 事件：当 TBCTR = 0 时将会生成 EPWMxSOCA 脉冲信号(未分频)TBD

7-4 保留

3 生成 EPWM_INT 中断使能

2-0 事件：当 TBCTR = 0 时将会生成 EPWMxINT 中断(未分频)

ETPS

15-14 略

13-12 关闭

11-10 略

9-8 事件发生一次就生成 EPWMxSOCA 脉冲信号

7-4 保留

3-2 略

1-0 事件发生一次就生成 EPWMxINT 中断

ETCLR

15-4 保留

3-0 清除所有事件的标志位

PC 模块关闭

TZ 模块

TZSEL

默认：关闭

8 对该 PWM 模块使能 TZ1 触发 OSH1(控制开关)

TZCTL

默认：设置为高阻态

3-2 拉高该 PWM 模块的 B 信号 TBD

1-0 拉高该 PWM 模块的 A 信号 TBD

TZEINT

默认：关闭

2 使能生成中断，OST 会触发 EPWMx_TZINT PIE 中断(控制总开关)

TZCLR

15-7 保留

6-0 清除全部触发标志位

TZFRC

默认：略

2 强制触发一次 OST 事件，OST 标志位置位，这里关波等待全局的控制命令

从动模块 EPwm2Regs 和 EPwm3Regs 的配置

TB 子模块:

TBCTL

15-14 不用仿真模式

13 同步事件之后相位增计数

12-10 对来自系统的时钟分频为 1

9-7 对来自系统的时钟分频为 1

6 不触发软同步信号

5-4 输出 EPWMxSYNC:同步信号（从动模式传递信号）

3 当 TBCTR=0 时把影子寄存器写入工作 TBPRD 寄存器

2 把相位 TBPHS 装入 TBCTR

1-0 增减计数

CMP 子模块:

CMPCTL

15:10 保留

9 略

8 略

7 保留

6 比较值 B 使用影子寄存器(激活但不用)

5 保留

4 比较值 A 使用影子寄存器

3:2 当 TBCTR 计数到 0 从 CMPB 影子寄存器装入工作寄存器(激活但不用)

1:0 当 TBCTR 计数到 0 从 CMPA 影子寄存器装入工作寄存器

AQ 模块:

AQCTLA

15:12 保留

11:10 略, 无动作

9:8 略, 无动作

7:6 当 TBCTR 减计数且等于 CMPA 时, 将 EPWMxA 输出置低

5:4 当 TBCTR 增计数且等于 CMPA 时, 将 EPWMxA 输出置高

3:2 略, 无动作

1:0 略, 无动作

DB 模块:

DBCTL

15 DB 模块时钟与 TB 模块时钟一致

14:6 保留

5:4 只用上一个模块的 EPWMxA 输出作为单相两桥臂的两路控制信号输入源

3:2 低有效, 互补波形。EPWMxA 输出翻转, EPWMxB 输出不翻转

1:0 上一个模块的输出输入到死区模块处理后输出, 不跳过死区模块

ET 模块: (2, 3 模块不用)

PC 模块关闭

TZ 模块

TZSEL

默认: 关闭

8 对该 PWM 模块使能 TZ1 触发 OSHT1(控制开关)

TZCTL

默认: 设置为高阻态

3-2 拉高该 PWM 模块的 B 信号

1-0 拉高该 PWM 模块的 A 信号

TZEINT (2, 3 模块不用)

默认: 关闭

2 使能生成中断, OST 会触发 EPWMx_TZINT PIE 中断(控制总开关)

TZCLR (2, 3 模块不用)

15-7 保留

6-0 清除全部触发标志位

TZFRC

默认：略

2 强制触发一次 OST 事件，OST 标志位置位。这里关波等待全局的控制命令

功能函数

函数：void OpenPwm(void)

描述：发波命令函数，使能逻辑时使用。通过软件清零信号 TZCLR 使得 PWM 模块使能，进而发波驱动功率电路，注意 TZ 的设定。

函数：void ClosePwm(void)

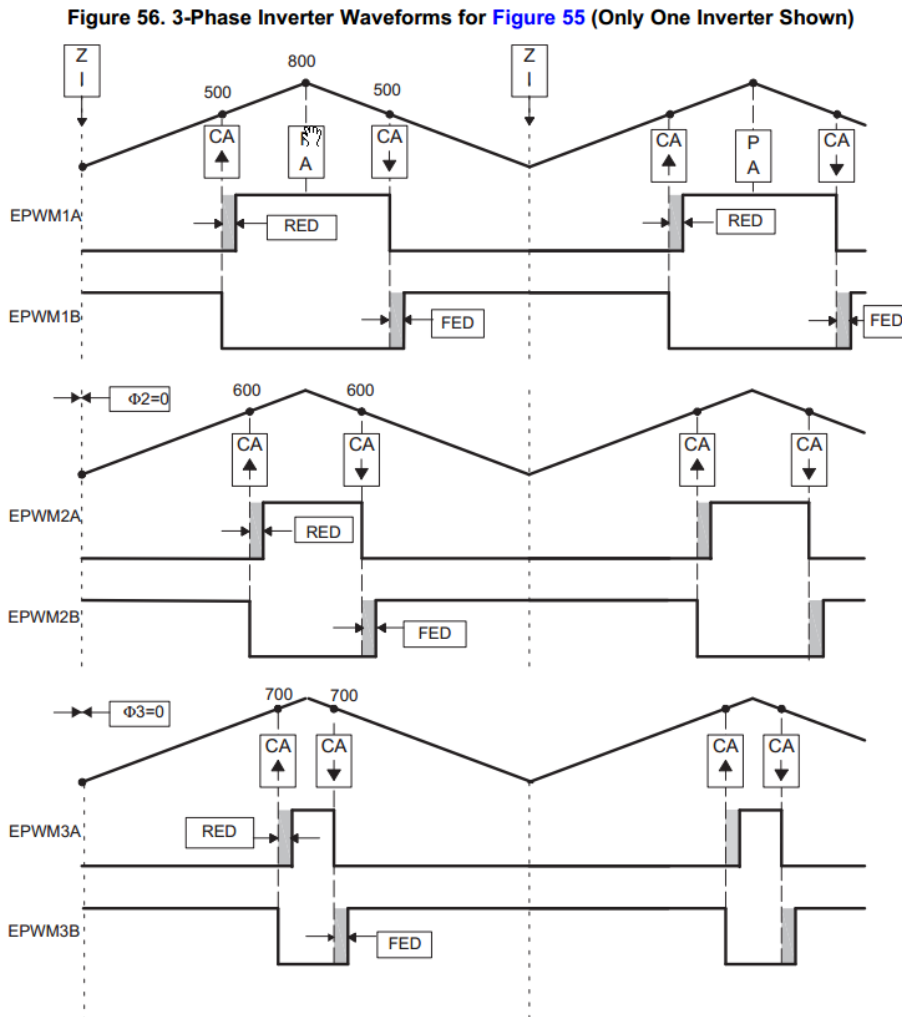
描述：关波命令函数，保护和待机逻辑时使用通过软件强制触发信号 FRC 使得 PWM 模块使能，进而关波关闭功率电路，注意 TZ 的设定。

函数：void PwmDutySet(int16 Cmp1, int16 Cmp2, int16 Cmp3)

描述：输入参数是 Q15 格式的占空比，乘以对应的 TBPRD 半周期值转化为对称波形的切换时刻计数值赋值到三路对应的比较寄存器。

函数：void InitEPwmFullCmp(void)

描述：初始化 EPWM 的配置，在主函数中调用。设置载波频率为 8k，模式如下图所示，考虑到电流采样为霍尔隔离的电流传感器采样，可以使用该模式。如果电流为使用电阻采样，则该模式需要更改，TBD。



函数：interrupt void Isr_TZ1Faults()

描述：为硬件过流保护电路触发，从而保护电路和系统。包含关波函数。

函数：interrupt void Isr_SvPWM()

描述：主中断，有 PWM 模块触发。包含位置，速度，转矩三环，一些逻辑控制。

发波模式为倒三角计数程序计算占空比和扇区判断的代码如下：

```
v.tmp1= v.Ubeta;
v.tmp2= _IQdiv2(v.Ubeta) + (_IQmpy(_IQ(0.866),v.Ualpha));
v.tmp3= v.tmp2 - v.tmp1;
```

如果单纯判断大于零小于零的时候，其实是以下三条直线：

$$y = 0$$

$$y + \sqrt{3}x = 0$$

$$y - \sqrt{3}x = 0$$

这里坐标轴的纵坐标 y 为 U_{β} ，横坐标 x 为 U_{α}

进行不等式区域判断可以辨别电压落在哪个扇区。横纵坐标对应静止两相正

交坐标。

整理如下表

扇区\判断条件	tmp2	Tmp3	Tmp1
1	>0	>0	>0
2	>0	<0	>0
3	<0	<0	>0
4	<0	<0	<0
5	<0	>0	<0
6	>0	>0	<0
三个条件无交集	>0	<0	<0
三个条件无交集	<0	>0	>0

倒三角增减计数模式下用程序定义的参考量表示的各扇区电机三相上桥导通时刻对应的占空比

扇区	电机 A 相占空比 Ta	电机 B 相占空比 Tb	电机 C 相占空比 Tc
1=U0, U60	$\frac{t_{aon}}{2}$ $\frac{1+tmp2}{2}$	$\frac{t_{bon}}{2}$ $\frac{1+tmp1-tmp3}{2}$	$\frac{t_{con}}{2}$ $\frac{1-tmp2}{2}$
2=U60, U120	$\frac{t_{bon}}{2}$ $\frac{1+tmp3+tmp2}{2}$	$\frac{t_{aon}}{2}$ $\frac{1+tmp1}{2}$	$\frac{t_{con}}{2}$ $\frac{1-tmp1}{2}$
3=U120, U180	$\frac{t_{con}}{2}$ $\frac{1+tmp3}{2}$	$\frac{t_{aon}}{2}$ $\frac{1-tmp3}{2}$	$\frac{t_{bon}}{2}$ $\frac{1-(tmp1+tmp2)}{2}$
4=U180, U240	$\frac{t_{con}}{2}$ $\frac{1+tmp2}{2}$	$\frac{t_{bon}}{2}$ $\frac{1+tmp1-tmp3}{2}$	$\frac{t_{aon}}{2}$ $\frac{1-tmp2}{2}$
5=U240, U300	$\frac{t_{bon}}{2}$ $\frac{1+tmp3+tmp2}{2}$	$\frac{t_{con}}{2}$ $\frac{1+tmp1}{2}$	$\frac{t_{aon}}{2}$ $\frac{1-tmp1}{2}$
6=U300, U0	$\frac{t_{aon}}{2}$ $\frac{1+tmp3}{2}$	$\frac{t_{con}}{2}$ $\frac{1-tmp3}{2}$	$\frac{t_{bon}}{2}$ $\frac{1-(tmp1+tmp2)}{2}$

```
if (v.VecSector==1 || v.VecSector==4)
```

```
{    v.Ta= v.tmp2;
```

```
    v.Tb= v.tmp1-v.tmp3;
```

```
\
```

```
\
```

```
\
```

```

        v.Tc=-v.tmp2;
    }

    else if(v.VecSector==2 || v.VecSector==5)
    {
        v.Ta= v.tmp3+v.tmp2;
        v.Tb= v.tmp1;
        v.Tc=-v.tmp1;
    }

    else
    {
        v.Ta= v.tmp3;
        v.Tb=-v.tmp3;
        v.Tc=-(v.tmp1+v.tmp2);
    }

```

函数：int16 MastIsrTimePercent(void)

描述：计算主中断的负载率，单位为%；最后上发上位机显示。

函数：void SvDataUpdate()

描述：根据功能码计算相应的 TBPRD，和死区时间，最后设置 PWM 的寄存器。如果电阻采样则需要加入一个最大的占空比。TBD.

函数：void DriveControl(void)

描述：放在主中断里执行，占空比计算后转换为比较寄存器的赋值，开关波的逻辑控制，无故障才发波。

函数 void SetMotorDriveIn(void)

描述：输入一些电机驱动的参数，注意有无限纲。这里有三种调试模式。零位测试，电角度固定为-90 度，给定 ud=0,uq=小值 2000（最后转换成 Q15 格式的占空比）；正常电流 PI 调节输出作为输入模式；电压 Udq 加电角度的电流开环运行。

ePWM4 母线泻放

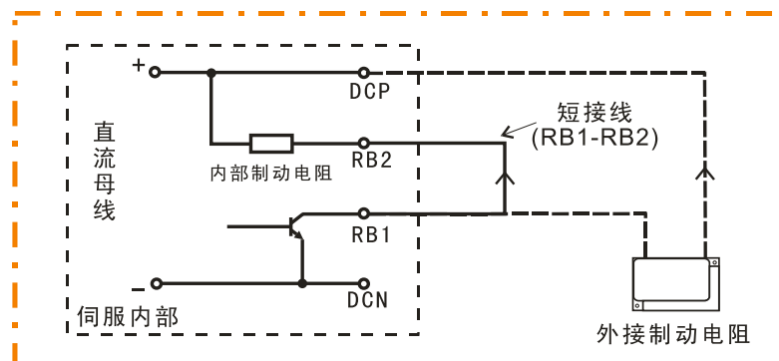
硬件接口及说明

硬件接口为 GPIO6, GPIO7 未使用; 为低电平(TBD)时制动有效。

220V 电压等级设置泻放点为 350V, 过压点为 400V

母线电压采样计算公式为 $\frac{500V * 10 * \frac{(\text{参数H01_30})}{1000}}{4096} * \text{ADC采样值}$, 单位为

0.1V。对应函数为 MonitCal();



根据设定斜率(所有电压放大 10 倍单位为 0.1V, 最大百分比为 95%对应 400V, 最小百分比为 5%对应 350V) 超过泻放点 350V 则按该比例计算占空比:

$$\frac{\text{百分比}-5}{U_{dc}-3500} = \frac{95-5}{4000-3500} \rightarrow \text{百分比} = \frac{95-5}{4000-3500} * (U_{dc}-3500) + 5$$

最后由百分比乘以 TBPRD (赋值为 6250), 对应函数 DcVoltageBrk();

外设配置

TBCLK=150MHz, TBPRD=9375, 那么 PWM 周期为: $(2*9375)/(150*10E6)=125\mu s$, 为 8kHz,

增减计数, 无相位, 周期立即作用, CMP 为影子模式计时器到零装载, 增计数到 CMP 时拉低(制动有效), 到零时拉高。不输出同步信号。

ePWM5 逐波限流

硬件接口及说明

硬件接口为 GPIO8-9, 作用为生成一个硬件限流的模拟电平信号。

用于控制电路领域, 尤其涉及一种变频器及其逐波限流保护电路, 通过在变频器中采用包括比较模块、逻辑模块、判断模块的逐波限流保护电路, 比较模块将硬件限流信号的电平与基准电平进行比较, 根据比较结果生成第一电平信号, 并将第一电平信号输出至逻辑模块; 逻辑模块将接收到的第一电平信号的电平与直流电平信号的电平进行逻辑处理, 生成第二电平信号, 并将第二电平信号输出至判断模块; 判断模块将接收到的第二电平信号与驱动开关信号进行逻辑处理, 生成驱动使能信号, (接入 PWM1 模块的 TZ 引脚 TBD) 控制变频器输出的电流, 从而解决了现有技术中无法解决的变频器逐波限流电路结构复杂、产品可靠性低以及用户体验差的问题。

硬件满输出为 3.3V, 电压系数推导: $\text{CMP 值}/\text{SIGNTBPRD_1500} = \text{输出电压} /$

满输出电压 3.3V=3300mV

$CMP \text{ 值} = \text{输出电压 mV} * (SIGNTBPRD_1500 / \text{满输出电压 } 3300\text{mV})$

那么电压系数 $VoltCoe_Q15 = (SIGNTBPRD_1500 / \text{满输出电压 } 3300\text{mV}) * 2^{15}$

最后需要 $Comp = SIGNTBPRD_1500 - (VoltCoe * Volt \gg 15)$;原因是 PWM 模式设置为比较值越大占空比越小，需要转换。

外设配置

TBCLK=150MHz, TBPRD=1500, 那么 PWM 周期为: $(1*1500)/(150*10E6)=10\mu s$, 为 100kHz,

增计数, 无相位, 周期立即作用, CMP 为影子模式计时器到零装载, 增计数到 CMP 时拉高(输出有效), 到零时拉低。不输出同步信号。

ePWM6 模拟输出

硬件接口及说明

硬件接口 GPIO8 和 9

硬件满输出为 10VTBD, 电压系数推导: $CMP \text{ 值} / SIGNTBPRD_1000 = \text{输出电压} / \text{满输出电压 } 10000\text{mV}$

$CMP \text{ 值} = \text{输出电压 mV} * (SIGNTBPRD_1000 / \text{满输出电压 } 10000\text{mV})$

那么电压系数 $VoltCoe_Q15 = (SIGNTBPRD_1000 / \text{满输出电压 } 10000\text{mV}) * 2^{15}$

最后需要 $Comp = SIGNTBPRD - (VoltCoe * Volt \gg 15)$;原因是 PWM 模式设置为比较值越大占空比越小，需要转换。

外设配置

TBCLK=150MHz, TBPRD=1500, 那么 PWM 周期为: $(1*1500)/(150*10E6)=10\mu s$, 为 100kHz,

增计数, 无相位, 周期立即作用, CMP 为影子模式计时器到零装载, 增计数到 CMP 时拉高(输出有效), 到零时拉低。不输出同步信号。

【4】FPGA 外部接口 XINTF 配置

硬件接口及说明

配置映射空间, 在相应的 CMD 文件添加一些定义的寄存器, 这里为 Zone7。初始地址为 0x200000, 其长度大小为 1M*16。

示例如下:

```
MEMORY
```

```
{
```

```
    PAGE 0:      /* Program Memory */
```

PAGE 1: /* Data Memory */

```
.....
/* external interface registers */
FPGACODE   : origin = 0x200000, length = 0x00000C   /* FPGA registers */
FPGAADCMD  : origin = 0x20000C, length = 0x000002   /* FPGA registers */
FPGAADFDB  : origin = 0x20000E, length = 0x000004   /* FPGA registers */
FPGAPOSC   : origin = 0x200012, length = 0x000007   /* FPGA registers */
.....
}
```

SECTIONS

```
{
.....
/** FPGA Data Register Structures */
FPGACodeRegsFile : > FPGACODE,    PAGE = 1
FPGAPosCRegsFile : > FPGAPOSC,    PAGE = 1
FPGAAdCmdRegFile : > FPGAADCMD,    PAGE = 1
FPGAAdFdbRegFile : > FPGAADFDB,    PAGE = 1
.....
}
```

不采用 XREADY 信号的同步采样，模块时钟 XTIMCLK = SYSCLKOUT/2。

外设配置

```
void InitXintf(void)
```

```
{
    // This shows how to write to the XINTF registers. The
    // values used here are the default state after reset.
    // Different hardware will require a different configuration.

    // For an example of an XINTF configuration used with the
    // F28335 eZdsp, refer to the examples/run_from_xintf project.

    // Any changes to XINTF timing should only be made by code
    // running outside of the XINTF.

    // All Zones-----
    // Timing for all zones based on XTIMCLK = 1/2 SYSCLKOUT
    // Make sure the XINTF clock is enabled
    SysCtrlRegs.PCLKCR3.bit.XINTFENCLK = 1;

    // Configure the GPIO for XINTF with a 16-bit data bus
```

```
// This function is in DSP2833x_Xintf.c
EALLOW;

GpioCtrlRegs.GPCMUX1.bit.GPIO64 = 3; // XD15
GpioCtrlRegs.GPCMUX1.bit.GPIO65 = 3; // XD14
GpioCtrlRegs.GPCMUX1.bit.GPIO66 = 3; // XD13
GpioCtrlRegs.GPCMUX1.bit.GPIO67 = 3; // XD12
GpioCtrlRegs.GPCMUX1.bit.GPIO68 = 3; // XD11
GpioCtrlRegs.GPCMUX1.bit.GPIO69 = 3; // XD10
GpioCtrlRegs.GPCMUX1.bit.GPIO70 = 3; // XD19
GpioCtrlRegs.GPCMUX1.bit.GPIO71 = 3; // XD8
GpioCtrlRegs.GPCMUX1.bit.GPIO72 = 3; // XD7
GpioCtrlRegs.GPCMUX1.bit.GPIO73 = 3; // XD6
GpioCtrlRegs.GPCMUX1.bit.GPIO74 = 3; // XD5
GpioCtrlRegs.GPCMUX1.bit.GPIO75 = 3; // XD4
GpioCtrlRegs.GPCMUX1.bit.GPIO76 = 3; // XD3
GpioCtrlRegs.GPCMUX1.bit.GPIO77 = 3; // XD2
GpioCtrlRegs.GPCMUX1.bit.GPIO78 = 3; // XD1
GpioCtrlRegs.GPCMUX1.bit.GPIO79 = 3; // XD0

GpioCtrlRegs.GPBMUX1.bit.GPIO40 = 3; // XA0/XWE1n
GpioCtrlRegs.GPBMUX1.bit.GPIO41 = 3; // XA1
GpioCtrlRegs.GPBMUX1.bit.GPIO42 = 3; // XA2
GpioCtrlRegs.GPBMUX1.bit.GPIO43 = 3; // XA3
GpioCtrlRegs.GPBMUX1.bit.GPIO44 = 3; // XA4
GpioCtrlRegs.GPBMUX1.bit.GPIO45 = 3; // XA5
GpioCtrlRegs.GPBMUX1.bit.GPIO46 = 3; // XA6
GpioCtrlRegs.GPBMUX1.bit.GPIO47 = 3; // XA7

GpioCtrlRegs.GPCMUX2.bit.GPIO80 = 3; // XA8
GpioCtrlRegs.GPCMUX2.bit.GPIO81 = 3; // XA9
GpioCtrlRegs.GPCMUX2.bit.GPIO82 = 3; // XA10
GpioCtrlRegs.GPCMUX2.bit.GPIO83 = 3; // XA11
GpioCtrlRegs.GPCMUX2.bit.GPIO84 = 3; // XA12
GpioCtrlRegs.GPCMUX2.bit.GPIO85 = 3; // XA13
GpioCtrlRegs.GPCMUX2.bit.GPIO86 = 3; // XA14
GpioCtrlRegs.GPCMUX2.bit.GPIO87 = 3; // XA15
//GpioCtrlRegs.GPBMUX1.bit.GPIO39 = 3; // XA16
//GpioCtrlRegs.GPAMUX2.bit.GPIO31 = 3; // XA17
//GpioCtrlRegs.GPAMUX2.bit.GPIO30 = 3; // XA18
//GpioCtrlRegs.GPAMUX2.bit.GPIO29 = 3; // XA19

GpioCtrlRegs.GPBMUX1.bit.GPIO34 = 3; // XREADY
GpioCtrlRegs.GPBMUX1.bit.GPIO35 = 3; // XRNW
GpioCtrlRegs.GPBMUX1.bit.GPIO38 = 3; // XWE0
```

```
//GpioCtrlRegs.GPBMUX1.bit.GPIO36 = 3; // XZCS0
GpioCtrlRegs.GPBMUX1.bit.GPIO37 = 3; // XZCS7
GpioCtrlRegs.GPAMUX2.bit.GPIO28 = 3; // XZCS6
EDIS;

EALLOW;
// All Zones-----
// Timing for all zones based on XTIMCLK = SYSCLKOUT
XintfRegs.XINTCNF2.bit.XTIMCLK = XTimClk;
// Buffer up to 3 writes
XintfRegs.XINTCNF2.bit.WRBUFF = 0; //3;
// XCLKOUT is enabled
XintfRegs.XINTCNF2.bit.CLKOFF = 0;
// XCLKOUT = XTIMCLK
XintfRegs.XINTCNF2.bit.CLKMODE = ClkMode;

// Zone 7-----
// When using ready, ACTIVE must be 1 or greater
// Lead must always be 1 or greater
// Zone write timing
XintfRegs.XTIMING7.bit.XWRLEAD = 3;
XintfRegs.XTIMING7.bit.XWRACTIVE = 7;
XintfRegs.XTIMING7.bit.XWRTRAIL = 3;
// Zone read timing
XintfRegs.XTIMING7.bit.XRDLEAD = 3;
XintfRegs.XTIMING7.bit.XRDACTIVE = 7;
XintfRegs.XTIMING7.bit.XRDTRAIL = 3;

// don't double all Zone read/write lead/active/trail timing
XintfRegs.XTIMING7.bit.X2TIMING = 1;

// Zone will not sample XREADY signal
XintfRegs.XTIMING7.bit.USEREADY = 0;
XintfRegs.XTIMING7.bit.READYMODE = 0;

// 1,1 = x16 data bus
// 0,1 = x32 data bus
// other values are reserved
XintfRegs.XTIMING7.bit.XSIZE = 3;
EDIS;

//Force a pipeline flush to ensure that the write to
//the last register configured occurs before returning.
```

```
asm(" RPT #7 || NOP");  
  
}
```

【5】CAN 通讯

硬件接口及说明

硬件接口为 GPIO12,GPIO13，分别对应 CANTX 和 CANRX。使用的是 CAN-B 模块

系统时钟为 SYSCLK/2(参考 SPRUFB0D 文献 P33)。若无特殊要求，则为了适应上位机配置频率为 250k。

功能函数

函数：void InitECanB(void)

描述：初始化 ECAN 的配置，在主函数中调用。设置波特率 250k，发送接收的配置，相关结构体初始化。

函数：void CanMaster(void)

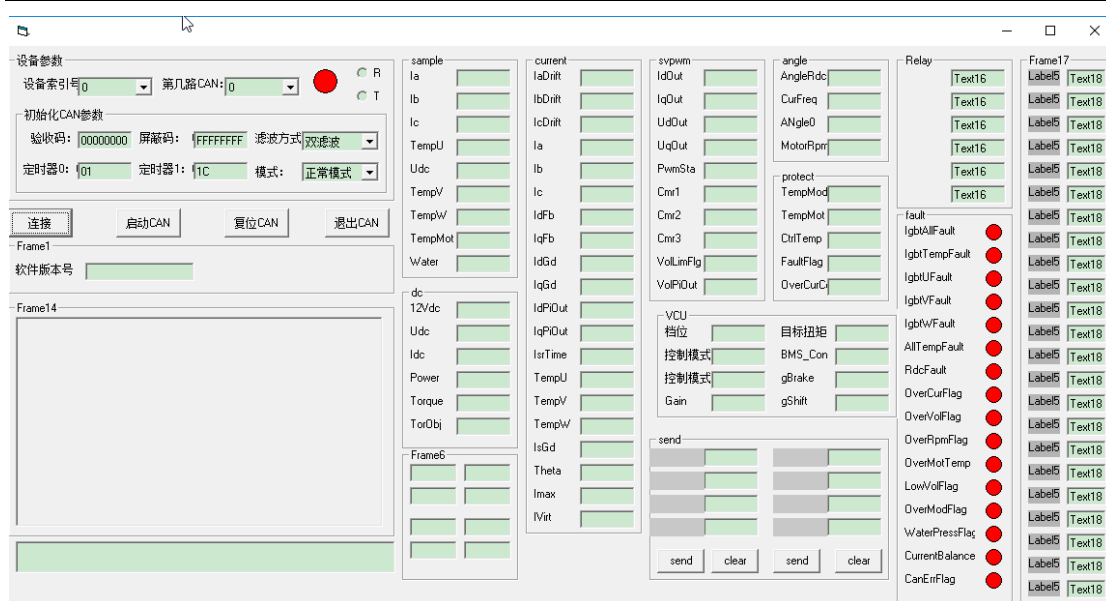
描述：在主中断里管理发送和接收的函数。

函数：void CanReceive(void)

描述：在主中断里查询管理接收到的消息，若与自定义的 ID 匹配，则将消息放到对应 ID 消息结构体 CanMboxUserRegsR1, CanMboxUserRegsR2 里。这里有 VCU 发送消息的 ID，上位机连接成功的 ID，上位机一号发送框的发送和复位 ID，以及二号发送框的发送和复位 ID。

函数：void CanSend(void)

描述：在主中断里周期性地用 CanMboxUserRegsT 发送要监控的变量值。若上位机连接成功则该函数负责发送控制器当前的状态消息和绘图的曲线信息。若没有连接上位机，则发送由用户定义的控制器的状态消息。



【备注】

配合 FPGA 完成测速度，测位置的功能。
通讯接口 SPI，SCI，I2C 暂不说明

PWM 模块

主控模块 EPwm1Regs 的配置

TB 子模块:

TBCTL

15-14 不用仿真模式

13 同步事件之后相位增计数

12-10 对来自系统的时钟分频为 1

9-7 对来自系统的时钟分频为 1

6 不触发软同步信号

5-4 当 TBCTR=0 时输出同步信号（主控模式）

3 当 TBCTR=0 时把影子寄存器写入工作 TBPRD 寄存器

2 不把相位 TBPHS 装入 TBCTR

1-0 增减计数

CMP 子模块:

CMPCTL

15:10 保留

9 略

8 略

7 保留

6 比较值 B 使用影子寄存器(激活但不用)

5 保留

4 比较值 A 使用影子寄存器

3:2 当 TBCTR 计数到 0 从 CMPB 影子寄存器装入工作寄存器(激活但不用)

1:0 当 TBCTR 计数到 0 从 CMPA 影子寄存器装入工作寄存器

AQ 模块:**AQCTLA**

15:12 保留

11:10 略, 无动作

9:8 略, 无动作

7:6 当 TBCTR 减计数且等于 CMPA 时, 将 EPWMxA 输出置低

5:4 当 TBCTR 增计数且等于 CMPA 时, 将 EPWMxA 输出置高

3:2 略, 无动作

1:0 略, 无动作

DB 模块:**DBCTL**

15 DB 模块时钟与 TB 模块时钟一致

14:6 保留

5:4 只用上一个模块的 EPWMxA 输出作为单相两桥臂的两路控制信号输入源

3:2 低有效, 互补波形。EPWMxA 输出翻转, EPWMxB 输出不翻转

1:0 上一个模块的输出输入到死区模块处理后输出, 不跳过死区模块

ET 模块:**ETSEL**

15 略, 关闭

14-12 略

11 使能 EPWMxSOCA 脉冲信号

10-8 事件: 当 TBCTR = 0 时将会生成 EPWMxSOCA 脉冲信号(未分频)

7-4 保留

3 生成 EPWM_INT 中断使能

2-0 事件: 当 TBCTR = 0 时将会生成 EPWMxINT 中断(未分频)

ETPS

15-14 略

13-12 关闭

11-10 略

9-8 事件发生一次就生成 EPWMxSOCA 脉冲信号

7-4 保留

3-2 略

1-0 事件发生一次就生成 EPWMxINT 中断

ETCLR

15-4 保留

3-0 清除所有事件的标志位

PC 模块关闭

TZ 模块

TZSEL

默认：关闭

8 对该 PWM 模块使能 TZ1 触发 OSHT1(控制开关)

TZCTL

默认：设置为高阻态

3-2 拉高该 PWM 模块的 B 信号

1-0 拉高该 PWM 模块的 A 信号

TZEINT

默认：关闭

2 使能生成中断，OST 会触发 EPWMx_TZINT PIE 中断(控制总开关)

TZCLR

15-7 保留

6-0 清除全部触发标志位

TZFRC

默认：略

2 强制触发一次 OST 事件，OST 标志位置位

从动模块 EPwm2Regs 和 EPwm3Regs 的配置

TB 子模块:

TBCTL

15-14 不用仿真模式

13 同步事件之后相位增计数

12-10 对来自系统的时钟分频为 1

9-7 对来自系统的时钟分频为 1

6 不触发软同步信号

5-4 输出 EPWMxSYNC:同步信号（从动模式传递信号）

3 当 TBCTR=0 时把影子寄存器写入工作 TBPRD 寄存器

2 把相位 TBPHS 装入 TBCTR

1-0 增减计数

CMP 子模块:

CMPCTL

15:10 保留

9 略

8 略

7 保留

6 比较值 B 使用影子寄存器(激活但不用)

5 保留

4 比较值 A 使用影子寄存器

3:2 当 TBCTR 计数到 0 从 CMPB 影子寄存器装入工作寄存器(激活但不用)

1:0 当 TBCTR 计数到 0 从 CMPA 影子寄存器装入工作寄存器

AQ 模块:

AQCTLA

15:12 保留

11:10 略，无动作

9:8 略，无动作

7:6 当 TBCTR 减计数且等于 CMPA 时，将 EPWMxA 输出置低

5:4 当 TBCTR 增计数且等于 CMPA 时，将 EPWMxA 输出置高

3:2 略，无动作

1:0 略，无动作

DB 模块:

DBCTL

15 DB 模块时钟与 TB 模块时钟一致

14:6 保留

5:4 只用上一个模块的 EPWMxA 输出作为单相两桥臂的两路控制信号输入源

3:2 低有效，互补波形。EPWMxA 输出翻转，EPWMxB 输出不翻转

1:0 上一个模块的输出输入到死区模块处理后输出，不跳过死区模块

ET 模块:（2，3 模块不用）

ETSEL

15 略，关闭

14-12 略

11 使能 EPWMxSOCA 脉冲信号

10-8 事件：当 TBCTR = 0 时将会生成 EPWMxSOCA 脉冲信号(未分频)

7-4 保留

3 生成 EPWM_INT 中断使能

2-0 事件：当 TBCTR = 0 时将会生成 EPWMxINT 中断(未分频)

ETPS

15-14 略

13-12 关闭

11-10 略

9-8 事件发生一次就生成 EPWMxSOCA 脉冲信号

7-4 保留

3-2 略

1-0 事件发生一次就生成 EPWMxINT 中断

ETCLR

15-4 保留

3-0 清除所有事件的标志位

PC 模块关闭

TZ 模块

TZSEL

默认：关闭

8 对该 PWM 模块使能 TZ1 触发 OSHT1(控制开关)

TZCTL

默认：设置为高阻态

3-2 拉高该 PWM 模块的 B 信号

1-0 拉高该 PWM 模块的 A 信号

TZEINT（2，3 模块不用）

默认：关闭

2 使能生成中断，OST 会触发 EPWMx_TZINT PIE 中断(控制总开关)

TZCLR（2，3 模块不用）

15-7 保留

6-0 清除全部触发标志位

TZFRC

默认：略

2 强制触发一次 OST 事件，OST 标志位置位

数据类型转换

通过以下程序在DSP中验证，注意正负符号位的转变，数值大小的考虑。在不同芯片平台上字长的定义不同。程序移植需要考量一定的兼容性。如不注意会引起一定的损坏或控制系统的异常。

```
int16 s16=0; //-2
int32 s32=0;
Uint16 u16=0;//65534
Uint32 u32=0;

int16 siv=0;
siv=sizeof(int16);//0001
//*****
//左移与右移
//*****

s16=13/7;//小数处理并非四舍五入，实际是取整
s16=12/7;
s16=11/7;

s16=-9;
s16=s16>>1;//带符号右移=-5
s16=-9;
s16=s16/2;//除以2=-4
s16=-9;
s16=s16>>2;//=-3
s16=-9;
s16=s16/4;//除以4=-2
s16=-9;
s16=s16>>4;//=-1
s16=-9;
s16=s16/16;//除以16=0
s16=-9;
s16=s16>>12;//=-1

u16=9;
u16=u16>>1;//不带符号右移=4

s16=9;
s16=s16<<1;
s16=9;
```

```
s16=s16<<2;
s16=9;
s16=s16<<11;
s16=9;
s16=s16<<12;//本应是36864实际为-28672溢出

s16=-9;
s16=s16<<1;
s16=-9;
s16=s16<<2;
s16=-9;
s16=s16<<11;
s16=-9;
s16=s16<<12;//本应是-36864实际为28672溢出

u16=9;
u16=u16<<1;
u16=9;
u16=u16<<2;
u16=9;
u16=u16<<11;
u16=9;
u16=u16<<12;//是36864
////////////////////////////////////
s16=9;
s16=s16>>1;//=4
s16=9;
s16=s16/2;//除以2=4
s16=9;
s16=s16>>2;//=2
s16=9;
s16=s16/4;//除以4=2
s16=9;
s16=s16>>4;//=0
s16=9;
s16=s16/16;//除以16=0
s16=9;
s16=s16>>12;//=0

//*****
//有符号与无符号位数相同之间互相赋值
//*****
//*****

s16=0;
```

```
u16=32767;
s16=u16;

s16=0;
u16=65535;
s16=u16;

//
u16=0;
s16=2;
u16=s16;

u16=0;
s16=-2;
u16=s16;

s32=0;
u32=65536;
s32=u32;

s32=0;
u32=0x80000000;
s32=u32;
//
u32=0;
s32=2;
u32=s32;

u32=0;
s32=-2;
u32=s32;

//*****
//位数少赋值给位数多；位数少的变量：有符号扩展后赋值，无符号不扩展空位补零赋值。
//*****
////////////////////////////////////
s16=0xfffe; //-2
u16=0xfffe;//65534
////////////////////////////////////
s32=4;
s32=-4;
s32=s16;//符号扩展
s32=4;
s32=-4;//0xFFFF FFFC
```

```
s32=u16;//无符号赋值给有符号，直接占低位
u32=5;
u32=-4;//有警告，但是按有符号32位的补码继续赋值，十六进制为0xFFFF FFFC十进制为
4294967292
u32=s16;//有符号16赋值给无符号32：符号扩展后再赋值给无符号32
u32=4;
u32=-4;//有警告，但是按有符号32位的补码继续赋值，十六进制为0xFFFF FFFC十进制为
4294967292
u32=u16;//无符号赋值给无符号，直接占低位
////////////////////////////////////
s16=0x2; //2
u16=0x2; //2
////////////////////////////////////
s32=4;
s32=-4;
s32=s16;//符号扩展
s32=4;
s32=-4;//0xFFFF FFFC
s32=u16;//无符号赋值给有符号，直接占低位
u32=5;
u32=-4;//有警告，但是按有符号32位的补码继续赋值，十六进制为0xFFFF FFFC十进制为
4294967292
u32=s16;//有符号16赋值给无符号32：符号扩展后再赋值给无符号32
u32=4;
u32=-4;//有警告，但是按有符号32位的补码继续赋值，十六进制为0xFFFF FFFC十进制为
4294967292
u32=u16;//无符号赋值给无符号，直接占低位

//*****
//位数多赋值给位数少，直接截断再考虑符号对数值的影响。
//*****
////////////////////////////////////
s32=0x00017fff; //98303
u32=0x00017fff; //98303
////////////////////////////////////
s16=4;
s16=-4;
s16=s32;//截断
s16=4;
s16=-4;//0xFFFC
s16=u32;
u16=5;
u16=-4;//有警告，但是按有符号16位的补码继续赋值，十六进制为0xFFFC十进制为65532
```

```
u16=s32;//截断
u16=4;
u16=-4;//有警告，但是按有符号16位的补码继续赋值，十六进制为0xFFFC十进制为65532
u16=u32;//截断
////////////////////////////////////
s32=0x0001fffe; //131070
u32=0x0001fffe;//131070
////////////////////////////////////
s16=4;
s16=-4;
s16=s32;//截断
s16=4;
s16=-4;//0xFFFC
s16=u32;//截断
u16=5;
u16=-4;//有警告，但是按有符号16位的补码继续赋值，十六进制为0xFFFC十进制为65532
u16=s32;//截断
u16=4;
u16=-4;//有警告，但是按有符号16位的补码继续赋值，十六进制为0xFFFC十进制为65532
u16=u32;//截断
////////////////////////////////////
s32=0xffff7fff; //-32769
u32=0xffff7fff;//-32769?=? 4294934527
////////////////////////////////////
s16=4;
s16=-4;
s16=s32;//截断
s16=4;
s16=-4;//0xFFFF FFFC
s16=u32;//截断
u16=5;
u16=-4;//有警告，但是按有符号16位的补码继续赋值，十六进制为0xFFFC十进制为65532
u16=s32;//截断
u16=4;
u16=-4;//有警告，但是按有符号16位的补码继续赋值，十六进制为0xFFFC十进制为65532
u16=u32;
////////////////////////////////////
s32=0xFFFFfffe; //-2
u32=0xFFFFfffe;//-2?=? 4294967294
////////////////////////////////////
s16=4;
s16=-4;
s16=s32;//截断
s16=4;
```

```
s16=-4;//0xFFFF FFFC
s16=u32;//截断
u16=5;
u16=-4;//有警告，但是按有符号32位的补码继续赋值，十六进制为0xFFFF FFFC十进制为
4294967292
u16=s32;//截断
u16=4;
u16=-4;//有警告，但是按有符号32位的补码继续赋值，十六进制为0xFFFF FFFC十进制为
4294967292
u16=u32;//截断

//
//各类型含多少字节判断
siv=10;
siv=sizeof(short);//1601
siv=10;
siv=sizeof(long);//3202
siv=10;
siv=sizeof(unsigned char);//801
siv=10;
siv=sizeof(unsigned short);//1601
siv=10;
siv=sizeof(unsigned long);//3202
siv=10;
siv=sizeof(float);//3202
siv=10;
siv=sizeof(long double);//6404

siv=10;
siv=sizeof(signed char);//0001
siv=10;
siv=sizeof(unsigned char);//0001
```

用户底层

驱动底层和控制底层都已配置好，就可以实现一定的功能。电机的标定，对拖测试或驻坡爬坡，电动及馈电，电机正反转及加速或刹车制动。把这些与档位油门刹车等等联系在一起就可以满足用户的需求。

对拖加载测试，验证效率等等也有一定的要求

电机弱磁标定

由于电机的相电压不便于测量，实际中只测量电机的线电压有效值，若母线电压设定为 U_{dc} ，则能提供给电机的线电压有效值为 $U_{dc}/\sqrt{2}$ ，为了保留一定的余量，实际上在标定的过程中，控制电机的线电压有效值在 $U_{dc}/\sqrt{2}$ 的 85%~90% 之间，然后调整电流矢量大小输出预定的转矩。增大或减小电流矢量与 d 轴的角度，找到较大的转矩点，这时候的角度是特定电流矢量下的 MPTA 或电压极限椭圆的角度。再增大或减小电流矢量大小使实际的转矩与预定的转矩相差较小。最后还可以重复调节角度和电流矢量大小。

在基速以下，电流矢量顶点轨迹沿着 MPTA 曲线。

在基速以上，忽略绕组上的压降，且电机处于稳态运行即电压方程中的微分项为零，受到母线电压的限制，可以得到以 i_d 和 i_q 为坐标系的电压极限方程：

$$(L_d i_d + \psi_f)^2 + (L_q i_q)^2 \leq \left(\frac{U_{s\max}}{\omega_e} \right)^2$$

目标：以(m,n)为中心坐标的椭圆方程 $\frac{(x-m)^2}{a^2} + \frac{(y-n)^2}{b^2} = 1$

展开二次项，同时除以 $L_d^2 L_q^2$ ：

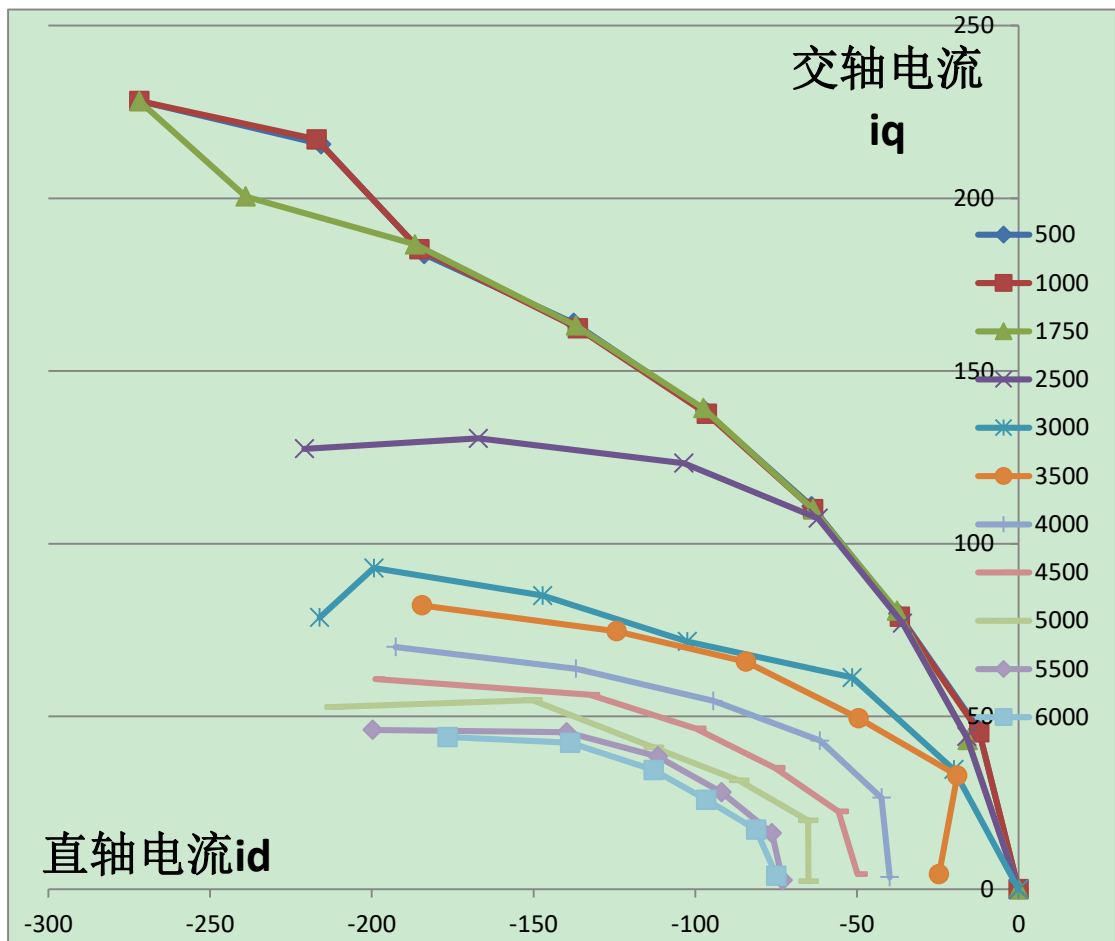
$$\frac{L_d^2 i_d^2 + 2L_d i_d \psi_f + \psi_f^2 + L_q^2 i_q^2}{L_d^2 L_q^2} \leq \left(\frac{U_{s\max}}{\omega_e L_d L_q} \right)^2$$

合并化简：

$$\begin{aligned} \frac{i_d^2}{L_q^2} + \frac{2i_d \psi_f}{L_d L_q^2} + \frac{\psi_f^2}{L_d^2 L_q^2} + \frac{i_q^2}{L_d^2} &\leq \left(\frac{U_{s\max}}{\omega_e L_d L_q} \right)^2 \\ \frac{1}{L_q^2} \left(i_d^2 + \frac{2i_d \psi_f}{L_d} + \frac{\psi_f^2}{L_d^2} \right) + \frac{i_q^2}{L_d^2} &\leq \left(\frac{U_{s\max}}{\omega_e L_d L_q} \right)^2 \\ \frac{\left(i_d + \frac{\psi_f}{L_d} \right)^2}{\left(\frac{U_{s\max}}{\omega_e L_d} \right)^2} + \frac{i_q^2}{\left(\frac{U_{s\max}}{\omega_e L_q} \right)^2} &\leq 1 \end{aligned}$$

与目标方程对比，由于 $L_d < L_q$ 所以椭圆的焦点落在 i_d 轴上，中心坐标为 $(-\frac{\psi_f}{L_d}, 0)$ ，

随着转速上升，椭圆逐渐缩小。如下图中的 3000~6000rpm 下的矢量顶点轨迹。



S1=====

加载 电机基本参数 以及 仿真或台架数据

Lq 或 Ld 数值			100
			50
			0
-100	-50	0	Id \\\ Iq

根据 较为细腻的步骤长 确定

矢量电流幅值扫描的向量 数组 Is_ScanVect_A

转矩扫描向量 Te_ScanVect_Nm

转速扫描向量 Spd_ScanVect_rpm

S2=====

用 thinplateinterp 方法平滑和滤波得到 Ld , Lq 电感表格 Ld_IdIq_Table 和 Lq_IdIq_Table

用 fourier8 方法滤波 iq-磁链 的关系得到 Flux_Iq_Table

S3=====

根据矢量电流幅值扫描的向量，

Beta 角度从 90 度到 180 度扫描。

根据 $t_e = p \left[\psi_f i_s \sin\beta + \frac{1}{2} (L_d - L_q) i_s^2 \sin 2\beta \right]$ 计算转矩

寻找同样电流大小 Is_ScanAmp 下不同角度对应的转矩值最大的那一个并存储记录，
最终呈现出一个扇形的转矩大小分布图 Te_Distribution

存储的相关信息命名：

同样大小的电流，最大的转矩 MTPA_Is_Te

同样大小的电流，最大的转矩时的 beta 角度 MTPA_Is_Beta

同样大小的电流，最大的转矩时的 Id MTPA_Is_Id

同样大小的电流，最大的转矩时的 Iq MTPA_Is_Iq

S4=====

用 gauss8 方法平滑和滤波得到 x 为电流扫描幅值大小，

y 为 Beta 角的 曲线 FilterTable_Is_Beta

用 smoothingspline 方法平滑和滤波得到 x 为扫描电流下的最大转矩，

y 为对应的电流幅值大小的 曲线 FilterTable_Te_Is

因为扫描是以电流 连续扫描的；但扫描计算得出【转矩 不连续】，需要按照转矩步长【重新拟合】方向是 Te=>Is，Is=>Beta

得到 最优扫描转矩 下的电流大小为 MTPA_Te_Is MTPA_Te_Id MTPA_Te_Iq

得到 最优扫描转矩 下的角度大小为 MTPA_Te_Beta

可以用 电流矢量扫描 代入

同样大小的电流，最大的转矩时的 beta 角度 MTPA_Is_Beta 曲线

算出 扫描时候和这次拟合滤波之后的差值，此处暂时不计算

拟合 按转矩扫描大小 得出的 Idq 矢量电流曲线 fitresult_MTPA_idiq 并绘图

S5=====

根据矢量电流幅值扫描的向量，

Beta 角度从 90 度到 180 度扫描。

根据电机 DQ 轴下考虑定子电阻的电压方程，

$$u_q = R_s i_q + \omega_r L_d i_d + \omega_r \psi_f$$

$$u_d = R_s i_d - \omega_r L_q i_q$$

$$|\mathbf{u}_s| = \sqrt{u_q^2 + u_d^2}$$

整理得出关于电频率 ω_e 的二元一次方程

可以用关于电频率 ω_e 的二元一次方程的根判别式 选取 实根

其物理含义为 当前电压最大利用率下，允许的最大电频率转速

最终呈现出一个扇形的 允许的转速大小 分布图 ω_e _Distribution

把 ω_e _Distribution 用 linearinterp 方法平滑和滤波得到 FilterTable_Id_Iq_Te

S6=====

01 转矩分布矩阵 T_e _Distribution 里画转矩扫描步长的 【恒转矩曲线】

选取第三象限的等转矩组合

用 smoothingspline 方法平滑和滤波得到 恒转矩的 IdIq 组合

存放于 fitresult_T_Distribution_curve

02 定转矩步长 从 IS 得出的最大转矩组合里 扫描算出的 Id,Iq 来画 【MTPA 曲线】

语法 contour(X,Y,Z,v)画出 Z 在向量 v 所有值处的等高线，

如只想画出 Z 在 i 处的等高线，则调用 contour (X,Y,Z, [i,i])。

03 从 求根得出的转速分布矩阵 来画 等转速曲线即为 【电压极限椭圆曲线】

选取第三象限的等转速电频率组合

用 smoothingspline 方法平滑和滤波得到 恒转速电频率的 IdIq 组合

存放于 fitresult_Wcontour_curve

选取等转速下的 id 的 最大值 和 最小值

%lf 最大值的物理含义： 椭圆的右端，即为该转速下需要的弱磁电流 Id 大小

%lf 最小值的物理含义：

% 因为转速只在电流圆内扇形扫描，转速升高椭圆缩小，可能不会描绘一个完整的椭圆！

% 电压极限椭圆左端 【或者】 电压极限椭圆 和 电流极限圆 的 交点对应下的 Id 值

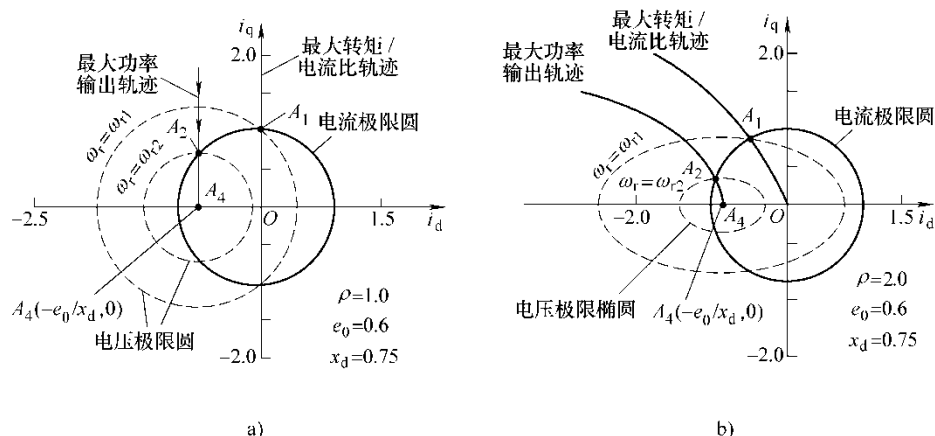
用于确认该等转速曲线下，在 id 横坐标扫描范围，找出转矩最大，就是 MTPV 策略！

04 绘出一个在第三象限的电流限制圆

S7===== 【TBD】

%LJF 在电压极限椭圆（等转速线）上，在 Id 最大最小值之间，寻找最小值（实际是最大

值，故函数里取负数)。间接性的找到 恒转矩曲线 和 电压极限椭圆 的切点，组合成最终的 MTPV 曲线



语法 `[x,fval] = fminbnd(fun,x1,x2,options)` % x 是取局部最值的 x ，局部最值是 $fval$ 。

%lf 找出 最大转速 下 最大的转矩

%按照电磁设计的最大转矩作为边界

%LJF 转置矩阵，使得行数为转速值，第一列即为当前转速下的转矩最大值

S8=====

控制策略和通讯

无位置传感器控制

电机控制程序源于以下网址：

<https://active-semi.com/zh-CN/products/intelligent-motor-control/pac5223>

为无传感方案，调试时对程序里的代码做的部分笔记以及查阅各大论坛的文章，整理出该文档。

1. 用PWM模块实现数模转换

用于电机控制领域的嵌入式芯片一般包含几组PWM模块，空闲的模块可以用于参数的调试，将定义好的Q格式小数变换为电压，外加示波器观察。

1.1 放大倍数

1.1.1 单极性正值

一般的量程，会在偏置以上显示

如果没有基值，可选一个比量程稍微大一点的2的N次幂数值（在程序上右移实际值便可完成计算）作为基值用于标么化

1.1.2 双极性数值

1.1.3 与示波器同量程的计算

若示波器有一个电流钳，且想把软件内部电流DAC波形与电流钳波形做对比，为了与电流钳统一量程，可以把软件内部电流DAC波形放大一定的倍数。即可实现用电流钳通道的光标去获取波形上任意两点的时间和幅值。

1.2 滤波用的电阻电容及截止频率

1.2.1 待定

单位

$$\frac{x(r)}{1(r)} = \frac{\Delta_{Q0}\pi}{2\pi} \Rightarrow x(r) = \frac{\Delta_{Q15} / 2^{15}}{2} \Rightarrow x(r) = \frac{\Delta_{Q15}}{2^{16}}$$

2. 差分放大电路与电流采样

该函数把对CURRENT_SENSOR进行标么化的Q15格式采样电流值（两相或三相）和旋变解码芯片（AD2S1205）得到的Q15格式的电角度值，经过克拉克变换和帕克变换后得到旋转两相坐标系的Id，Iq当前采样反馈值。

2.1 放大倍数与偏置电压

以下

2.2 电流对称采样以及标么化

具体

3. 查表法计算反正切

由于无传感的算法涉及反正切计算，部分芯片可能没有完整的数学库，可以

自己设计一个查表法实现。

文章源于以下网址：

http://blog.sina.com.cn/s/blog_75d3a9030101646a.html

修订了文章的错误并分析函数。

3.1 从原理及分析到实现

如图目标：实现反正切函数的查表算法，精度 0.5° 。

分析：

$f(x) = \arctan(x)$ 函数单调但非线性，越接近原点处，斜率越大。

90° 比较特殊，其值为正无穷大，只要得到的正切值 $\tan\alpha$ 大于114.59（ 89.5° 度对应的正切值），就可以认为是 90° 度。

假如 $0-89.5^\circ$ 用一个查找表实现，索引为正切值，值为角度值。 $\tan 89.5^\circ$ 为115，假如用N点uint16_t型查找表，需要保证

$$\arctan(115 / N) < 0.5^\circ$$

$$\text{即 } 115 / N < 0.0087$$

可选的最小值为13219，意味着至少要建立一张 $13K \times 16$ 的表，占用26KB的Flash，对于一般的MCU来说不可接受。

如果 $0-45^\circ$ 用一个查找表实现，由于 $\tan 45^\circ = 1$ ，根据上面的计算公式，可以得出N的最小值为114，这个N值对一般的MCU均可以接受，甚至可以增加至256点来提高精度。对于 45° 以上的角度，可以根据两角和的三角公式来得到，推导过程如下：

根据两角和公式：

$$\tan(\alpha + \beta) = \frac{\tan\alpha + \tan\beta}{1 - \tan\alpha\tan\beta}$$

$$\text{设 } \beta = \frac{\pi}{4}, \tan(\alpha + \beta) = K, \text{ 有}$$

$$K = \frac{1 + \tan\alpha}{1 - \tan\alpha}$$

解得

$$\tan\alpha = \frac{K - 1}{K + 1}$$

进一步得到

$$\alpha = \arctan\left(\frac{K - 1}{K + 1}\right)$$

如果测量的正切值K大于1，那么根据最后的公式，可以求出比 45° 大的那个角度 α ，再加上 45° 就是实际的角度。而 α 的计算可以由256点查找表来快速得到，这样就实现了用一个小容量查找表计算 90° 度以内的反正切值。

查找表涉及两个数值：表索引`idx`和表的角度值`angle[idx]`。

索引值`idx`对应 $0-45^\circ$ 的正切值，由于这些值均小于1，采用整体扩大256倍的方法，将正切值就映射到了 $0-256$ 的区间，从而索引`idx`的范围为 $0-255$ 。

角度值`angle[]`是`uint16_t`型的，而实际的范围为 $0-45$ ，根据 $65536/45=1456$ ，可以用Q12的表示方式来提高分辨率。应该为Q10格式

表格和程序片段如下：

```
static const uint16_t tb_atan[256] = {
0, 229, 458, 687, 916, 1145, 1374, 1603,
1832, 2061, 2290, 2519, 2748, 2976, 3205, 3433,
3662, 3890, 4118, 4346, 4574, 4802, 5029, 5257,
5484, 5711, 5938, 6165, 6391, 6618, 6844, 7070,
7296, 7521, 7746, 7971, 8196, 8421, 8645, 8869,
9093, 9317, 9540, 9763, 9986, 10208, 10431, 10652,
10874, 11095, 11316, 11537, 11757, 11977, 12197, 12416,
12635, 12853, 13071, 13289, 13507, 13724, 13940, 14157,
14373, 14588, 14803, 15018, 15232, 15446, 15660, 15873,
16085, 16297, 16509, 16720, 16931, 17142, 17352, 17561,
17770, 17979, 18187, 18394, 18601, 18808, 19014, 19220,
19425, 19630, 19834, 20038, 20241, 20444, 20646, 20848,
21049, 21250, 21450, 21649, 21848, 22047, 22245, 22443,
22640, 22836, 23032, 23227, 23422, 23616, 23810, 24003,
24196, 24388, 24580, 24771, 24961, 25151, 25340, 25529,
25717, 25905, 26092, 26278, 26464, 26649, 26834, 27018,
27202, 27385, 27568, 27750, 27931, 28112, 28292, 28471,
28650, 28829, 29007, 29184, 29361, 29537, 29712, 29887,
30062, 30236, 30409, 30582, 30754, 30925, 31096, 31266,
31436, 31605, 31774, 31942, 32109, 32276, 32442, 32608,
32773, 32938, 33101, 33265, 33428, 33590, 33751, 33913,
34073, 34233, 34392, 34551, 34709, 34867, 35024, 35180,
35336, 35492, 35646, 35801, 35954, 36107, 36260, 36412,
36563, 36714, 36864, 37014, 37163, 37312, 37460, 37607,
37754, 37901, 38047, 38192, 38337, 38481, 38624, 38768,
38910, 39052, 39194, 39335, 39475, 39615, 39754, 39893,
40032, 40169, 40307, 40443, 40580, 40715, 40850, 40985,
41119, 41253, 41386, 41519, 41651, 41782, 41913, 42044,
42174, 42303, 42432, 42561, 42689, 42817, 42944, 43070,
43196, 43322, 43447, 43572, 43696, 43819, 43943, 44065,
44188, 44309, 44431, 44551, 44672, 44792, 44911, 45030,
45148, 45266, 45384, 45501, 45618, 45734, 45849, 45965,
};
```

```
static int16_t calc_phase(int16_t real, int16_t imag)
{
```

```
int16_t abs_real;
int16_t abs_imag;
int16_t phase;
uint16_t zone;
uint32_t tmp_tan;
uint32_t delta_tan;
if (real < 0)
{
    abs_real = -real;
    zone = 1U;
}
else
{
    abs_real = real;
    zone = 0U;
}
if (imag < 0)
{
    abs_imag = -imag;
    zone += 2U;
}
else
{
    abs_imag = imag;
    zone += 0U;
}
if (abs_imag <= abs_real)
{
    tmp_tan = (uint32_t)abs_imag<<8U;
    tmp_tan /= abs_real;
    phase = tb_atan[tmp_tan]>>3U;
}
else if ((abs_imag>>7U) <= abs_real)
{
    delta_tan = ((uint32_t)(abs_imag - abs_real))<<15U;
    tmp_tan = (uint32_t)abs_imag + (uint32_t)abs_real;
    delta_tan /= tmp_tan;
    delta_tan >>= 7U;
    phase = tb_atan[delta_tan]>>3U;
    phase += 5760;
}
else
{
    phase = 11520;
```

```
}
switch (zone)
{
    case 0U:
    {
        break;
    }
    case 1U:
    {
        phase = 23040 - phase;
        break;
    }
    case 2U:
    {
        phase = -phase;
        break;
    }
    case 3U:
    {
        phase = phase - 23040;
        break;
    }
    default:
    {
        break;
    }
}
return(phase);
}
```

3.2 数值的精度和象限的判断

分析表格中的数值，存储类型为16位无符号值， $\lg(45) / \lg(2) = 6$ ，uint16_t有16位，6位表示0~45度范围，剩下10位表示小数，所以为Q10格式。

4. 有传感器下的电角度的获取

该函数对电流进行PI调节，使之满足与之对应的转矩大小。

4.1 光电编码器Z脉冲与磁链d轴夹角的关系

一般

4.2 用求模取余的方法对电角度进行转换

程序，大于等于每电角度的对应脉冲数，则减去此脉冲数，正如时钟是一圈12个小时，13点为 $13-12=1$ 点(P.M.)，24点为 $24-12=12$ ； $12-12=0$ 点(A.M.)。

4.3 霍尔UVW信号与磁极的初始定位

可以先用示波器观察波形并记录，按照轴向逆时针，慢慢旋转，直至霍尔组合状态发生改变，记录一圈不重复的值。给定一个开环电压与电角度，如 $U_q=0, U_d=\text{小值}$ ，电角度分别给定为在霍尔扇区中间的角度此时读取霍尔状态值便可判断之前记录的值是否正确。

5.初始位置的获取原理与电控

过程

6.七段SVPWM占空比分析

程序

6.1 不变的数值计算分析

由于

6.2 正三角与倒三角载波对应的占空比

自动

7.弱磁阈值与线性计算

先后

8.母线电压转换和计算

该函数

9.外同步启动用的周期弧度增量计算

该函数

10.反电动势的获取及分析

分析无位置传感器控制的无刷直流电机在静止，顺风，逆风情况下的状态，介绍一种基于硬件反电动势检测的开环启动方法，该方法通过电机两相反电动势的检测，识别出电机当前转向及转速，获取转子位置角度，以当前角度拖动电机启动，当电机运行转速达到稳定后，切入闭环正弦波控制。此方法需要硬件反电动势检测电路可检测电压范围宽，计算量小，方法易于实现，适用于一般的无位置传感器的电机控制系统。

10.1 反电动势采样的分析

资料来源: <http://www.doc88.com/p-9973176845962.html>

根据硬件电路检测得到的两相反电动势采样值作为电机启动状态的判断依据，并根据公式转换获取此时的转子角度与位置，以进行顺逆风启动。如下图1为基于硬件反电动势检测的原理图。

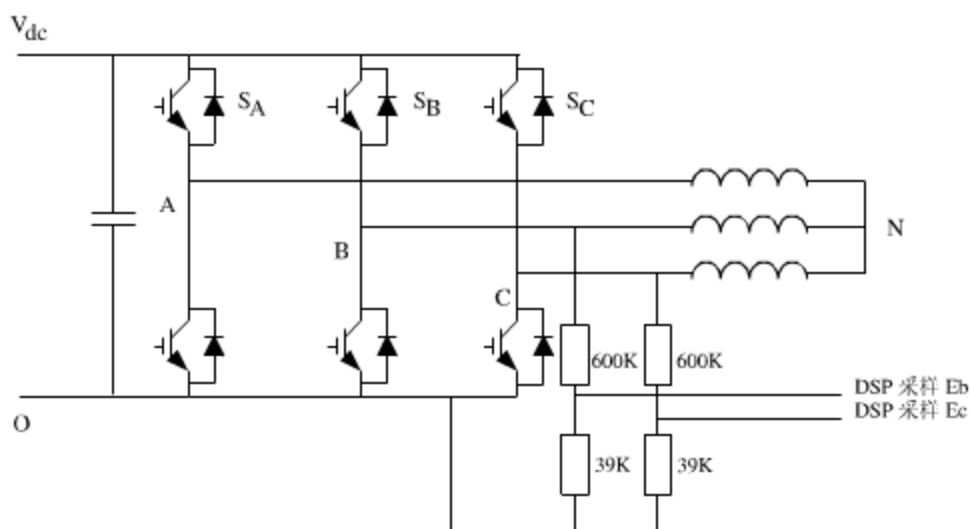


图1 反电动势检测硬件原理图

Fig. 1 schematic diagram of EMF detection

根据以下公式（1-1），可以根据端电压 U_{BO} ， U_{CO} 得到电机当前线反电动势 U_{BC} ，利用该线反电动势，可以得到当前转子位置与转速。

$$\begin{aligned} U_{BO} &= U_{BN} + U_{NO} \\ U_{CO} &= U_{CN} + U_{NO} \\ U_{BC} &= U_{BN} - U_{CN} = U_{BO} - U_{CO} \end{aligned} \quad (\text{式1-1})$$

1.1 实际端电压

以电机被吹动值400rpm时的状态为例说明。如下图2，3，4所示，绿色为B相端电压 U_{BO} ，红色为C相端电压 U_{CO} ，蓝色为中性点电压 U_{NO} ，橙色为线反电动势 U_{BC}

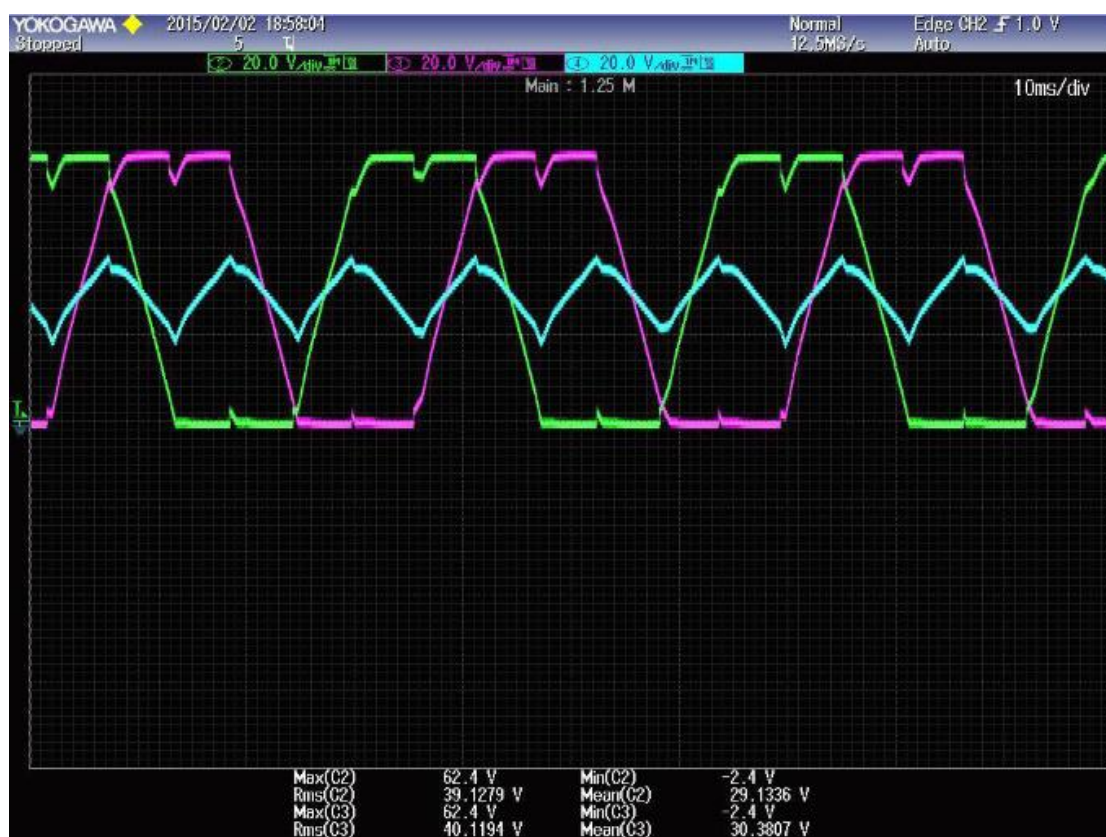


图2 驱动器未上电时的端电压与中性点电压



图3 驱动器上310VDC时的端电压与中性点电压

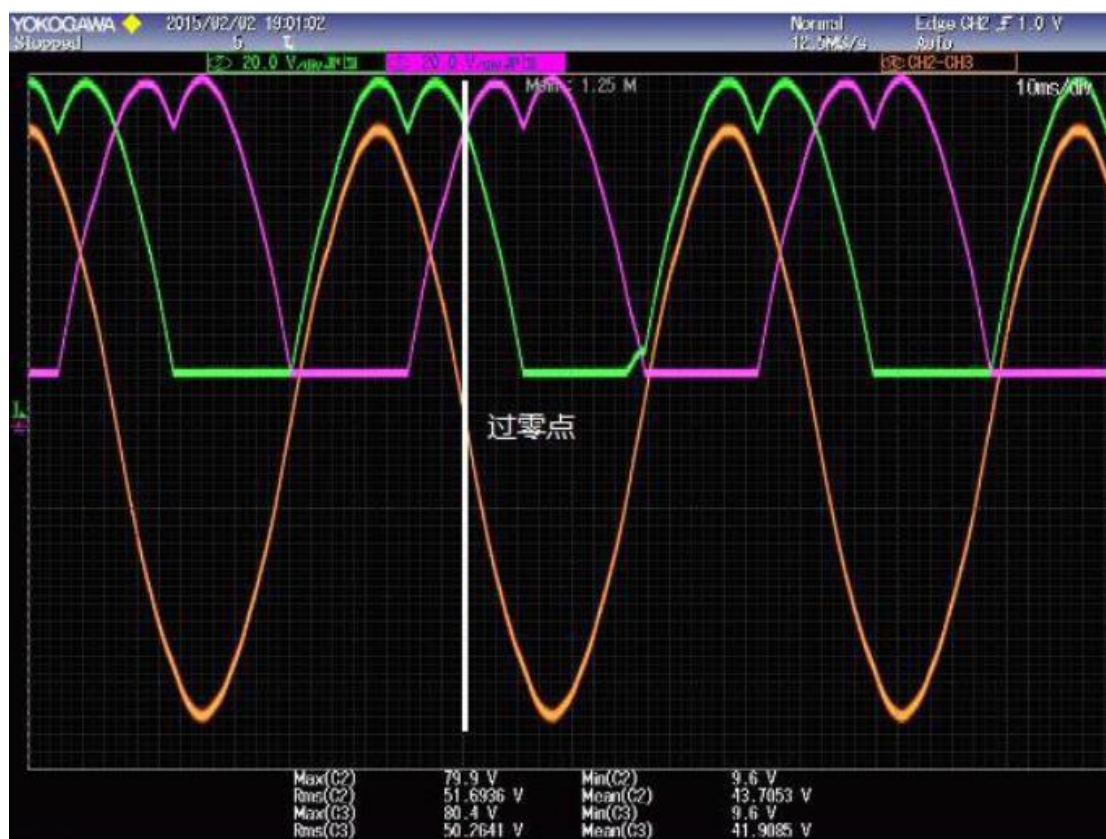


图4 端电压 U_{BO} ， U_{CO} 与其线电压 U_{BC} 波形

1.2 转子角度与速度计算

端电压 U_{BO} 和端电压 U_{CO} 相减，得到其线反电动势 U_{BC} 及其过零点。通过线反电动势的正向过零点与反向过零点（或者过零点前 U_{BO} 和 U_{CO} 的正负）判断得到转子的转向：初始阶段端电压 U_{BO} ， U_{CO} 加和，若在正向过零点时小于反向过零点，转子为正方向旋转；若在正向过零点时大于反向过零点，转子为反向旋转。

转子的转速可以根据两个过零点的间隔时间计算得到，从而获取转子位置角度。

10.2 反电动势采样的滤波与补偿

(1) 低通滤波器设计

本系统设计要求电机转速范围 1000RPM-3000RPM。由于电机为 4 极电机，因此反电动势频率范围可根据转速可得 33Hz-100Hz。同时系统中的 PWM 驱动信号频率为 10KHz。因此低通滤波器的通带边界为 100Hz，阻带边界为 10KHz。

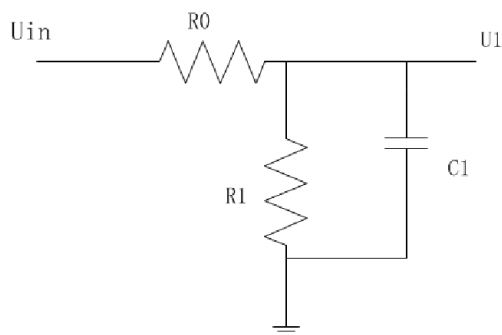


图4.22 低通滤波器电路

如图 4.22 为端电压分压网络与低通滤波电路。其中，两个电阻构成分压网络，共同与电容形成一阶低通滤波网络。

图中节点电压方程：

$$\frac{U_{in}(s) - U_1(s)}{R_0} = \frac{U_1(s)}{R_1 + \frac{1}{sC_1}}$$

$$U_{in}(s) = U_1(s) \left(\frac{R_0}{R_1 + \frac{1}{sC_1}} + 1 \right)$$

由此可得低通滤波网络的传递函数：

$$A_1(s) = \frac{U_1(s)}{U_{in}(s)} = \frac{R_1}{R_0 R_1 C_1 s + R_0 + R_1}$$

10.3 逆时针（负序）相与线反电动势关系

程序

11. 无传感器SMO调试

该函数对电流进行PI调节，使之满足与之对应的转矩大小。

11.1 外同步启动和I_{cur}/frq控制

一般

11.2 角度估算误差与转速电流双闭环

程序

11.3 脉振高频注入法

一般

参考资料

文献与书籍:

10.1-一种无位置传感器无刷直流电机的开环启动方法_张倩等

10.2-基于XMC1302的无传感器风机驱动系统设计_杨济韩

8.6-TMS320x2803x Piccolo ePWM Module Reference Guide_spruge9e

软件工具:

MathType-6.9a

Visio_Premium_2010-32bits

GeoGebra-Windows-Installer-5-0-332-0

controlSUITE-v3.3.3

Code Composer Studio-5.5.0.00077

MATLAB R2014a-8.3.0.532-64bits

用相应的字号并加粗加红色

正文中文小四，代码小五

$$F_k = \begin{pmatrix} \frac{\partial f_1(x_k)}{\partial x_1} & \frac{\partial f_1(x_k)}{\partial x_2} & \cdots & \frac{\partial f_1(x_k)}{\partial x_n} \\ \frac{\partial f_2(x_k)}{\partial x_1} & \frac{\partial f_2(x_k)}{\partial x_2} & \cdots & \frac{\partial f_2(x_k)}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m(x_k)}{\partial x_1} & \frac{\partial f_m(x_k)}{\partial x_2} & \cdots & \frac{\partial f_m(x_k)}{\partial x_n} \end{pmatrix}$$

12. 扩展卡尔曼滤波器EKF算法

1公式推导

1-1 表征系统实际准确值

扩展卡尔曼滤波EKF的状态转移方程和观测方程(系统实际准确值)为

$$(1) \quad x_{k+1} = f(x_k) + V_k \quad \text{其中 } E\{V_k\} = 0; \text{ 而 } \eta(V) \sim (0, Q); \quad Q = E\{VV^T\};$$

$$(2) \quad y_k = C(x_k) + W_k \quad \text{其中 } E\{W_k\} = 0; \text{ 而 } \eta(W) \sim (0, R); \quad R = E\{WW^T\};$$

1-2 非线性系统线性化

利用泰勒展开式对(1)式在上一次的估计后验值 $\hat{x}_k$ 处展开得:

$$x_{k+1} = f(x_k) + V_k = f(\hat{x}_k) + F_k * (x_k - \hat{x}_k) + V_k$$

再利用泰勒展开式对(2)式在本轮的状态预测先验值 $\tilde{x}_{k+1}$ 处展开得:

$$y_{k+1} = C(x_{k+1}) + W_{k+1} = C(\tilde{x}_{k+1}) + H_{k+1} * (x_{k+1} - \tilde{x}_{k+1}) + W_{k+1}$$

其中 F_k 和 H_{k+1} 分别表示函数 $f(x)$ 和 $C(x)$ 在 $\hat{x}_k$ 和 $\tilde{x}_{k+1}$ 处的雅克比矩阵, 这里非线性函数做泰勒展开式(线性化)只保留一阶导, 二阶导及其之后的舍去, 同样也有较好的结果。预测先验未校正, 假设噪声均为加性高斯噪声, 高斯分布的变量经过线性变换后仍然是高斯分布。

整理得到以下两个关系等式, 后续的公式推导用到以下两条式子:

$$(6) \quad f(x_k) - f(\hat{x}_k) = F_k * (x_k - \hat{x}_k)$$

$$(7) \quad C(x_{k+1}) - C(\tilde{x}_{k+1}) = H_{k+1} * (x_{k+1} - \tilde{x}_{k+1})$$

1-3 预测和估计

EKF状态估计大致分为两个阶段: 第一个阶段是预测阶段, 第二个阶段是校正阶段

在第一阶段, 首先由第 k 次的估计结果 $\hat{x}_k$ 来推算下一次估计的预测先验值 $\tilde{x}_{k+1}$, 符号“ $\sim$ ”表示预测值, “预测”的含义是由式(6-118)确定的还没有被校正环节校正的预测量, 即有

$$\tilde{x}_{k+1} = f(\hat{x}_k) \quad (6-119)$$

此预测量 $\tilde{x}_{k+1}$ 对应的预测先验输出 $\tilde{y}_{k+1}$ 为

$$\tilde{y}_{k+1} = C(\tilde{x}_{k+1}) \quad (6-120)$$

由于系统噪声 V_k 和 W_k 是零均值的噪声, 所以没有出现在式(6-120)中。

考虑到EKF的反馈校正环节, 即引入反馈后, 计算出估计后验值 $\hat{x}_{k+1}$:

$$\hat{x}_{k+1} = \tilde{x}_{k+1} + K_{k+1}(y_{k+1} - \tilde{y}_{k+1}) = \tilde{x}_{k+1} + K_{k+1}[y_{k+1} - C(\tilde{x}_{k+1})]$$

1-4 估计后验协方差矩阵

EKF对 K_{k+1} 的选择原则, 是使 $(x_{k+1} - \hat{x}_{k+1})$ 均方差矩阵取得极小, 若令

$$J = E\{(x_{k+1} - \hat{x}_{k+1})^T (x_{k+1} - \hat{x}_{k+1})\} \quad (6-123)$$

则应使 J 取得极小, 式中的 x_{k+1} 为准确值, $(x_{k+1} - \hat{x}_{k+1})$ 为估计误差。

显然, J 取得极小才最有可能使式(6-122)中的 $\hat{x}_{k+1}$ 获得准确结果, 而增益矩阵 K_{k+1} 的选择原则就是使 J 能取得极小。通常, 利用估计后验协方差矩阵 $\hat{P}_{k+1}$ 来推导 K_{k+1} , 因为 J 取得极小可等同于 $\hat{P}_{k+1}$ 取得极小。

定义估计后验误差 $\hat{e}_{k+1} = (x_{k+1} - \hat{x}_{k+1})$

$$\text{估计后验协方差矩阵 } \hat{P}_{k+1} = E\{\hat{e}_{k+1} \hat{e}_{k+1}^T\} = E\{(x_{k+1} - \hat{x}_{k+1})(x_{k+1} - \hat{x}_{k+1})^T\}$$

两者的关系为 $\eta(\hat{e}) \sim (0, P)$, 而 $P = P^T = E\{\hat{e}\hat{e}^T\}$

当估计后验协方差矩阵 $\hat{P}_{k+1}$ 的迹最小时, 方差最小

把具体的矩阵代入定义有:

$$\begin{aligned}\hat{e}_{k+1} &= (x_{k+1} - \hat{x}_{k+1}) = x_{k+1} - [\tilde{x}_{k+1} + K_{k+1} * (y_{k+1} - \tilde{y}_{k+1})] \\ &= x_{k+1} - \{\tilde{x}_{k+1} + K_{k+1} * [C(x_k) + W_{k+1} - C(\tilde{x}_{k+1})]\}\end{aligned}$$

将线性化的关系式 (7) 代入得

$$\begin{aligned}&= x_{k+1} - \{\tilde{x}_{k+1} + K_{k+1} * [H_{k+1} * (x_{k+1} - \tilde{x}_{k+1}) + W_{k+1}]\} \\ &= (x_{k+1} - \tilde{x}_{k+1}) - K_{k+1} * [H_{k+1} * (x_{k+1} - \tilde{x}_{k+1}) - W_{k+1}] \\ &= (I - K_{k+1}H_{k+1})(x_{k+1} - \tilde{x}_{k+1}) - K_{k+1}W_{k+1} \quad \text{注意 } \mathbf{I} \text{ 为单位矩阵, 相当于单式的“1”}\end{aligned}$$

定义预测先验误差 $\tilde{e}_{k+1} = (x_{k+1} - \tilde{x}_{k+1})$

$$\hat{e}_{k+1} = (I - K_{k+1}H_{k+1})\tilde{e}_{k+1} - K_{k+1}W_{k+1}$$

那么具体的

$$\begin{aligned}\hat{P}_{k+1} &= E\{\hat{e}_{k+1}\hat{e}_{k+1}^T\}; \text{ 用到的矩阵运算 } (AB)^T = B^T A^T \text{ 和 } (A+B)^T = A^T + B^T \\ &= E\{ [(I - K_{k+1}H_{k+1})\tilde{e}_{k+1} - K_{k+1}W_{k+1}] [(I - K_{k+1}H_{k+1})\tilde{e}_{k+1} - K_{k+1}W_{k+1}]^T \} \\ &= E\{ [(I - K_{k+1}H_{k+1})\tilde{e}_{k+1} - K_{k+1}W_{k+1}][\tilde{e}_{k+1}^T(I - K_{k+1}H_{k+1})^T - W_{k+1}^T K_{k+1}^T] \} \\ &= E\{ \\ &\quad (I - K_{k+1}H_{k+1})\tilde{e}_{k+1}\tilde{e}_{k+1}^T(I - K_{k+1}H_{k+1})^T \\ &\quad - (I - K_{k+1}H_{k+1})\tilde{e}_{k+1}W_{k+1}^T K_{k+1}^T \\ &\quad - K_{k+1}W_{k+1}\tilde{e}_{k+1}^T(I - K_{k+1}H_{k+1})^T \\ &\quad + K_{k+1}W_{k+1}W_{k+1}^T K_{k+1}^T \\ &\quad \} \end{aligned}$$

因为 $(I - K_{k+1}H_{k+1})$ 和 K_{k+1}^T ; K_{k+1} 和 $(I - K_{k+1}H_{k+1})^T$ 为常数

若 **A** 和 **B** 相互独立, 则 $E\{AB\} = E\{A\}E\{B\}$

再者 $E\{\hat{e}_{k+1}\} = 0$, $E\{\tilde{e}_{k+1}^T\} = 0$; $E\{W_{k+1}\} = 0$, $E\{W_{k+1}^T\} = 0$; 则有:

$$\begin{aligned}(I - K_{k+1}H_{k+1})E\{\tilde{e}_{k+1}W_{k+1}^T\}K_{k+1}^T &= 0 \\ K_{k+1}E\{W_{k+1}\tilde{e}_{k+1}^T\}(I - K_{k+1}H_{k+1})^T &= 0\end{aligned}$$

单位矩阵的转置等于它本身, 即 $I^T = I$

测量噪声协方差矩阵 $R = E\{WW^T\}$, 其为常数, 不受预测或估计步阶次数影响。

定义预测先验协方差矩阵 $\tilde{P}_{k+1} = E\{\tilde{e}_{k+1}\tilde{e}_{k+1}^T\}$

$$\begin{aligned}\hat{P}_{k+1} &= E\{\hat{e}_{k+1}\hat{e}_{k+1}^T\} \\ &= (I - K_{k+1}H_{k+1})E\{\tilde{e}_{k+1}\tilde{e}_{k+1}^T\}(I - K_{k+1}H_{k+1})^T + K_{k+1}E\{W_{k+1}W_{k+1}^T\}K_{k+1}^T \\ &= (I - K_{k+1}H_{k+1})\tilde{P}_{k+1}(I - K_{k+1}H_{k+1})^T + K_{k+1}RK_{k+1}^T \\ &= (\tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1})(I - H_{k+1}^T K_{k+1}^T) + K_{k+1}RK_{k+1}^T\end{aligned}$$

$$\begin{aligned}
&= (\tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1}) - (\tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1})(H_{k+1}^TK_{k+1}^T) \\
&\quad + K_{k+1}RK_{k+1}^T \\
&= (\tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1}) - (\tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T - K_{k+1}H_{k+1}\tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T) \\
&\quad + K_{k+1}RK_{k+1}^T
\end{aligned}$$

展开得到式 (8) :

$$\begin{aligned}
\hat{P}_{k+1} &= \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1} - \tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T \\
&\quad + K_{k+1}H_{k+1}\tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T + K_{k+1}RK_{k+1}^T
\end{aligned}$$

1-5 卡尔曼增益

求估计后验协方差矩阵 $\hat{P}_{k+1}$ 的迹:

因为 $tr(A) = tr(A^T)$; 估计后验协方差矩阵的转置等于它本身, 即 $\hat{P}_{k+1} = \hat{P}_{k+1}^T$

$$tr(\tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T) = tr[(\tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T)^T] = tr(K_{k+1}H_{k+1}\tilde{P}_{k+1})$$

又因为 $tr(A+B) = tr(A) + tr(B)$, 求 $\hat{P}_{k+1}$ 的迹则为:

$$\begin{aligned}
tr(\hat{P}_{k+1}) &= tr(\tilde{P}_{k+1}) - 2tr(K_{k+1}H_{k+1}\tilde{P}_{k+1}) \\
&\quad + tr(K_{k+1}H_{k+1}\tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T) + tr(K_{k+1}RK_{k+1}^T)
\end{aligned}$$

为了要让估计值更接近于真实值, 就要使上面的迹尽可能的小, 因此要取得合适的卡尔曼增益 K_{k+1} , 使得迹得到最小, 即 $\hat{P}_{k+1}$ 的迹对 K_{k+1} 的偏导为 $\mathbf{0}$, 这样就能算出合适的卡尔曼增益 K_{k+1} 了。

利用一下矩阵迹的求导公式:

$$\frac{\partial tr(AB)}{\partial A} = B^T; \quad \frac{\partial tr(ABA^T)}{\partial A} = 2AB$$

那么 $tr(\hat{P}_{k+1})$ 对 K_{k+1} 的偏导为 $\mathbf{0}$ 写为:

$$\frac{\partial tr(\hat{P}_{k+1})}{\partial K_{k+1}} = 0 = 0 - 2(H_{k+1}\tilde{P}_{k+1})^T + 2K_{k+1}H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + 2K_{k+1}R$$

整理得:

$$-2(H_{k+1}\tilde{P}_{k+1})^T + 2K_{k+1}(H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R) = 0$$

$$K_{k+1} = \frac{2(H_{k+1}\tilde{P}_{k+1})^T}{2(H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R)} = \frac{\tilde{P}_{k+1}H_{k+1}^T}{H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R}$$

书写为 $K_{k+1} = \tilde{P}_{k+1}H_{k+1}^T(H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R)^{-1}$

注意这里用到了矩阵的求逆运算, 后续加入说明以便于编写代码

将上式代入式 (8) 得:

$$\begin{aligned}
\hat{P}_{k+1} &= \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1} - \tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T \\
&\quad + K_{k+1}H_{k+1}\tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T + K_{k+1}RK_{k+1}^T
\end{aligned}$$

$$\begin{aligned}
&= \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1} - \tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T \\
&\quad + K_{k+1}(H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R)K_{k+1}^T \\
&= \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1} - \tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T \\
&\quad + \frac{\tilde{P}_{k+1}H_{k+1}^T}{H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R}(H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R)K_{k+1}^T \\
&= \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1} - \tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T + \tilde{P}_{k+1}H_{k+1}^TK_{k+1}^T \\
&= \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1}
\end{aligned}$$

最后得到估计后验协方差矩阵 $\hat{P}_{k+1}$ 最终的表达式

$$\hat{P}_{k+1} = \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1} = (I - K_{k+1}H_{k+1})\tilde{P}_{k+1}$$

1-6 预测先验协方差矩阵

那么现在推导预测先验协方差矩阵 $\tilde{P}_{k+1} = E\{\tilde{e}_{k+1}\tilde{e}_{k+1}^T\}$ 的具体表达式

$$\tilde{P}_{k+1} = E\{\tilde{e}_{k+1}\tilde{e}_{k+1}^T\}$$

其中 $\tilde{e}_{k+1} = (x_{k+1} - \hat{x}_{k+1}) = f(x_k) + V_k - f(\hat{x}_k)$

将线性化的关系式（6）代入得

$$\tilde{e}_{k+1} = F_k * (x_k - \hat{x}_k) + V_k = F_k\hat{e}_k + V_k$$

那么则有

$$\begin{aligned}
\tilde{P}_{k+1} &= E\{\tilde{e}_{k+1}\tilde{e}_{k+1}^T\} = E\{(F_k\hat{e}_k + V_k)(F_k\hat{e}_k + V_k)^T\} \\
&= E\{(F_k\hat{e}_k + V_k)(\hat{e}_k^TF_k^T + V_k^T)\} \\
&= E\{F_k\hat{e}_k\hat{e}_k^TF_k^T + F_k\hat{e}_kV_k^T + V_k\hat{e}_k^TF_k^T + V_kV_k^T\}
\end{aligned}$$

由于 $x_{k+1} = f(x_k) + V_k$, V_k 是作用于 x_{k+1} 上的, 而由 $\hat{e}_k = x_k - \hat{x}_k$ 可知, 和 $\hat{e}_k$ 相关的是 x_k , 所以 $\hat{e}_k$ 和 V_k 是相互独立的, 所以有:

$$E\{\hat{e}_k\}E\{V_k^T\} = E\{V_k\}E\{\hat{e}_k^T\} = 0 * 0 = 0$$

再者, 系统噪声协方差矩阵 $Q = E\{VV^T\}$, 其为常数, 不受预测或估计步阶次数影响。那么预测先验协方差矩阵 $\tilde{P}_{k+1}$ 表达式为:

$$\tilde{P}_{k+1} = F_kE\{\hat{e}_k\hat{e}_k^T\}F_k^T + E\{VV^T\} = F_k\hat{P}_kF_k^T + Q$$

书写为 $\tilde{P}_{k+1} = F_k\hat{P}_kF_k^T + Q$

1-7 算法执行顺序

至此, 已经推导得出扩展卡尔曼滤波器的五个递推公式:

$$\hat{x}_{k+1} = f(\hat{x}_k)$$

$$\tilde{P}_{k+1} = F_k\hat{P}_kF_k^T + Q$$

$$K_{k+1} = \tilde{P}_{k+1}H_{k+1}^T(H_{k+1}\tilde{P}_{k+1}H_{k+1}^T + R)^{-1}$$

$$\begin{aligned}\hat{x}_{k+1} &= \tilde{x}_{k+1} + K_{k+1}(y_{k+1} - \tilde{y}_{k+1}) = \tilde{x}_{k+1} + K_{k+1}[y_{k+1} - C(\tilde{x}_{k+1})] \\ \hat{P}_{k+1} &= \tilde{P}_{k+1} - K_{k+1}H_{k+1}\tilde{P}_{k+1} = (I - K_{k+1}H_{k+1})\tilde{P}_{k+1}\end{aligned}$$

2在永磁同步电机无位置传感器控制的应用

2-1状态方程的表示

表贴式三相PMSM在静止坐标系下的电压方程为：

$$\begin{aligned}u_\alpha &= Ri_\alpha + L_s \frac{di_\alpha}{dt} - \omega_e \psi_f \sin\theta_e \\ u_\beta &= Ri_\beta + L_s \frac{di_\beta}{dt} + \omega_e \psi_f \cos\theta_e\end{aligned}$$

变换为电流方程，可得：

$$\begin{aligned}\frac{di_\alpha}{dt} &= -\frac{R}{L_s}i_\alpha + \omega_e \frac{\psi_f}{L_s} \sin\theta_e + \frac{u_\alpha}{L_s} \\ \frac{di_\beta}{dt} &= -\frac{R}{L_s}i_\beta - \omega_e \frac{\psi_f}{L_s} \cos\theta_e + \frac{u_\beta}{L_s}\end{aligned}$$

在数字化系统中，由于采样周期很短，在每个采样周期内，都可以认为 ω_e 是恒定一样的。则有如下所示关系式：

$$\begin{aligned}\frac{d\omega_e}{dt} &= 0 \\ \frac{d\theta_e}{dt} &= \omega_e\end{aligned}$$

可获得如下的状态方程

$$\begin{aligned}\frac{dx}{dt} &= f(x) + Bu \\ y &= C(x)\end{aligned}$$

$$\begin{aligned}\frac{d}{dt} \begin{pmatrix} i_\alpha \\ i_\beta \\ \omega_e \\ \theta_e \end{pmatrix} &= \begin{pmatrix} -\frac{R}{L_s}i_\alpha + \omega_e \frac{\psi_f}{L_s} \sin\theta_e \\ -\frac{R}{L_s}i_\beta - \omega_e \frac{\psi_f}{L_s} \cos\theta_e \\ 0 \\ \omega_e \end{pmatrix} + \begin{pmatrix} \frac{1}{L_s} & 0 \\ 0 & \frac{1}{L_s} \\ 0 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} u_\alpha \\ u_\beta \end{pmatrix} \\ \begin{pmatrix} i_\alpha \\ i_\beta \end{pmatrix} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} i_\alpha \\ i_\beta \\ \omega_e \\ \theta_e \end{pmatrix}\end{aligned}$$

2-2向前差分离散化

由上述可知，这个是非线性系统，将状态方程合并为 $f(x)$ 并采用向前差分的形式离散化：

$$\begin{aligned}\frac{i_{\alpha,k+1} - i_{\alpha,k}}{T_s} &= -\frac{R}{L_s} i_{\alpha,k} + \omega_{e,k} \frac{\psi_f}{L_s} \sin\theta_{e,k} + \frac{u_{\alpha,k}}{L_s} \\ i_{\alpha,k+1} - i_{\alpha,k} &= -\frac{T_s R}{L_s} i_{\alpha,k} + T_s \omega_{e,k} \frac{\psi_f}{L_s} \sin\theta_{e,k} + \frac{T_s u_{\alpha,k}}{L_s} \\ i_{\alpha,k+1} &= i_{\alpha,k} - \frac{T_s R}{L_s} i_{\alpha,k} + T_s \omega_{e,k} \frac{\psi_f}{L_s} \sin\theta_{e,k} + \frac{T_s u_{\alpha,k}}{L_s}\end{aligned}$$

重写上式，并同理可得出以下式子，其中
状态方程为：

$$\begin{cases} i_{\alpha,k+1} = i_{\alpha,k} - \frac{T_s R}{L_s} i_{\alpha,k} + T_s \omega_{e,k} \frac{\psi_f}{L_s} \sin\theta_{e,k} + \frac{T_s u_{\alpha,k}}{L_s} \\ i_{\beta,k+1} = i_{\beta,k} - \frac{T_s R}{L_s} i_{\beta,k} - T_s \omega_{e,k} \frac{\psi_f}{L_s} \cos\theta_{e,k} + \frac{T_s u_{\beta,k}}{L_s} \\ \omega_{e,k+1} = \omega_{e,k} \\ \theta_{e,k+1} = \theta_{e,k} + T_s \omega_{e,k} \end{cases}$$

输出方程为：

$$\begin{cases} i_{\alpha,k+1} = i_{\alpha,k+1} \\ i_{\beta,k+1} = i_{\beta,k+1} \end{cases}$$

2-3非线性系统线性化

非线性系统线性化需要求出状态方程 $f(x)$ 和输出方程 $C(x)$ 的雅克比矩阵分别用 F_k 和 H_{k+1} 表示。

虽然输出方程是线性的，但为了说明 EKF，同样写成函数形式，求 H_{k+1} 用到的 $C(x_{k+1})$ 和 x_{k+1} 为：

$$\begin{aligned}C(x_{k+1}) &= \begin{pmatrix} C_1(x_{k+1}) \\ C_2(x_{k+1}) \end{pmatrix} = \begin{pmatrix} i_{\alpha,k+1} & +0 & +0 & +0 \\ 0 & +i_{\beta,k+1} & +0 & +0 \end{pmatrix} \\ \text{而 } x_{k+1} &= \begin{pmatrix} x_{1,k+1} \\ x_{2,k+1} \\ x_{3,k+1} \\ x_{4,k+1} \end{pmatrix} = \begin{pmatrix} i_{\alpha,k+1} \\ i_{\beta,k+1} \\ \omega_{e,k+1} \\ \theta_{e,k+1} \end{pmatrix}\end{aligned}$$

需要说明的是矩阵 $C(x_k)$ 有 2 个元素，矩阵 x 有 4 个元素，所以输出方程的雅克比矩阵 H_{k+1} 是 2×4 的矩阵

$$H_{k+1} = \begin{pmatrix} \frac{\partial C_1(x_{k+1})}{\partial x_{1,k+1}} & \frac{\partial C_1(x_{k+1})}{\partial x_{2,k+1}} & \frac{\partial C_1(x_{k+1})}{\partial x_{3,k+1}} & \frac{\partial C_1(x_{k+1})}{\partial x_{4,k+1}} \\ \frac{\partial C_2(x_{k+1})}{\partial x_{1,k+1}} & \frac{\partial C_2(x_{k+1})}{\partial x_{2,k+1}} & \frac{\partial C_2(x_{k+1})}{\partial x_{3,k+1}} & \frac{\partial C_2(x_{k+1})}{\partial x_{4,k+1}} \end{pmatrix}$$

最后求得 $H_{k+1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} = C_{k+1}$

可以看到线性方程的雅克比矩阵还是其线性系数本身，这里做了算法上的融合，即非线性的扩展卡尔曼滤波 EKF 的预测，估计和更新的公式和线性的卡尔曼滤波 KF 公式是一致的。

状态方程是非线性的，求 F_k 用到的 $f(x)$ 和 x 为：

$$f(x_k) = \begin{pmatrix} f_1(x_k) \\ f_2(x_k) \\ f_3(x_k) \\ f_4(x_k) \end{pmatrix} = \begin{pmatrix} i_{\alpha_k} - \frac{T_s R}{L_s} i_{\alpha_k} + T_s \omega_{e_k} \frac{\psi_f}{L_s} \sin \theta_{e_k} + \frac{T_s u_{\alpha_k}}{L_s} \\ i_{\beta_k} - \frac{T_s R}{L_s} i_{\beta_k} - T_s \omega_{e_k} \frac{\psi_f}{L_s} \cos \theta_{e_k} + \frac{T_s u_{\beta_k}}{L_s} \\ \omega_{e_k} \\ \theta_{e_k} + T_s \omega_{e_k} \end{pmatrix}; \quad x_k = \begin{pmatrix} i_{\alpha_k} \\ i_{\beta_k} \\ \omega_{e_k} \\ \theta_{e_k} \end{pmatrix}$$

同理，矩阵 $f(x_k)$ 有 4 个元素，矩阵 x 也有 4 个元素，所以状态方程的雅克比矩阵 F_k 是 4×4 的矩阵

$$F_k = \begin{pmatrix} \frac{\partial f_1(x_k)}{\partial i_{\alpha_k}} & \frac{\partial f_1(x_k)}{\partial i_{\beta_k}} & \frac{\partial f_1(x_k)}{\partial \omega_{e_k}} & \frac{\partial f_1(x_k)}{\partial \theta_{e_k}} \\ \frac{\partial f_2(x_k)}{\partial i_{\alpha_k}} & \frac{\partial f_2(x_k)}{\partial i_{\beta_k}} & \frac{\partial f_2(x_k)}{\partial \omega_{e_k}} & \frac{\partial f_2(x_k)}{\partial \theta_{e_k}} \\ \frac{\partial f_3(x_k)}{\partial i_{\alpha_k}} & \frac{\partial f_3(x_k)}{\partial i_{\beta_k}} & \frac{\partial f_3(x_k)}{\partial \omega_{e_k}} & \frac{\partial f_3(x_k)}{\partial \theta_{e_k}} \\ \frac{\partial f_4(x_k)}{\partial i_{\alpha_k}} & \frac{\partial f_4(x_k)}{\partial i_{\beta_k}} & \frac{\partial f_4(x_k)}{\partial \omega_{e_k}} & \frac{\partial f_4(x_k)}{\partial \theta_{e_k}} \end{pmatrix}$$

$$\text{最后求得 } F_k = \begin{pmatrix} 1 - \frac{T_s R}{L_s} & 0 & T_s \frac{\psi_f}{L_s} \sin \theta_{e_k} & T_s \omega_{e_k} \frac{\psi_f}{L_s} \cos \theta_{e_k} \\ 0 & 1 - \frac{T_s R}{L_s} & -T_s \frac{\psi_f}{L_s} \cos \theta_{e_k} & T_s \omega_{e_k} \frac{\psi_f}{L_s} \sin \theta_{e_k} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & T_s & 1 \end{pmatrix}$$

3矩阵求逆

Step1行列式P14

上述定义表明三阶行列式含 6 项, 每项均为不同行不同列的三个元素的乘积再冠以正负号, 其规律遵循图 1.2 所示的对角线法则: 图中有三条实线看做是平行于主对角线的连线, 三条虚线看做是平行于副对角线的连线, 实线上三元素的乘积冠正号, 虚线上三元素的乘积冠负号.

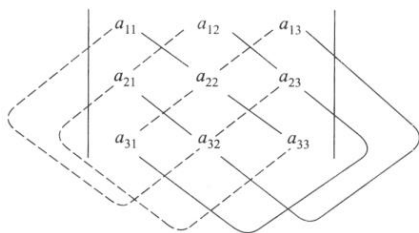


图 1.2

例 2 计算三阶行列式

$$D = \begin{vmatrix} 1 & 2 & -4 \\ -2 & 2 & 1 \\ -3 & 4 & -2 \end{vmatrix}.$$

解 按对角线法则, 有

$$\begin{aligned} D &= 1 \times 2 \times (-2) + 2 \times 1 \times (-3) + (-4) \times (-2) \times 4 - \\ &\quad 1 \times 1 \times 4 - 2 \times (-2) \times (-2) - (-4) \times 2 \times (-3) \\ &= -4 - 6 + 32 - 4 - 8 - 24 = -14. \end{aligned}$$

Step2 余子式和代数余子式 P27

在 n 阶行列式中, 把 (i, j) 元 a_{ij} 所在的第 i 行和第 j 列划去后, 留下来的 $n-1$ 阶行列式叫做 (i, j) 元 a_{ij} 的余子式, 记作 M_{ij} ; 记

$$A_{ij} = (-1)^{i+j} M_{ij},$$

A_{ij} 叫做 (i, j) 元 a_{ij} 的代数余子式.

例如四阶行列式

$$D = \begin{vmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{vmatrix}$$

中 $(3, 2)$ 元 a_{32} 的余子式和代数余子式分别为

$$M_{32} = \begin{vmatrix} a_{11} & a_{13} & a_{14} \\ a_{21} & a_{23} & a_{24} \\ a_{41} & a_{43} & a_{44} \end{vmatrix},$$

$$A_{32} = (-1)^{3+2} M_{32} = -M_{32}.$$

Step3 伴随矩阵 P49

注意伴随矩阵的元素标号 和 代数余子式的标号, (相当于加了符号转置了)

例 10 行列式 $|A|$ 的各个元素的代数余子式 A_{ij} 所构成的如下的矩阵

$$A^* = \begin{pmatrix} A_{11} & A_{21} & \cdots & A_{n1} \\ A_{12} & A_{22} & \cdots & A_{n2} \\ \vdots & \vdots & & \vdots \\ A_{1n} & A_{2n} & \cdots & A_{nn} \end{pmatrix},$$

称为矩阵 A 的伴随矩阵, 简称伴随阵. 试证

Step4求逆P52

例 11 求二阶矩阵 $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ 的逆矩阵.

解 $|A| = ad - bc, A^* = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix},$

利用逆矩阵公式(8), 当 $|A| \neq 0$ 时, 有

$$A^{-1} = \frac{1}{|A|} A^* = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

例 12 求方阵

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 2 & 1 \\ 3 & 4 & 3 \end{pmatrix}$$

的逆矩阵.

解 求得 $|A| = 2 \neq 0$, 知 A^{-1} 存在. 再计算 $|A|$ 的余子式

$$M_{11} = 2, \quad M_{12} = 3, \quad M_{13} = 2,$$

$$M_{21} = -6, \quad M_{22} = -6, \quad M_{23} = -2,$$

$$M_{31} = -4, \quad M_{32} = -5, \quad M_{33} = -2,$$

得

$$A^* = \begin{pmatrix} M_{11} & -M_{21} & M_{31} \\ -M_{12} & M_{22} & -M_{32} \\ M_{13} & -M_{23} & M_{33} \end{pmatrix} = \begin{pmatrix} 2 & 6 & -4 \\ -3 & -6 & 5 \\ 2 & 2 & -2 \end{pmatrix},$$

所以

$$A^{-1} = \frac{1}{|A|} A^* = \begin{pmatrix} 1 & 3 & -2 \\ -\frac{3}{2} & -3 & \frac{5}{2} \\ 1 & 1 & -1 \end{pmatrix}.$$

注释如下:

```
//分母为 [3*3]矩阵的行列式|A|即一个数值, 分子是伴随矩阵对应元素的 代数余子式 A_ij = (-1)^(i+j) * M_ij
//      (+M11  -M21  +M31)
// 伴随矩阵 A^* = (-M12  +M22  -M32) 注意伴随矩阵的元素标号 和 代数余子式的标号, (相当于加了符号转置了)
//      (+M13  -M23  +M33)
//
//      (+M11/|A|  -M21/|A|  +M31/|A|)
// 求逆矩阵 A^-1 = 1/|A| * A^* = (-M12/|A|  +M22/|A|  -M32/|A|)
//      (+M13/|A|  -M23/|A|  +M33/|A|)
```

4其他的矩阵运算

$$\begin{cases} y_1 = a_{11}x_1 + a_{12}x_2 + a_{13}x_3, \\ y_2 = a_{21}x_1 + a_{22}x_2 + a_{23}x_3, \end{cases} \quad (4)$$

$$\begin{cases} x_1 = b_{11}t_1 + b_{12}t_2, \\ x_2 = b_{21}t_1 + b_{22}t_2, \\ x_3 = b_{31}t_1 + b_{32}t_2, \end{cases} \quad (5)$$

若想求出从 t_1, t_2 到 y_1, y_2 的线性变换, 可将(5)代入(4), 便得

$$\begin{cases} y_1 = (a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31})t_1 + (a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32})t_2, \\ y_2 = (a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31})t_1 + (a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32})t_2. \end{cases} \quad (6)$$

线性变换(6)可看成是先作线性变换(5)再作线性变换(4)的结果. 我们把线性变换(6)叫做线性变换(4)与(5)的乘积, 相应地把(6)所对应的矩阵定义为(4)与(5)所对应的矩阵的乘积, 即

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{pmatrix} = \begin{pmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} & a_{11}b_{12} + a_{12}b_{22} + a_{13}b_{32} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} & a_{21}b_{12} + a_{22}b_{22} + a_{23}b_{32} \end{pmatrix}.$$

矩阵的乘法

一般地, 我们有

定义 4 设 $A = (a_{ij})$ 是一个 $m \times s$ 矩阵, $B = (b_{ij})$ 是一个 $s \times n$ 矩阵, 那么规定矩阵 A 与矩阵 B 的乘积是一个 $m \times n$ 矩阵 $C = (c_{ij})$, 其中

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{is}b_{sj} = \sum_{k=1}^s a_{ik}b_{kj} \quad (i=1, 2, \cdots, m; j=1, 2, \cdots, n), \quad (7)$$

并把此乘积记作

$$C = AB.$$

按此定义, 一个 $1 \times s$ 行矩阵与一个 $s \times 1$ 列矩阵的乘积是一个 1 阶方阵, 也就是一个数

$$(a_{i1}, a_{i2}, \cdots, a_{is}) \begin{pmatrix} b_{1j} \\ b_{2j} \\ \vdots \\ b_{sj} \end{pmatrix} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{is}b_{sj} = \sum_{k=1}^s a_{ik}b_{kj} = c_{ij},$$

由此表明乘积矩阵 $AB=C$ 的 (i, j) 元 c_{ij} 就是 A 的第 i 行与 B 的第 j 列的乘积.

必须注意: 只有当第一个矩阵(左矩阵)的列数等于第二个矩阵(右矩阵)的行数时, 两个矩阵才能相乘.

参考资料

文献与书籍:

- 1, 现代电机控制技术第2版_王成元等
- 2, 现代永磁同步电机控制原理及MATLAB仿真_袁雷等
- 3, 工程数学线性代数第六版_同济大学数学系编

4, Bilibili门户UP主DR_CAN的卡尔曼滤波器1~5集视频教程_网址为
https://www.bilibili.com/video/BV1hC4y1b7K7/?spm_id_from=333.788.recommend_more_video.0
5, 论坛用户GHeIpU的CSDN博文_网址为
https://blog.csdn.net/weixin_42647783/article/details/89054641

软件工具:

1, 知乎用户的数学公式快速输入教程
<https://zhuanlan.zhihu.com/p/215440677>

2, 公式拍照识别latex应用
<https://latexlive.com/>

书签笔记

根据浏览器书签整理出该文档。

1. 电流检测中AD采样值处理的高效方法

标签: 硬件: 电流采样 软件: 标幺化 C语言: 数值的转换

在电机控制软件的编写过程中,经常要处理由AD采样回来的电流值。由于电流有正有负,电流传感器输出地电压也是以0为 midpoint,而一般AD的输入只能为正电压,所以电流传感器出来的电压量需要通过加减运算电路加上一个偏移值再送入AD,以保证进入AD的值恒为正。

这里AD以TI的2812的片内AD为例,其输入范围是0~3V,电流传感器以LEM的LA25NP为例,量程正负25A,通常采用的电路是将LEM传感器的输出经过一个100欧的电阻接到地,获得一个以0为 midpoint的电压信号,然后在送入比例运算电路,在比例运算电路的另一个输入端加入1.5V的偏移量,这样在满量程范围内使输入AD的电压信号始终保持在0~3V之内。电流为0的时候送入AD的电压是1.5V

在程序中进行电流运算时,一般要把AD采样结果取回来转换成ADC端口的电压,然后再减去1.5V的偏移值,这样的到一个有符号数,然后再进行运算。

在TI提供的电机驱动库代码时,发现里边使用了一种高效的处理方法,代码很简单,是这样的:

```
DatQ15 = AdcRegs.ADCRESULT0^0x8000;
```

2812的AD是12位的,AD采样结果16位变量左对齐存储在高12位。因此当AD满量程时,ADCRESULT0中的值是0xFFF0。这段代码仅仅通过一个按位异或运算

就将AD的值减去了1.5V的偏移，并变成了有符号数。

举例说明，当电压为3V时，采样值是0xFFF0，异或运算后最高位的1变成了0，其余位不变，接结果是7ff0，如果将最高位看做符号位，剩下的15位作为数据位，即强制把无符号16位变量转换成有符号16位变量，也就是变成了Q15格式的数值，电流基值为上述所说25A。

如果电流采样为电阻采样，电流基值为 $U_{adc}/(2*R_{shunt}*Gain)$ ，如Uadc为3V，采样电阻R_shunt为0.05欧姆，电路放大倍数Gain为2，则电流基值为15A。

再举一个负电流的例子，如电流采样为电阻采样，Uadc为3V，采样电阻R_shunt为0.05欧姆，电路放大倍数Gain为2，则电流基值为 $U_{adc}/(2*R_{shunt}*Gain)=15A$ 。现假设ADC端口电压为0.75V，ADC采样值为1024，十六进制为0x0400，由于左对齐即左移了四位为0x4000，异或了0x8000后，为0xC000（无符号为49152，有符号为-（65536-49152）=-16384），这个作为Q15数值十进制为-16384，那么它代表的电流大小为 $15A*(-16384/32768)=-7.5A$

列举以下表格，以便明白双极性电流到有符号Q格式代表的小数转换，并用于内部控制。

实际 电流 (A)	0.05 欧姆 放大 2 倍 偏置 1.5V	3V-12 位 ADC 数值 DEC 进制	十六进 制	左对齐 的十六 进制	异或 0x8000	U16 转 S16	软件计算电 流大小 (A)
15	3	4095	0x0FFF	0xFFF0	0x7FF0	32752	14.993
...	...	...	...	...	...	...	...
7.5	2.25	3071	0x0BFF	0xBFF0	0x3FF0	16368	7.493
...	...	...	...	...	...	...	...

0	1.5	2048	0x07FF	0x7FF0	0xFFF0	-16	-0.007
...	...	...	...	...	...	...	...
-7.5	0.75	1024	0x03FF	0x3FF0	0xBFF0	-1640 0	-7.507
...	...	...	...	...	...	...	...
-15	0	0	0x0000	0x0000	0x8000	-3276 8	-15.000

2. 两种结构体成员初始化方式：顺序和乱序

软件:电机参数, PID等 C语言: 数值的转换

C语言中的结构体类型实例在进行成员初始化时, 我们通常采用顺序初始化的方式。在ST的官方开源代码中, 发现还可以采用乱序的方式进行初始化, 并且这种方式可以明确指定每个成员的值, 看起来比较清晰。

在几个算法中都有运用到PID模块, 如电流控制, 转速控制, 弱磁控制, 观测器PLL输出角度等等。在弱磁FW算法PID模块中结构体声明如下:

```
typedef struct PID_Handle
{
    int16_t  hDefKpGain;          /**< Default @f$K_{pg}@f$ gain */
    int16_t  hDefKiGain;          /**< Default @f$K_{ig}@f$ gain */
    int16_t  hKpGain;             /**< @f$K_{pg}@f$ gain used by PID component */
    int16_t  hKiGain;             /**< @f$K_{ig}@f$ gain used by PID component */
    int32_t  wIntegralTerm;       /**< integral term */
    int32_t  wUpperIntegrallLimit;
    int32_t  wLowerIntegrallLimit;
    int16_t  hUpperOutputLimit;   /**< Upper limit used to saturate the PI output */
    int16_t  hLowerOutputLimit;   /**< Lower limit used to saturate the PI output */
    uint16_t hKpDivisor;
    uint16_t hKiDivisor;
    uint16_t hKpDivisorPOW2;
    uint16_t hKiDivisorPOW2;
    int16_t  hDefKdGain;          /**< Default Kd gain */
    int16_t  hKdGain;             /**< Kd gain used by PID component */
    uint16_t hKdDivisor;
    uint16_t hKdDivisorPOW2;
    int32_t  wPrevProcessVarError;
} PID_Handle_t;
```

在弱磁FW算法PID模块中结构体实例初始化如下,

另外需要注意的是: (作者注)

1, 结构体乱序初始化是C99中的新特性, 需要支持C99规范的编译器支持, 比如VS2013编译器cl.exe, 但是VS2010编译器就不支持C99规范(支持C89规范)。

2, linux内核中就是采用乱序这种方式初始化struct。

弱磁FW算法里PID输出为负数的Id, 如果正常情况下输出为零, 所以上限最大值为0, 下限最小值为-INT16_MAX, 这个是与其它PID模块初始化不一样的地方, 需要仔细地阅读。

```
PID_Handle_t PIDFluxWeakeningHandle_M1 =
{
    .hDefKpGain          = (int16_t)FW_KP_GAIN,
    .hDefKiGain          = (int16_t)FW_KI_GAIN,
    .wUpperIntegralLimit = 0,
    .wLowerIntegralLimit = (int32_t)(-NOMINAL_CURRENT) * (int32_t)FW_KIDIV,
    .hUpperOutputLimit   = 0,
    .hLowerOutputLimit   = -INT16_MAX,
    .hKpDivisor           = (uint16_t)FW_KPDIV,
    .hKiDivisor           = (uint16_t)FW_KIDIV,
    .hKpDivisorPOW2       = (uint16_t)FW_KPDIV_LOG,
    .hKiDivisorPOW2       = (uint16_t)FW_KIDIV_LOG,
    .hDefKdGain           = 0x0000U,
    .hKdDivisor           = 0x0000U,
    .hKdDivisorPOW2       = 0x0000U,
};
```